Collection
Ressources Informatiques

VBA Access 2007

Programmer sous Access

Michèle AMELOT



Microsoft GOLD CERTIFIED Partner

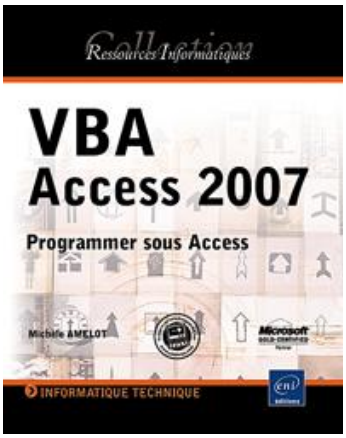
INFORMATIQUE TECHNIQUE

eni éditions

VBA Access 2007

Programmer sous Access

Michèle AMELOT



Résumé

A la fois simple, pratique et complet, ce livre s'adresse **aux développeurs et aux utilisateurs avertis** souhaitant créer des applications professionnelles conviviales, fiables et performantes sous Access. Outre les éléments de base du langage VBA vous permettant de créer vos propres procédures et fonctions, vous apprendrez à manipuler vos données en utilisant les objets **DAO** ou **ADO** et le langage **SQL**, à personnaliser vos formulaires et états, à concevoir un ruban Access spécifique à votre application, à piloter les autres applications Office 2007 via la **technologie Automation**, à exporter des informations sur Internet (au format XML ou HTML) et à utiliser les **API Windows**.
En plus des nombreux exemples fournis dans cet ouvrage - en téléchargement sur notre espace Livres (www.eni-livres.com) - le dernier chapitre vous guide dans la création d'une mini-application Access.

L'auteur

Consultante et formatrice, **Michèle Amelot** est spécialisée dans la création de logiciels spécifiques autour des technologies Microsoft. Elle accompagne des équipes de développement dans la conduite de leurs projets. C'est, entre autres, cette connaissance approfondie des besoins des utilisateurs et des développeurs, qui lui permet dans ses ouvrages de faciliter l'apprentissage et la pratique de la programmation en langage VBA.

Ce livre numérique a été conçu et est diffusé dans le respect des droits d'auteur. Toutes les marques citées ont été déposées par leur éditeur respectif. La loi du 11 Mars 1957 n'autorisant aux termes des alinéas 2 et 3 de l'article 41, d'une part, que les "copies ou reproductions strictement réservées à l'usage privé du copiste et non destinées à une utilisation collective", et, d'autre part, que les analyses et les courtes citations dans un but d'exemple et d'illustration, "toute représentation ou reproduction intégrale, ou partielle, faite sans le consentement de l'auteur ou de ses ayants droit ou ayant cause, est illicite" (alinéa 1er de l'article 40). Cette représentation ou reproduction, par quelque procédé que ce soit, constituerait donc une contrefaçon sanctionnée par les articles 425 et suivants du Code Pénal. Copyright Editions ENI

Introduction

Access 2007 dispose du langage de programmation Visual Basic Edition Application (VBA) qui lui permet d'étendre ses fonctionnalités pour vous offrir le meilleur de ses performances.

C'est un langage puissant, qui ne présente aucune difficulté pour être compris par les débutants. Les fonctions de VBA sont dérivées en grande partie de Visual Basic auxquelles s'ajoutent des instructions (objets, méthodes, fonctions...) spécifiques, nécessaires à la manipulation des objets d'Access.

Pour bien exploiter le contenu de ce manuel, il est impératif d'avoir manipulé Access 2007 en tant qu'utilisateur, c'est-à-dire savoir gérer (créer, modifier, supprimer, manipuler) les tables, les requêtes, les formulaires, les pages d'accès aux données et les états. Des connaissances générales en programmation sont également pré-requises.

© ENI Editions - All rigths reserved

Rappels

Une base Access est constituée d'un unique fichier portant l'extension .ACCDB et contenant les différents objets de l'application. Ces objets sont :

Les tables

Elles contiennent les données de la base et peuvent être en liaison les unes avec les autres.

Les requêtes

Elles permettent d'effectuer :

- des sélections et des tris sur les tables,

- des calculs simples et des analyses croisées,

- des actions (ajout, suppression, mise à jour d'enregistrements, création de tables),

- des opérations SQL.

Les formulaires

Ils sont utilisés pour consulter ou mettre à jour les données.

Les états

Ils sont utilisés pour imprimer des rapports, depuis la simple liste d'une table jusqu'à des rapports plus complexes comprenant des regroupements d'enregistrements et des calculs.

Les macros

Elles permettent l'automatisation de plusieurs tâches avec un langage particulier mais ne permettent pas d'atteindre la puissance de développement de VBA.

Les modules

Ils contiennent une partie du code VBA de l'application, à savoir des déclarations, des fonctions, des procédures...

© ENI Editions - All rigths reserved

Principes fondamentaux de VBA

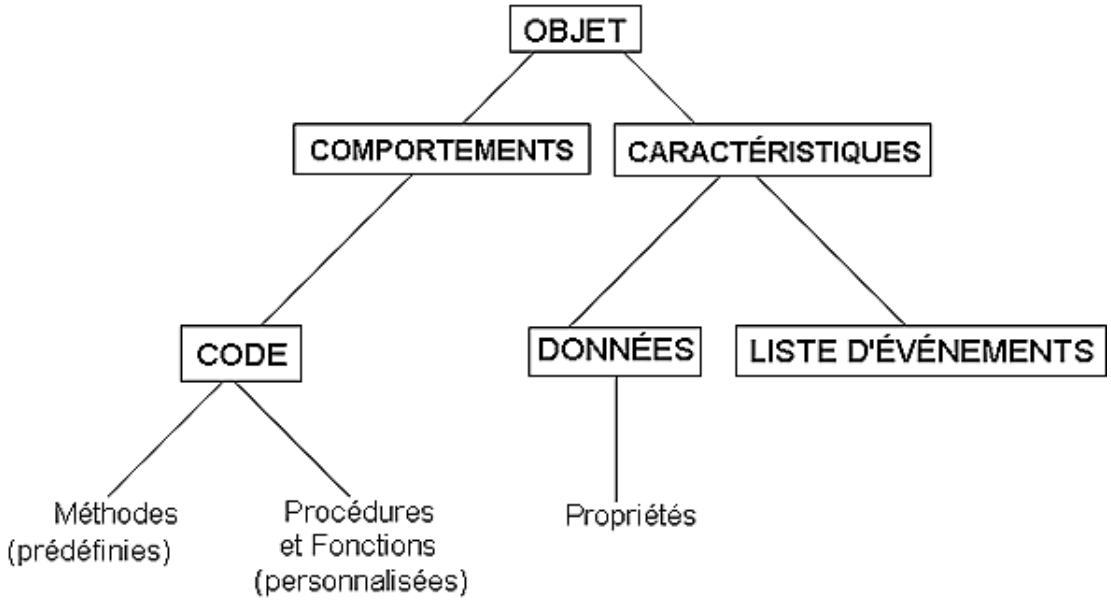
VBA présente un environnement de programmation orienté objet.

Un objet est une entité autonome possédant des caractéristiques (propriétés : taille, couleur, valeur...) et des comportements (méthodes) qui lui sont propres. Certains objets possèdent une représentation visuelle (Formulaire, Etats...), tandis que d'autres ne sont accessibles qu'en code VBA (DBEngine, Container...).

La plupart des objets visuels ont la possibilité de réagir à des événements (sur ouverture, après mise à jour...).

La programmation VBA permet d'induire, en fonction des événements, des comportements particuliers sur les objets par le code : méthodes, procédures et fonctions. Par ce biais, elle facilite l'écriture de code car elle repose sur une conception modulaire. Elle permet ainsi une grande souplesse de fonctionnement et une maintenance plus aisée.

Structure d'un objet



Événement : Action utilisateur ou système agissant sur un objet et déclenchant le code correspondant.

Méthode : Code prédéfini se rapportant à un type d'objet.

Procédure : Séquence d'instructions ne renvoyant pas de valeur.

Fonction : Séquence d'instructions renvoyant une valeur.

© ENI Editions - All rigths reserved

Les nouveautés d'Access 2007

Access 2007 fait partie de la suite logicielle Microsoft Office 2007 (également appelée Office 12) et bénéficie par conséquent de la plupart des améliorations de cette version :

- Nouvelle interface utilisateur : ruban et onglets regroupant toutes les commandes et remplaçant les menus et barres d'outils.

- Partage efficace des informations entre applications Office (notamment échange d'informations entre Access et Outlook et Access et SharePoint).

- Thèmes Office permettant de personnaliser aisément la présentation des documents.

- Enregistrement des fichiers au format pdf et xps...

D'autres nouveautés ont été apportées à Access 2007 :

- Nouveaux modèles d'applications complètes ;

- Volet de navigation ;

- Tris et filtrages optimisés ;

- Dispositions permettant de personnaliser la présentation des formulaires et états ;

- Nouveaux types de champs : champs à plusieurs valeurs, type de données "Pièce jointe" permettant de stocker des informations provenant de tout type de fichier, texte enrichi dans les champs mémo...

Contrairement à l'interface d'Access, l'environnement de développement a conservé ses menus et barres d'outils. Toutefois, la documentation en ligne a été considérablement améliorée : aspect de Visual Studio, exemples de code de tâche et modèle objet enrichi (informations sur les changements par rapport aux versions précédentes).

© ENI Editions - All rigths reserved

Passage des macros à VBA

Les macros représentent une manière facile de gérer des enchaînements d'actions simples, tels que l'ouverture et la fermeture de formulaires.

Toutefois VBA présente bien d'autres avantages :

- il facilite la maintenance des bases de données car les procédures événementielles Visual Basic sont intégrées dans la définition du formulaire ou de l'état,
- il conforte et améliore la gestion des erreurs,
- il permet d'accomplir des actions au niveau du système,
- il autorise la manipulation d'un seul enregistrement à la fois,
- il accepte le passage d'arguments au code en cours d'exécution.

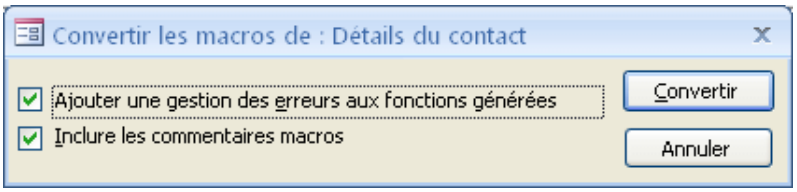
» Toute macro peut être convertie automatiquement en code VBA.

Conversion de macros dans un formulaire ou un état

- Dans le volet de navigation, cliquez avec le bouton droit de la souris sur le formulaire ou l'état, puis cliquez sur **Mode création**.
- Activez l'onglet **Outils de base de données** du ruban.
- Cliquez sur l'icône **Convertir les macros de formulaire en Visual Basic** ou **Convertir les macros d'état en Visual Basic** située en bas à droite du groupe **Macros** :



La boîte de dialogue suivante s'affiche alors :



- Désactivez si vous le souhaitez les options proposées par défaut. Il est toutefois recommandé de conserver la gestion des erreurs pour la fiabilité de l'application, et les commentaires pour simplifier la lecture du code.
- Cliquez sur **Convertir** pour continuer.

Conversion de macros globales

- Dans le volet de navigation, sélectionnez la macro que vous souhaitez convertir.
- Activez l'onglet **Outils de base de données** du ruban.
- Cliquez sur l'icône **Convertir les macros en Visual Basic** située en bas à droite du groupe **Macros**.

© ENI Editions - All rigths reserved

- Dans la boîte de dialogue, sélectionnez les options souhaitées.

- Cliquez sur **Convertir** pour continuer : la macro est convertie en code VBA dans un module intitulé "Macro convertie" suivi du nom de la macro.

Les macros et la sécurité

Les paramètres de sécurité des macros permettent de contrôler ce qui se produit lorsque vous ouvrez une base de données Access contenant une macro.

Modification des paramètres de sécurité

Par défaut, lorsque vous installez Access, les macros sont désactivées. Si vous ouvrez un classeur contenant des macros, l'avertissement suivant est affiché dans la barre des messages :



Si vous cliquez sur **Options**, une boîte de dialogue de sécurité s'affiche pour vous permettre d'activer les macros contenues dans votre base Access. Vous pouvez alors sélectionner l'option **Activer ce contenu** et cliquer sur le bouton **OK** : les macros sont alors activées et l'alerte de sécurité disparaît.

Vous pouvez également modifier les paramètres de sécurité pour toutes les bases de données Access de la façon suivante :

- Cliquez sur le bouton Microsoft Office  , puis sur le bouton 

(situé en bas de la boîte de dialogue).

- Sélectionnez la catégorie **Centre de gestion de la confidentialité** dans le menu de gauche.

- Cliquez sur le bouton de commande **Paramètres du Centre de gestion de la confidentialité**.

- Sélectionnez la catégorie **Paramètres des macros** dans le menu de gauche.

» En cas de changement des paramètres de sécurité, la nouvelle option choisie s'appliquera à toutes les bases de données Access à l'exclusion de la base actuellement active. Pour l'appliquer à cette base, vous devez la fermer puis l'ouvrir à nouveau.

Description des différentes options de sécurité

Désactiver toutes les macros sans notification

Toutes les macros et toutes les alertes de sécurité sont désactivées.

Désactiver toutes les macros avec notification

Option par défaut. Les macros sont désactivées mais l'alerte de sécurité est affichée.

Désactiver toutes les macros à l'exception des macros signées numériquement

Si les macros d'un classeur sont signées numériquement par un émetteur approuvé, les macros peuvent être exécutées. Si l'émetteur n'a pas été approuvé, une notification est affichée : il est alors possible d'activer les macros signées ou d'approuver l'émetteur. Les macros non signées numériquement ne pourront pas être activées.

Activer toutes les macros

Si cette option est sélectionnée, toutes les macros sont activées. Il n'est pas recommandé d'utiliser ce paramètre en permanence.

» Quelle que soit l'option choisie, si un logiciel antivirus fonctionnant avec Microsoft Office System 2007 est installé, les classeurs

© ENI Editions - All rigths reserved

contenant des macros sont analysés avant d'être ouverts.

Editeurs approuvés

Les applications Access développées par des éditeurs approuvés se caractérisent par les critères suivants :

- Le code est signé par le développeur à l'aide d'une signature numérique ;

- La signature numérique est valide ;

- La signature numérique est effective (non expirée) ;

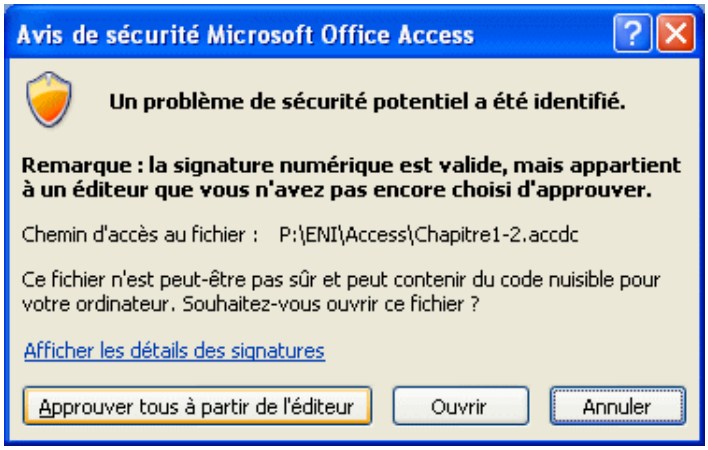
- Le certificat associé à la signature numérique a été émis par une autorité de certification reconnue.

Pour ajouter un éditeur à la liste **Editeurs approuvés** :

- Activez l'option **Désactiver toutes les macros avec notification** dans les paramètres de sécurité (cf. **Généralités - Passage des macros à VBA - Les macros et la sécurité - Modification des paramètres de sécurité** ci-dessus).

- Ouvrez une base de données Access dont le code est signé par un Editeur approuvé.

- Cliquez sur le bouton de commande **Approuver tous à partir de l'éditeur**.



Vous pouvez à tout moment connaître la liste des éditeurs approuvés de la façon suivante :

- Cliquez sur le bouton Microsoft Office, puis sur le bouton Options Access (situé en bas de la boîte de dialogue).

- Sélectionnez la catégorie **Centre de gestion de la confidentialité** dans le menu de gauche.

- Cliquez sur le bouton de commande **Paramètres du Centre de gestion de la confidentialité**.

- Sélectionnez la catégorie **Editeurs approuvés** dans le menu de gauche.

Il est également possible de désactiver un éditeur approuvé en cliquant sur le bouton **Supprimer**.

Emplacements approuvés

Tout fichier enregistré dans un emplacement approuvé peut être ouvert sans l'intervention du Centre de gestion de la confidentialité.

Vous pouvez à tout moment ajouter un dossier dans la liste des **Emplacements approuvés** de la façon suivante :

© ENI Editions - All rigths reserved

- Cliquez sur le bouton Microsoft Office  , puis sur le bouton 

(situé en bas de la boîte de dialogue).
- Sélectionnez la catégorie **Centre de gestion de la confidentialité** dans le menu de gauche.
- Cliquez sur le bouton de commande **Paramètres du Centre de gestion de la confidentialité**.
- Sélectionnez la catégorie **Emplacements approuvés** dans le menu de gauche.
- Cliquez sur **Ajouter un nouvel enregistrement**.
- Complétez les informations de la boîte de dialogue **Emplacement de Microsoft Office approuvé.**



- Cliquez sur le bouton **OK** pour ajouter l'emplacement.

© ENI Editions - All rigths reserved

L'environnement de développement IDE

L'IDE (*Integrated Development Environment*) est l'environnement dans lequel vous pouvez saisir, modifier et tester votre code VBA. Cet environnement est également appelé VBE (*Visual Basic Editor*).

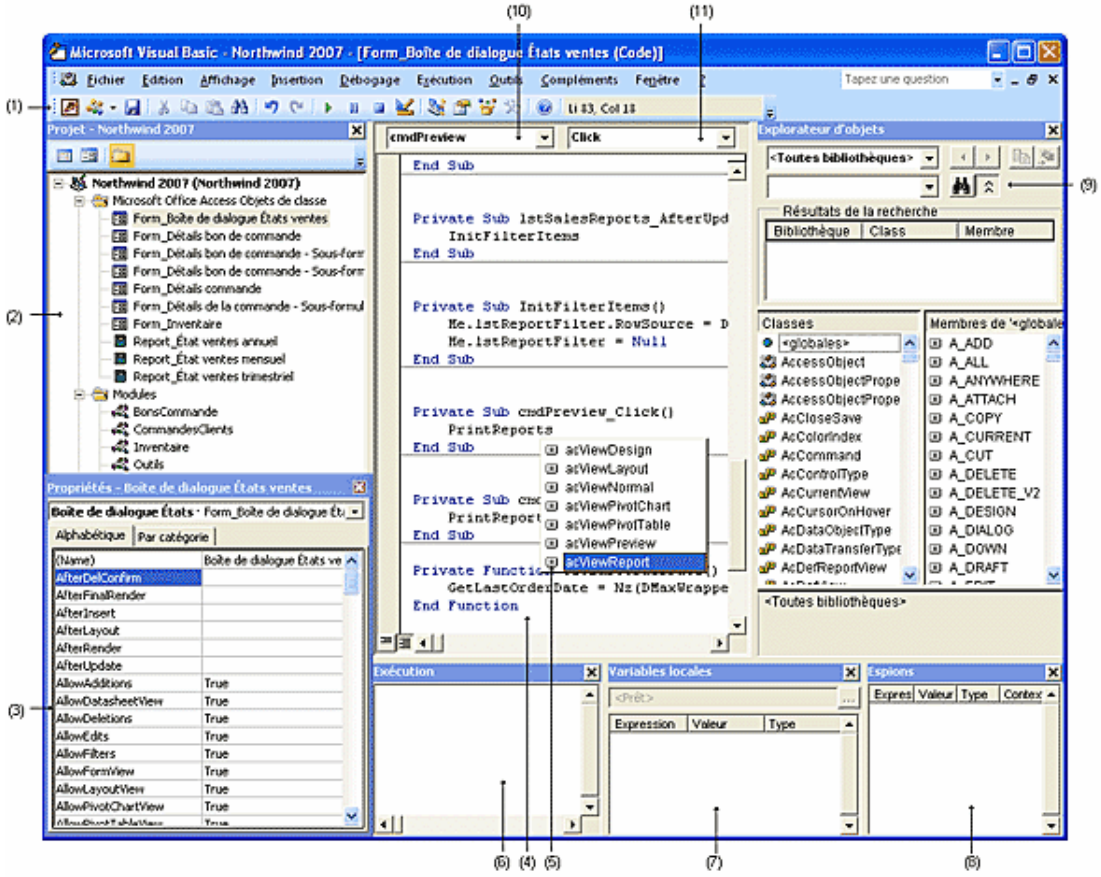
L'IDE met à votre disposition de nombreux outils qui facilitent la programmation et la mise au point du code VBA : outils de débogage, assistance à la saisie, explorateur d'objets...

- En utilisant la combinaison de touches **Ctrl F11**.

- En cliquant sur l'icône située dans l'onglet **Outils de base de données**.

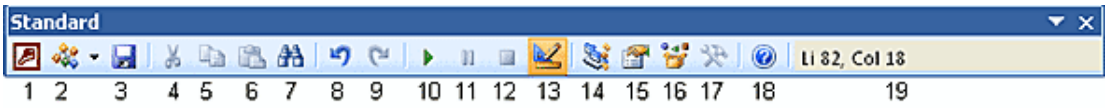
- En sélectionnant une procédure évènementielle depuis une feuille de propriétés dans un formulaire ou un état.

Fenêtre de l'environnement IDE



» Toutes les fenêtres de l'environnement IDE peuvent être affichées à partir du menu **Affichage**.

La barre d'outils Standard (1)



1 Afficher Access
2 Insérer un module, un module de classe ou une procédure (Function ou Sub)
3 Enregistrer
4 Couper

© ENI Editions - All rigths reserved

5 Copier
6 Coller
7 Rechercher
8 Annuler
9 Rétablir
10 Exécuter
11 Arrêt
12 Réinitialiser
13 Mode création
14 Afficher l'explorateur de projets
15 Afficher la fenêtre Propriétés
16 Afficher l'explorateur d'objets
17 Afficher la boîte à outils
18 Aide sur Microsoft Visual Basic
19 Position courante dans la fenêtre de Code

L'Explorateur de projet (2)

Tous les modules de la base de données ou du projet VBA sont affichés dans l'explorateur de projets, selon une structure arborescente. Ils sont regroupés en trois ensembles :

- les modules de formulaires ou d'états,

- les modules standards,

- les modules de classe indépendants.

La fenêtre Propriétés (3)

Elle permet d'afficher les propriétés des différents modules. Pour afficher les propriétés d'un formulaire ou d'un état, celui-ci doit être ouvert dans Access ; si ce n'est pas le cas, sélectionnez le formulaire ou l'état dans l'explorateur de projet et choisissez l'option **Objet** du menu **Affichage**.

La fenêtre Code (4)

Dans cette fenêtre se trouvent deux zones de liste déroulantes :

- la zone objet (**10**) affiche la liste les objets du module,

- la zone procédure (**11**) affiche les procédures ou les événements de l'objet sélectionné dans la zone objet. Les événements déjà utilisés apparaissent en gras.

Le complément automatique d'instructions (5)

Le complément automatique d'instructions présente les informations qui sont le complément logique de l'instruction au niveau du point d'insertion.

Par exemple, la liste déroulante des méthodes et propriétés disponibles pour un objet s'affiche automatiquement lorsque vous tapez le nom de l'objet suivi d'un point.

Si cette liste n'est pas active, sélectionnez **Options** dans le menu **Outils** et cochez la case **Complément automatique des instructions** de l'onglet **Éditeur**.

La fenêtre Exécution (6)

Elle permet d'afficher les valeurs des variables, de les modifier et d'exécuter des instructions.

» Les fenêtres **Exécution**, **Variables locales** et **Espions** sont utilisées lors du débogage de l'application.

© ENI Editions - All rigths reserved

La fenêtre Variables locales (7)

Elle contient toutes les valeurs des variables accessibles dans la portée en cours.

La fenêtre Espions (8)

Elle affiche les valeurs des variables qui ont été définies comme variables espions.

L'Explorateur d'objets (9)

Il permet de visualiser, pour chaque objet, ses propriétés, méthodes et constantes.

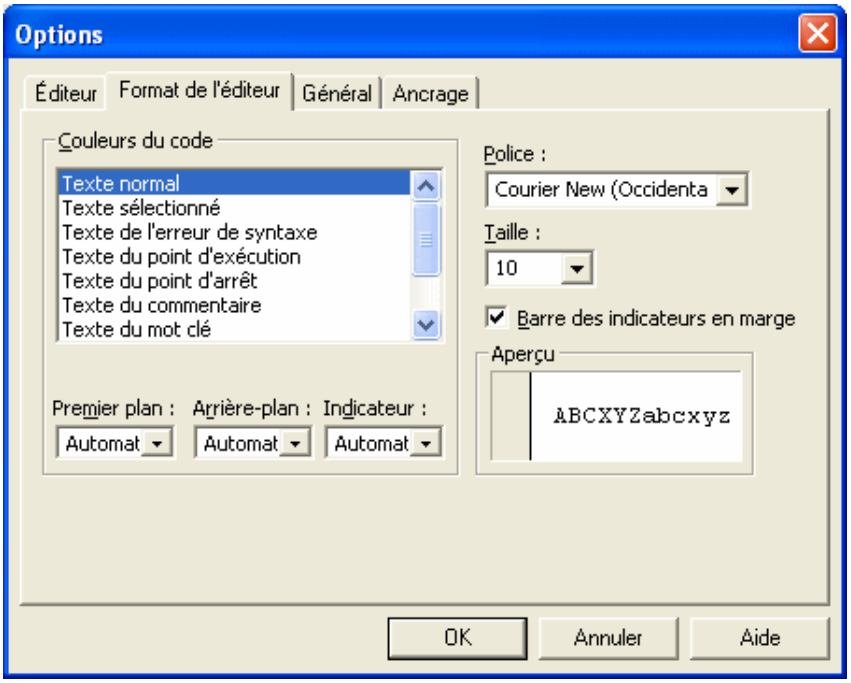
© ENI Editions - All rigths reserved

Configuration de l'éditeur VBA

Paramétrage des couleurs de l'éditeur

Les mots clés, les fonctions et les instructions VBA sont affichées en **bleu**, les objets, méthodes et propriétés en **noir** et les commentaires en **vert**. Les instructions contenant des erreurs sont mises en évidence en **rouge**.

Vous pouvez modifier le style (couleurs, police, taille) des différentes parties du code. Pour cela, sélectionnez **Options** dans le menu **Outils** et cliquez sur l'onglet **Format de l'éditeur**.

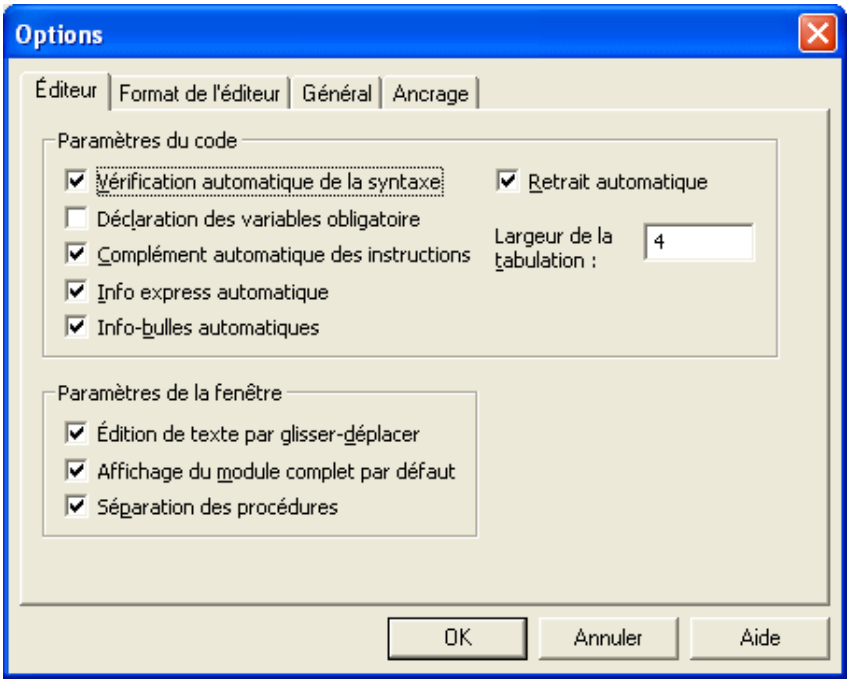


Paramétrage de la présentation du code

Différents outils, comme la vérification automatique de la syntaxe, la déclaration obligatoire des variables, le complément automatique d'instructions... facilitent la saisie et la mise au point du code VBA.

Pour activer ces outils, sélectionnez **Options** dans le menu **Outils** et cliquez sur l'onglet **Éditeur**.

© ENI Editions - All rigths reserved

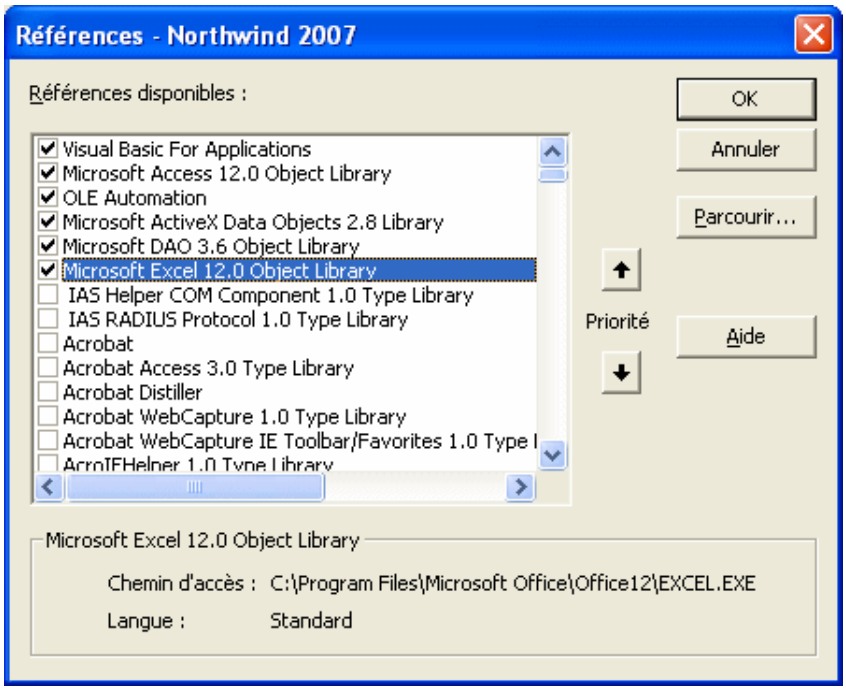


© ENI Editions - All rigths reserved

Référence aux objets

Pour pouvoir utiliser un objet d'accès aux données, un objet d'automation, ou un contrôle ActiveX spécifique, vous devez référencer la bibliothèque d'objet correspondante. Une bibliothèque d'objets est généralement un fichier DLL ou OCX qui contient le code de l'objet.

Pour activer une référence à une bibliothèque, sélectionnez l'option **Références** dans le menu **Outils**.



» Vous pouvez ajouter des références en cliquant sur le bouton de commande **Parcourir**.

© ENI Editions - All rigths reserved

Création d'une procédure en VBA

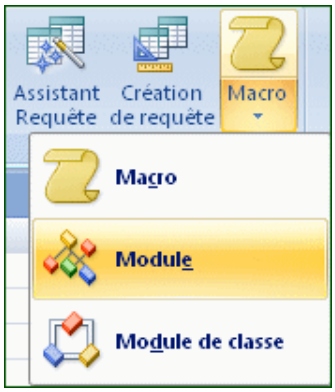
Cette section va vous guider pas à pas dans la création d'une première procédure en VBA.

Cette procédure demande à l'utilisateur de saisir une date sous forme abrégée (19/06/07), contrôle la date et l'affiche dans un format date complet (mardi 19 juin 2007).

Étape 1 : création d'un nouveau module

Depuis Access :

- Activez l'onglet **Créer** du ruban.
- Cliquez sur la flèche située en bas de l'icône **Macro** puis choisissez l'option **Module**.



Depuis l'environnement VBE :

- Sélectionnez l'option **Module** du menu **Insertion**

ou

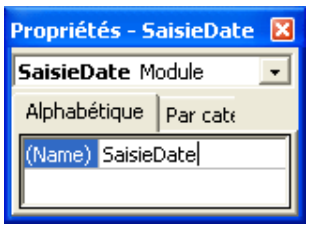
depuis la fenêtre **Explorateur de projets**, cliquez avec le bouton droit de la souris et sélectionnez **Insertion** puis **Module** à partir du menu contextuel.

Étape 2 : attribuer un nom au module

Par défaut, le module créé est intitulé Module1 (ou Module suivi du premier numéro disponible si d'autres modules ont déjà étés créés et non renommés).

Pour modifier le nom du module :

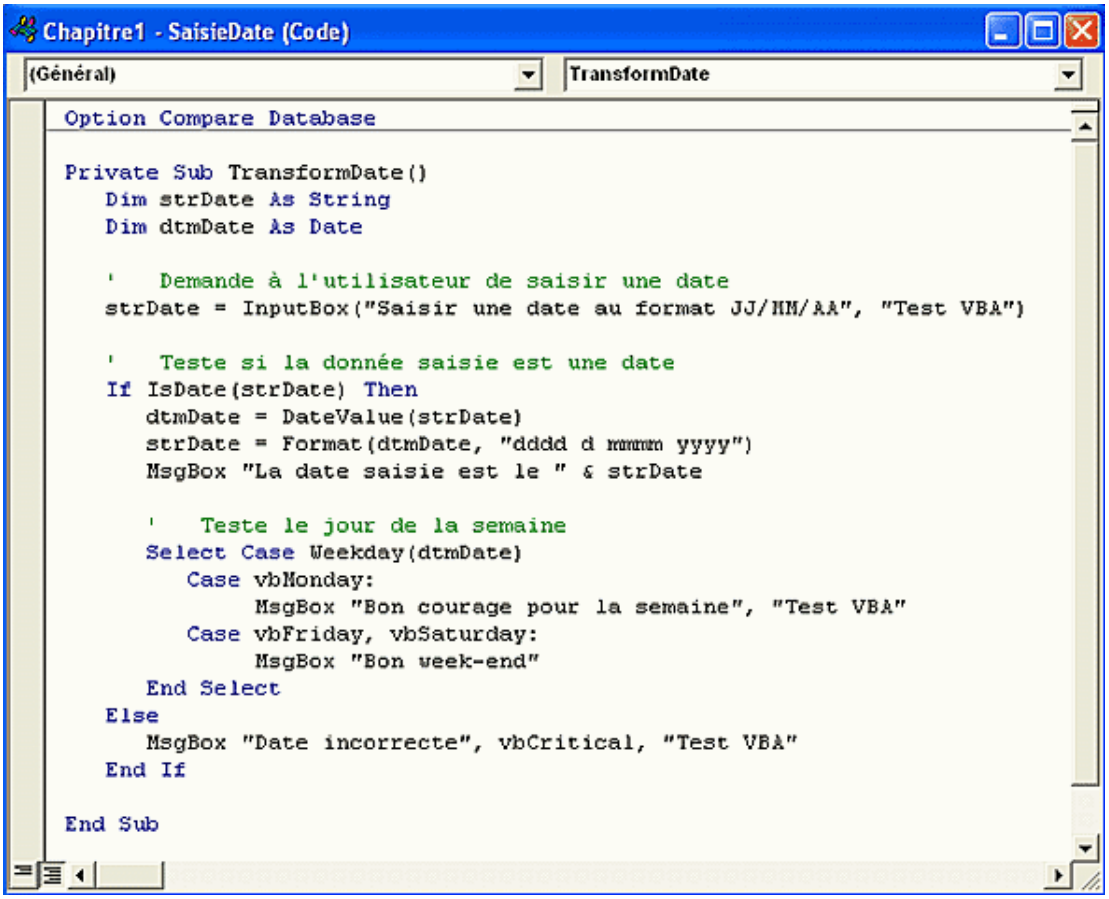
- Sélectionnez le module dans la fenêtre Explorateur de projet.
- Dans la fenêtre **Propriétés** (cliquez sur **F2** si la fenêtre Propriétés n'est pas affichée), modifiez la valeur de la propriété **(Name)**.



Étape 3 : Création d'une procédure

- Double cliquez sur le module pour afficher la fenêtre de VBA associée.
- Saisissez le code suivant dans la fenêtre VBA :

© ENI Editions - All rigths reserved

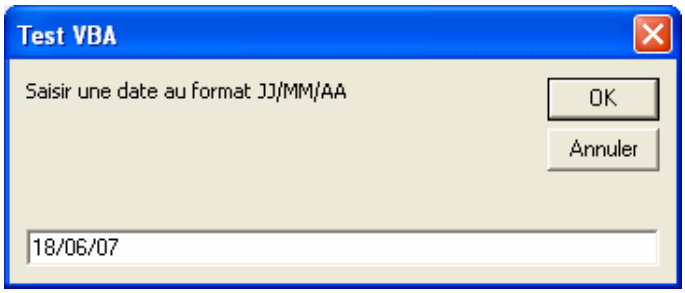


- Cliquez sur l'icône de la barre d'outils **Standard** pour enregistrer la procédure.

Étape 4 : Test de la procédure

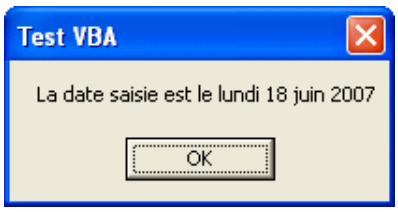
Pour exécuter le code de la procédure :

- Positionnez le curseur de la souris sur le code de la procédure saisie.
- Cliquez sur l'icône de la barre d'outils **Standard** ou utilisez la touche **F5**.
- Saisissez une date dans la boîte de dialogue affichée et cliquez sur le bouton de commande **OK** :



Le message suivant est alors affiché :

© ENI Editions - All rigths reserved



- Cliquez sur le bouton de commande **OK**.

Le jour de la semaine étant un lundi, le message suivant apparaît :



© ENI Editions - All rigths reserved

Recommandations

Afin d'optimiser les performances de VBA, certaines techniques peuvent contribuer à accélérer la vitesse d'exécution de votre code.

- Les variables doivent toujours être explicitement déclarées.

- Le type des variables doit être le plus spécifique possible.

- Pour faire référence aux propriétés, contrôles et objets d'accès aux données, les variables sont préconisées.

- Le mot réservé **Me** doit être utilisé pour faire référence au formulaire ou à l'état au sein d'une procédure événementielle.

- L'emploi des constantes est recommandé chaque fois que possible.

- Les modules doivent être organisés pour ne pas surcharger la mémoire. Un module est mis en mémoire uniquement si une de ses procédures est appelée par d'autres codes.

- Le code doit être régulièrement compilé.

© ENI Editions - All rigths reserved

Les modules

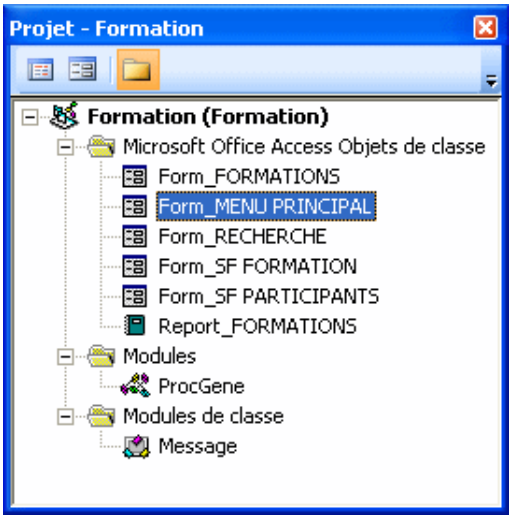
On distingue trois types de modules :

- les modules Microsoft Access Objets de classe,

- les modules standards,

- les modules de classe indépendants.

La liste de ces modules est affichée de façon hiérarchique dans l'explorateur de projets de l'interface VBA.



» Si l'explorateur de projet n'est pas affiché, choisissez l'option **Explorateur de projets** du menu **Affichage** ou utilisez le raccourci-clavier **Ctrl R**.

Les modules **Microsoft Access Objets de classe** contiennent les procédures événementielles associées aux formulaires et états.

Les modules standards (ou modules) sont composés d'une ou de plusieurs procédures pouvant être appelées en de nombreuses circonstances à partir des différents modules.

» Seuls les modules standards apparaissent dans la liste des modules du volet de navigation.

» Il est recommandé de regrouper dans un ou plusieurs modules les procédures "utilitaires" pouvant être utilisées par différentes applications (ex : traitement des dates). Ces modules pourront ensuite être exportés dans d'autres bases Access à partir de l'option **Exporter un fichier** du menu **Fichier**.

Les modules de classe indépendants sont utilisés lors de la création de classes d'objets. La programmation à base d'objets sera détaillée dans le chapitre **Objets et collections**.

Les éléments du langage Visual Basic décrits dans ce chapitre peuvent être utilisés dans les différents modules.

© ENI Editions - All rigths reserved

Les procédures

Les procédures sont des **sous-programmes** qui permettent de décomposer une tâche de programmation complexe en une suite de tâches plus petites et plus simples.

Elles permettent d'organiser le code à l'intérieur des modules afin d'obtenir un code plus facile à maintenir et facilement réutilisable.

Dans VBA Access 2007, il existe trois types de procédures :

- les procédures **Sub** (pour sub routine) appelées sous-programmes,

- les procédures **Function** appelées fonctions,

- les procédures **Property** appelées procédures de propriété.

Dans ce chapitre, nous utiliserons uniquement les procédures Sub et Function qui sont les plus courantes (les procédures de propriété seront présentées dans le chapitre **Objets et collections**).

Il existe des points communs entre ces deux procédures :

- elles contiennent des instructions et/ou des méthodes VBA,

- elles acceptent des arguments,

- elles peuvent être appelées depuis d'autres fonctions ou procédures Sub.

Certaines caractéristiques sont spécifiques aux fonctions :

- elles peuvent retourner des valeurs,

- elles peuvent être appelées depuis une macro.

Les procédures Sub

Il existe deux types de procédures Sub :

- les procédures Sub générales

- les procédures Sub événementielles.

Une **procédure Sub générale** est une procédure déclarée dans un module standard ou au niveau général d'un module de formulaire ou d'état. L'appel d'une telle procédure est explicitement défini dans le code du programme.

Exemple

Cette procédure générale demande à l'utilisateur de confirmer son souhait de quitter l'application, et quitte Access si l'utilisateur répond Oui.

```
 Private Sub Quitter_Appli()
   If Msgbox ("Voulez-vous quitter l'application ?",_
     VbQuestion + VbYesNo) = vbYes Then
     Quit
   End If
End Sub
```

» Ce code peut être appelé à partir de tout bouton de commande ou option de menu permettant de quitter l'application.

© ENI Editions - All rigths reserved

Une **procédure Sub événementielle** est une procédure associée à un événement d'un objet. D'ailleurs son nom est composé du nom de l'objet activé, suivi du caractère souligné (_) et du nom de l'événement. L'appel d'une telle procédure est implicite, c'est-à-dire que la procédure n'est exécutée que lorsque l'événement associé a lieu.

Exemple

*Cette procédure événementielle permet d'afficher le formulaire "**Employés**" lorsque l'utilisateur clique sur un bouton de commande **cmdEmployés**.*

```
Private Sub cmdEmployés_Click()
   Docmd.OpenForm "Employés"
End Sub
```

» Certaines procédures Sub événementielles peuvent être générées automatiquement à partir d'assistants (ex : assistant boutons de commande).

Les procédures Function

Les **procédures Function**, plus couramment appelées **fonctions**, renvoient une valeur, telle que le résultat d'un calcul. La valeur retournée doit porter le nom de la fonction.

» Ce sont les seules procédures pouvant être appelées depuis une macro.

Le langage Visual Basic comporte de nombreuses fonctions intégrées telles que les fonctions se rapportant aux dates (day, week, year, format...).

En plus de ces fonctions intégrées, vous pouvez créer vos propres fonctions personnalisées.

Exemple

Cette fonction demande à l'utilisateur de confirmer son souhait de quitter l'application, et renvoie True si l'utilisateur répond Oui et False sinon.

```
Function Quitter_Appli() As Boolean
  If MsgBox("Voulez-vous quitter l'application ?", _
    vbQuestion + vbYesNo) = vbYes Then
    Quitter_Appli = True
  Else
    Quitter_Appli = False
  End If
End Function
```

Déclaration des procédures

Syntaxe d'une procédure Sub

```
[Private | Public | Friend] [Static] Sub NomProc
([liste d'arguments])
    <Séquences d'instructions>
End Sub
```

Syntaxe d'une procédure Function

```
[Private | Public | Friend] [Static] Function
NomProc ([liste d'arguments]) [As Type]
    <Séquences d'instructions>
End Function
```

Pour créer une procédure Sub ou Function, vous devez respecter les étapes suivantes :

- déterminez la **portée** de la procédure,

- déclarez la procédure en fonction de son type avec le mot clé **Sub** ou **Function**, suivi du nom de la procédure,

© ENI Editions - All rigths reserved

- définissez les **arguments** que vous voulez passer en paramètres à la procédure en les indiquant entre parenthèses après le nom de la procédure,

- s'il s'agit d'une fonction, précisez éventuellement le type de la valeur retournée après le mot clé **As**,

- rédigez le code permettant d'effectuer le traitement souhaité. Utilisez éventuellement **Exit Sub** ou **Exit Function** pour sortir de la procédure. S'il s'agit d'une fonction, affectez le résultat au nom de la fonction,

- terminez la procédure par **End Sub** ou **End Function**.

Portée des procédures

La portée d'une procédure définit l'étendue de son utilisation.

Une procédure **publique** (déclarée avec le mot clé **Public**) peut être appelée depuis tout module, toute procédure événementielle ou toute macro de la base de données.

Une procédure **privée** (déclarée avec le mot clé **Private**) ne peut être appelée que depuis une procédure au sein du même module.

La procédure **Friend** est utilisée uniquement dans les modules de classe.

Le mot clé **Static** indique que les variables locales de la procédure sont préservées entre les appels.

En l'absence des mentions **Public**, **Private** ou **Friend**, les procédures sont publiques par défaut.

Arguments des procédures

Les arguments sont utilisés pour transmettre aux procédures des paramètres sous forme de données. Le nombre d'arguments peut varier de zéro à plusieurs.

Pour déclarer un argument, vous pouvez simplement spécifier son nom. Néanmoins, la syntaxe complète de déclaration d'un argument est la suivante :

```
[Optional] [ByVal | Byref] [ParamArray]
<variable> [As type]
```

`Optional` : indique que l'argument est facultatif. Tous les arguments facultatifs doivent être situés à la fin de la liste des arguments et être de type variant.

`Byval` : indique que l'argument est passé par valeur. La procédure accède à une copie de la variable ; sa valeur initiale n'est donc pas modifiée par la procédure.

`Byref (option par défaut)` : indique que l'argument est passé par référence. La procédure peut ainsi accéder à la variable proprement dite ; sa valeur réelle peut, de ce fait, être modifiée par la procédure.

`ParamArray` : utilisé uniquement pour le dernier argument de la liste, indique que celui-ci est un tableau facultatif d'éléments de type **Variant**. Il ne peut être utilisé avec les mots clés **ByVal**, **ByRef** ou **Optional**.

`Variable` : précise le nom de l'argument**.** Pour les variables tableau, il ne faut pas préciser les dimensions.

`Type` : précise le type de données de l'argument passé à la procédure (Byte, Boolean, Integer, Long...).

Les arguments nommés

Le passage d'arguments à une procédure en tenant compte de leur ordre d'apparition est parfois difficile à mettre en œuvre, notamment lorsque certains paramètres sont facultatifs. De même, la lisibilité des appels de procédures comportant plusieurs paramètres n'est pas toujours évidente.

» Les arguments nommés facilitent le passage des arguments en présentant les avantages suivants :

© ENI Editions - All rigths reserved

- l'ordre des arguments nommés n'a pas d'importance,
- les arguments facultatifs peuvent être omis.

La syntaxe des arguments nommés est :

```
NomArgument := Valeur
```

Exemple

```
If MsgBox("Voulez-vous quitter l'application ?", _
vbYesNo + vbQuestion, "Gestion des ventes") = vbYes Then
    Application.Quit
End If
```

peut être transformé en :

```
If MsgBox(Prompt:="Voulez-vous quitter l'application ?", _
    Buttons:=vbYesNo + vbQuestion, _
    Title:="Gestion des ventes") = vbYes Then
    Application.Quit
End If
```

De même l'ordre des paramètres peut être modifié :

```
If MsgBox(Prompt:="Voulez-vous quitter l'application ?", _
    Title:="Gestion des ventes", _
    Buttons:=vbYesNo + vbQuestion) = vbYes Then
    Application.Quit
End If
```

Appel d'une procédure

Syntaxe

```
[Call] NomProc [liste d'arguments]
```

Si le mot clé `Call` est indiqué, vous devez placer la liste d'arguments entre parenthèses.

Pour stocker le résultat d'une fonction dans une variable, utilisez la syntaxe suivante :

```
<variable> = NomProc ( [liste d'arguments] )
```

Exemples de procédures

Lorsque l'utilisateur clique sur le bouton de commande cmdCalcul du formulaire de saisie d'une facture, le montant de la remise doit être calculé de la façon suivante :

- 5% du montant de la facture si celui-ci est supérieur ou égal à 5000,
- 2% du montant de la facture si celui-ci est compris entre 1000 et 5000,
- 0 si le montant de la facture est inférieur à 1000.

Procédure événementielle

```
Private Sub cmdCalcul_Click()
  txtMntRem = Remise (txtMntFact)
End Sub
```

» La fonction Remise est appelée, avec comme argument, le montant de la facture (contenu dans la zone de texte txtMntFact du formulaire). La remise calculée sera directement affichée dans la zone de texte txtMntRem.

Fonction

© ENI Editions - All rigths reserved

```
Private Function Remise(Montant as Currency) as Currency
Const Taux1 = 0.05
Const Taux2 = 0.02

  If Montant >= 5000 then
    Remise = Montant * Taux1
  Else
    If Montant >= 1000 then
      Remise = Montant * Taux2
    End If
  End If
End Function
```

» Si le montant de la facture est inférieur à 1000, Remise conserve sa valeur initiale, soit zéro.

La fonction MsgBox

Cette fonction affiche un message dans une boîte de dialogue, éventuellement accompagné d'une icône et de un à trois boutons.

Syntaxe de l'instruction

Utilisée lorsqu'il n'y a qu'un seul bouton.

```
MsgBox <message> [, [<type>][, <titre>]]
```

Syntaxe de la fonction

Utilisée lorsqu'il y a plus d'un bouton. Elle permet de savoir quel bouton a été activé grâce à sa valeur de retour.

```
MsgBox (<message> , [<boutons>][, <titre>]
[,helpfile, context])
```

`message` : Texte du message dans la boîte de dialogue.

`boutons` : Expression numérique qui représente la somme des valeurs spécifiant les boutons à afficher, le style d'icône à utiliser, l'identité du bouton par défaut, ainsi que la modalité.

`titre` :Texte dans la barre de titre.

`helpfile` : Fichier d'aide à utiliser.

`Context` : Rubrique de l'aide concernée.

Valeurs de l'argument Boutons

Constante symbolique	Valeur	Signification
Nombre et type de boutons		
vbOKOnly	0	Affiche le bouton **OK** uniquement.
vbOKCancel	1	Affiche les boutons **OK** et **Annuler**.
vbAbortRetryIgnore	2	Affiche les boutons **Abandonner**, **Recommencer** et **Ignorer**.
vbYesNoCancel	3	Affiche les boutons **Oui**, **Non** et **Annuler**.
vbYesNo	4	Affiche les boutons **Oui** et **Non**.
vbRetryCancel	5	Affiche les boutons **Recommencer** et **Annuler**.
vbMsgBoxHelpButton	16384	Affiche un bouton d'aide.
Type d'icône		

© ENI Editions - All rigths reserved

vbCritical	16	Affiche l'icône  .
vbQuestion	32	Affiche l'icône  .
vbExclamation	48	Affiche l'icône  .
vbInformation	64	Affiche l'icône  .
Bouton par défaut		
vbDefaultButton1	0	Premier bouton.
vbDefaultButton2	256	Deuxième bouton.
vbDefaultButton3	512	Troisième bouton.
vbDefaultButton4	768	Quatrième bouton.
Modalité		
vbApplicationModal	0	Application modale. L'utilisateur doit répondre au message affiché dans la zone de message avant de pouvoir continuer à travailler dans l'application en cours.
vbSystemModal	4 096	Système modal. Toutes les applications sont interrompues jusqu'à ce que l'utilisateur réponde au message affiché dans la zone de message.
Présentation		
vbMsgBoxHelpButton	16384	Ajoute le bouton "Aide" à la zone de message.
vbMsgBoxSetForeground	65536	Affiche la fenêtre message au premier plan.
vbMsgBoxRight	524288	Aligne le texte à droite.
vbMsgBoxRtlReading	1048576	Définit un ordre de lecture de droite à gauche pour les systèmes hébreux et arabes.

Les valeurs de retour possibles sont également définies par des constantes :

Constante	Valeur de retour	Bouton choisi
vbOK	1	**OK**
vbCancel	2	**Annuler**
vbAbort	3	**Abandonner**
vbRetry	4	**Recommencer**
vbIgnore	5	**Ignorer**
vbYes	6	**Oui**
vbNo	7	**Non**

Exemples : utilisation de la fonction MsgBox

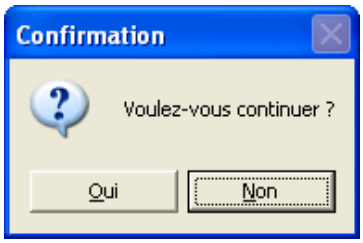
```
StrRep = MsgBox ("Voulez-vous continuer ?", 292, _
"Confirmation")
```

ou

```
StrRep = MsgBox ("Voulez-vous continuer ?", _
vbYesNo + vbQuestion + vbDefaultButton2, _
```


© ENI Editions - All rigths reserved

```
"Confirmation")
```



Utilisation de l'instruction MsgBox

```
MsgBox "Résultat faux",vbCritical,"Erreur"
```



© ENI Editions - All rigths reserved

Les variables

Les types de variables

Les variables permettent de stocker des valeurs intermédiaires à tout moment de l'exécution du code VBA afin de les exploiter ultérieurement pour effectuer des calculs, des comparaisons, des tests…

Les variables sont identifiées par un **nom**, permettant de faire référence à la valeur qu'elles contiennent, et un type déterminant la nature des données qu'elles peuvent stocker.

Numériques

>

Type	Étendue	Taille en octets
Byte	0 à 255	1
Integer (entier)	-32 768 à 32 767	2
Long (entier long)	-2 147 483 648 à 2 147 483 647	4
Single (réel simple à virgule flottante)	-3,402823E38 à 1,401298E-45 (valeurs négatives) 1,401298E-45 à 3,402823E38 (valeurs positives)	4
Double (réel double à virgule flottante)	-1,79769313486231E308 à 4,94065645841247E-324 (valeurs négatives) 4,94065645841247E-324 à 1,79769313486231E308 (valeurs positives)	8
Currency (monétaire à virgule fixe)	-922 337 203 685 477,5808 à 922 337 203 685 477,5807	8
Decimal	+/-79 228 162 514 264 337 593 543950 335 sans séparateur décimal ; +/-7,9228162514264337593543950335 avec 28 chiffres à droite du séparateur décimal ; le plus petit nombre différent de zéro est +/-0.0000000000000000000000000001	12

Chaînes de caractères

Le type est **String**. Il existe deux types de chaînes :

- les chaînes de longueur variable peuvent contenir environ 2 milliards (2^{31}) de caractères.

- Les chaînes de longueur fixe peuvent contenir de 1 à environ 64000 (2^{16}) caractères.

Exemple

```
'Chaîne de longueur variable
Dim Adresse As String
'Chaîne de longueur fixe (20 caractères)
Dim Nom As String * 20
```

Booléen

© ENI Editions - All rigths reserved

Le type est **Boolean**. La variable peut prendre la valeur True (Vrai) ou False (Faux) qui est sa valeur par défaut. Elle occupe deux octets.

Date

Le type est **Date**. La variable peut prendre les valeurs de date et d'heure du 1er janvier 100 au 31 décembre 9999. Elle occupe huit octets.

Variant

Le type est **Variant**. La variable permet de stocker des valeurs de données de toutes sortes : dates, nombres à virgule flottante, chaînes de caractères, références à des objets... Elle occupe seize octets, plus un octet par caractère si la valeur est une chaîne.

Objet

Le type est **Object**. La variable permet de stocker la référence à un objet sous forme d'une adresse de quatre octets.
C'est un type générique, mais pour obtenir des performances supérieures, il est souhaitable d'utiliser plutôt le nom de l'objet tel qu'il est défini dans la zone **Classes** de l'explorateur d'objets.

Exemple

Ouverture de base de données Access en utilisant le type générique Objet ou à partir de l'objet Database.

```
Dim dbsEmp As Object
Dim dbsArt As Database
Set dbsEmp = OpenDatabase("C:\Application\" _
      & "Employés.accdb")
Set dbsArt = OpenDatabase("C:\Application\" _
      & "Articles.accdb")
```

Défini par l'utilisateur

Ce type est utilisé surtout pour la déclaration de variables contenant plusieurs types d'informations. La définition du type ne peut se faire que dans la section de déclaration d'un module.

Exemples

Déclaration d'un nouveau type Employés :

```
Public Type Employés
    Matricule As String * 5
    Nom As String * 20
    Age As Integer
    Service As String * 10
End Type
```

Utilisation du type Employés :

```
Dim NewEmployé As Employés
NewEmployé.Matricule = "15253"
...
```

Les déclarations de variables

Il existe deux façons de déclarer des variables : implicitement et explicitement.

Déclaration implicite

Dès que VBA rencontre une variable qui n'a pas fait l'objet d'une déclaration explicite, il la déclare de façon implicite en lui donnant un type **Variant**.

© ENI Editions - All rigths reserved

Le type Variant permet de stocker successivement dans la même variable des valeurs de type différent. Le type de la variable évolue en fonction de la valeur qui lui est affectée.

```
Private Sub Affectation()

' Le type de la variable devient String
var = "Bonjour"
MsgBox TypeName(var)

' Le type de la variable devient Integer
var = 12
MsgBox TypeName(var)

' Le type de la variable devient Double
var = 6.55957
MsgBox TypeName(var)

' Le type de la variable devient Boolean
var = True
MsgBox TypeName(var)

End Sub
```

Déclaration explicite

La variable est déclarée avant son utilisation.

Il est possible d'imposer la déclaration explicite des variables en utilisant dans la section de déclaration de chaque module, l'instruction **Option Explicit**. Pour que cette instruction soit définie par défaut dans VBA, activez l'option **Déclaration des variables obligatoire** du menu **Outils - Options** - onglet **Éditeur**.

Les déclarations explicites de variables peuvent se faire dans la section de déclarations des modules, ou dans le corps d'une procédure ou d'une fonction.

La déclaration est réalisée par l'une des quatre instructions suivantes utilisant la même syntaxe :

- au niveau d'une procédure, d'une fonction ou d'un module :

```
Dim <Nomvariable> [As Type]
Static <Nomvariable> [As Type]
```

- au niveau d'un module :

```
Private <Nomvariable> [As Type]
Public <Nomvariable> [As Type]
```

À chaque fois que l'instruction **Dim** est exécutée, la variable est réinitialisée (0 pour numérique, vide pour **String**, **Empty** pour **Variant**). S'il est nécessaire de garder la précédente valeur, il faut utiliser **Static** à la place de **Dim**.

» Si aucun type n'est précisé (ex : dim var), le type par défaut Variant est affecté à la variable.

La portée des variables

Une variable est dite **locale** à une fonction ou à une procédure, si elle est déclarée dans le corps de la fonction ou de la procédure avec **Dim** ou **Static**. Cette variable n'est pas connue (visible) en dehors de la fonction ou de la procédure.

Une variable peut être visible de toutes les procédures ou fonctions d'un module (standard, de classe, formulaire ou état), si elle est déclarée dans la section de déclaration du module avec **Private** (ou **Dim**).

Une variable est **publique** à toute l'application (c'est-à-dire visible depuis n'importe quel endroit de l'application), si elle est déclarée dans la section de déclaration d'un module avec l'instruction **Public**.

» Pour faire référence, à partir d'un module, à une variable publique se trouvant dans un autre module de formulaire ou d'état, il convient d'utiliser la syntaxe suivante :
`Forms!<NomFormulaire>.<NomVariable>`

© ENI Editions - All rigths reserved

```
Reports!<NomEtat>.<NomVariable>
```

Les tableaux

Un tableau se déclare de la même façon que les variables, avec des instructions **Dim**, **Static**, **Private** ou **Public**.

Un tableau est une variable comportant plusieurs compartiments permettant de stocker autant de valeurs. Par opposition, une variable classique désigne un compartiment destiné à une seule valeur.

On peut faire référence au tableau globalement ou accéder individuellement à ses éléments en utilisant des indices.

Quel que soit le type des éléments, un tableau nécessite 20 octets de mémoire, auxquels viennent s'ajouter quatre octets pour chaque dimension et le nombre d'octets occupés par les données.

Si le type de données n'est pas spécifié, les éléments du tableau déclaré prennent le type **Variant**.

Le premier poste d'un tableau se voit attribuer l'indice 0 par défaut. Toutefois, il est possible de mettre cette valeur par défaut à 1 en utilisant, en section de déclaration, l'instruction **Option Base 1**.

L'utilisation d'une clause **To** permet également de définir explicitement les indices de début et de fin.

Exemples

Déclaration de différents tableaux :

```
' 12 éléments indicés de 0 à 11
Dim Remise (11) As Single
' 12 éléments indicés de 1 à 12
Dim Frais (1 To 12) As Single
' 2 dimensions : la première indicée de 0 à 10 et
' la seconde de 1 à 15
Dim Train (10,1 To 15) As String
```

```
Option Base 1
' 12 éléments indicés de 1 à 12
Dim Frais (12)
```

Utilisation d'un tableau à deux dimensions.
L'exemple suivant permet de créer une table de multiplication : chaque élément du tableau ***Multiplie*** *est égal au résultat de la multiplication de ses indices. Chaque ligne de la table est affichée dans la fenêtre exécution (méthode Print de l'objet Debug).*

```
Sub Affiche_Tableau()
Dim Multiplie(1 To 10, 1 To 10) As Integer
Dim i, j As Integer
Dim strResult As String

' 1ère ligne de la table de multiplication
For i = 1 To 10
    strResult = Format(i, "000") & " "
Next i
Debug.Print strResult
' Initialisation du tableau
For i = 1 To 10
    strResult = Format(i, "00")
    For j = 1 To 10
      Multiplie(i, j) = i * j
      strResult = strResult & " " & Format(Multiplie(i, j),
"000")
    Next j
    Debug.Print strResult
Next i

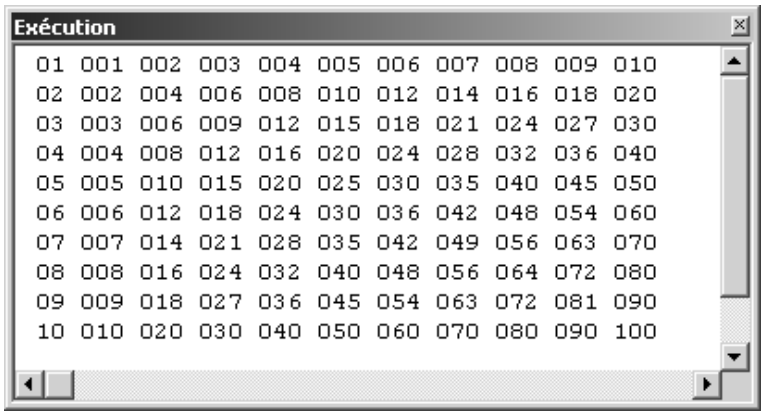
' Renvoie le résultat de 4 * 7
MsgBox "4 * 7 = " & Multiplie(4, 7)

' Renvoie le résultat de 7 * 4
MsgBox "7 * 4 = " & Multiplie(7, 4)
```


© ENI Editions - All rigths reserved

```
End Sub
```

Résultat dans la fenêtre d'exécution.



Les tableaux dynamiques

Il s'agit de tableaux dont les dimensions sont définies à l'exécution. Ils permettent l'optimisation de la place occupée en mémoire.

Déclaration :

```
Dim Tableau ()
```

Utilisation :

Avant de l'utiliser, il faut le dimensionner à l'aide de l'instruction ***Redim*** *:*

```
ReDim Tableau (15 To 50)
```

Il est possible d'utiliser l'instruction **ReDim** pour modifier le tableau dynamique aussi souvent que nécessaire. Par défaut, les valeurs présentées dans le tableau sont écrasées. L'instruction **ReDim Preserve** permet d'étendre un tableau tout en conservant les valeurs existantes.

```
ReDim Preserve Tableau (15 To 70)
```

Les constantes

Une constante représente une valeur numérique ou de type chaîne qui ne doit pas être modifiée.

L'instruction **Const** permet de déclarer une constante et de définir sa valeur fixe. Une fois déclarée, la constante ne peut être modifiée ni recevoir une nouvelle valeur.

La déclaration d'une constante est réalisée dans une procédure ou dans la section déclaration d'un module.

Les constantes de niveau module sont **privées** par défaut.

Pour déclarer une constante de niveau module publique, il est nécessaire de faire précéder l'instruction **Const** du mot clé **Public**.

Exemple

```
acForm
vbCurrency
```

Les constantes intrinsèques

© ENI Editions - All rigths reserved

Les constantes intrinsèques sont des constantes mises à disposition par les applications elles-mêmes. C'est ainsi qu'Access 2003 possède ses propres constantes intrinsèques qui sont visibles à partir de l'explorateur d'objets.

Une constante intrinsèque peut être utilisée dans une macro ou dans du code Visual Basic.

Les constantes intrinsèques sont caractérisées par un préfixe de deux lettres identifiant la bibliothèque d'objets où est définie la constante.

Préfixe des constante

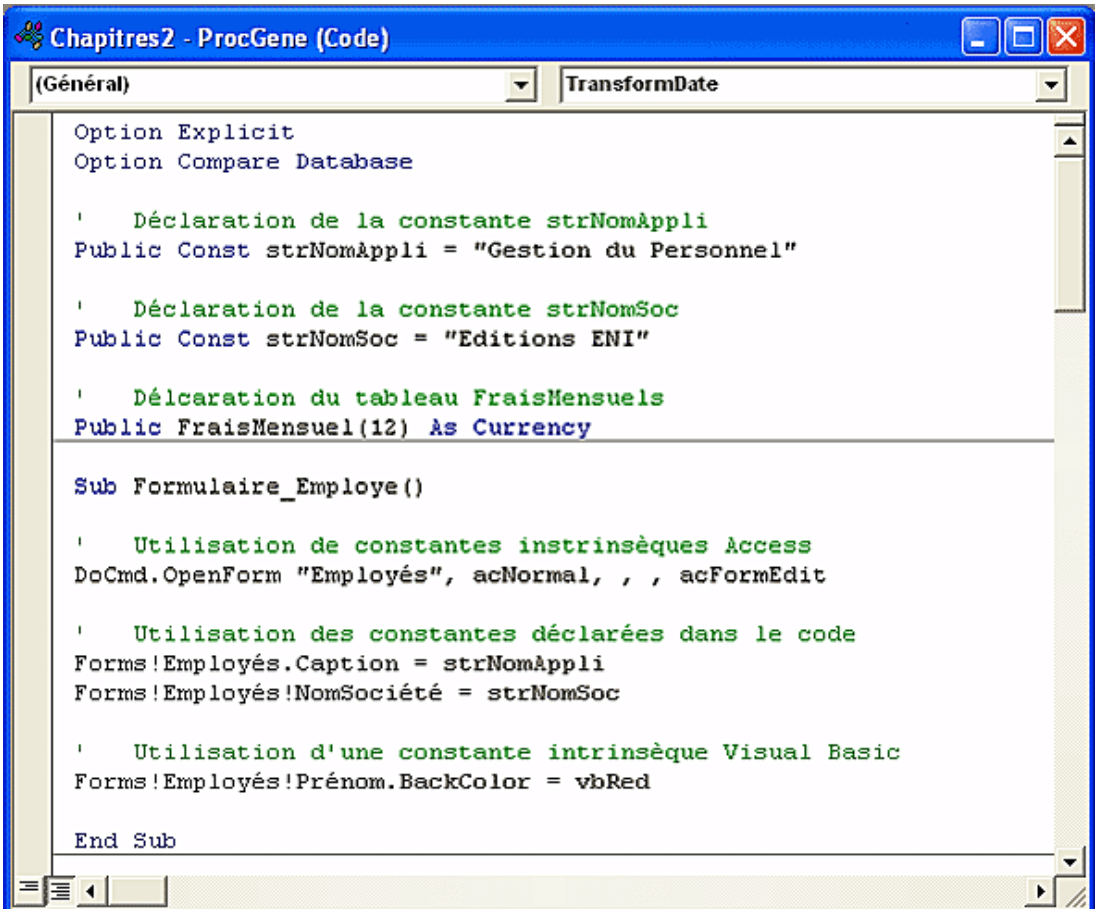
Bibliothèque	Préfixe	Exemples
Microsoft Access	ac	acForm, acNormal
Visual Basic	vb	vbCurrency, vbYellow
Microsoft Office	mso	msoSortByFileName

» Les constantes de la bibliothèque Visual Basic sont décrites dans l'annexe B de cet ouvrage. Pour utiliser les constantes de Microsoft Office, vous devez sélectionner la référence "Microsoft Office 12.0 Object Library" (option **Références** du menu **Outils**).

» Il existe plusieurs catégories de constantes intrinsèques (constantes d'action, constantes de procédures événementielles...).

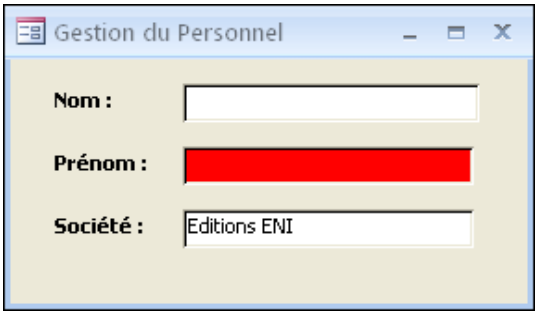
Exemple

L'exemple suivant montre comment afficher un formulaire et le personnaliser. Il utilise les constantes strNomAppli et strNomSoc déclarées dans le code et plusieurs constantes intrinsèques.



Résultat dans Access : affichage du formulaire employé personnalisé.

© ENI Editions - All rigths reserved



© ENI Editions - All rigths reserved

Les structures de décision

Il est souvent nécessaire de tester des conditions spécifiques avant d'exécuter des instructions.

Les structures de décisions, appelées aussi alternatives ou branchements conditionnels, permettent, au terme d'une évaluation, d'opter pour un bloc de code ou un autre.

On distingue deux instructions de branchement conditionnel :

- If ... Then ... Else

- Select ... Case

» La fonction IIf peut également être utilisée pour définir une valeur en fonction d'une condition.

L'instruction IF

- If

Permet d'exécuter des instructions en fonction du résultat d'une condition.

Les instructions **If...Then...Else** peuvent être imbriquées autant de fois que nécessaire.

» La syntaxe multiligne, avec une indentation appropriée, présente l'avantage d'une plus grande lisibilité.

Syntaxe 1 (sur une ligne)

```
If <cond.> Then <inst.> [Else <inst.>]
```

`<inst.>` peut être constitué de plusieurs instructions séparées par le caractère (:).

Syntaxe 2 (sur plusieurs lignes)

```
If <condition 1> Then
    <séquence d'instructions 1>
[ElseIf <condition 2> Then
    <séquence d'instructions 2>]
[Else
    <séquence d'instructions 3>]
End If
```

Autre possibilité

```
If <condition 1> Then
    <séquence d'instructions 1>
[Else
    If <condition 2> Then
      <séquence d'instructions 2>
    [Else
      <séquence d'instructions 3>]
    End If]
End If
```

Exemple

Affiche le résultat de la comparaison entre A et B :

```
If A>B Then
    MsgBox "A>B"
Else
    If A = B Then
      MsgBox "A=B"
    Else
```

© ENI Editions - All rigths reserved

```
      MsgBox "A<B"
    End If
End If
```

L'instruction Select Case

- Select Case

Exécute des séquences d'instructions spécifiées en fonction de la valeur d'une expression.

L'instruction **Select Case** permet de remplacer avantageusement le **ElseIf** dans des instructions **If...Then...Else** lors de la comparaison entre une expression et plusieurs valeurs.

Syntaxe

```
Select Case <expression à tester>
[Case <liste d'expressions 1>
<séquence d'instructions 1>]
[Case <liste d'expressions 2>
<séquence d'instructions 2>]
[Case Else
<séquence d'instructions 3>]
End Select
```

`<liste d'expression>` peut prendre les formes suivantes :

- valeur (ex : `Case 10`)

- liste de valeurs (ex : Case 1, 5, 10)

- plage de valeurs (ex : `Case 1 To 5`)

- expression conditionnelle (ex : `Case Is >= 5`)

Exemple

Affiche un commentaire sur la température.

```
Select Case Température
  Case 0
    MsgBox "Glace"
  Case 1 To 10
    MsgBox "Froid"
  Case 11 To 16
    MsgBox "Frais"
  Case 17, 18, 19
    MsgBox "Température intérieure " _
      & "d'une maison en hiver"
  Case 20 To 25
    MsgBox "Agréable"
  Case Is > 25
    MsgBox "Chaud"
  Case Else
    MsgBox "Température non referencée"
End Select
```

La fonction IIf

- IIf

Retourne une valeur en fonction d'une condition.

Syntaxe

```
IIf (<condition>, <valeur si vrai>, <valeur si faux>)
```


© ENI Editions - All rigths reserved

Exemple

```
Dim bln As Boolean
bln = IIf(A = B, True, False)
```

Il est possible d'imbriquer plusieurs appels de fonction IIf.

Exemple

```
Dim IntI As Integer
IntI = IIf(A = B, IIf(B = C, 1, 0), 0)
```

© ENI Editions - All rigths reserved

Les structures en boucles

Les structures en boucles (ou répétitives) permettent de répéter l'exécution d'un ensemble d'instructions.

On distingue plusieurs types de structures en boucles :

```
Do...Loop

While...Wend

For...Next

For Each...Next
```

Do...Loop et **While...Wend** répètent un traitement jusqu'à ce qu'une certaine condition soit réalisée.

For...Next effectue un traitement un nombre de fois donné, en fonction d'un compteur.

For Each...Next permet de parcourir les éléments d'une collection.

L'instruction Do...Loop

- Do...Loop

Exécute une séquence d'instructions tant que (**While**) ou jusqu'à ce que (**Until**) la condition spécifiée soit vérifiée.

Syntaxe 1

Séquence d'instructions éventuellement non exécutée (condition testée avant la séquence).

```
Do [{While | Until} <condition>]
    <séquence d'instructions>
    [Exit Do]
    [<séquence d'instructions>]
Loop
```

Syntaxe 2

Séquence d'instructions exécutée au moins une fois (condition testée après la séquence).

```
Do
    <séquence d'instructions>
    [Exit Do]
    [<séquence d'instructions>]
Loop [{While | Until} <condition>]
```

» **Exit Do** est souvent utilisé après l'évaluation d'une condition interne à la boucle et permet de sortir de la boucle sans que la condition de contrôle de la boucle ne soit vérifiée.

Exemple

Calcule et affiche le montant des frais annuels en fonction d'un tableau (FraisMensuel) de douze postes chargés antérieurement.

```
Dim intCpt As Integer
Dim FraisAnnuel As Currency

FraisAnnuel = 0
IntCpt = 0
Do
  IntCpt = IntCpt + 1
  FraisAnnuel = FraisAnnuel + FraisMensuel (IntCpt)
Loop Until IntCpt = 12
```

© ENI Editions - All rigths reserved

```
MsgBox "Frais Annuels = " & FraisAnnuel
```

L'instruction While...Wend

- While...Wend

Exécute une séquence d'instructions tant que la condition spécifiée est vérifiée.

Syntaxe

```
While <condition>
  <séquence d'instructions>
Wend
```

Exemple

Calcule et affiche le montant des frais annuels en fonction d'un tableau (FraisMensuel) de douze postes chargés antérieurement.

```
Dim intCpt As Integer
Dim FraisAnnuel As Currency

FraisAnnuel = 0
intCpt = 1
While intCpt <= 12
  FraisAnnuel = FraisAnnuel + FraisMensuel (intcpt)
  intCpt = intCpt + 1
Wend

MsgBox "Frais Annuels = " & FraisAnnuel
```

L'instruction For...Next

- For...Next

Permet de répéter l'exécution d'une séquence d'instructions en fonction d'un compteur.

Syntaxe

```
For <compteur> = <début> To <fin> [Step <incrément>]
  <séquence d'instructions>
  [Exit For]
  <séquence d'instructions>
Next [<compteur> [,<compteur2>] [, ...]]
```

» **Exit For** souvent placée après l'évaluation d'une condition, permet de sortir précocement de la boucle.

Exemple

Calcule et affiche le montant des frais annuels en fonction d'un tableau (FraisMensuel) de douze postes chargés antérieurement.

```
Dim intCpt As Integer
Dim FraisAnnuel As Currency

FraisAnnuel = 0
For intCpt = 1 To 12
  FraisAnnuel = FraisAnnuel + FraisMensuel (intCpt)
Next

MsgBox "Les frais annuels sont de :" & FraisAnnuel
```

Le mot clé **Step** permet d'incrémenter ou de décrémenter la variable de compteur d'un pas spécifié.

Exemple

La variable de compteur j est incrémentée de 5 à chaque itération. À la fin de la boucle, total correspond à la somme de 5, 10, 15,

© ENI Editions - All rigths reserved

et 20.

```
Dim j As Integer
Dim total As Integer

total = 0
For j = 5 To 20 Step 5
  total = total + j
Next j

MsgBox "Le total est de " & total
```

L'instruction For Each...Next

- For Each ... Next

Permet de parcourir tous les éléments d'un tableau ou d'une collection.

Syntaxe

```
For Each <élément> In <tableau>/<collection>
  <séquence d'instructions>
  [Exit For]
  <séquence d'instructions>
Next [élément]
```

Exemple

Affiche les frais de chaque mois (cf. exemple For Next) :

```
Dim Frais As Currency

For Each Frais In FraisMensuel
  MsgBox Frais
Next
```

© ENI Editions - All rigths reserved

Les opérateurs

Les opérateurs permettent d'effectuer des opérations arithmétiques sur des variables et/ou des constantes, de comparer des variables entre elles, de tester plusieurs conditions...

On distingue plusieurs catégories d'opérateurs :

- les opérateurs arithmétiques,

- les opérateurs de comparaison,

- les opérateurs logiques,

- l'opérateur de concaténation.

» L'opérateur d'affectation est le signe d'égalité. La valeur de l'expression située à droite du signe égal est affectée à la variable située à gauche du signe. (exemple : IntA = 12, 12 est affecté à la variable IntA, IntA = IntB * 12, la valeur d'IntB multipliée par 12 est affectée à IntA).

Les opérateurs arithmétiques

Ils permettent d'effectuer des calculs arithmétiques à partir de variables et/ou de constantes numériques.

Opérateur	Calcul réalisé
+	Addition
-	Soustraction
/	Division avec comme résultat un nombre à virgule flottante
Mod	Reste de la division de deux nombres
\	Division avec comme résultat un nombre entier
*	Multiplication
^	Élévation à la puissance

Les opérateurs de comparaison

Ils permettent de comparer deux valeurs numériques ou deux chaînes de caractères.

Opérateur	Calcul réalisé
<	Inférieur à
<=	Inférieur ou égal à
>	Supérieur à
>=	Supérieur ou égal à
=	Égal à
<>	Différent de

L'instruction **Option compare** utilisée au niveau module permet de définir la méthode de comparaison par défaut

© ENI Editions - All rigths reserved

qu'il convient d'utiliser lors de la comparaison de chaînes de caractères. Elle peut prendre trois valeurs :

Compare Binary

(option par défaut) fournit des comparaisons de chaînes basées sur un ordre de tri dérivé de la représentation binaire interne des caractères :
A<B<E<Z<a<b<e<z<À<Ê<Ø<à<ê...

Compare Text

fournit des comparaisons de chaînes basées sur un ordre de tri qui ne distingue pas les majuscules des minuscules :
(A=a)<(À=à)<(B=b)<(E=e)<(Ê=ê)<(Z=z)<(Ø=ø)...

Compare Database

fournit des comparaisons de chaînes basées sur l'ordre de tri déterminé par l'identificateur de paramètres régionaux de la base de données courante.

Les opérateurs logiques

Ils permettent de tester simultanément deux (ou plusieurs) valeurs booléennes ou expressions renvoyant ce type valeur. Ils sont généralement utilisés avec l'instruction **If**.

Opérateur	Calcul réalisé
And	Si toutes les expressions ont la valeur True, le résultat est True. Si l'une des expressions a la valeur False, le résultat est False.
Or	Si l'une au moins des expressions a pour valeur True, le résultat est True (ou inclusif).
Xor	Si une et une seule des expressions a pour valeur True, le résultat est True (ou exclusif).
Not	Renvoie le contraire de l'expression.
Eqv	Renvoie True si les deux expressions sont identiques.

Exemple

(A>= 1) And (A<= 9) renvoie True si A est compris entre 1 et 9,

Not (A >= 10) renvoie True si A est strictement inférieur à 10,

(A>0) Or (B>0) Or (C>0) renvoie True si au moins l'une des valeurs est positive.

L'opérateur de concaténation

L'opérateur de concaténation est le signe **&**. Il permet d'assembler des chaînes de caractères, des valeurs et des expressions. Le résultat est une chaîne de caractères.

Exemple

Concaténation du nom et du prénom séparés par un espace :

```
StrNomPre = Nom & " " & Prenom
```

Priorité des opérateurs

Lorsque plusieurs opérateurs sont contenus dans une même expression, chacun est évalué dans un ordre prédéfini, appelé priorité des opérateurs.

© ENI Editions - All rigths reserved

Les opérateurs sont évalués dans l'ordre suivant : opérateurs arithmétiques, opérateurs de comparaison, opérateurs logiques. Les opérateurs de comparaison ont la même priorité ; c'est-à-dire qu'ils sont évalués dans leur ordre d'apparition, de gauche à droite.

Les opérateurs arithmétiques et logiques sont évalués dans l'ordre de priorité ci-dessous (priorité décroissante de gauche à droite) :

Arithmétique : ^, *, /, Mod, +, -

Logique : Not, And, Or, Xor, Eqv

Exemple

*L'expression "3 + 4 * 5" donne comme résultat 23. La multiplication (4 * 5) est effectuée avant l'addition (+ 3) :*

» L'utilisation de parenthèses permet de modifier l'ordre de priorité afin qu'un élément d'une expression soit évalué avant les autres. Les opérations situées à l'intérieur de parenthèses sont toujours traitées avant les autres. Exemple : l'expression "(3 + 4) * 5" renvoie 35. L'addition est effectuée en priorité. Il est conseillé d'utiliser des parenthèses pour une meilleure lisibilité du code.

© ENI Editions - All rigths reserved

Les règles d'écriture du code

Les commentaires

Les commentaires permettent de documenter un programme afin de le rendre plus lisible.

Syntaxe

```
Rem <commentaire>
```

ou

```
'<commentaire>
```

Exemple

```
'=====================================================================
Rem Procédure permettant de comparer deux zones
Rem de texte txtA et txtB
'=====================================================================
Private Sub cmdRésultat_Click ()
  'si txtA supérieur à txtB
  If txtA > txtB Then
    lblRésultat.Caption = "A>B"
  Else
    'si txtA égal txtB
    If txtA = txtB Then
      lblRésultat.Caption = "A=B"
    Else
      'txtA inférieur à txtB
      lblRésultat.Caption = "A<B"
    End If
  End If
End Sub
```

Le caractère de continuation

Une instruction VBA peut être écrite sur plusieurs lignes en utilisant un caractère de continuation : le caractère de soulignement (_) précédé d'un espace.

Exemple

```
If A _
> _
b then _
```

Bien sûr, cet exemple n'est pas digne d'une programmation structurée, mais il met en évidence le fait qu'il est possible de scinder une ligne de code à n'importe quel endroit.

L'indentation

Il est important d'aligner sur le même retrait les instructions de même niveau. L'en-tête et le pied de procédure étant alignés à gauche, le contenu doit être décalé d'un cran à droite à l'aide d'une tabulation.

Ces indentations aèrent la structure de la procédure qui devient ainsi plus lisible.

Les noms des procédures, variables et constantes

Les noms des procédures, variables et constantes doivent respecter les règles suivantes :

- ils doivent commencer par une lettre.

- Ils ne doivent pas contenir plus de 255 caractères.

© ENI Editions - All rigths reserved

- Ils peuvent être composés de lettres, de chiffres et du caractère "blanc souligné" (_).

- Ils ne peuvent comprendre ni caractères de ponctuation ni espaces.

- Ils ne doivent pas correspondre à des mots réservés.

- Ils doivent être uniques à l'intérieur d'une même portée.

Exemples

```
Nbclient
Nb_client
Nbclient1
```


© ENI Editions - All rigths reserved

Les conventions d'appellation

Il est recommandé d'utiliser des conventions d'appellation pour les variables, les contrôles et les objets. Elles permettent de standardiser le code et par conséquent de le rendre plus lisible et plus facile à maintenir par différents développeurs.

Convention d'appellation des variables

Il est recommandé d'utiliser deux préfixes :

- le premier pour préciser la portée de la variable : g pour globale, m pour module, aucun préfixe pour locale,

- le deuxième pour préciser son type.

Exemple

`gstrNom` variable globale de type String

`intQuantité` variable locale de type Integer

Type de variable	Préfixe
Booléen	bln
Byte	byt
Currency	cur
Date, heure	dtm
Double	dbl
Error	err
Integer	int
Long	lng
Object	obj
Single	sng
String	str
Défini par l'utilisateur	udt
Variant	var

Convention d'appellation des contrôles

De la même façon que pour les variables, les noms de contrôles des formulaires et des états doivent être préfixés selon leur type.

Contrôle	Préfixe
Données ADO	ado

© ENI Editions - All rigths reserved

Case à cocher	chk
Zone de liste modifiable (combo)	cbo
Bouton de commande	cmd
Cadre (frame)	fra
Groupe d'option	grp
Image	img
Étiquette	lbl
Bouton d'option	opt
Zone de texte	txt

Convention d'appellation des objets

Les objets Access ou les variables objets se rapportant aux objets Access peuvent aussi être préfixés.

Contrôle	**Préfixe**
Table	tbl
Requête	qry
Formulaire	frm
État	rpt
Macro	mac
Index	idx
Champ	fld
Collection	col
Propriété	pty
Pages d'accès aux données	pag

© ENI Editions - All rigths reserved

Présentation

VBA Access est un langage de programmation orienté objet, même s'il ne dispose pas de toutes les fonctionnalités des langages de ce type.

La plupart des éléments manipulés dans Access sont des objets : les tables, les champs, les enregistrements, les formulaires...

Les objets sont organisés selon un **modèle hiérarchique** : certains objets contiennent des objets qui en contiennent eux-mêmes d'autres... Par exemple, l'objet **Application** est composé de plusieurs objets **Form** (formulaires de l'application), qui sont eux-mêmes composés de plusieurs objets **Control** (contrôles d'un formulaires).

Un ensemble d'objets de même nature constitue une **collection** (collection **Forms** : ensemble des formulaires ou objets **Form** de l'application).

Un objet dispose d'un ensemble de caractéristiques appelées **propriétés** (ex : taille, couleur pour un contrôle dans un formulaire...) et de comportements ou actions appelées **méthodes** (ex : la méthode close de l'objet Formulaire permet de fermer celui-ci). Il répond à des **événements** provoqués par l'utilisateur (ex : clic sur un bouton de commande) ou par le système.

Les **classes** sont des modèles permettant de créer des objets de même nature. Les objets issus d'une même classe héritent systématiquement de toutes les méthodes, propriétés et événements de leur classe d'origine. Il est possible de créer des classes d'objets avec VBA Access en utilisant des modules de classe.

Il existe plusieurs ensembles d'objets dans Access :

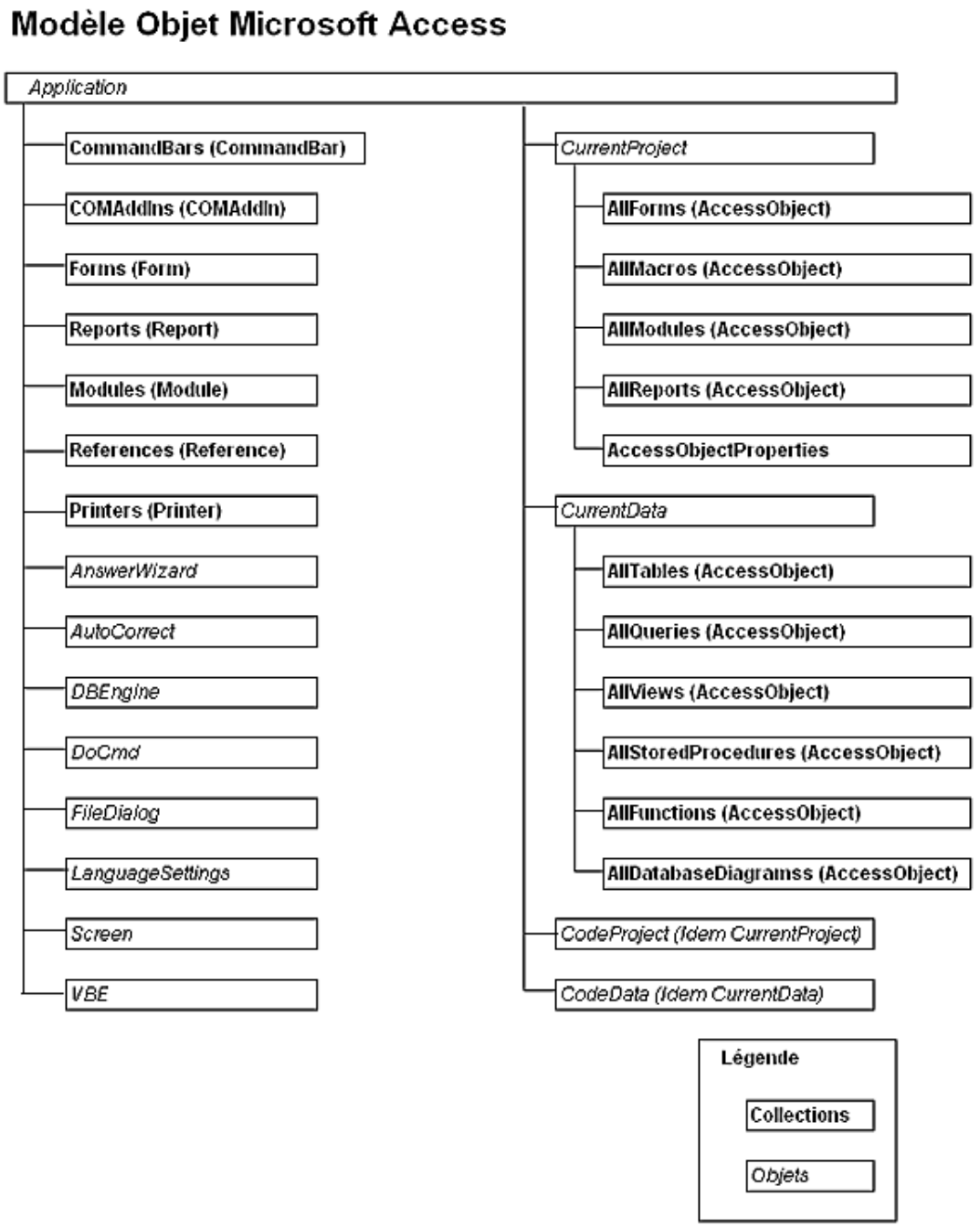
- les objets généraux (formulaires, états ...),
- les objets d'accès aux données (modèle DAO),
- les objets de données ActiveX (modèle DAO).

Chaque ensemble peut être représenté sous forme d'un modèle hiérarchique. Ce chapitre décrit plus particulièrement les objets généraux d'Access, les objets d'accès aux données seront abordés dans le chapitre suivant.

© ENI Editions - All rigths reserved

Le modèle objet d'Access

Le diagramme ci-après permet de distinguer les principaux objets globaux d'Access.



Principales collections

CommandBars	Collection d'objets **CommandBar** qui représentent **les barres de commandes** de l'application active.
COMAddIns	Collection d'objets **COMAddIn** qui fournissent des **informations sur un complément COM** inscrit dans le Registre Windows.
Forms	Collection contenant tous **les formulaires** (objets Form) ouverts dans la base de données active. Pour répertorier tous les formulaires de la base de données, qu'ils soient ouverts ou fermés, utilisez la collection **AllForms** de l'objet

© ENI Editions - All rigths reserved

	CurrentProject.
Reports	Collection contenant tous **les états** (objets Report) ouverts dans la base de données active. Pour répertorier tous les états de la base de données, qu'ils soient ouverts ou fermés, utilisez la collection **AllReports** de l'objet **CurrentProject**.
Modules	Collection contenant tous **les modules standards** et modules de classe ouverts dans la base de données active.
References	Collection contenant toutes **les références à des bibliothèques d'objets** appartenant à d'autres applications (fichiers dll, contrôles ActiveX...) actuellement sélectionnées (les références sont sélectionnées à partir de la boîte de dialogue **Référence** du menu **Outils**).
Printers	Collection contenant les objets **Printer** représentant toutes **les imprimantes** disponibles sur le système.

Autres collections

DependencyObjects	Collection des objets Access contenus dans un objet **DependencyInfo** (informations de dépendance d'un objet AccessObject). Pour renvoyer une collection Dependency-Objects, utilisez la propriété Dependants ou Dependencies de l'objet DependencyInfo.
AccessObjects Properties	Collection d'objets **AccessObjectsProperty** représentant une caractéristique intégrée ou définie par l'utilisateur d'un objet **AccessObject** (objet Access).
SmartTags	Collection d'objets **SmartTag** représentant **une balise active** associée à un contrôle dans un formulaire, un rapport ou une page d'accès aux données.
SmartTagActions	Collection d'**actions pour une balise active** individuelle ou pour un type de balise active. Les actions de balise active sont des processus programmés dans des balises actives qui permettent aux utilisateurs d'effectuer certaines fonctions liées à la balise active. Par exemple, pour une balise active, une action peut consister à accéder à un site Web, tandis qu'une autre action insère des informations de contact provenant de **Microsoft Outlook**, et qu'une autre encore affiche une carte et des instructions de conduite.
SmartTagProperties	Collection d'objets SmartTagProperty représentant les **propriétés liées à une balise active**. Pour renvoyer la collection SmartTagProperties d'une balise active, utilisez la propriété Properties de l'objet **SmartTag**.
ImportExport Specifications	Collection d'objets contenant toutes les informations dont Access a besoin pour effectuer automatiquement une opération d'importation ou d'exportation.

Objets du Modèle

AnswerWizard	Objet représentant l'aide intuitive de Microsoft Office.
Application	Objet faisant référence à l'application Microsoft Access active.
Assistant	Objet représentant le Compagnon Office de Microsoft. Vous pouvez utiliser la propriété Visible pour afficher le Compagnon, la propriété On pour activer le Compagnon, et la propriété FileName pour modifier le Compagnon.
AutoCorrect	Objet représentant les options de correction automatique d'Access.
DBEngine	Objet représentant le moteur de base de données Microsoft Jet. En tant qu'objet principal à la racine de la hiérarchie, l'objet **DBEngine** contient et contrôle tous les autres objets

© ENI Editions - All rigths reserved

	d'accès aux données.
DependencyInfo	Objet représentant les informations de dépendance d'un objet Access.
DoCmd	Objet permettant de convertir en Visual Basic des actions Macro. La plupart des actions macros peuvent être ainsi traduites en VBA.
FileDialog	Objet permettant d'accéder à des fonctionnalités similaires à celles des boîtes de dialogue standards **Ouvrir** et **Enregistrer**.
LanguageSettings	Objet renvoyant des informations sur les paramètres linguistiques de l'application.
Screen	Objet permettant d'accéder au formulaire, à l'état ou au contrôle actuellement actif.
TempVar	Objet permettant d'échanger facilement des données entre des procédures VBA et des macros.
VBE	Objet représentant l'éditeur Microsoft Visual Basic Édition Applications.
CurrentProject	Objet regroupant plusieurs collections d'objets Access spécifiques (**AllForms** : collection de tous les formu-laires de la base)...
CurrentData	Objet regroupant plusieurs collections d'objets Access d'accès aux données. (**AllTables** : collection de toutes les tables de la base)...

© ENI Editions - All rigths reserved

Principes d'utilisation des objets et des collections

Les propriétés

Les propriétés servent à décrire un objet. Certaines propriétés sont en lecture seule et ne peuvent donc pas être modifiées par du code VBA.

Syntaxe

```
{<objet> | <variable objet>}.<propriété>
```

Exemple

```
Dim strVersion As String
' Récupération de la propriété version
' de l'objet application
' Cette propriété est en lecture seule
strVersion = Application.Version
' Modification du pointeur de la souris
' 0 : pointeur par défaut - 11 : Sablier
If Screen.MousePointer = 0 Then
  Screen.MousePointer = 11
Else
  Screen.MousePointer = 0
End If
End Sub
```

Propriétés représentant des objets

Les objets globaux et les objets instanciés dans le code à partir de classes fournies par VBA possèdent des propriétés dont la valeur est mise à jour automatiquement par le système.

Propriété	De l'objet	Contenu
ActiveControl	Screen	Contrôle actif.
ActiveDataAccessPage	Screen	Page active ou page contenant le contrôle actif.
ActiveForm	Screen	Formulaire actif.
ActiveReport	Screen	État actif.
Application	Objets multiples	L'objet Application d'Access.
DBEngine	Application	L'objet DBEngine.
Form	Contrôle sous-formulaire	Objet Form associé au contrôle de sous-formulaire.
Me	Form ou Report	L'objet Form ou Report dont le code est en cours d'exécution.
Module	Form ou Report	Module de l'objet Form ou Report.
Parent	Objets multiples	Objet ou collection qui contient l'objet.
PreviousControl	Screen	L'objet Control précédemment actif.
RecordsetClone	Form	Un Recordset clone du jeu d'enregistrements sous-jacent du formulaire.
Report	Contrôle sous-état	Objet Report associé au contrôle de sous-état.
Section	Form, Report	Section d'un formulaire ou d'un état.

Les méthodes

Les méthodes permettent d'effectuer des actions propres aux objets.

© ENI Editions - All rigths reserved

Elles se présentent comme des procédures :

- elles peuvent ou non utiliser des arguments,

- certaines méthodes peuvent renvoyer une valeur au même titre que les procédures **Function**, d'autres non au même titre que les procédures **Sub**.

Syntaxe de méthode ne renvoyant pas de valeur

```
{<objet> | <variable objet>}.<méthode> [<Liste d'arguments>]
```

Exemple

```
' Modification de l'option "Compacter lors de la
fermeture"
' Méthode SetOption ne renvoyant pas de valeur
Application.SetOption "Auto Compact", True
```

Les différents arguments de la méthode doivent être séparés par des virgules. Si un argument facultatif n'est pas défini explicitement, la méthode utilisera une valeur par défaut.

Syntaxe de méthode renvoyant une valeur

```
<variable> = {<objet> | <variable objet>}.<méthode>
([<Liste d'arguments>])
```

Exemple

```
Private Sub cmdMethodes_Click()
' Variable booléenne
Dim blnCompact As Boolean
' Récupération de l'option
' "Compacter lors de la fermeture"
' Méthode GetOption renvoyant une valeur booléenne
blnCompact = Application.GetOption("Auto Compact")
' Modification de l'option
' Méthode SetOption ne renvoyant pas de valeur
  If blnCompact Then
  Application.SetOption "Auto Compact", False
  MsgBox "La base ne sera pas compactée lors de la
fermeture", _
   vbExclamation
Else
  Application.SetOption "Auto Compact", True
  MsgBox "La base sera compactée lors de la fermeture", _
  vbExclamation
End If
End Sub
```

Les événements

Un événement est une **action spécifique** qui se produit sur un objet. Microsoft Access est en mesure de répondre à plusieurs types d'événements : ouverture ou fermeture de formulaires, clics de souris, modification de données, etc. Les événements résultent généralement d'une action de l'utilisateur.

L'utilisation d'une procédure événementielle vous permet d'associer votre propre code en réponse à un événement qui se produit dans un formulaire, un état, un contrôle...

Exemple

Lorsque l'utilisateur clique sur le bouton de commande "Quitter", une boîte de dialogue demandant une confirmation est affichée.

```
Private Sub cmdQuitter_Click()
' Demande à l'utilisateur de confirmer
```


© ENI Editions - All rigths reserved

```
' son souhait de quitter l'application
If MsgBox("Voulez-vous quitter l'application ?", _
  vbQuestion + vbYesNo) = vbYes Then
  DoCmd.Quit
End If
End Sub
```

» La gestion des événements étant l'un des aspects les plus importants dans le développement d'applications Access, le chapitre **Gestion des événements** est entièrement consacré à ce sujet.

Les collections

Pour faire référence à un objet d'une collection, vous pouvez utiliser l'une des syntaxes suivantes :

```
NomCollection!NomObjet
NomCollection![NomObjet]
NomCollection("NomObjet")
NomCollection(var)
```
où var représente une variable de type string contenant le nom de l'objet.
```
NomCollection(index)
```
où index représente le numéro d'index de l'objet dans la collection.

» Afin d'assurer une meilleure lisibilité du code, il est conseillé d'utiliser toujours la même syntaxe. Les 3ième et 5ième syntaxes sont recommandées car elles permettent d'activer l'assistant de l'éditeur de code. De plus la syntaxe 5 est très utile pour parcourir les objets d'une collection.

» Attention le premier élément de la plupart des collections a pour index 0. N'utilisez les index que pour parcourir une collection. Évitez par exemple `Me.controls(5)` pour faire référence à un contrôle car l'index du contrôle peut changer si le formulaire est modifié.

Exemple

```
Private Sub cmdCollections_Click()
Dim ctl As Control
Dim intI As Integer

' Ouvre le formulaire "Employes"
' et le masque (propriété Visible)
DoCmd.OpenForm "Employes"
Forms("Employes").Visible = False

' Fait référence au contrôle Prénom
' du Formulaire "Employes"
' Collections Forms et Controls
MsgBox Forms("Employes").Controls("Prénom").Value
MsgBox Forms!Employes.Controls![Prénom].Value

' Date de création d'une table de la base courante
' Collections AllTables
MsgBox CurrentData.AllTables![Employes].DateCreated
MsgBox CurrentData.AllTables("Employes").DateCreated
MsgBox CurrentData.AllTables(0).DateCreated
' Modifie la police et de la couleur de tous les
contrôles
' "zones de texte" du formulaire
For intI = 0 To Forms("Employes").Controls.Count - 1
  Set ctl = Forms("Employes").Controls(intI)
  If TypeOf ctl Is TextBox Then
    ctl.ForeColor = vbRed
    ctl.FontItalic = True
    ctl.FontBold = True
  End If
Next intI

' Affiche le formulaire
Forms("Employes").Visible = True
End Sub
```

Collections par défaut

Définition

© ENI Editions - All rigths reserved

Une collection par défaut est une collection contenant des objets auxquels il est possible de faire référence à travers le nom de l'objet conteneur de la collection, c'est-à-dire sans faire référence à la collection elle-même.

Objets possédant une collection par défaut

Objet	Collection par défaut
Container	Documents
Database	TableDefs
DBEngine	Workspaces
Form	Controls
Group	Users
Index	Fields
QueryDef	Parameters
Recordset	Fields
Relation	Fields
Report	Controls
TableDef	Fields
User	Groups
Workspace	Databases

Exemple

Référence au contrôle txtDateDeb du formulaire Formations (la collection Controls est la collection par défaut d'un formulaire).

```
Forms![Formations].[txtDateDeb].Value
Ou
Forms!Formations.txtDateDeb.Value
ou
Forms!Formations!txtDateDeb.Value
```

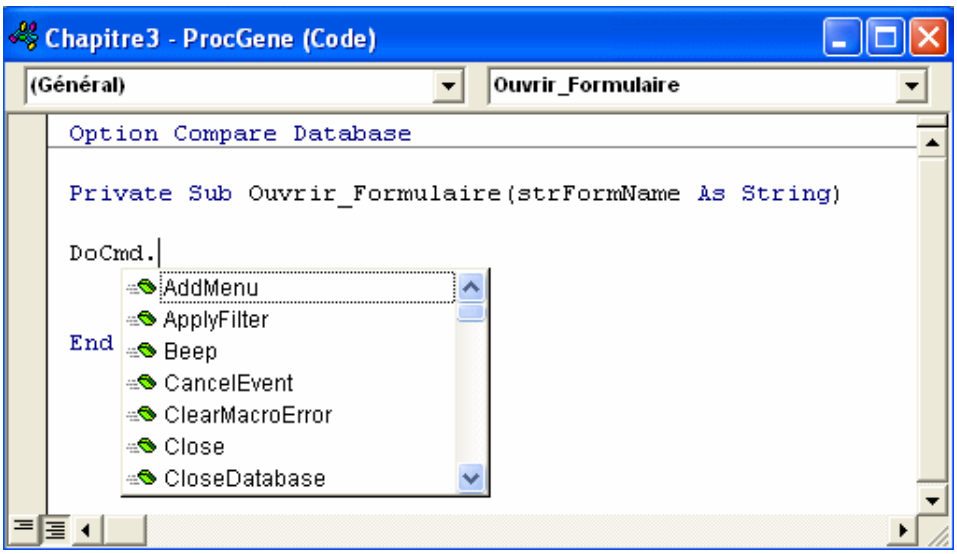
Affichage automatique d'instructions

L'éditeur VBA dispose d'une technologie permettant de vous assister lors de l'utilisation d'objets. Dès que vous tapez un nom d'objet reconnu par VBA suivi d'un point, la liste déroulante des méthodes et propriétés de cet objet est alors affichée. Si vous sélectionnez une méthode, l'assistant vous aide également à saisir les différents arguments qu'elle comporte.

Exemple

- Tapez le nom d'objet **DoCmd** suivi d'un point, la liste déroulante suivante est affichée.

© ENI Editions - All rigths reserved



» L'icône  représente les méthodes, l'icône  représente les propriétés.

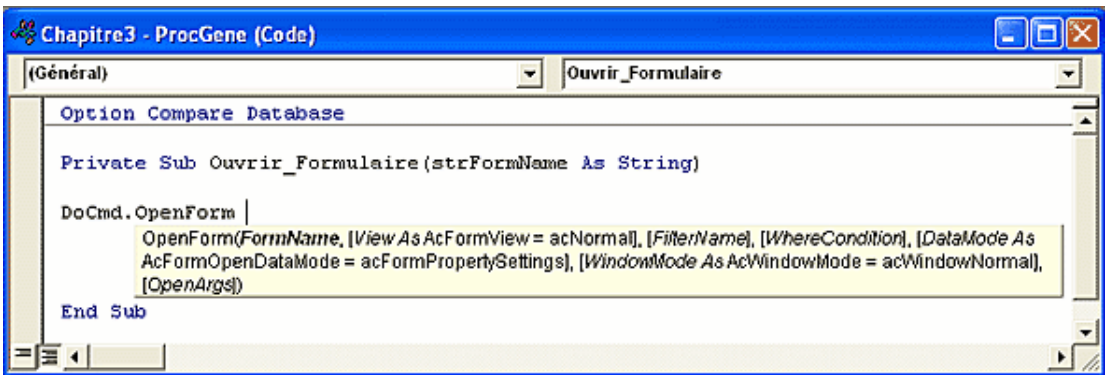
Vous pouvez faire défiler les éléments de la liste déroulante en tapant les premières lettres de la méthode, propriété ou collection recherchée ou à l'aide de l'ascenseur. Pour sélectionner un élément de la liste, double cliquez dessus.

Tapez un point si vous venez de sélectionner un objet et si vous souhaitez voir la liste des propriétés et méthodes s'y rapportant. Si vous avez sélectionné une méthode, tapez un espace pour saisir la liste des paramètres de la méthode.

Pour poursuivre l'exemple :

- Sélectionnez la méthode OpenForm.

- Puis tapez un espace :

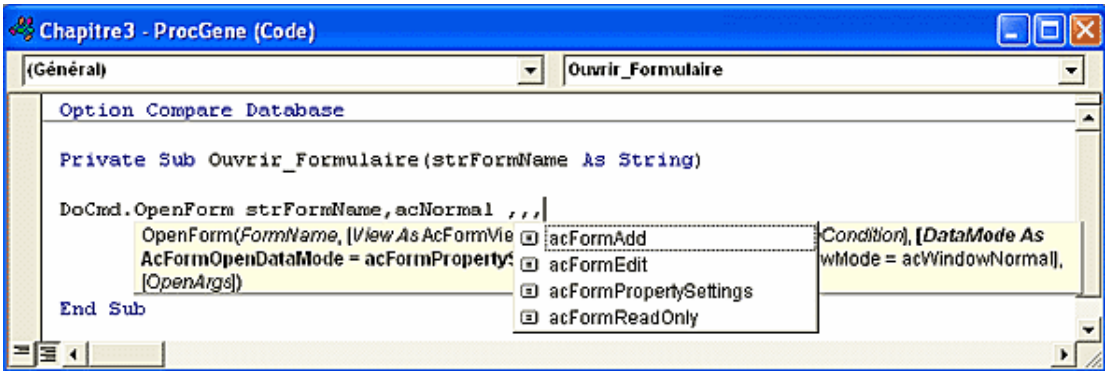


La liste des arguments de la méthode est alors affichée et s'adapte au fur et à mesure que vous les saisissez.

Les arguments facultatifs sont entre crochets. L'argument courant est en gras. Si pour un argument donné, il existe une liste de valeurs prédéfinies, la liste déroulante des constantes correspondantes est alors affichée.

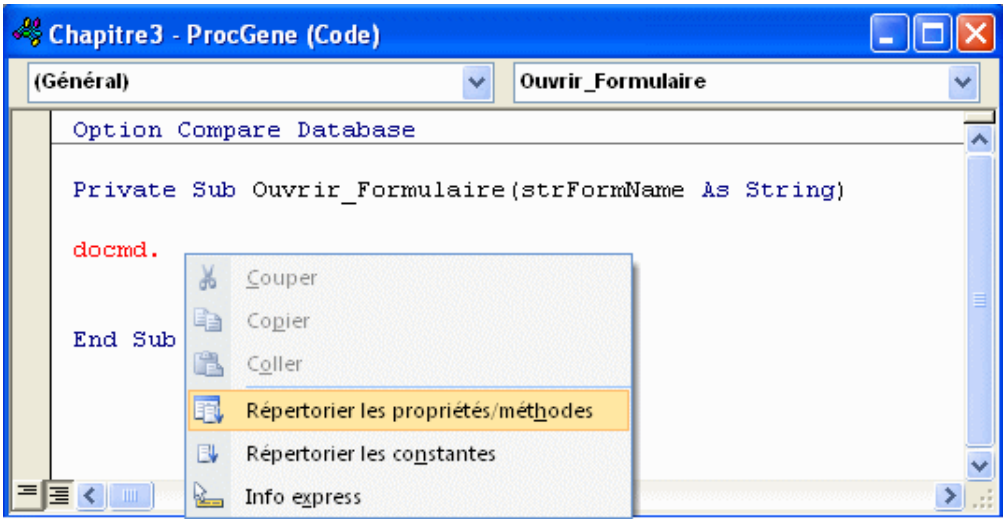
- Saisissez maintenant le nom du paramètre transmis dans la fonction, à savoir strFormName, suivi d'une virgule et de l'argument acNormal, et de 3 virgules (pour les deux arguments optionnels).

© ENI Editions - All rigths reserved



Dans l'exemple la liste des valeurs possibles pour l'argument DataMode est affichée.

» Pour obtenir la liste des propriétés et méthodes d'un objet, vous pouvez également procéder de la façon suivante :
- placez le curseur derrière le point (situé après le nom de l'objet),
- cliquez sur le bouton droit de la souris et sélectionnez l'option **Répertorier les propriétés/méthodes** du menu contextuel ou utilisez le raccourci-clavier **Ctrl J**.



» De même, pour obtenir la liste des constantes disponibles comme argument d'une méthode, vous pouvez procéder de la façon suivante :
- placez le curseur à la place de l'argument (après la méthode ou après une virgule),
- cliquez sur le bouton droit de la souris et sélectionnez l'option **Répertorier les constantes** du menu contextuel.

© ENI Editions - All rigths reserved

Instructions utilisées avec les objets

L'instruction With

L'instruction **With** permet d'accéder plusieurs fois au même objet en le nommant une seule fois.

Elle offre plusieurs avantages :

- optimisation du temps d'exécution du code,

- gain de temps sur le travail d'écriture,

- meilleure lisibilité du code.

Syntaxe

```
With <objet>
  <code utilisant des méthodes et propriétés>
  <se rapportant à l'objet>
End With
```

Exemple

```
' Ouvre le formulaire "Employes"
DoCmd.OpenForm "Employes"
' Modifie le titre du formulaire
' Positionne le curseur sur le nom
With Forms("Employes")
  .Caption = "FICHE DE L'EMPLOYE " & _
  .Controls("Prénom").Value & " " & _
  UCase(.Controls("Nom").Value)
  .Controls("Nom").SetFocus
End With
```

L'instruction For each...Next

L'instruction **For Each...Next** permet de passer en revue les objets d'une collection ou les éléments d'un tableau.

Syntaxe

```
For Each <élément > In {<tableau>|<collection>}
  <séquence d'instructions>
  [Exit For]
  <séquence d'instructions>
Next <élément>
```

Exemple

L'exemple suivant permet d'affecter une couleur de police à chaque contrôle du formulaire "employé" en fonction de son type (propriété ControlType). Il utilise la collection par défaut de l'objet Form (Screen.ActiveForm équivaut à Screen.ActiveForm.Controls).

```
Dim ctl As Control
' Parcours des contrôles du formulaire actif
For Each ctl In Screen.ActiveForm
  With Ctl
    Select Case ControlType
    ' Etiquettes
    Case acLabel
      .ForeColor = vbBlue
    ' Zones de texte
    Case acTextBox
      .ForeColor = vbYellow
    ' Listes déroulantes
    Case acListBox, acComboBox
      .ForeColor = vbRed
```

© ENI Editions - All rigths reserved

```
    End Select
  End With
Next ctl
```

L'instruction If TypeOf

L'instruction **If TypeOf** permet de tester le type d'un objet.

Syntaxe

```
If TypeOf <objet> Is <TypeObjet> Then
  <code utilisant des méthodes et propriétés>
  <se rapportant à l'objet>
End If
```

Exemple

```
If TypeOf ctl Is acListBox Then ...
```

L'instruction Set

L'instruction **Set** permet d'attribuer une référence d'objet à une variable, appelée **variable objet**.

Cette instruction peut être utilisée pour créer une référence vers un nouvel objet (en utilisant éventuellement une méthode permettant de créer l'objet) ou pour affecter une référence à un objet déjà existant.

Syntaxes

Création d'une référence vers un nouvel objet

```
Set <NomObjet> = New <expression objet>
ou
Set <NomObjet> = <méthode permettant de créer l'objet>
```

`<NomObjet>` nom de la variable objet
`<expression objet>` nom d'un objet ou d'une variable objet de même type

» Le mot clé **New** permet de créer une nouvelle instance de la classe. Si la variable objet contient déjà une référence, cette dernière est alors perdue.

» Vous ne pouvez utiliser le mot clé New pour référencer un objet que si le composant ActiveX de l'objet fournit une bibliothèque de type (ex : objets ADO, objets Excel...).

Affectation d'une référence à un objet existant

```
Set <NomObjet> = <expression objet>
```
`<NomObjet>` nom de la variable objet
`<expression objet>` nom d'un objet ou d'une variable objet de même type

Réinitialisation des variables objets

```
Set <NomObjet> = Nothing
```
`Nothing` permet de réinitialiser la variable objet et de libérer l'ensemble des ressources système et mémoire associées à cet objet.

Exemples :

Création de références vers de nouveaux objets.

*Les exemples 1 et 2 permettent de créer une nouvelle table et de l'ajouter à la base de données courante. Pour exécuter ces exemples, vous devez sélectionner la référence Microsoft DAO 3.6 Object Library au projet (**Outils - Références**).*

Exemple 1 : utilisation du mot clé New

© ENI Editions - All rigths reserved

```
Dim tbl As Object
Dim tdfClient As DAO.TableDef
Dim fld As DAO.Field

' Vérifie si la table existe déjà
' et propose de la supprimer
For Each tbl In CurrentData.AllTables
  If tbl.Name = "CLIENTS" Then
    If MsgBox("La table existe déjà." & _
      " Voulez-vous la supprimer ?", _
      vbYesNo & vbQuestion) = vbYes Then
      DoCmd.DeleteObject acTable, "CLIENTS"
    Else
      Exit Sub
    End If
  End If
Next tbl

' Crée la définition de table
Set tdfClient = New DAO.TableDef
tdfClient.Name = "CLIENTS"
' Crée le 1er champ
Set fld = New DAO.Field
With fld
  .Name = "Cli_Nom"
  .Type = dbText
  .Size = 40
End With
tdfClient.Fields.Append fld
' Crée le 2ème champ
Set fld = New DAO.Field
With fld
  .Name = "Cli_Effectif"
  .Type = dbInteger
End With
tdfClient.Fields.Append fld
' Ajoute la table à la base de donnée courante
Application.CurrentDb.TableDefs.Append tdfClient
' Réinitialise les variables objet
Set tdfClient = Nothing
Set fld = Nothing
```

Exemple 2 : même traitement en utilisant des méthodes

```
Dim tbl As Object
Dim tdfClient As DAO.TableDef
Dim fld As DAO.Field

' Vérifie si la table existe déjà
' et propose de la supprimer
For Each tbl In CurrentData.AllTables
  If tbl.Name = "CLIENTS" Then
    If MsgBox("La table existe déjà." & _
      " Voulez-vous la supprimer ?", _
      vbYesNo + vbQuestion) = vbYes Then
      DoCmd.DeleteObject acTable, "CLIENTS"
    Else
      Exit Sub
    End If
    End If
Next tbl

' Crée la définition de table
Set tdfClient = CurrentDb.CreateTableDef("Clients")
With tdfClient
  ' Crée le 1er champ
   Set fld = .CreateField("Cli_Nom", dbText, 40)
  .Fields.Append fld
   ' Crée le 2ème champ
   .Fields.Append .CreateField("Cli_Effectif", dbInteger)
End With
' Ajoute la table à la base de donnée courante
CurrentDb.TableDefs.Append tdfClient
' Réinitialise les variables objet
Set fld = Nothing
Set tdfClient = Nothing
```

© ENI Editions - All rigths reserved

Exemple 3 :

Création d'un formulaire avec une zone de texte et une étiquette

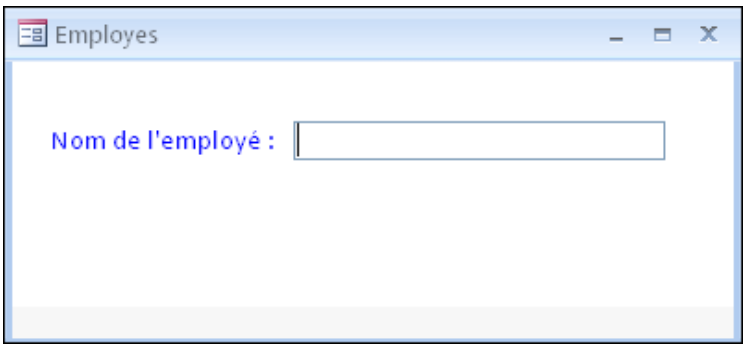
```
' Objet Formulaire
Dim frmEmploye As Form
' Nom du formulaire
Dim strName As String
' Objet contrôle
Dim ctl As Control
' Création d'un nouveau formulaire
Set frmEmploye = Application.CreateForm
With frmEmploye
  ' Titre du formulaire
  .Caption = "Employes"
  ' Largeur et hauteur du formulaire
  .Width = 5000
  .Section(acDetail).Height = 2000
  ' Supprime les boutons de déplacement
  .NavigationButtons = False
  ' Supprime le sélecteur d'enregistrement
  .RecordSelectors = False
  ' Centre le formulaire dans l'application
  .AutoCenter = True
  ' Nom du formulaire
  strName = frmEmploye.Name
End With

' Ajout de la zone de texte Nom de l'employé
Set ctl = Application.CreateControl(strName, _
    acTextBox, , "", "", 2000, 500, 2500, 300)
' Mise en forme de la zone de texte
With ctl
  ' Nom de la zone de texte
  .name = "txtNom"
  ' Couleurs
  .BackColor = vbWhite
  .ForeColor = vbBlack
  .FontBold = True
End With

' Ajout de l'étiquette Nom de l'employé
Set ctl = Application.CreateControl(strName, _
    acLabel, , "", "", 500, 500, 1500, 300)
' Mise en forme de l'étiquette
With ctl
  ' Nom de l'étiquette
  .Name = "lblNom"
  ' Titre de l'étiquette
  .Caption = "Nom de l'employé : "
  ' Couleurs
  .BackColor = vbWhite
  .ForeColor = vbBlue
End With

' Sauvegarde du formulaire
DoCmd.Save , "Fiche_Employe"
Docmd.Close
```

Formulaire créé par ce code :



© ENI Editions - All rigths reserved

© ENI Editions - All rigths reserved

Les classes d'objets

Description

Une **classe** est en quelque sorte un moule d'où naissent les **objets**. Par exemple, il est possible de réaliser de petits santons en plâtre avec un moule adéquat. On peut dire que tous les santons réalisés sont des **Objets**, chacun d'eux pouvant avoir à terme des couleurs et un habillage différents, et que les moules sont des **Classes**.

Ainsi, les objets issus d'une même classe héritent systématiquement de toutes les méthodes (codes), les propriétés (données) et les événements de leur classe d'origine. Par la suite, les caractéristiques et le comportement de chacun d'eux pourront être adaptés en fonction des besoins de l'application.

En fait, tout objet Access est issu d'une classe.

Les modules de classe

Les modules de classe vous permettent de créer et de manipuler dans vos applications vos propres types d'objets.

Les modules de classes présentent les caractéristiques suivantes :

- ils portent le nom de l'objet,
- ils comportent des procédures publiques **Sub** et **Function** correspondant aux méthodes propres de l'objet,
- ils comportent des procédures **Property Get** permettant de lire les valeurs de propriétés de l'objet et des procédures **Property Set/Property Let** permettant de fixer les valeurs de ces propriétés.

Une fois l'ensemble de ces procédures décrites au sein du module de classe, il est possible de créer un nouvel objet du type défini. Pour cela il suffit de créer une nouvelle instance de la classe par le biais d'une variable objet du type de la classe :
Dim <nom de l'objet> As New <nom de la classe>

Les modules de classe sont peu utilisés en programmation VBA. Ils s'adressent avant tout aux développeurs confirmés.

Exemple de modules de classe

Cet exemple vous guidera pas à pas dans la création et l'utilisation d'un module de classe. Le module créé permettra d'afficher les messages utilisateurs.

Ce module comporte les éléments suivants :

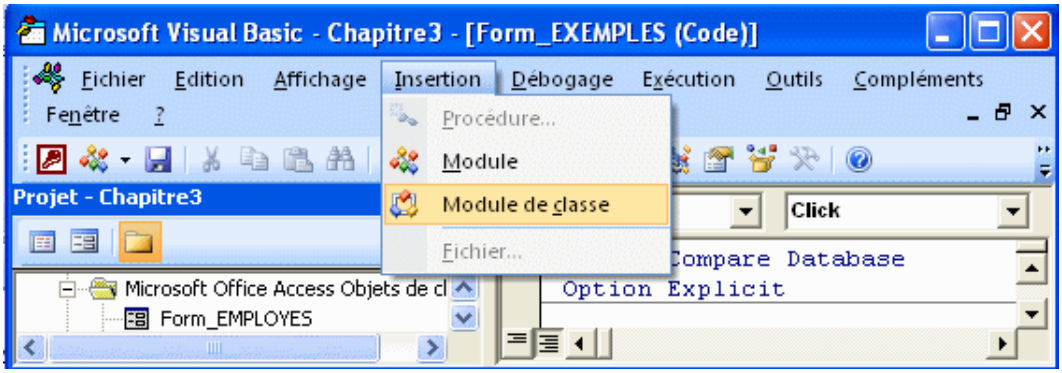
- une méthode **Confirmation** qui affiche une boîte de dialogue invitant l'utilisateur à répondre par "oui" ou "non" à une question, et récupère la réponse de l'utilisateur,
- deux propriétés : la propriété **IsBeep** indique si un beep doit être émis avant l'affichage du message, la propriété **Title** contient le titre de la boîte de dialogue.

Exemple

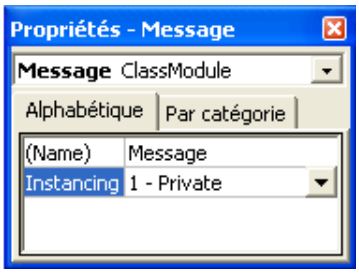
Pour créer un module de classe :

- sélectionnez l'option **Module de classe** du menu **Insertion**.

© ENI Editions - All rigths reserved



- Définissez ensuite le nom de la classe dans le champ Name.

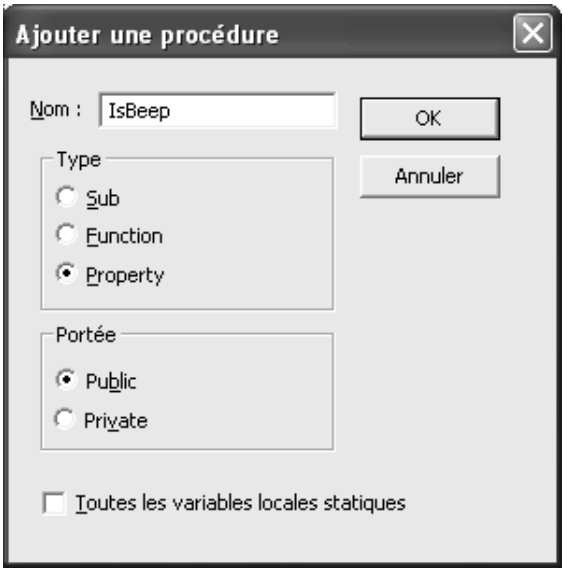


- Si la fenêtre **Propriétés** n'est pas affichée, choisissez l'option **Fenêtre propriétés** du menu **Affichage** ou utilisez le raccourci-clavier **F4**.

- Définissez les variables nécessaires pour l'utilisation des propriétés dans le module de classe Message. La variable strTitle est associée à la propriété Title, la variable blnBeep à la propriété IsBeep.

```
Option Compare Database
Option Explicit
' Emission d'un beep
Dim blnBeep As Boolean
' Titre de la fenêtre MsgBox
Dim strTitle As String
```

- Créez les propriétés **IsBeep** et **Title** en utilisant l'option **Procédure** du menu **Insertion**.



*Pour chaque propriété, les procédures **Property Get** et **Property Set** sont créées :*

- Modifiez-les afin d'obtenir le code suivant :

© ENI Editions - All rigths reserved

```
Public Property Get IsBeep() As Boolean
  IsBeep = blnBeep
End Property
_____________________________________________________________________

Public Property Let IsBeep(ByVal blnNewValue As Boolean)
  blnBeep = blnNewValue
End Property
_____________________________________________________________________

Public Property Get Title() As String
  Title = strTitle
End Property
_____________________________________________________________________

Public<+>Property Let Title(ByVal strNewValue As String)
  strTitle = strNewValue
End Property
```

- Créez maintenant la méthode confirmation en utilisant l'option **Procédure** du menu **Insertion**.



- Modifiez la procédure Confirmation afin d'obtenir le code suivant.

```
Public Function Confirmation(strMsg) As Boolean
  If MsgBox(strMsg, vbYesNo, strTitle) = vbYes Then
    Confirmation = True
  Else
    Confirmation = False
  End If
  If blnBeep Then Beep
End Function
```

Nous voyons que cette méthode utilise les deux variables **strTitle** et **blnBeep**. La méthode a été créée en tant que fonction car elle renvoie la réponse de l'utilisateur sous la forme d'une variable booléenne.

Maintenant que cette classe est créée, nous allons l'utiliser à partir d'un formulaire.

- Créez pour cela un formulaire **Employés** avec deux boutons de commande **cmdFermer** et **cmdQuitter**.

- Affectez le code ci-dessous aux événements **click**.

```
Private Sub CmdFermer_Click()
' Objet message
Dim msg As New Message

' Pas d'émission de beep
msg.IsBeep = False
```

© ENI Editions - All rigths reserved

```
' Titre de la boîte de dialogue
msg.Title = "Application Gestion du personnel"
' Demande de confirmation
If msg.Confirmation("Voulez-vous fermer " _
  & "le formulaire " & Me.Caption & " ?") Then
  DoCmd.Close
End If
End Sub
```

```
Private Sub CmdQuitter_Click()
' Objet message
Dim msg As New Message

' Emission d'un beep
msg.IsBeep = True
' Titre de la boîte de dialogue
msg.Title = "Application Gestion du personnel"
' Demande de confirmation
If msg.Confirmation("Voulez-vous quitter " _
  & "l'application ? ") Then
  DoCmd.Quit
End If

End Sub
```

- Testez ensuite le formulaire.

La boîte de message affichée par la méthode **Confirmation** utilise les deux propriétés et renvoie la réponse de l'utilisateur.

» Il est possible d'enrichir le module de classe **Message** en définissant de nouvelles propriétés et de nouvelles méthodes (ex : messages d'erreur...).

© ENI Editions - All rigths reserved

L'explorateur d'objets

Présentation

Compte tenu du nombre important d'objets Access et de leur diversité, il est utile de pouvoir rechercher rapidement des informations se rapportant aux objets.

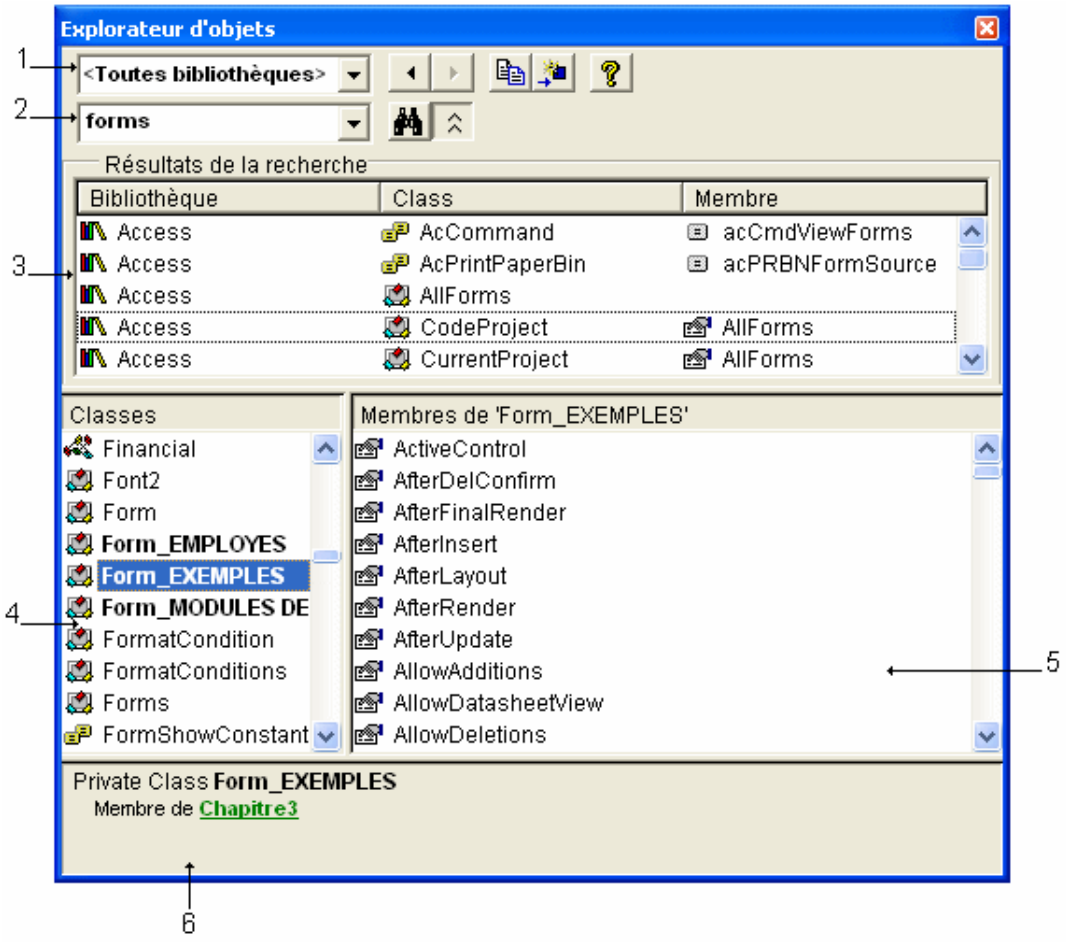
L'explorateur d'objets permet d'afficher des informations relatives aux objets, méthodes, propriétés, événements et constantes.

Vous pouvez l'afficher de différentes façons :

- choisissez l'option **Explorateur d'objets** du menu **Affichage**,

- utilisez le raccourci-clavier **F2**,

- cliquez sur l'icône  .



1. Liste des bibliothèques d'objets actuellement chargées.

2. Élément recherché : objet, propriété, collection, événement, méthode...

3. Résultat de la recherche : liste des classes d'objets (objets et collections) et des membres les composant (objet, collection, propriété, événement ou méthode). Le mot recherché peut être dans la liste des classes ou dans celle des membres.

© ENI Editions - All rigths reserved

4. Objets de la bibliothèque ; la classe d'objet sélectionnée dans la liste Résultat est encadrée.

5. Méthodes, propriétés, événements et constantes se rapportant à l'objet sélectionné ou encadré dans la liste de gauche. Le membre sélectionné dans la liste Résultat est encadré.

6. Détail de l'élément sélectionné.

Recherche dans l'explorateur d'objets

Pour effectuer une recherche dans l'explorateur d'objets, procédez de la manière suivante :

- Saisissez le mot recherché dans la 2ème liste déroulante.
- Cliquez sur l'icône  .
- Si la fenêtre des Résultats affiche plusieurs lignes, déplacez-vous sur celle qui vous intéresse (la partie basse de la fenêtre est alors réactualisée).

Le module de classe créé précédemment apparaît dans la liste des classes avec les méthodes et propriétés rattachées.



© ENI Editions - All rigths reserved

Propriétés et méthodes d'objets Access

Cette partie décrit les propriétés et méthodes des objets couramment utilisés dans Access. Les objets **Form** et **Report** seront décrits dans le chapitre **Personnalisation des formulaires et états**.

Pour obtenir la liste exhaustive des objets, des propriétés et méthodes, vous pouvez rechercher dans l'aide VBA Access ou utiliser l'explorateur d'objets.

L'objet Application

L'objet **Application** fait référence à l'application Microsoft Access active.

Propriétés

AutomationSecurity	Renvoie ou définit une constante **MsoAutomationSecurity** qui représente le mode de sécurité utilisé par Microsoft Access lorsqu'il ouvre des fichiers par programmation.
BrokenReference	Renvoie une valeur de type **Boolean** indiquant si la base de données active possède des références rompues vers des bases de données ou des bibliothèques de types.
Build	Renvoie une valeur de type **Long** qui représente le numéro de la copie de Microsoft Office Access 2007 actuellement installée.
CodeContextObject	Renvoie l'objet dans lequel une macro ou du code Visual Basic est en cours d'exécution.
CurrentObjectName	Retourne le nom de l'objet base de données actif (table, requête, formulaire, état, macro, module).
CurrentObjectType	Retourne le type de l'objet base de données actif.
FeatureInstall	Spécifie ou détermine la manière dont Microsoft Access gère les appels de méthodes et de propriétés qui nécessitent des fonctionnalités non installées.
IsCompiled	Retourne une valeur de type booléen qui indique si le projet Visual Basic se trouve dans un état compilé.
MenuBar	Détermine la barre de menus à utiliser pour la base de données.
Name	Retourne le nom de l'application ("Microsoft Access").
Parent	Permet de faire référence à l'objet père.
ProductCode	Détermine l'identificateur universel unique de Microsoft Access.
ShortcutMenuBar	Spécifie le menu contextuel qui apparaît lorsque l'utilisateur clique avec le bouton droit de la souris.
UserControl	Permet de déterminer si l'application Microsoft Access en cours a été lancée par l'utilisateur ou par une autre application au moyen de l'automation.
Version	Retourne le numéro de version de Microsoft Access.
Visible	Permet de masquer l'application.

Méthodes

AccessError	Retourne la chaîne descriptive associée à une erreur Microsoft Access ou à une erreur DAO.
AddToFavorites	Ajoute une adresse de lien hypertexte dans le dossier Documents favoris.
BuildCriteria	Permet de construire facilement les critères d'un filtre.

© ENI Editions - All rigths reserved

CloseCurrentDatabase	Ferme une base de données Access à partir d'une autre application utilisant la technologie Automation (application Excel par exemple). La base de données doit avoir été préalablement ouverte, par cette même application, au moyen de la méthode **OpenCurrentDatabase**.
ColumnHistory	Fournit l'historique des valeurs qui ont été stockées sans un champ Mémo.
CompactRepair	Compacte et répare la base de données (.mdb) ou le projet Microsoft Access (.adp) spécifiés. Renvoie un Boolean prenant la valeur True si l'opération a réussi.
ConvertAccessProject	Convertit la version du fichier Microsoft Access spécifié dans une autre version.
CreateAccessProject	Permet de créer un nouveau projet Microsoft Access (.adp) sur disque.
CreateAdditionalData	Permet de créer un objet AdditionalData pouvant être utilisé pour ajouter des tables et des requêtes supplémentaires dans la table parente qui est en cours d'exportation par la méthode ExportXML.
CreateControl	Permet de créer un contrôle dans le formulaire ouvert spécifié.
CreateForm	Permet de créer un formulaire et retourne un objet de type Form.
CreateGroupLevel	Permet de spécifier un champ ou une expression sur lequel (ou laquelle) seront regroupées ou triées les données d'un état.
Create NewWorkgroupFile	Crée un nouveau fichier de groupe de travail pour permettre aux utilisateurs d'accéder de manière sécurisée à une base de données.
CreateReport	Permet de créer un état et retourne un objet État.
CreateReportControl	Permet de créer un contrôle dans un état ouvert.
CurrentDb	Renvoie une variable objet faisant référence à la base de données actuellement ouverte dans Microsoft.
CurrentUser	Permet de retourner le nom de l'utilisateur actuel de la base de données.
DefaultWorkspace Clone	Crée un nouvel objet Workspace sans obliger l'utilisateur à se connecter à nouveau.
DeleteControl	Permet de supprimer un contrôle déterminé dans un formulaire.
DeleteReportControl	Permet de supprimer un contrôle déterminé dans un état.
Echo	Spécifie si Microsoft Access doit redessiner l'écran.
ExportXML	Permet d'exporter des données XML, des schémas et des informations de présentation à partir de Microsoft SQL Server 2000 Desktop Engine (MSDE 2000), Microsoft SQL Server, version 6.5 ou ultérieure, ou encore du moteur de base de données Microsoft Jet.
FollowHyperLink	Ouvre le document ou la page Web spécifié par une adresse de lien hypertexte.
GetOption	Retourne la valeur en cours d'une option de la boîte de dialogue **Options**.
HyperlinkPart	Renvoie les informations relatives aux données stockées sous la forme de lien hypertexte.
ImportNavigationPane	Charge une configuration enregistrée du volet de navigation à partir du disque.
ImportXML	Permet d'importer des données XML et/ou des informations de présentation dans Microsoft SQL Server 2000 Desktop Engine (MSDE 2000), Microsoft SQL Server, version 7.0 ou ultérieure, ou encore dans le moteur de base de données Microsoft Jet.

© ENI Editions - All rigths reserved

LoadCustomUI	Charge le code XML qui représente un ruban personnalisé.
LoadPicture	Charge une image dans un contrôle activeX.
NewAccessProject	Permet de créer et d'ouvrir un nouveau projet Microsoft Access (.adp) comme projet Access actif.
Nz	Transforme une valeur nulle en zéro, chaîne vide ou une autre valeur spécifiée.
NewCurrentDatabase	Crée une nouvelle base de données dans la fenêtre Microsoft Access.
OpenCurrentDatabase	Ouvre une base de données Access comme base de données courante à partir d'une autre application utilisant la technologie Automation (application Excel par exemple).
Quit	Permet de quitter Access.
RefreshDatabase Window	Permet de mettre à jour la fenêtre **Base de données** lorsqu'une table, une requête, un formulaire, un état, une macro ou un module ont été créés, supprimés ou renommés.
RefreshTitleBar	Actualise la barre de titre de Microsoft Access.
Run	Permet d'exécuter une procédure Function ou Sub spécifiée par Microsoft Access ou définie par l'utilisateur.
RunCommand	Permet d'exécuter une commande intégrée de menu ou de barre d'outils.
SetDefaultWorkgroup File	Définit le fichier spécifié comme fichier de groupe de travail.
SetHiddenAttribute	Active l'attribut caché d'un objet Access.
SetOption	Définit la valeur en cours d'une option dans la boîte de dialogue **Options**.
SysCmd	Permet d'afficher une jauge de progression ou un texte spécifié facultatif dans la barre d'état, de retourner des informations sur Microsoft Access et les fichiers associés, ou l'état d'un objet de base de données spécifié (objet ouvert, nouveau, ou modifié mais non enregistré).
TransformXML	Applique une feuille de style XSL (*eXtensible Style-sheet Language*) à un fichier de données XML et écrit le langage XML qui en résulte dans un fichier de données XML.

Exemple

Bouton de commande permettant de quitter Access en sauvegardant tous les objets.

```
Sub cmdQuitter_Click()
  Application.Quit acSaveYes
End Sub
```

Exemple

*Le code suivant permet de modifier les couleurs par défaut des feuilles de données (contenues dans l'onglet **Feuilles de données** de la boîte de dialogue **Options** d'Access).*

```
With Application
   ' Police bleue foncé
   .SetOption "Default Font Color", 4
   ' Arrière plan : blanc
   .SetOption "Default Background Color", 15
   ' Quadrillage bleu clair
   .SetOption "Default Gridlines Color", 12
End With
```

Fonctions applicables à un jeu d'enregistrements

© ENI Editions - All rigths reserved

Ces fonctions sont généralement utilisées dans les formulaires et états :

DAvg	Calcule la moyenne d'un ensemble de valeurs dans un jeu d'enregistrements.
DCount	Détermine le nombre d'enregistrements dans un jeu d'enregistrements.
DFirst	Renvoie le premier enregistrement d'un jeu d'enregistrements.
DLast	Renvoie le dernier enregistrement d'un jeu d'enregistrements.
DLookUp	Recherche la valeur d'un champ particulier qui fait partie d'un jeu d'enregistrements.
DMin	Détermine la valeur minimale d'un champ pour un jeu d'enregistrements.
DMax	Détermine la valeur maximale d'un champ pour un jeu d'enregistrements.
DStDev	Évalue l'écart type pour un échantillon de population pour un jeu d'enregistrements.
DstDevP	Estime l'écart-type d'une population dans un jeu d'enregistrements.
DSum	Calcule la somme d'un ensemble de valeurs dans un jeu d'enregistrements.
DVar	Évalue la variance pour un échantillon dans un jeu d'enregistrements.
DVarP	Calcule la variance d'une population dans un jeu d'enregistrements.
Eval	Évalue une expression (combinaison d'opérateurs, de noms de champ, de fonctions, de littérales et de constantes) et retourne le résultat sous la forme d'une chaîne de caractère ou d'une valeur numérique.

L'objet DoCmd

L'objet **DoCmd** permet d'exécuter à partir de Visual Basic la plupart des actions exécutables via les macros.

À chaque action macro correspond une méthode. Les arguments de l'action deviennent les arguments de la méthode. L'objet **DoCmd** ne dispose pas de propriétés.

» Les actions macro correspondant aux méthodes sont indiquées entre parenthèses.

Exécution d'actions

CancelEvent	`(AnnulerEvénement)` Annule un événement.
Quit	`(Quitter)` Quitte ACCESS.
RunCommand	`(ExécuterCommande)` Exécute une commande d'un menu ou d'une barre d'outils.
RunMacro	`(ExécuterMacro)` Exécute une macro.
RunSql	`(ExécuterSql)` Exécute un ordre SQL.

Données des formulaires et états

ApplyFilter	`(AppliquerFiltre)` Applique un filtre à une table, un formulaire, un état.
FindNext	`(TrouverSuivant)` Continue la recherche.
FindRecord	`(TrouverEnregistrement)` Recherche d'un enregistrement.

© ENI Editions - All rigths reserved

GotoControl	(AtteindreContrôle) Déplace le focus sur un contrôle.
GotoPage	(AtteindrePage) Déplace le focus sur le premier contrôle de la page indiquée du formulaire actif.
GotoRecord	(AtteindreEnregistrement) Atteint un enregistrement.
SearchForRecord	(TrouverEnregistrement) Permet de rechercher un enregistrement spécifique dans une table, une requête, un formulaire ou un état.

Import/Export de données

CopyDatabaseFile	(CopierFichierBaseDeDonnées) Copie la base de données connectée au projet actif à un fichier de base de données Microsoft SQL Server pour l'exporter.
OutputTo	(CopierVers) Permet d'exporter un objet Access (table, formulaire) dans une autre base Access ou un autre type de fichier (Excel, XML...).
SendObject	(EnvoyerObjet) Envoie un objet vers la messagerie.
TransferDatabase	(TransférerBase) Importe ou exporte une base de données.
TransferSpread Sheet	(TransférerFeuilleCalcul) Importe ou exporte une feuille de calcul.
TransferShare PointList	(TransférerListeSharePoint) Permet d'importer ou de lier des données à partir d'un site Microsoft Windows SharePoint Services 3.0.
TransferSQL Database	(TransférerBaseDeDonnéesSQL) Transfère l'intégralité de la base de données Microsoft SQL Server spécifiée vers une autre base de données SQL Server.
TransferText	(TransférerTexte) Importe ou exporte du texte au format ASCII.

Manipulation d'objets

Close	(Fermer) Ferme un objet (formulaire, état...).
CloseDatabase	(FermerBaseDonnées) Ferme la base de données ouverte dans l'interface Access. Si la base de données contient des objets qui n'ont pas été enregistrés lorsque vous utilisez la méthode **CloseDatabase**, des boîtes de dialogues identiques à celles qui s'affichent lorsque vous cliquez sur **Fermer la base de données** apparaissent.
CopyObject	(CopierObjet) Permet de copier un objet Access dans la même base ou dans une autre base.
DeleteObject	(SupprimerObjet) Supprime un objet.
Maximize	(Agrandir) Agrandit une fenêtre.
Minimize	(Réduire) Réduit une fenêtre en icône.
MoveSize	(DéplacerDimensionner) Déplace ou dimensionne une fenêtre.
OpenDataAccess Page	(OuvrirPageAccèsDonnées) Ouvre une page d'accès aux données.
OpenDiagram	(OuvrirSchéma) Ouvre un schéma de base de données.

© ENI Editions - All rigths reserved

OpenForm	(OuvrirFormulaire) Ouvre un formulaire.
OpenFunction	(OuvrirFonction) Ouvre une fonction utilisateur dans une base de données Microsoft SQL Server pour l'afficher dans Microsoft Access.
OpenModule	(OuvrirModule) Ouvre un module VBA.
OpenQuery	(OuvrirRequête) Ouvre une requête.
OpenReport	(OuvrirEtat) Ouvre un état.
OpenStored Procedure	(OuvrirProcédureStockée) Ouvre une procédure stockée en mode Feuilles de données, en mode Création ou en Aperçu avant impression.
OpenTable	(OuvrirTable) Ouvre une table.
OpenView	(OuvrirPageAccèsDonnées) Ouvre une page d'accès aux données.
PrintOut	(Imprimer) Imprime sur papier.
Rename	(Renommer) Renomme un objet.
RepaintObject	(RedessinerObjet) Redessine un objet (formulaire...).
Requery	(Actualiser) Met à jour les données de l'écran (relance la requête source de l'objet).
Restore	(Restaurer) Redonne à une fenêtre sa taille normale.
Save	(Enregistrer) Enregistre l'objet spécifié ou l'objet actif.
SelectObject	(SélectionnerObjet) Sélectionne un objet de base de données.
ShowAllRecords	(AfficherTousEnreg) Désactive un filtre et réaffiche tous les enregistrements.

Modification de l'interface Access

AddMenu	(AjouterMenu) Permet de créer une barre de menu ou un menu contextuel.
Hourglass	(Sablier) Affiche le pointeur de souris en sablier ou non.
LockNavigation Pane	(VerrouillerVoletNavigation) Permet d'empêcher les utilisateurs de supprimer des objets de base de données qui sont affichés dans le volet de navigation.
SetDisplayed Categories	(AccéderA) Précise les catégories qui sont affichées sous **Atteindre la catégorie** dans la barre de titre du volet de navigation.
SetMenuItem	(DéfinirElémentMenu) Permet de définir l'état (actif ou inactif, coché ou non-coché) d'éléments de menu personnalisé ou global pour la fenêtre active.
SetWarnings	(Avertissements) Active/désactive l'affichage des messages d'avertissement standards.
ShowToolbar	(AfficherBarreOutils) Affiche/masque une barre d'outils.

Divers

Beep	(Bip) Emet un bip sonore.

© ENI Editions - All rigths reserved

ClearMacroError	(EffacerMacroErreur) Supprime les informations relatives à une erreur stockées dans un objet **MacroErreur**.
DoMenuItem	(AfficherBarreOutils) Affiche la commande de menu appropriée pour Microsoft Access.
Echo	(Echo) Masque ou affiche les résultats d'une macro lors de son exécution.
PlainText	Supprime la mise en forme de texte enrichi d'une chaîne et renvoie une chaîne de texte sans mise en forme.
RunSavedImportExport	(ExécuterImportationExportationSauvegardée) Exécute une spécification d'importation ou d'exportation enregistrée, créée à partir de l'Assistant Importation ou de l'Assistant Exportation.
SetDefaultWorkgroupFile	Définit le fichier spécifié comme fichier de groupe de travail par défaut.
SetHiddenAttribute	Active l'attribut caché d'un objet Access.
SingleStep	(PasAPas) Suspend l'exécution de la macro active et ouvre la boîte de dialogue Pas à pas.

Actions de macros non prises en compte par l'objet DoCmd

BoiteMsg	Utilisez la fonction ou l'instruction MsgBox.
ExécuterApplication	Utilisez la fonction Shell.
ExécuterCode	Appelez directement la fonction dans Visual Basic.
EnvoiTouches	Utilisez l'instruction SendKeys.

» Les actions **ArrêtToutesMacros** et **ArrêtMacro** n'ont pas d'équivalent en Visual Basic.

L'objet Screen

L'objet **Screen** fait référence à l'objet actif qu'il s'agisse d'un formulaire, d'une page d'accès aux données, d'un état ou d'un contrôle.

L'objet **Screen** est surtout utilisé dans les procédures événementielles de ses différents objets (Control, Data Access Page, Form, Report). Cet objet ne comporte que des propriétés :

Propriétés

ActiveControl	Fournit une référence au contrôle actif.
ActiveDataAccessPage	Fournit une référence à la page d'accès aux données active.
ActiveDataSheet	Fournit une référence à la feuille de données active.
ActiveForm	Fournit une référence au formulaire actif.
ActiveReport	Fournit une référence à l'état actif.
Application	Permet d'accéder à l'objet Application et à ses propriétés.
MousePointer	Permet de spécifier le type de pointeur de souris.
Parent	Fournit une référence au père (objet conteneur) d'un contrôle, d'une section, d'un sous-formulaire...
PreviousControl	Fournit une référence au dernier contrôle actif avant le contrôle en cours.

Exemple

© ENI Editions - All rigths reserved

```
' Affiche sub cmd Screen-click le nom du contrôle
précédent
Dim ctl As Control
Set ctl = Screen.PreviousControl
MsgBox ctl.Name
' Transforme le pointeur de souris en sablier
Screen.MousePointer = 11
' Réaffiche le pointeur par défaut
Screen.MousePointer = 0

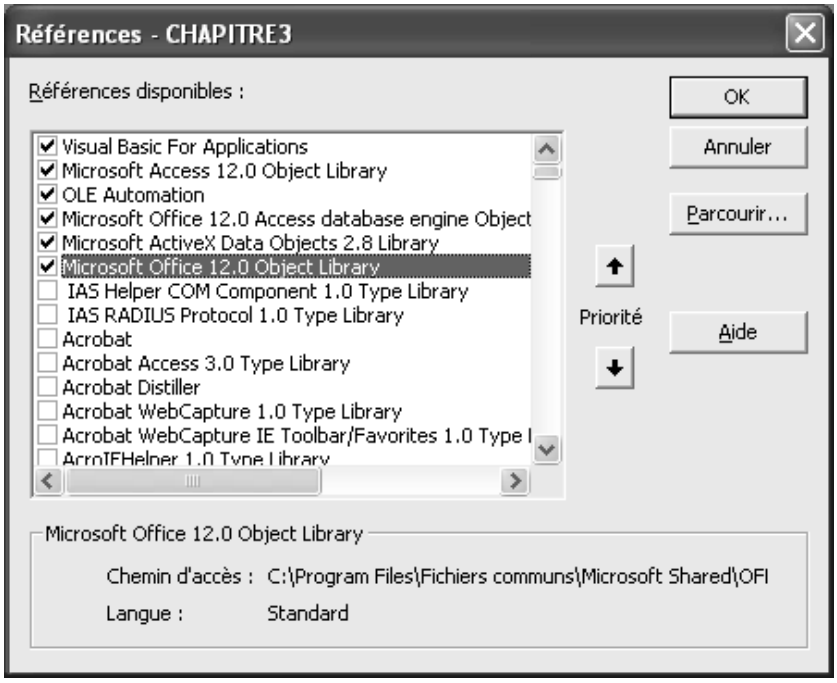
' Affiche le nom du formulaire actif
Dim frm As Form
Set frm = Screen.ActiveForm
MsgBox frm.Name
' Modifie le titre du formulaire
frm.Caption = "Formulaire de saisie des employés"
End Sub
```

La collection References

La collection **References** contient les objets **Reference** correspondant aux bibliothèques d'objets provenant d'autres applications.

Pour ajouter (ou supprimer) une référence à une bibliothèque d'objets, choisissez l'option **Références** du menu **Outils**.

La fenêtre suivante vous permet alors de sélectionner les références souhaitées :



Propriétés

Count	Détermine le nombre d'objets références.
Parent	Non utilisée.

Méthodes

AddFromFile	Crée une référence à une bibliothèque à partir d'un nom de fichier.
AddFromGuid	Crée une référence à une bibliothèque à partir de son GUID (identificateur global unique dans le registre Windows).
Item	Renvoie une référence en fonction de sa position dans la collection **Références**.

© ENI Editions - All rigths reserved

Remove	Supprime un objet référence dans la collection **References**.

Propriétés de l'objet Reference

BuiltIn	Indique si une référence par défaut qui est nécessaire pour le bon fonctionnement d'Access.
FullPath	Renvoie le chemin d'accès et le nom du fichier de la bibliothèque référencée.
Guid	Renvoie le GUID, identificateur de la bibliothèque dans le Registre Windows.
IsBroken	Indique si une référence valide est affectée à l'objet dans le Registre Windows.
Kind	Indique le type de référence.
Major	Indique le numéro de version principal de l'application à laquelle est associée la référence.
Mineur	Indique le numéro de version mineur de l'application.
Name	Indique le nom de la référence.

Exemple

```
Private Sub cmdReferences_Click()
Dim ref As Reference
Dim i As Integer

' Affiche le nombre total de références
MsgBox "Le nombre total de références est de : " &
References.Count
' Parcourt les références et affiche leur nom et chemin
d'accès
i = 0
For Each ref In Application.References
  MsgBox " Nom : " & ref.Name & " " & Chr(13) & Chr(13) _
  & " Chemin : " & ref.FullPath
Next ref

' Ajoute les bibliothèques d'objet Excel et Word
' Erreur si bibliothèque déjà référencée
On Error Resume Next
Application.References.AddFromFile "Excel.exe"
Application.References.AddFromFile "MsWord.olb"
On Error GoTo 0
' Affiche le nombre total de références
MsgBox "Le nombre total de références est de : " &
References.Count
End Sub
```

La collection Printers

La collection **Printers** contient des objets **Printer** représentant chacun une imprimante disponible sur le système en cours.

» L'objet **Printer** n'existait pas dans les versions précédentes d'Access.

Propriétés de l'objet Printer

BottomMargin	Marge basse d'une page imprimée.
ColorMode	Indique si l'imprimante doit imprimer en couleur (**acPRCMColor**) ou non (**acPRCMMonochrome**).
ColumnSpacing	Indique l'espacement vertical des sections Détail d'un formulaire.
Copies	Indique le nombre de copies à imprimer.
DataOnly	Permet de n'imprimer que les données d'une table ou d'une requête en mode Feuille de données.

© ENI Editions - All rigths reserved

DefaultSize	Renvoie la valeur **True** si la taille de la section de détail en mode Création est utilisée lors de l'impression d'un état ou d'un formulaire.
DeviceName	Renvoie une valeur de type **String** qui indique le nom du périphérique d'impression spécifié.
DriverName	Indique le nom du pilote utilisé par l'imprimante.
Duplex	Indique la manière dont l'imprimante gère l'impression recto verso.
ItemLayout	Indique si l'imprimante trace les colonnes horizontalement, puis verticalement, ou bien verticalement puis horizontalement.
ItemsAcross	Renvoie ou définit une valeur de type **Long** indiquant le nombre de colonnes à imprimer dans le cas d'états ou d'étiquettes sur plusieurs colonnes.
ItemSizeHeight	Renvoie ou définit une valeur de type **Long** indiquant la hauteur de la section détaillée d'un formulaire ou d'un état en twips.
ItemSizeWidth	Renvoie ou définit une valeur de type **Long** indiquant la largeur de la section détaillée d'un formulaire ou d'un état en twips.
LeftMargin	Marge gauche d'une page imprimée.
Orientation	Indique l'orientation de l'impression (**acPRORPortrait** ou **acPRORLandscape**).
PaperBin	Indique le bac que doit utiliser l'imprimante.
PaperSize	Indique la taille de papier à utiliser lors de l'impression.
Port	Indique le port auquel est connectée l'imprimante.
PrintQuality	Indique la résolution à utiliser par l'imprimante pour les travaux d'impression.
RightMargin	Marge droite d'une page imprimée.
RowSpacing	Renvoie ou définit une valeur de type **Long** qui représente l'espacement horizontal des sections détaillées en twips.
TopMargin	Marge haute d'une page imprimée.

» Les autres propriétés (**ColumnSpacing**, **RowSpacing**, **ItemSizeHeight**...) permettent d'effectuer le formatage de la section **Détail** d'un formulaire ou d'un état.

© ENI Editions - All rigths reserved

Objets d'accès aux données DAO et ADO

Les objets d'accès aux données sont très utiles : ils permettent, via VBA, de manipuler les données (recherches, mises à jour, ajouts d'enregistrements...) de bases de données **locales** ou **distantes**.

Dans les versions antérieures à Access 2000, le seul modèle d'accès aux données était le modèle DAO **: Data Access Objects**.

Avec Access 2000 est apparu le modèle ADO **: ActiveX Data Objects**.

Microsoft recommande d'utiliser le modèle ADO pour les raisons suivantes :

- meilleure prise en charge de SQL Server,

- meilleures performances en environnement client/serveur,

- code VBA plus court et plus simple.

Si vous développez **une nouvelle application**, vous pouvez utilisez le modèle ADO même si votre application accède à une base de données locale de type Access. Si vous souhaitez par la suite migrer vos données sous MSDE (*Microsoft Data Engine*) ou SQL Server, votre code fonctionnera de façon optimale.

Si vous faites évoluer **une application existante** utilisant le modèle DAO, il est possible de modifier votre code afin de transformer les objets DAO en objets ADO.

© ENI Editions - All rigths reserved

Le modèle d'accès aux données DAO

Présentation

La bibliothèque d'objets DAO (*Data Access Objects*) prend en charge deux environnements de bases de données différents, appelés espaces de travail :

- **les espaces de travail Microsoft Jet** : ils permettent d'accéder à des bases de données de type Access, à des serveurs de bases de données ODBC (*Open Database Connectivity*) et à des bases de données externes telles que dBASE, Microsoft Excel et Paradox accessibles via un pilote ISAM (*Indexed Sequential Access Method*),

- **les espaces de travail ODBCDirect** : ils permettent d'accéder à des serveurs de base de données ODBC de façon "directe", c'est-à-dire sans charger le moteur de base de données Microsoft Office 12. Cet espace de travail est par conséquent recommandé pour exécuter des requêtes, des procédures stockées, ou des fonctions spécifiques à ODBC sur un serveur distant de type SQL Server.

Pour pouvoir utiliser la bibliothèque d'objets DAO, il est nécessaire de cocher la référence **Microsoft DAO 3.6 Object Library** dans la liste des références aux bibliothèques d'objets (pour afficher cette liste, sélectionnez l'option **Références** du menu **Outils**).

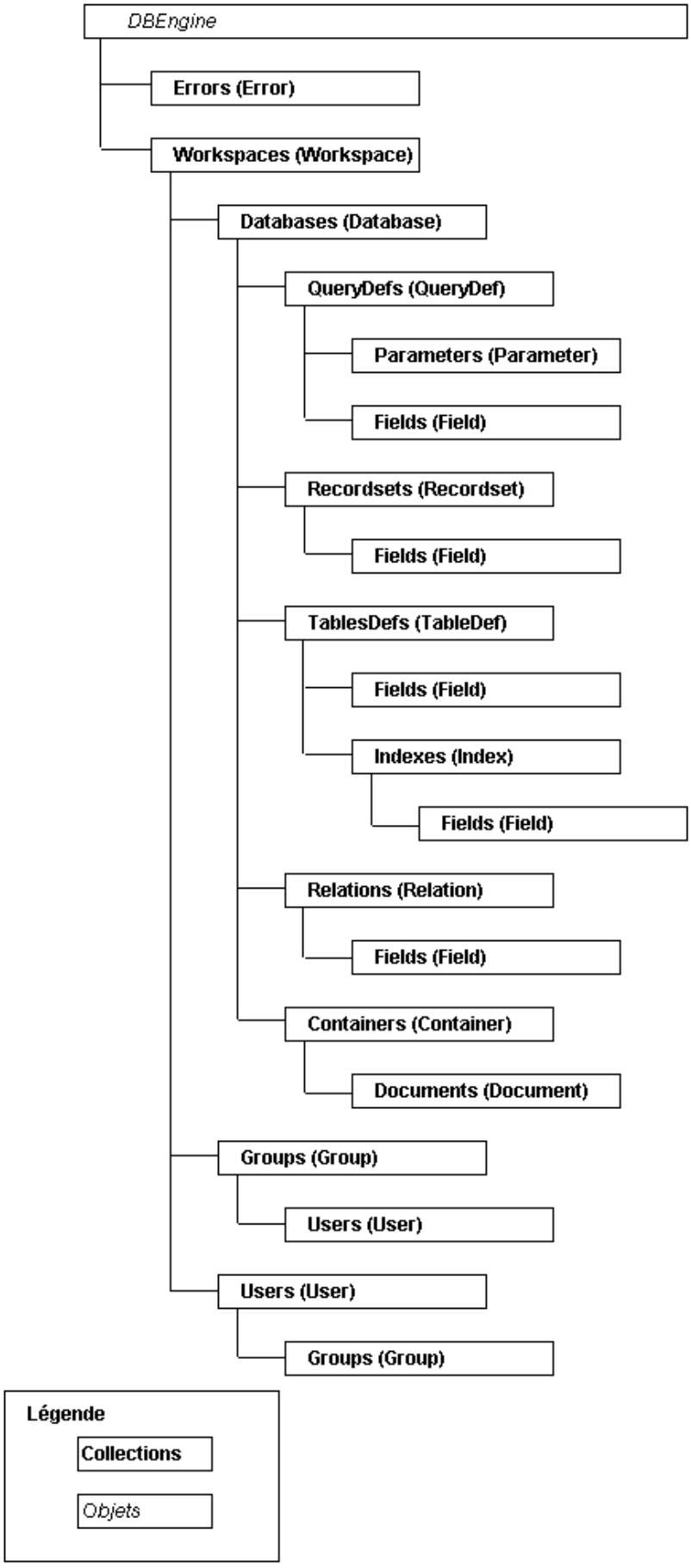
Si cette référence n'est pas disponible, vous devez sélectionner le fichier DAO360.dll à partir du bouton de commande **Parcourir**. Ce fichier est généralement situé dans le répertoire "C:\Program Files\Fichiers communs\Microsoft Shared\DAO".

Le fichier d'aide DAO360.chm, situé dans le répertoire C:\Program Files\Fichiers Communs\Microsoft Shared\Office12\1036 vous permet d'obtenir des informations détaillées sur l'utilisation du modèle DAO.

Hiérarchie des objets DAO

© ENI Editions - All rigths reserved

Modèle Objet Microsoft Data Access Objects (DAO) 3.6



© ENI Editions - All rigths reserved

Description des collections

Une collection existe pour chacun des types d'objet DAO, à l'exception de l'objet **DBEngine**.

Toute collection inclut tous les objets d'un type donné. Par exemple, la collection **DataBases** contient tous les objets **DataBase** ouverts.

Tout objet DAO possède une collection **Properties**, à l'exception des objets **Error**.

La plupart des objets DAO disposent de collections par défaut. Par exemple, la collection par défaut d'un objet Recordset est la collection Fields. Le code peut ainsi être simplifié.

Containers : Ensemble des objets **Container** d'une base de données.

Databases : Contient toutes les bases de données (objets **DataBase**) ouvertes dans un Workspace.

Documents : Ensemble des objets **Document** d'un Container.

Errors : Ensemble des erreurs (**Error**) du moteur d'accès aux données (**DBEngine**).

Fields : Ensemble des champs (**Field**) d'une table (**TableDef**), d'une requête (**QueryDef**), d'un index (**Index**), d'un jeu d'enregistrements (**Recordset**) ou d'une relation (**Relation**).

Groups : Ensemble des groupes d'utilisateurs (**Group**) d'un **Workspace** ou ensemble des groupes d'utilisateurs (**Group**) auxquels appartient un utilisateur (**User**).

Indexes : Ensemble des index (**Index**) d'une table (**TableDef**).

Parameters : Ensemble des paramètres (**Parameter**) d'une requête (**QueryDef**).

Properties : Ensemble des propriétés (**Property**) d'un objet.

QueryDefs : Contient toutes les définitions de requête (**QueryDef**) d'une base de données (**Database**).

Recordsets : Ensemble des jeux d'enregistrements ouverts (**Recordset**) d'une base de données (**Database**).

Relations : Ensemble des relations (**Relation**) d'une base de données (**Database**).

TableDefs : Contient toutes les définitions de table (**TableDef**) d'une base de données (**Database**).

Users : Ensemble des utilisateurs (**User**) d'un **Workspace** ou d'un groupe d'utilisateurs (**Group**).

Workspaces : Contient tous les objets **Workspace** actifs.

Propriétés

Count : Renvoie le nombre d'éléments dans la collection.

Item : Renvoie un membre spécifique d'une collection en fonction de sa position ou de sa clé.

» Toutes les collections peuvent utiliser ces propriétés.

Exemple

Liste des tables de la base de données courante.

```
Dim intI As Integer
For intI = 0 To CurrentDb.TableDefs.Count - 1
  MsgBox CurrentDb.TableDefs.Item(intI).Name
Next intI
```

© ENI Editions - All rigths reserved

Méthodes

Append : Ajoute un objet à la collection.

Delete : Supprime un objet de la collection.

Refresh : Réactualise le contenu de la collection.

Les collections **Errors**, **Databases**, **Containers**, **Documents**, **Parameters** et **Recordsets** ne peuvent utiliser les méthodes **Append** et **Delete**. Leur contenu est directement géré par le moteur de la base de données.

L'objet DBEngine

Il représente le moteur d'accès aux données Microsoft Jet. Il contient et contrôle l'ensemble des objets DAO.

Propriétés

DefaultType : Définit le type d'espace de travail, Microsoft Jet ou ODBCDirect.

DefaultUser et **DefaultPassword** : Nom d'utilisateur et mot de passe utilisés par le moteur de base de données Microsoft Jet quand un objet Workspace est créé sans nom d'utilisateur ni mot de passe spécifique.

InitPath : Définit ou renvoie des informations sur la clé du registre Windows qui contient des valeurs relatives au moteur de base de données Microsoft Jet.

SystemDB : Définit ou renvoie le chemin d'accès au fichier d'informations sur le groupe de travail en cours (MDW).

Version : Valeur qui indique la version de la base de données ou du moteur de base de données associé à l'objet.

Méthodes

CompactDatabase : Compacte une base de données fermée.

CreateWorkspace : Crée un nouvel objet **Workspace**.

Idle : Interrompt le traitement, permettant ainsi au moteur de base de données Microsoft Jet de terminer les tâches en attente.

SetOption : Permet de modifier temporairement les options du moteur d'accès aux données Microsoft Jet.

Collections

Workspaces, **Properties**, **Errors**

Exemple

Compactage de la base de données Devis en Devis2.

```
DBEngine.CompactDatabase "c:\devis\devis.accdb", _
                         "c:\devis\devis2.accdb"
```

Affichage de la version du moteur Microsoft Jet.

```
MsgBox "Numéro de version du moteur Microsoft Jet :" &
DBEngine.Version,_
        vbExclamation
```

Les objets Workspace

© ENI Editions - All rigths reserved

Un objet **Workspace** (espace de travail) permet de définir une session de travail pour un utilisateur.

Propriétés

Name : Renvoie le nom de l'objet Workspace.

Type : Renvoie ou définit le type de connexion de l'objet Workspace (constante dbUseJet si l'objet Workspace est connecté au moteur de bases de données Microsoft Jet, constante dbUseODBC si l'objet Workspace est connecté à une source de données ODBC).

UserName : Renvoie le nom du propriétaire de l'objet Workspace.

Autres propriétés

DefaultCursorDriver, IsolateODBCTrans, LoginTimeout

Méthodes

BeginTrans : Démarre une nouvelle transaction.

Close : Ferme le **Workspace**.

CommitTrans : Termine et valide la transaction en cours.

CreateDatabase : Crée un nouvel objet **Database**.

CreateGroup : Crée un nouveau groupe d'utilisateurs.

CreateUser : Crée un nouvel utilisateur.

OpenDatabase : Ouvre une base de donnés.

RollBack : Annule et termine une transaction en cours et y met fin.

Collections

Databases, **Users**, **Groups**, **Properties**

Exemple

Ouverture de la base de données Devis dans l'espace de travail actif.

```
Dim wrk As Workspace
Dim dbs As Database
Set wrk = DBEngine.Workspaces(0)
Set dbs = wrk.OpenDatabase("C:\devis\devis.accdb")
```

Les objets Database

Un objet Database représente une base de données ouverte.

Propriétés

CollatingOrder : Définit la méthode de comparaison des chaînes de caractères selon la langue.

Connect : Détermine les informations nécessaires à l'ouverture d'une base de données externe ou à l'attachement d'une table d'une base externe.

DesignMasterID : Détermine le GUID de la structure maître d'un jeu de réplicas.

Name : Renvoie le nom de la base.

© ENI Editions - All rigths reserved

RecordsAffected : Indique le nombre d'enregistrements affectés par la dernière méthode **Execute**.

Replicable : Indique si la base de données peut faire l'objet d'une réplication.

ReplicaID : Renvoie l'identifiant d'un réplica.

Transactions : Indique si la base de données de la connexion prend en compte les transactions (ODBCDirect).

Updatable : Indique si les modifications de l'objet Database sont autorisées.

Version : Identifie la version du format de la base de données.

V1xNullBehavior : Indique si les chaînes de longueur nulle sont converties en valeurs de type **Null**.

Méthodes

Close : Ferme l'objet **Database**.

CreateProperty : Crée un nouvel objet **Property**.

CreateQueryDef : Crée un nouvel objet **QueryDef**.

CreateRelation : Crée un nouvel objet **Relation**.

CreateTableDef : Crée un nouvel objet **TableDef**.

Execute : Exécute une requête action ou une instruction SQL.

MakeReplica : Crée un Réplica de la base de données.

NewPassword : Définit un nouveau mot de passe pour une base de données Microsoft Jet.

OpenRecordset : Crée un nouvel objet **Recordset**.

PopulatePartial : Synchronise les modifications apportées à un Réplica partiel par rapport à un Réplica intégral.

Synchronize : Synchronise deux Réplica.

Collections

TableDefs, **QueryDefs**, **Relations**, **Containers**, **Recordsets**, **Properties**

Exemple

Initialisation d'une référence sur la base de données courante.

```
Dim dbDevis as DAO.Database
Set dbDevis = CurrentDb()
```

Ouverture d'une base de données externe.

```
Dim dbsDevis as DAO.Database
Set dbsDevis = DBEngine.OpenDatabase _
("C:\Clients\Clients.accdb")
```

Création d'un objet RecordSet de type table en lecture seule sur la table Clients de la base de données courante.

```
Dim dbs As Database
Dim rstCli As DAO.Recordset
Set dbs = CurrentDb
```


© ENI Editions - All rigths reserved

```
Set rstCli = dbs.OpenRecordset("Clients", dbOpenTable, _
dbReadOnly)
```

Les objets Recordset

Un objet **Recordset** représente les enregistrements d'une table ou ceux issus d'une requête.

Il existe quatre types d'objets **Recordset** :

Recordset	Définition	Intérêt	Constante de déclaration
Table	Représente une table.	Permet d'utiliser les index.	dbOpenTable
Dynaset (feuille de réponse dynamique)	Représente le résultat d'une requête dont les enregistrements peuvent être mis à jour.	Permet de travailler sur les données provenant de plusieurs tables.	dbOpen Dynaset
Recordset	**Définition**	**Intérêt**	**Constante de déclaration**
ForwardOnly (En Avant seulement)	Idem Snapshot mais défilement des enregistrements uniquement vers l'avant.	Rapidité	dbOpen ForwardOnly
Snapshot (instantané)	Représente le résultat d'une requête en lecture seule.	Rapidité par rapport au Dynaset.	dbOpen Snapshot

» Les types d'objets Recordset concernés par chaque propriété ou méthode sont représentés par ***T*** (Table), ***D*** (Dynaset), ***S*** (Snapshot), ***F*** (Forward Only).

Propriétés

AbsolutePosition : Détermine la position relative d'un enregistrement dans un recordset (**DS**).

BOF : Indique un positionnement au début du **Recordset** (**DSTF**).

Bookmark : Renvoie ou définit un signet qui identifie de façon unique un enregistrement (**DST**).

Bookmarkable : Indique si le **Recordset** prend en charge les signets (**DST**).

CacheStart : Définit ou renvoie le signet du premier enregistrement à placer en mémoire cache (**D**).

DateCreated : Renvoie la date et l'heure de création de la table sous-jacente (**T**).

EditMode : Indique l'état de modification de l'enregistrement en cours (**DT**).

EOF : Indique un positionnement à la fin du **Recordset** (**DSTF**).

Filter : Permet de définir le filtre à appliquer (**DSF**).

Index : Définit l'index à utiliser (**T**).

LockEdits : Définit la condition de verrouillage (**DTS**).

LastModified : Renvoie un signet indiquant le dernier enregistrement modifié (**DT**).

LastUpdated : Renvoie la date et l'heure de dernière modification de la table d'origine sous-jacente (**T**).

Name : Renvoie le nom du **Recordset** (**DSTF**).

© ENI Editions - All rigths reserved

NoMatch : Indique si l'enregistrement recherché a été trouvé (**DST**).

PercentPosition : Retourne la position en cours d'un enregistrement en pourcentage du nombre total d'enregistrements (**DST**).

RecordCount : Indique le nombre d'enregistrements sollicités (**DSTF**).

Restartable : Indique si le **Recordset** prend en charge la méthode **Requery** qui exécute à nouveau la requête sous-jacente (**DSFT**).

Sort : Définit l'ordre de tri (**DS**).

Transactions : Indique si le **Recordset** supporte les transactions (**DTSF**).

Type : Indique le type de **Recordset** (**DSTF**).

Updatable : Indique les mises à jour autorisées (**DTSF**).

ValidationText : Spécifie le message d'erreur si la condition de validation est fausse (**DSTF**).

ValidationRule : Spécifie la condition de validation du **Recordset** (**DSTF**).

Méthodes

AddNew : Crée un nouvel enregistrement (**DT**).

CancelUpdate : Annule les mises à jour en attente (**DT**).

Clone : Crée une copie du **Recordset** (**DST**).

Close : Ferme le **Recordset** (**DSTF**).

CopyQueryDef : Crée une copie de l'objet **QueryDef** associé au **Recordset** (**DSF**).

Delete : Supprime l'enregistrement courant (**DT**).

Edit : Passe l'enregistrement courant en mode Edition (**DT**).

FindFirst : Localise le premier enregistrement répondant au critère et en fait l'enregistrement courant (**DS**).

FindLast : Localise le dernier enregistrement répondant au critère et en fait l'enregistrement courant (**DS**).

FindNext : Localise l'enregistrement suivant répondant au critère et en fait l'enregistrement courant (**DS**).

FindPrevious : Localise l'enregistrement précédent répondant au critère et en fait l'enregistrement courant (**DS**).

GetRows : Extrait plusieurs lignes d'un **Recordset** et les copie dans un tableau (**DSTF**).

MoveFirst : Localise le premier enregistrement et en fait l'enregistrement courant (**DST**).

MoveLast : Localise le dernier enregistrement et en fait l'enregistrement courant (**DST**).

MoveNext : Localise l'enregistrement suivant et en fait l'enregistrement courant (**DST**).

MovePrevious : Localise l'enregistrement précédent et en fait l'enregistrement courant (**DST**).

Move : Change la position de l'enregistrement courant (**DST**).

OpenRecordset : Crée un nouveau **Recordset** (**DST**).

© ENI Editions - All rigths reserved

Requery : Met à jour les enregistrements d'un **Recordset** en réexécutant la requête sous-jacente (**DSF**).

Seek : Localise par rapport à un **Index** sélectionné, le premier enregistrement répondant aux critères spécifiés et en fait l'enregistrement courant (**T**).

Update : Met à jour l'enregistrement (**DT**).

Exemple

Transformation en majuscules des caractères du champ Cli_Societe dans la table Clients.

```
Dim dbs As Database
Dim rstCli As DAO.Recordset

' Création du Recordset
Set dbs = CurrentDb
Set rstCli = dbs.OpenRecordset("Clients", dbOpenTable)
rstCli.MoveFirst
' Parcourt des enregistrements
With rstCli
  Do Until .EOF
    .Edit
    rstCli("Cli_Societe") = UCase(rstCli("Cli_Societe"))
    ' Mise à jour
    .Update
  ' Enregistrement suivant
    .MoveNext
  Loop
End With
rstCli.Close
```

Utilisation de la méthode Seek : recherche d'un client à partir de son code.

```
Dim dbs As Database
Dim rstCli As DAO.Recordset
Dim strCodeCli As String

' Création du Recordset
Set dbs = CurrentDb
Set rstCli = dbs.OpenRecordset("Clients", dbOpenTable)

With rstCli
  ' Saisie du code client
  strCodeCli = InputBox("Saisissez le code client")
  ' Index courant : clé primaire
  .Index = "PrimaryKey"
  ' Recherche
  .Seek "=", strcodecli
  ' Enregistrement non trouvé
  If .NoMatch Then
    MsgBox "Client non trouvé"
  Else
  ' Enregistrement trouvé
    MsgBox "Société : " & rstCli("Cli_Societe")
  End If
End With
rstCli.Close
```

Utilisation de la méthode Find : recherche des clients dont le nom de Société commence par une valeur donnée.

```
Dim dbs As DAO.Database
Dim rstCli As DAO.Recordset
Dim strSoc As String
Dim strCrit As String

' Création du Recordset
Set dbs = CurrentDb
Set rstCli = dbs.OpenRecordset("Clients", dbOpenSnapshot)

With rstCli
  ' Saisie du code client
  strSoc = InputBox("Saisissez les 1ère lettres de la
Société")
```

© ENI Editions - All rigths reserved

```
  .MoveLast
  ' Recherche
  strCrit = "Cli_Societe Like '" & strSoc & "*'"
  .FindFirst strCrit
  ' Enregistrement non trouvé
  If .NoMatch Then
    MsgBox "Client non trouvé"
  Else
  ' Si trouve : recherche des suivants
    Do While True
      MsgBox rstCli("Cli_Societe")
      .FindNext strCrit
      If .NoMatch Then Exit Do
    Loop
  End If
End With
```

Les objets Tabledef

Un objet TableDef représente la définition enregistrée d'une table ou d'une table attachée créée dans une base externe.

Propriétés

Attributes : Détermine les caractéristiques de la table décrite par l'objet **TableDef** (exclusivité, attachement, système...).

ConflictTable : Renvoie le nom de la table de conflits contenant les enregistrements entrés en conflit pendant une opération de synchronisation.

Connect : Retourne les informations sur la source de l'objet **TableDef**.

DateCreated : Indique la date et l'heure de création de la table sous-jacente.

KeepLocal : Détermine si la table ne doit pas faire partie d'une opération de réplication.

LastUpdated : Indique la date et l'heure de dernière modification de la table sous-jacente.

Name : Renvoie le nom de la table.

RecordCount : Indique le nombre d'enregistrements dans la table.

Replicable : Détermine si la table doit faire partie d'une opération de réplication.

ReplicaFilter : Définit les enregistrements d'une table devant faire partie d'une opération de réplication.

SourceTableName : Spécifie le nom de la table attachée.

Updatable : Indique si la modification de la structure est autorisée.

ValidationText : Spécifie le message d'erreur si la condition de validation est fausse.

ValidationRule : Spécifie la condition à respecter pour la validation de l'objet.

Méthodes

CreateField : Crée un nouveau champ (**Field**).

CreateIndex : Crée un nouvel **Index**.

CreateProperty : Crée un nouvel objet **Property**.

OpenRecordset : Crée un nouveau **Recordset**.

© ENI Editions - All rigths reserved

RefreshLink : Met à jour les informations à partir d'une table attachée.

Collections

Fields, **Indexes**, **Properties**

Exemple

Ajout d'un champ de type Mémo dans la table Clients.

```
Dim dbs As DAO.Database
Dim tdf As DAO.Tabl eDef
Dim fld As DAO.Field
Set dbs = CurrentDb
Set tdf = dbs.TableDefs("Clients")
Set fld = tdf.CreateField("Cli_Comment", dbMemo)
tdf.Fields.Append fld
```

Les objets Field et Index

- Field

Un objet **Field** représente un champ d'une table (**TableDef**), d'une relation (**Relation**), d'une requête (**QueryDef**), d'un jeu d'enregistrements (**Recordset**) ou d'un index (**Index**).

Propriétés

AllowZeroLength : Définit si une chaîne vide est autorisée comme valeur du champ.

Attributes : Indique certaines caractéristiques du champ.

CollatingOrder : Spécifie les caractéristiques de comparaison de chaînes de caractères et l'ordre de tri.

DataUpdatable : Indique si le champ peut être mis à jour.

DefaultValue : Définit la valeur par défaut du champ.

FieldSize : Renvoie la taille d'un champ **Memo** ou Binaire Long (**Objet OLE**).

ForeignName : Spécifie, dans une **Relation**, le nom du champ d'une table externe (table liée) qui correspond à un champ (clé primaire) de la table source (table de référence).

Name : Renvoie le nom du champ.

OrdinalPosition : Spécifie l'ordre d'affichage des champs.

Required : Définit si le champ doit contenir obligatoirement une valeur.

Size : Définit ou renvoie la taille du champ.

SourceField : Indique le nom du champ source des données.

SourceTable : Indique le nom de la table source des données.

Type : Indique le type de donnée du champ.

Value : Renvoie la valeur du champ.

ValidationText : Spécifie le message d'erreur renvoyé si la condition de validation du champ est fausse.

ValidationOnSet : Spécifie si la validation de valeur du champ est immédiate lors de sa modification.

© ENI Editions - All rigths reserved

ValidationRule : Spécifie la condition de validation du champ.

Méthodes

AppendChunk : Ajoute des données d'une expression chaîne à un champ de type **Memo** ou **Objet OLE**.

CreateProperty : Crée un nouvel objet **Property**.

GetChunk : Lit tout ou une partie d'un champ **Memo** ou **Objet OLE**.

Collections

Properties

Exemple

Affiche la liste de tous les champs (nom et type) de la table Clients dans la fenêtre Exécution.

```
Dim dbs As DAO.Database
Dim tdf As DAO.TableDef
Dim fld As DAO.Field
Set dbs = CurrentDb
Set tdf = dbs.TableDefs("Clients")
For Each fld In tdf.Fields
  Debug.Print fld.Name & " " & fld.Type
Next
```

- Index

Un objet **Index** représente un index associé à un objet **TableDef** ou **Recordset** de type table.

Propriétés

Clustered : Indique si l'index représente l'index de regroupement d'une table.

DistinctCount : Renvoie le nombre de valeurs uniques pour cet index dans la table associée.

Foreign : Indique si l'index est une clé externe de la table.

IgnoreNulls : Indique s'il y a création d'une entrée d'index pour les enregistrements dont les champs indexés contiennent la valeur **Null**.

Name : Renvoie le nom de l'index.

Primary : Indique si l'index est la clé primaire.

Required : Définit si tous les champs de l'index doivent contenir obligatoirement une valeur.

Unique : Indique si les valeurs de l'index sont uniques (pas de doublons).

Méthodes

CreateField : Crée un nouveau champ pour l'index.

CreateProperty : Crée une nouvelle propriété.

Collections

Fields, **Properties**

Exemple

© ENI Editions - All rigths reserved

Création de l'index Societe sur le champ Cli_Societe de la table Clients.

```
Dim dbs As DAO.Database
Dim tdf As DAO.TableDef
Dim fld As DAO.Field
Dim idx As DAO.Index

Set dbs = CurrentDb
Set tdf = dbs.TableDefs("Clients")
Set idx = tdf.CreateIndex("Societe")
Set fld = idx.CreateField("Cli_Societe")
idx.Unique = False
idx.Fields.Append fld
tdf.Indexes.Append idx
```

Les objets QueryDef et Parameter

- QueryDef

L'objet **QueryDef** représente la définition enregistrée d'une requête dans une base de données.

Propriétés

Connect : Donne les informations sur la base de données utilisée dans la requête.

DateCreated : Indique la date et l'heure de création de la requête.

KeepLocal : Détermine si la requête doit faire partie ou non d'une opération de réplication.

LastUpdated : Indique la date et l'heure de dernière mise à jour de la requête.

MaxRecords : Définit le nombre maximum d'enregistrements que la requête doit retourner.

Name : Renvoie le nom de la requête.

RecordsAffected : Indique le nombre d'enregistrements affectés par la dernière méthode **Execute**.

Replicable : Détermine si la requête doit faire partie d'une opération de réplication.

ReturnsRecords : Indique si la requête a retourné des enregistrements.

SQL : Définit ou renvoie l'instruction SQL associée à la requête.

Type : Définit le type de la requête (sélection, ajout...).

Updatable : Indique si les mises à jour sont autorisées à travers la requête.

Méthodes

Close : Ferme la requête.

CreateProperty : Crée une nouvelle propriété.

Execute : Lance une requête Action ou exécute une instruction SQL.

OpenRecordset : Crée un nouveau **Recordset**.

Collections

Fields, **Parameters**, **Properties**

Exemple

© ENI Editions - All rigths reserved

Création d'une nouvelle requête basée sur une instruction SQL.

```
Dim dbs As Database
Dim qdf As QueryDef

Set dbs = CurrentDb
Set qdf = dbs.CreateQueryDef("Liste des " _
  & "employés triés par date d'embauche")
qdf.SQL = "Select * from Employes " _
    & "Order by [Date Embauche]"
```

- Parameter

Un objet **Parameter** représente un paramètre d'un objet **QueryDef** paramétré.

Propriétés

Name : Définit ou renvoie le nom du paramètre.

Type : Définit ou renvoie le type de données du paramètre.

Value : Définit ou renvoie la valeur du paramètre.

Collections

Properties

Exemple

Création d'une requête paramétrée et affichage du nombre de paramètres.

```
Dim dbs As Database
Dim qdf As QueryDef
Dim prm As Parameter

Set dbs = CurrentDb
Set qdf = dbs.CreateQueryDef("Liste des Employés" _
  & " embauchés à une période donnée")
qdf.SQL = "Select * from Employes Where " _
  & "[Date embauche] >= [Entrez la date de début] " _
  & " And [Date embauche] <= [Entrez la date de fin] "
MsgBox qdf.Parameters.Count
```

Les objets Relation

Un objet **Relation** représente une relation entre les champs de deux objets **TableDef**.

Propriétés

Attributes : Indique certaines caractéristiques de la relation (un à plusieurs, intégrité...).

ForeignTable : Définit ou renvoie le nom de la table externe.

Name : Définit ou renvoie le nom de la relation.

PartialReplica : Définit si la relation doit être prise en compte pour le remplissage d'un **Réplica** partiel à partir d'un **Réplica** intégral.

Table : Définit ou renvoie le nom de la **TableDef** ou du **QueryDef** source.

Méthodes

CreateField : Crée un nouvel objet **Field**.

© ENI Editions - All rigths reserved

Collections

Fields, **Properties**

Exemple

Enregistrement dans une table des différentes relations et de leurs propriétés. La table Relations contient 3 champs texte (Rel_Source, Rel_Externe, Rel_Typerel et 3 champs booléen (Rel_Integrite, Rel_Maj_Cascade, Rel_Sup_Cascade).

```
Dim dbs As DAO.Database
Dim rst As DAO.Recordset
Dim rel As Relation

' Création du Recordset
' et suppression des enregistrements existants
Set dbs = CurrentDb
Set rst = dbs.OpenRecordset("Relations", dbOpenTable)
Do While Not rst.EOF
  rst.Delete
  rst.MoveNext
Loop

' Stockage des différentes relations du modèle
' relationnel dans la table Relations
For Each rel In dbs.Relations
  rst.AddNew
  rst("REL_SOURCE") = rel.Table
  rst("REL_EXTERNE") = rel.ForeignTable
  If rel.Attributes And dbRelationUnique Then
    rst("REL_TYPEREL") = "1 à 1"
  Else
    rst("REL_TYPEREL") = "1 à n"
  End If
  rst("REL_INTEGRITE") = IIf(rel.Attributes And _
    dbRelationDontEnforce, False, True)
  rst("REL_MAJ_CASCADE") = IIf(rel.Attributes And _
    dbRelationUpdateCascade, True, False)
  rst("REL_SUP_CASCADE") = IIf(rel.Attributes And _
    dbRelationDeleteCascade, True, False)
  rst.Update
Next
rst.Close
```

Visualisation dans Access de la table RELATIONS mise à jour :

RELATIONS

REL_IDREL	REL_SOURCE	REL_EXTERNE	REL_TYPEREL	REL_INTEGRITE	REL_MAJ_CASCADE	REL_SUP_CASCADE
29	COMMANDES	DETAIL COMMANDES	1 à n	☑	☐	☑
30	CATEGORIES	PRODUITS	1 à n	☑	☐	☐
31	PRODUITS	DETAIL COMMANDES	1 à n	☑	☐	☐
32	MESSAGERS	COMMANDES	1 à n	☑	☐	☐
33	FOURNISSEURS	PRODUITS	1 à n	☑	☐	☐
34	CLIENTS	COMMANDES	1 à n	☑	☐	☐
35	EMPLOYES	COMMANDES	1 à n	☑	☐	☐
(Nouv.)				☐	☐	☐

Enr : 1 sur 7 | Aucun filtre | Rechercher

Les objets Container et Document

- Container

Un objet **Container** représente l'ensemble des informations portant sur une base de données et sur chaque type d'objets qu'elle contient.

Propriétés

AllPermissions : Retourne toutes les autorisations qui s'appliquent à la propriété **UserName** de l'objet Container en cours.

© ENI Editions - All rigths reserved

Inherit : Indique si les nouveaux objets **Document** hériteront, ou non, d'une valeur pour la propriété **Permissions**.

Name : Indique le nom du container.

Owner : Indique le propriétaire du container.

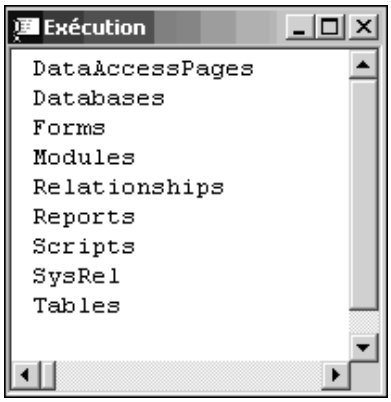
Permissions : Définit les permissions d'un utilisateur ou d'un groupe d'utilisateurs identifiés par la propriété **UserName** de l'objet Container.

UserName : Définit le nom de l'utilisateur.

Exemple

Affichage de la liste des noms de tous les objets Container de la base dans la fenêtre Exécution.

```
Dim dbs As DAO.Database
Set dbs = CurrentDb
Dim ctn As Container
For Each ctn In dbs.Containers
  Debug.Print ctn.Name
Next
```



- Document

Un objet **Document** représente l'ensemble des informations portant sur un type d'objet (base de données, table, requête ou relation enregistrée).

Propriétés

AllPermissions (3.0) : Retourne toutes les autorisations qui s'appliquent à la propriété **UserName** en cours de l'objet **Document**.

Container : Retourne le nom de l'objet **Container** auquel appartient le **Document**.

DateCreated : Retourne la date et l'heure de création de la table sous-jacente.

KeepLocal : Détermine si le document doit ou non faire partie d'une opération de réplication.

LastUpdated : Renvoie la date et l'heure de dernière modification de la table sous-jacente.

Name : Renvoie le nom du Document.

Owner : Renvoie le nom du propriétaire du Document.

Permissions : Définit les permissions de l'utilisateur ou du groupe d'utilisateurs identifiés par la propriété **UserName** de l'objet **Document**.

© ENI Editions - All rigths reserved

Replicable : Détermine si le document doit faire partie d'une opération de réplication.

UserName : Définit le nom de l'utilisateur.

Collections

Properties

Exemple

Affichage de la liste de tous les formulaires dans la fenêtre Exécution.

```
Dim dbs As DAO.Database
Dim ctn As Container
Dim doc As Document

Set dbs = CurrentDb
For Each ctn In dbs.Containers
  If ctn.Name = "Forms" Then
    For Each doc In ctn.Documents
      Debug.Print doc.Name
    Next
  End If
Next
```

Les objets Group et User

- User

Un objet **User** représente un compte utilisateur avec ses autorisations.

Propriétés

Name : Définit ou renvoie le nom du compte utilisateur.

PID : Renvoie le numéro d'identification personnel du compte utilisateur.

PassWord : Renvoie le mot de passe du compte utilisateur.

Méthodes

CreateGroup : Crée un nouvel objet **Group**.

NewpassWord : Définit un nouveau mot de passe pour l'utilisateur.

Collections

Groups, **Properties**

- Group

Un objet **Group** représente un groupe d'utilisateurs possédant les mêmes autorisations.

Propriétés

Name : Définie ou renvoie le nom du groupe d'utilisateurs.

PID : Définit le numéro personnel du compte de groupe.

Méthodes

CreateUser : Crée un nouvel objet User.

© ENI Editions - All rigths reserved

Collections

Users, **Properties**

Exemple

Pour chaque groupe, affichage dans la fenêtre Exécution des noms de groupes et d'utilisateurs du groupe.

```
Dim wrk As Workspace
Dim usr As User
Dim grp As Group
Set wrk = DBEngine.Workspaces(0)
For Each grp In wrk.Groups
  Debug.Print grp.Name
  With grp
    For Each usr In .Users
      Debug.Print usr.Name
    Next
  End With
Next
```

L'objet Error

Un objet **Error** représente le détail d'une erreur d'accès aux données.

Propriétés

Description : Renvoie le texte de l'erreur.

HelpContext : Fournit un identificateur de contexte d'aide, sous la forme d'une variable de chaîne, correspondant à une rubrique d'un fichier d'aide Microsoft Windows.

HelpFile : Indique le chemin d'accès complet au fichier d'aide considéré.

Number : Retourne le numéro de l'erreur.

Source : Fournit le nom de l'objet ou de l'application qui a généré l'erreur.

Exemple

Déclenchement intentionnel d'une erreur et affichage de toutes les propriétés de l'objet Error.

```
Private Sub cmdErreur_Click()

On Error GoTo Gest_Erreur
   ' Déclenche intentionnellement une erreur
   DBEngine.Workspaces(0).OpenDatabase _
  ("C:\Devis\devis1.accdb")
  Exit Sub

Gest_Erreur:
  Dim strError As String
  Dim errLoop As Variant

' Parcourt les éléments de la collection Errors
' Affiche un message avec la description de l'erreur
For Each errLoop In Errors
  With errLoop
    strError = _
      "Erreur #" & .Number & vbCr
    strError = strError & _
      " " & .Description & vbCr
    strError = strError & _
      "(Source: " & .Source & ")" & vbCr
    strError = strError & _
      "Appuyez sur F1 pour afficher " & _
      "la rubrique " & .HelpContext & vbCr
    strError = strError & _
```


© ENI Editions - All rigths reserved

```
      "dans le fichier " & .HelpFile & "."
  End With
  MsgBox strError
Next

End Sub
```

Le message affiché est le suivant :



Les objets Property

Un objet **Property** représente une propriété d'un objet.

Propriétés

Inherited : Indique si la propriété est héritée.

Name : Définit ou renvoie le nom de la propriété.

Type : Définit ou renvoie le type de la propriété.

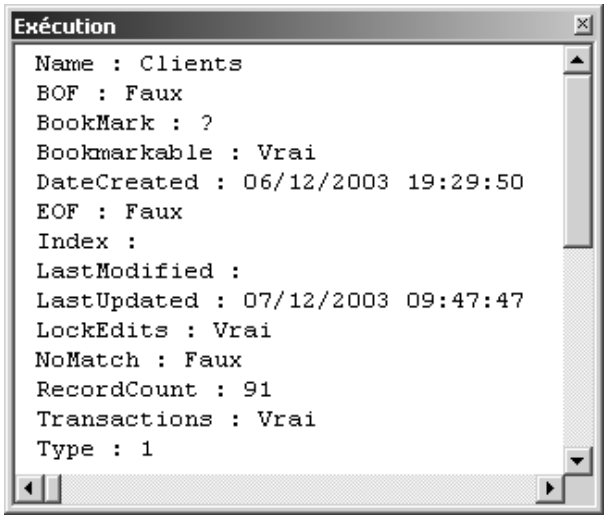
Value : Définit ou renvoie la valeur de la propriété.

Exemple

Affichage du nom et de la valeur de chaque propriété d'un objet Recordset dans la fenêtre Exécution.

```
Dim dbs As DAO.Database
Dim rst As DAO.Recordset
Dim prp As DAO.Property
Set dbs = CurrentDb
Set rst = dbs.OpenRecordset("Clients")
' Parcourt les propriétés
For Each prp In rst.Properties
  On Error Resume Next
  Debug.Print prp.Name & ":" & prp.Value
Next
```

© ENI Editions - All rigths reserved



© ENI Editions - All rigths reserved

Le modèle d'accès aux données ADO

Présentation

La bibliothèque d'objets ADO **(ActiveX Data Objects)** vous permet d'écrire une application qui accède à des données situées sur un serveur de bases de données et de les manipuler par le biais d'un fournisseur OLE DB.

ADO a l'avantage d'être facile d'emploi, performant, et d'utiliser peu de mémoire et d'espace disque. ADO offre les fonctionnalités de base permettant de créer des applications client/serveur et des applications Web.

Pour pouvoir utiliser la bibliothèque d'objets ADO, il est nécessaire de cocher la référence **Microsoft ActiveX Data Objects 2.x Library** dans la liste des références aux bibliothèques d'objets. Si cette référence n'est pas disponible, vous devez sélectionner le fichier MSADO15.dll à partir du bouton de commande **Parcourir**. Ce fichier est généralement situé dans le répertoire "C:\Program Files\Fichiers communs\System\ADO".

Vous pouvez également utiliser les extensions ADO en sélectionnant la référence **Microsoft ADO Ext 2.x for DLL and Security** (fichier MSADOX.DLL). Ces extensions vous permettent d'accéder à des objets complémentaires (catalog, users, views...) au modèle ADO.

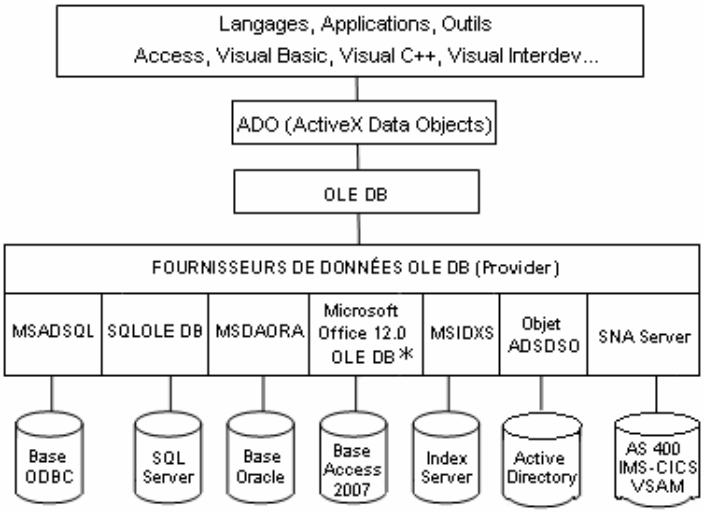
Le fichier d'aide ADO210.chm vous permet d'obtenir des informations détaillées sur l'utilisation des modèles ADO.

ADO et OLE DB

OLE DB est une technologie permettant un accès uniforme aux données stockées dans diverses sources d'informations : bases de données relationnelles ou non relationnelles, messageries, systèmes de fichiers… Quasiment toutes les données de l'entreprise sont accessibles par OLE DB.

ADO (*ActiveX Data Objects*) est l'interface de programmation utilisée pour accéder aux données de tous les fournisseurs de données OLE DB.

Architecture technologique

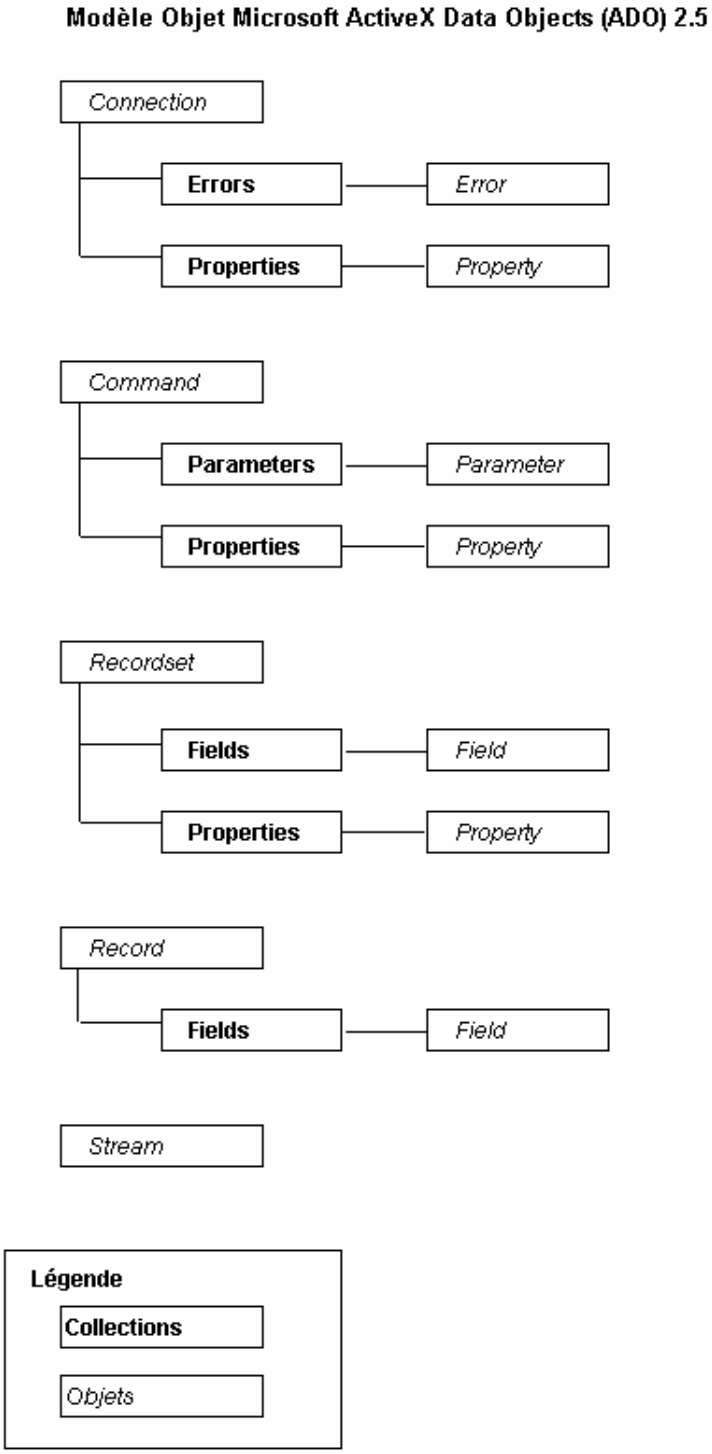


* Microsoft Office 12.0 Access Database Engine OLE DB Provider remplace Microsoft Jet 4.0 OLE DB Provider.

» Les fournisseurs de données ou **Providers** sont des pilotes permettant de communiquer avec les sources de données.

Hiérarchie des objets ADO

© ENI Editions - All rigths reserved



Modèle de base de programmation ADO

ADO vous permet d'exécuter la séquence d'actions suivante :

- établir une connexion à une base de données OLE DB ou ODBC au moyen de l'objet Connection.

- créer une commande permettant d'envoyer une requête SQL au serveur en utilisant l'objet Command,

- exécuter la commande. Si la commande renvoie des lignes, les stocker dans un objet Recordset,

- exploiter les données de l'objet Recordset : trier, filtrer, modifier, supprimer… les enregistrements,

© ENI Editions - All rigths reserved

- mettre à jour les données : valider les modifications éventuelles de l'objet Recordset, incorporer les mises à jour dans une transaction,

- si une transaction a été utilisée : accepter ou rejeter les mises à jour apportées au cours de cette dernière.

Exemple

```
Dim cnn As New ADODB.Connection
Dim cmd As New ADODB.Command
Dim rs As New ADODB.Recordset

' Etape 1 : établir une connexion à une source de
données
' SQL Server au moyen de la technologie OLEDB
cnn.Open "Provider=SQLOLEDB.1;Data Source=SRV;" _
    & "Initial Catalog=Devis;" _
    & "Integrated Security=SSPI;Persist Security Info=False;"

' Etape 2 : créer une commande
Set cmd.ActiveConnection = cnn
cmd.CommandText = "SELECT * FROM Clients"

' Etape 3 : exécuter la commande
rs.CursorLocation = adUseClient
rs.Open cmd, , adOpenStatic, adLockBatchOptimistic

' Etape 4 : manipuler les données
rs.Filter = "CLI_VILLE LIKE 'Nantes'"
Do While Not rs.EOF
  rs("CLI_PAYS") = "F"
  rs.MoveNext
Loop

' Etape 5 : mettre à jour les données modifiées
cnn.BeginTrans
rs.UpdateBatch

' Etape 6 : terminer la mise à jour de la base SQL
If MsgBox("Confirmez-vous les mises à jour ?", _
  vbYesNo) = vbYes Then
  cnn.CommitTrans
Else
  cnn.RollbackTrans
End If
```

Description des objets ADO

Connection : Représente une connexion avec une source de données.

Command : Définit une commande spécifique à exécuter sur une source de données.

Recordset : Représente le jeu d'enregistrements complet d'une table de bases de données ou les résultats d'une commande (ex : résultats d'une requête SQL).

Record : Fonctionne essentiellement comme un objet Recordset d'une seule ligne. La source des données dans un objet Record peut être une commande qui renvoie une ligne de données à partir du fournisseur. L'utilisation d'objets Record à la place d'objets Recordset pour recevoir les résultats d'une requête renvoyant une seule ligne de données élimine la charge de traitement liée à l'instanciation de l'objet Recordset, lequel est plus complexe.

Stream : Fournit le moyen de lire, écrire et gérer un flux d'octets constitué de texte ou de données binaires.

Les collections ADO

Parameters : Collection de tous les objets **Parameter** d'un objet **Command**. L'objet **Parameter** représente un paramètre ou argument associé à un objet **Command** basé sur une requête paramétrée ou une procédure stockée paramétrée.

Properties : Collection de tous les objets **Property** pour un objet donné (**Connection**, **Command**, **Recordset**). L'objet **Property** représente une caractéristique dynamique d'un objet ADO défini par l'utilisateur.

© ENI Editions - All rigths reserved

Errors : Collection de tous les objets **Error** créés en réponse à un échec unique lié au fournisseur OLE DB. L'objet **Error** contient des informations relatives à une erreur d'accès aux données.

Fields : Collection de tous les objets **Field** correspond à une colonne (ou champ) d'un objet Recordset.

Les objets Connection

Méthodes

BeginTrans : Commence une nouvelle transaction.

Cancel : Interrompt et annule l'exécution d'une requête lancée en mode asynchrone à l'aide des méthodes **Execute** ou **Open**.

Close : Ferme une connexion ouverte ainsi que tous les objets dépendant de celle-ci.

CommitTrans : Enregistre les modifications effectuées sur les données et arrête la transaction en cours.

Execute : Exécute la requête, l'instruction SQL, la procédure stockée ou le texte propre au fournisseur OLE DB.

Open : Ouvre une connexion à une source de données, afin de pouvoir exécuter des commandes.

OpenSchema : Obtient des informations, en provenance du fournisseur OLE DB, relatives au schéma de la base de données.

RollbackTrans : Annule les modifications effectuées sur les données durant la transaction en cours et arrête celle-ci.

Propriétés

Attributes : Définit ou renvoie une ou plusieurs caractéristiques d'un objet **Connection**. La valeur par défaut est zéro.

CommandTimeout : Indique le délai à respecter, en secondes, lors d'une tentative d'exécution d'une commande avant d'interrompre cette tentative et de générer un message d'erreur. La valeur par défaut est 30.

ConnectionString : Définit ou renvoie les informations utilisées pour établir une connexion avec la source de données.

ConnectionTimeout : Indique le délai à respecter, en secondes, lors d'une tentative de connexion avant d'y mettre un terme et de générer un message d'erreur. La valeur par défaut est 15.

CursorLocation : Définit ou renvoie le type de curseur à utiliser par défaut pour cette connexion (curseur côté client ou curseur côté serveur).

DefaultDatabase : Indique la base de données par défaut d'un objet **Connection**.

IsolationLevel : Indique le niveau d'isolation d'un objet **Connection**.

Mode : Indique les autorisations de modification de données d'un objet **Connection**.

Provider : Indique le nom du fournisseur OLE DB de l'objet **Connection**.

State : Indique si l'objet **Connection** est ouvert ou fermé. En lecture seule.

Version : Indique le numéro de version d'ADO. En lecture seule.

Exemple

© ENI Editions - All rigths reserved

Création d'une référence sur la base de données courante.

```
Dim cncDevis as ADODB.Connection
Set cncDevis = CurrentProject.Connection
```

Ouverture d'une base de données externe.

```
Dim cncDevis as ADODB.Connection
Set cncDevis = New ADODB.Connection
cncDevis.Open "Provider=Microsoft.ACE.OLEDB.12.0;" _
      & "Data Source = C:\Devis\Devis.accdb"
```

Utilisation des méthodes CommitTrans et RollbackTrans pour confirmer ou annuler la mise à jour d'enregistrements.

```
Dim cnc As ADODB.Connection
Dim strSQL As String
Dim strMessage As String

' Ouvre la connexion.
Set cnc = CurrentProject.Connection
' Début de la transaction
cnc.BeginTrans

' Requêtes SQL de mises à jour
strSQL = "UPDATE CLIENTS SET CLI_PAYS = 'France'" _
    & "WHERE CLI_VILLE = 'Paris'"
cnc.Execute strSQL
strSQL = "UPDATE CLIENTS SET CLI_PAYS = 'Espagne'" _
    & "WHERE CLI_VILLE = 'Madrid'"
cnc.Execute strSQL

' Demande à l'utilisateur s'il veut valider les mises à jour
If MsgBox("Confirmez-vous la mise à jour des pays ?", vbYesNo) _
  = vbYes Then
  ' Enregistre les modifications
  cnc.CommitTrans
Else

  ' Annule les modifications
  cnc.RollbackTrans
End If
```

Les objets Command

Méthodes

Cancel : Interrompt et annule l'exécution d'une requête lancée en mode asynchrone à l'aide de la méthode **Execute**.

CreateParameter : Crée un nouvel objet **Parameter** et spécifie ses propriétés.

Execute : Exécute la requête, l'instruction SQL ou la procédure stockée spécifiée par la propriété **CommandText**.

Propriétés

ActiveConnection : Indique l'objet **Connection** auquel est associée la commande.

CommandText : Contient le texte de la commande à envoyer au fournisseur OLE DB.

CommandTimeout : Indique le délai (en secondes) à respecter, lors d'une tentative d'exécution de la commande avant d'interrompre cette tentative et de générer un message d'erreur. La valeur par défaut est 30.

CommandType : Indique le type de l'objet **Command**.

Name : Indique le nom de l'objet **Command**.

© ENI Editions - All rigths reserved

Prepared : Indique s'il faut enregistrer une version compilée de la commande sur le serveur avant exécution.

State : Indique si l'objet **Command** est ouvert ou fermé. En lecture seule.

Exemple

Utilisation de l'objet Command pour créer un jeu d'enregistrements.

```
Dim cnc As New ADODB.Connection
Dim cmd As New ADODB.Command
Dim rst As New ADODB.Recordset

' Ouvre la connexion
cnc.Open "Provider=Microsoft.Jet.OLEDB.4.0.;" _
  & "Data Source = C:\Devis\Devis.mdb"
' Exécute la commande Select
Set cmd.ActiveConnection = cnc
cmd.CommandType = adCmdText
cmd.CommandText = "SELECT * FROM CLIENTS WHERE " _
    & " CLI_VILLE = 'Paris'"

Set rst = cmd.Execute
' Affiche le nom de la Société
' dans la fenêtre Exécution
Do While Not rst.EOF
  Debug.Print rst("CLI_SOCIETE")
  rst.MoveNext
Loop
rst.Close
cnc.Close
```

Les objets Recordset

Méthodes

AddNew : Crée un nouvel enregistrement pour un objet **Recordset** pouvant être mis à jour.

Cancel : Annule l'exécution d'une opération **Open** asynchrone en attente.

CancelBatch : Annule une mise à jour par lots en attente.

CancelUpdate : Annule toutes les modifications apportées à l'enregistrement en cours ou à un nouvel enregistrement avant d'appeler la méthode **Update**.

Clone : Crée un objet **Recordset** en double à partir d'un objet **Recordset** existant.

Close : Ferme l'objet **Recordset** ouvert ainsi que tous les objets dépendant de celui-ci.

CompareBookmarks : Compare deux signets et renvoie une valeur indiquant leurs positions relatives.

Delete : Supprime l'enregistrement en cours ou un groupe d'enregistrements.

Find : Recherche dans un objet **Recordset** l'enregistrement correspondant à des critères spécifiques.

GetRows : Extrait plusieurs enregistrements d'un objet **Recordset** et les place dans un tableau.

GetString : Renvoie un objet **Recordset** sous forme de chaîne.

Move : Déplace la position de l'enregistrement en cours dans un objet **Recordset**.

MoveFirst : Déplace la position de l'enregistrement en cours jusqu'au premier enregistrement de l'objet **Recordset**.

MoveLast : Déplace la position de l'objet en cours jusqu'au dernier enregistrement dans l'objet **Recordset**.

© ENI Editions - All rigths reserved

MoveNext : Déplace la position de l'enregistrement en cours jusqu'à l'enregistrement suivant dans l'objet **Recordset**.

MovePrevious : Déplace la position de l'enregistrement en cours jusqu'à l'enregistrement précédent dans l'objet **Recordset**.

NextRecordset : Renvoie le jeu d'enregistrements suivant dans le cas d'une commande comportant plusieurs requêtes d'extraction.

Open : Ouvre un curseur qui représente les enregistrements d'une table ou les résultats d'une requête.

Requery : Met à jour les données d'un objet **Recordset** en réexécutant la requête sur laquelle l'objet est basé.

Resync : Actualise les données de l'objet **Recordset** en cours à partir de la base de données sous-jacente.

Save : Enregistre l'objet **Recordset** dans un fichier.

Seek : Recherche dans l'index d'un objet **Recordset** un enregistrement correspondant à une valeur, et remplace l'enregistrement en cours par celui trouvé. (Il s'agit d'une nouvelle fonctionnalité d'ADO 2.1).

Supports : Détermine si un objet **Recordset** prend en charge un type de fonctionnalité particulier.

Update : Enregistre les modifications apportées à l'enregistrement en cours de l'objet **Recordset**.

UpdateBatch : Enregistre toutes les mises à jour par lots en attente sur le disque.

Propriétés

AbsolutePage : Indique la page de l'enregistrement en cours.

AbsolutePosition : Indique la position ordinale de l'enregistrement en cours dans un objet **Recordset**.

ActiveCommand : Indique l'objet **Command** qui a créé l'objet **Recordset** correspondant. En lecture seule.

ActiveConnection : Indique à quel objet **Connection** appartient l'objet **Recordset**.

BOF : Indique si le pointeur d'enregistrement est positionné avant le premier enregistrement de l'objet **Recordset**. En lecture seule.

Bookmark : Renvoie un signet qui identifie de manière unique l'enregistrement en cours dans l'objet **Recordset** ou déplace l'enregistrement en cours sur un enregistrement identifié par un signet valide.

CacheSize : Indique le nombre d'enregistrements d'un objet **Recordset** stocké en mémoire cache.

CursorLocation : Définit ou renvoie le type de curseur à utiliser par défaut pour cette connexion (curseur côté client ou curseur côté serveur).

CursorType : Indique le type de curseur utilisé pour cet objet **Recordset**.

DataMember : Indique le nom du membre de données à extraire de l'objet défini par la propriété DataSource. En lecture seule.

DataSource : Indique qu'un objet contenant des données doit être représenté comme objet **Recordset**. En lecture seule.

EditMode : Indique le statut de modification de l'enregistrement en cours. En lecture seule.

EOF : Indique si le pointeur d'enregistrement est positionné après le dernier enregistrement dans l'objet **Recordset**. En lecture seule.

© ENI Editions - All rigths reserved

Filter : Définit un filtre de données dans le **Recordset**.

Index : Indique le nom de l'index en cours pour le **Recordset**.

LockType : Indique le type de verrouillage des enregistrements lors des modifications.

MarshalOptions : Indique les enregistrements à renvoyer au serveur.

MaxRecords : Indique le nombre maximal d'enregistrements à renvoyer à un objet Recordset pour une même requête. La valeur par défaut est zéro (aucune limite).

PageCount : Indique le nombre de pages de données contenues dans l'objet **Recordset**. En lecture seule.

PageSize : Indique le nombre d'enregistrements contenus dans une page du **Recordset**.

Properties : Référence la **Collection** des objets **Property** de l'objet **Recordset** en cours.

RecordCount : Indique le nombre actuel d'enregistrements dans l'objet **Recordset**. En lecture seule.

Sort : Spécifie un ou plusieurs noms de champs selon lesquels l'objet **Recordset** est trié, et détermine l'ordre de tri de chaque champ.

Source : Indique la source des données d'un objet **Recordset**.

State : Indique l'état du **Recordset** : ouvert, fermé ou opération asynchrone en cours. En lecture seule.

Status : Indique le statut de l'enregistrement en cours en ce qui concerne les mises à jour par lots ou autres opérations globales. En lecture seule.

StayInSync : Indique, dans un objet **Recordset** hiérarchique, si la ligne parent doit changer lorsque l'ensemble des enregistrements enfants sous-jacents change. En lecture seule.

Exemple

Transformation en majuscules des caractères du champ Cli_Societe dans la table Clients.

```
Dim cnc As New ADODB.Connection
Dim rstCli As New ADODB.Recordset

' Création du Recordset
Set cnc = CurrentProject.Connection
rstCli.Open "Select * From Clients", cnc, _
    adOpenKeyset, adLockOptimistic
' Parcours des enregistrements
With rstCli
  Do Until .EOF
    rstCli("Cli_Societe") = _
      UCase(rstCli("Cli_Societe"))
     ' Mise à jour
    .Update
     ' Enregistrement suivant
     .MoveNext
  Loop
End With
rstCli.Close
cnc.Close
```

*Utilisation de la méthode **Seek** : recherche d'un client à partir de son code.*

```
Dim cnc As New ADODB.Connection
Dim rstCli As New ADODB.Recordset
Dim strCodeCli As String

' Création du Recordset
Set cnc = CurrentProject.Connection
```


© ENI Editions - All rigths reserved

```
rstCli.Open "Clients", cnc, adOpenKeyset, _
    adLockReadOnly, adCmdTableDirect
With rstCli
   ' Saisie du code client
   strCodeCli = _
  InputBox("Saisissez le code client")
   ' Index courant : clé primaire
   .Index = "PrimaryKey"
   ' Recherche
   .Seek strCodeCli, adSeekFirstEQ
   ' Enregistrement non trouvé
   If .EOF Then
    MsgBox "Client non trouvé"
  Else
   ' Enregistrement trouvé
     MsgBox "Société : " & rstCli("Cli_Societe")
  End If
End With
rstCli.Close
cnc.Close
```

*Utilisation de la méthode **Find** : recherche des clients dont le nom de Société commence par une valeur donnée.*

```
Dim cnc As New ADODB.Connection
Dim rstCli As New ADODB.Recordset
Dim strSoc As String
Dim strCrit As String
Dim bkm As Variant

' Création du Recordset
Set cnc = CurrentProject.Connection
rstCli.Open "Clients", cnc, adOpenKeyset, _
  adLockReadOnly, adCmdTableDirect
With rstCli
   ' Saisie du code client
   strSoc = InputBox("Saisissez les 1ères " _
    & "lettres de la Société")
   ' Recherche
   strCrit = "Cli_Societe Like '" & strSoc & "%'"
  .Find strCrit, , adSearchForward
   ' Enregistrement non trouvé
   If .EOF Then
    MsgBox "Client non trouvé"
  Else
   ' Si trouve : recherche des suivants
   Do While Not .EOF
     MsgBox rstCli("Cli_Societe")
     ' Position en cours.
     bkm = .Bookmark
     .Find strCrit, 1, adSearchForward, bkm
    Loop
  End If
End With
rstCli.Close
cnc.Close
```

La collection Fields

Méthodes

Append : Ajoute un objet **Field** à une collection **Fields**.

Delete : Supprime un objet **Field** de la collection **Fields**.

Refresh : Met à jour les objets **Field** dans la collection **Fields**.

Les objets Field

Méthodes

AppendChunk : Ajoute des données à un objet **Field** de type binaire ou de grande taille.

© ENI Editions - All rigths reserved

GetChunk : Renvoie tout ou une partie du contenu d'un objet **Field** de type binaire ou de grande taille.

Propriétés

ActualSize : Indique la longueur de la valeur d'un champ. En lecture seule.

Attributes : Indique une ou plusieurs caractéristiques d'un objet **Field**.

DefinedSize : Indique la taille de l'objet **Field**. En écriture seule.

Name : Indique le nom de l'objet **Field**.

NumericScale : Indique l'échelle des valeurs numériques de l'objet **Field**. En écriture seule.

OriginalValue : Indique la valeur de l'objet **Field** avant modification. En lecture seule.

Precision : Indique le degré de précision des valeurs de type numérique dans l'objet **Field**. En lecture seule.

Properties : Comprend tous les objets **Property** d'un objet **Field**.

Type : Indique le type de données de l'objet **Field**.

UnderlyingValue : Indique la valeur en cours de l'objet **Field** dans la base de données. En lecture seule.

Value : Indique la valeur de l'objet **Field**.

La collection Parameters

Méthodes

Append : Ajoute un objet **Parameter** à la collection **Parameters**.

Delete : Supprime un objet **Parameter** de la collection **Parameters**.

Refresh : Met à jour les objets **Parameter** dans la collection **Parameters**.

Exemple

Affichage de la liste de tous les champs (nom et type) de la table Clients dans la fenêtre Exécution.

```
Dim cnc As New ADODB.Connection
Dim rstCli As New ADODB.Recordset
Dim fld As ADODB.Field

' Création du Recordset
Set cnc = CurrentProject.Connection
rstCli.Open "Employes", cnc, adOpenKeyset, adLockReadOnly, _
adCmdTableDirect
For Each fld In rstCli.Fields
  Debug.Print fld.Name & " " & fld.Type
Next
rstCli.close
```

Les objets Parameter

Méthodes

AppendChunk : Ajoute des données à un objet **Parameter** de type binaire ou texte de grande taille.

Propriétés

Attributes : Indique une ou plusieurs caractéristiques d'un objet **Parameter**.

© ENI Editions - All rigths reserved

Direction : Indique si l'objet **Parameter** correspond à un paramètre d'entrée, de sortie, ou les deux, ou si le paramètre est le type de retour d'une procédure stockée.

Name : Indique le nom de l'objet **Parameter**.

NumericScale : Indique l'échelle des valeurs numériques de l'objet **Parameter**.

Precision : Indique le degré de précision des valeurs de type **Numeric** de l'objet **Parameter**.

Properties : Comprend tous les objets **Property** d'un objet **Parameter**.

Size : Indique la taille maximale, en octets ou en caractères, d'un objet **Parameter**.

Type : Indique le type de données de l'objet **Parameter**.

Value : Indique la valeur attribuée à l'objet **Parameter**.

La collection Properties

Méthodes

Refresh : Met à jour les objets **Property** dans la collection **Properties** à partir des informations du fournisseur.

Les objets Property

Propriétés

Attributes : Indique une ou plusieurs caractéristiques d'un objet **Property**.

Name : Indique le nom de l'objet **Property**. En lecture seule.

Type : Indique le type de données de l'objet **Property**.

Value : Indique la valeur de l'objet **Property**.

La collection Errors

Méthodes

Clear : Supprime l'ensemble des objets **Error** de la collection **Errors**.

Refresh : Met à jour les objets **Error** à partir des informations du fournisseur OLE DB.

Propriétés

Count : Indique le nombre d'objets **Error** de la collection **Errors**. En lecture seule.

Item : Permet l'indexation de la collection **Errors** afin de référencer un objet **Error** spécifique. En lecture seule.

Les objets Error

Propriétés

Description : Renvoie la chaîne descriptive associée à l'objet **Error**. En lecture seule.

HelpContext : Indique le **ContextID** du fichier d'aide associé à l'objet **Error**. En lecture seule.

HelpFile : Indique le nom du fichier d'aide associé à l'objet **Error**. En lecture seule.

© ENI Editions - All rigths reserved

NativeError : Indique le code d'erreur spécifique du fournisseur associé à l'objet **Error**. En lecture seule.

Number : Indique le numéro identifiant de manière unique l'erreur portée par l'objet **Error**. En lecture seule.

Source : Indique le nom de l'objet ou de l'application à l'origine d'une erreur. En lecture seule.

SQLState : Indique la valeur renvoyée par le fournisseur OLE DB. Chaîne de cinq caractères conforme à la norme SQL ANSI. En lecture seule.

© ENI Editions - All rigths reserved

Présentation

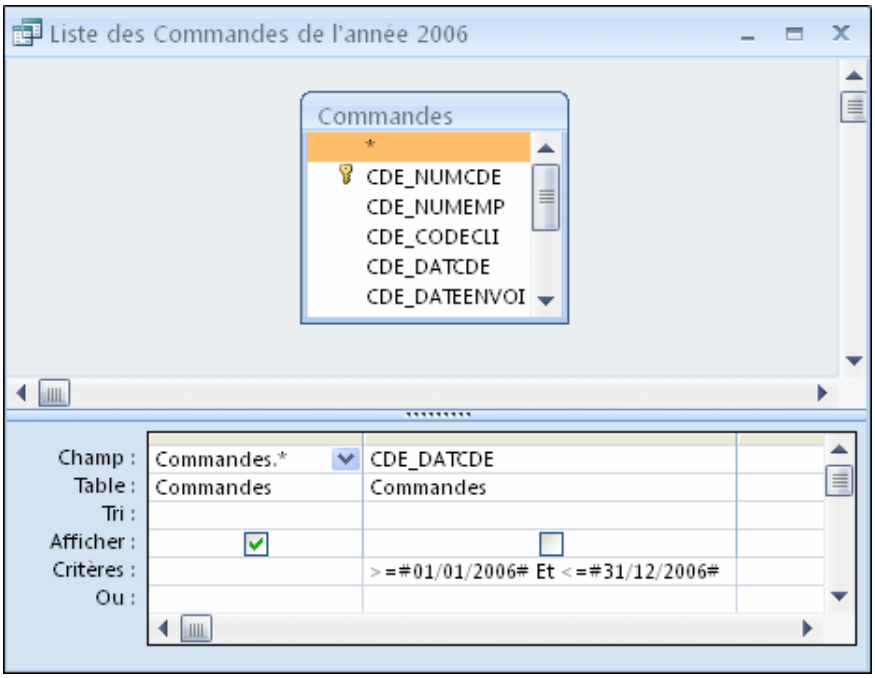
Le langage SQL (**Structured Query Langage** ou langage de requête structuré) est un langage d'interrogation et de gestion de bases de données relationnelles.

Access 2007 utilise SQL comme langage de requête. Lorsque vous créez une requête en mode Création de requête, Access construit en tâche de fond les instructions SQL équivalentes. En fait, la plupart des propriétés de la requête en mode Création de requête ont des clauses équivalentes et des options accessibles en langage SQL.

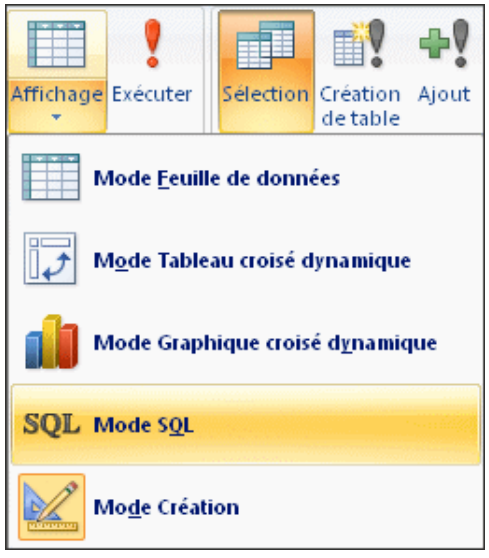
Il est possible d'afficher ou de modifier l'instruction SQL correspondant à une requête en passant en mode SQL.

Exemple

- Ouvrez une requête existante en mode création.

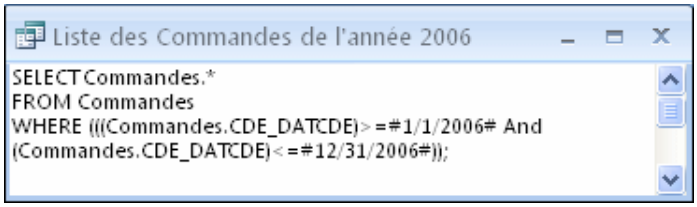


- Cliquez ensuite sur l'option **Mode SQL** de l'icône **Affichage**.



Vous obtenez alors l'instruction SQL correspondant à votre requête :

© ENI Editions - All rigths reserved



» Cette manipulation est très intéressante notamment si vous n'êtes pas coutumier du langage SQL : vous pouvez créer vos requêtes dans Access et récupérer l'instruction SQL générée pour l'inclure dans votre code VBA.

Certaines requêtes SQL, nommées requêtes spécifiques à SQL, ne peuvent être créées dans la grille de création de requêtes mais doivent être saisies directement en mode SQL. Il s'agit :

- des "requêtes **directes**" qui vous permettent de travailler directement avec les tables sur le serveur,

- des "requêtes **définition des données**" qui vous permettent de créer, supprimer ou modifier la structure des tables,

- des "requêtes **Union**" qui vous permettent de regrouper les champs de deux tables (ou plus) dans un même champ résultat de la requête. Ces requêtes sont très utiles pour fusionner des enregistrements provenant de tables différentes.

© ENI Editions - All rigths reserved

Langage SQL et VBA

Les requêtes SQL s'intègrent parfaitement à VBA et peuvent être utilisées à partir des méthodes de différents objets :

- méthode **OpenRecordset** des objets **DataBase** et **Connection**,

- méthode **CreateQueryDef** des objets **DataBase** et **Connection**,

- méthode **Execute** des objets **Command** et **Connection**,

- etc.

Les requêtes SQL sont aussi très utilisées dans les formulaires :

- propriété **RecordSource** d'un formulaire ou d'un état,

- propriété **RowSource** d'une liste déroulante,

- etc ...

La plupart de ces propriétés peuvent être modifiées par du code VBA.

Les avantages de l'utilisation d'instructions SQL en VBA sont les suivants :

Vitesse d'exécution : une instruction **Select** est plus rapide que le parcours séquentiel d'une table.

Performance accrue : en mode client/serveur, les requêtes sont compilées par le SGBD qui optimise leurs performances.

Maintenance plus facile : le code des instructions SQL est plus court, donc plus lisible que l'équivalent VBA.

Standardisation : le langage SQL est le standard d'interrogation des bases de données relationnelles.

© ENI Editions - All rigths reserved

Description du langage SQL

L'instruction SELECT

SELECT est l'instruction SQL la plus utilisée : elle correspond à la requête Sélection sous Access. Elle permet d'extraire des champs d'un ensemble d'enregistrements répondant à certains critères.

Syntaxe

```
SELECT [ALL | DISTINCT] <liste de champs>
FROM <liste de tables>
[WHERE... ]
[GROUP BY...]
[HAVING... ]
[ORDER BY... ]
```

`All` (valeur par défaut) : Tous les enregistrements répondant aux critères sont pris en compte.

`Distinct` : Les enregistrements pour lesquels les champs sélectionnés contiennent des données en double sont omis.

`<liste de champs>` : Liste des champs sélectionnés, séparés par des virgules, en respectant la syntaxe suivante :

```
* | table.* | [table.]champ1 [AS alias1] [,
[table.]champ2 [AS alias2] [, ...]]
```

`*` : Indique que tous les champs de la (des) table(s) spécifiée(s) sont sélectionné(s).

`alias1, alias2` : Indiquent les noms à utiliser comme en-têtes de colonne à la place des noms de colonnes originaux dans table.

`<liste de tables>` : Nom de la ou des tables contenant les données à extraire. Si la requête utilise plusieurs tables, le mode de jointure doit également être précisé dans la clause FROM.

Exemples

```
- SELECT * FROM Produits
- SELECT Produits.* FROM Produits
- SELECT Prod_ref, Prod_desig, Prod_prix, Prod_tva
FROM Produits
- SELECT Prod_ref As Reference, Prod_desig As Designation,
Prod_Prix as Prix, Prod_tva as Tva From Produits
```

La clause FROM

La clause FROM spécifie les tables ou les requêtes dont sont extraites les données.

Syntaxe

```
FROM table1 [ LEFT | RIGHT | INNER ] JOIN table2
ON table1.champ1 <Opérateur de comparaison> table2.champ2
```

[LEFT | RIGHT | INNER] :Ces opérateurs permettent de définir le type de jointure entre les tables.

LEFT JOIN : Utilisez une opération LEFT JOIN pour créer une jointure externe gauche. Le résultat de la jointure externe gauche comprend tous les enregistrements issus de la première table (celle de gauche : <table 1>) même ceux pour lesquels il n'existe pas de valeur correspondante dans la deuxième table (celle de droite : <table 2>).

RIGHT JOIN : Utilisez une opération RIGHT JOIN pour créer une jointure externe droite. Le résultat de la jointure externe droite comprend tous les enregistrements issus de la deuxième table (celle de droite : <table 2>) même ceux pour lesquels il n'existe pas de valeur correspondante dans la première table (celle de gauche : <table 1>).

© ENI Editions - All rigths reserved

INNER JOIN : Utilisez INNER JOIN pour créer une jointure ne comprenant que les enregistrements dans lesquels les données des champs reliés sont identiques.

<opérateur de comparaison> : Utilisez tout opérateur de comparaison relationnelle tel que : =, <, >, <=, >=,<> (différent de). L'opérateur = est le plus couramment utilisé.

Exemples

Liste des commandes avec le nom et le n° de téléphone du messager.

Premier exemple

Seules les commandes auxquelles un messager a été affecté sont sélectionnées.

```
SELECT Commandes.Cde_Numcde, Commandes.Cde_DateEnvoi,
Messagers.
Mes_Nom, Messagers.Mes_Telephone
FROM Commandes INNER JOIN Messagers ON Commandes.Cde_NumMess
=
Messagers.Mes_NumMes
```

Deuxième exemple

Toutes les commandes sont sélectionnées.

```
SELECT Commandes.Cde_Numcde, Commandes.Cde_DateEnvoi,
Messagers.Mes_Nom, Messagers.Mes_Telephone
FROM Commandes LEFT JOIN Messagers ON Commandes.Cde_NumMess
= Messagers.Mes_NumMes
```

La clause WHERE

La clause WHERE permet de filtrer les enregistrements à partir de critères.

Les conditions de la clause WHERE s'expriment avec les opérateurs suivants :

- opérateurs de comparaison : =, <, >, <=, >=, <> (différent de), Is [Not] Null, [Not] In, Between.

- opérateurs logiques : And, Or, Not.

Exemples

```
SELECT * FROM Produits WHERE [Prod_Prix] = 500
SELECT * FROM Produits WHERE [Prod_Prix]
    BETWEEN 100 AND 200
SELECT * FROM Clients WHERE [Cli_pays] IN
('France','Italie','Espagne')
```

La clause GROUP BY

La clause GROUP BY correspond aux "**Totaux**" des requêtes Access (appelés Regroupements dans les anciennes versions). Elle permet de regrouper des données sur un ou plusieurs champs. Elle est principalement utilisée pour effectuer des fonctions d'agrégation SQL telles que Sum (somme), Count (compte), Avg (moyenne)...

Exemples

Cette requête renvoie le nombre de commandes passées par client.

```
SELECT Clients.Cli_Societe, Count(Commandes.Cde_Numcde) AS
Nb_Cdes
```


© ENI Editions - All rigths reserved

```
FROM Clients INNER JOIN Commandes ON Clients.Cli_NumCli=
Commandes.Cde_CodeCli
GROUP BY Clients.Cli_Societe
```

Cette requête renvoie le montant total de chacune des commandes.

```
SELECT Commandes.Cde_Numcde, Commandes.Cde_DateCde,
Sum([Cde_Prix]*[Cde_Qte]) AS TotalCde
FROM Commandes INNER JOIN [Détails commandes] ON
Commandes.Cde_Numcde = [Détails commandes].Cde_NumCde
GROUP BY Commandes.Cde_Numcde, Commandes.Cde_DateCde
```

La clause HAVING

La clause HAVING spécifie les enregistrements regroupés à afficher dans une instruction SELECT dotée d'une clause GROUP BY. Une fois le regroupement effectué suivant la liste des champs du GROUP BY, seuls sont conservés les enregistrements qui vérifient la condition spécifiée dans la clause HAVING.

Exemple

Cette requête extrait les commandes dont le montant total est supérieur à 10 000 € .

```
SELECT Commandes.Cde_Numcde, Commandes.Cde_DateCde,
Sum([Cde_Prix]*[Cde_Qte]) AS TotalCde
FROM Commandes INNER JOIN [Détails commandes] ON
Commandes.Cde_Numcde = [Détails commandes].Cde_NumCde
GROUP BY Commandes.Cde_Numcde, Commandes.Cde_DateCde
HAVING (Sum([Cde_Prix]*[Cde_Qte])>10000)
```

La clause ORDER BY

La clause ORDER BY trie, par ordre croissant ou décroissant, les enregistrements résultants d'une requête en fonction du ou des champs spécifiés.

Syntaxe

```
[ORDER BY champ1 [ASC | DESC ][, champ2 [ASC | DESC ]][, ...]]]
```

ASC : Ordre croissant.

DESC : Ordre décroissant.

Exemple

Cette requête renvoie le nombre de commandes passées par client, trié par ordre décroissant du nombre de commande.

```
SELECT Clients.Cli_Societe, Count(Commandes.Cde_Numcde) AS
Nb_Cdes
FROM Clients INNER JOIN Commandes ON
Clients.Cli_NumCli=Commandes.Cde_CodeCli
GROUP BY Clients.Cli_Societe
ORDER BY Count(Commandes.Cde_Numcde) DESC
```

L'instruction UPDATE

L'instruction UPDATE correspond à une requête de Mise à jour qui modifie les valeurs des champs dans la table spécifiée, pour les enregistrements qui répondent au critère fourni (facultatifs).

Syntaxe

```
UPDATE table
SET champ1 = nouvellevaleur1[, champ2 = nouvellevaleur2]...
[WHERE critère]
```

© ENI Editions - All rigths reserved

» Si aucune clause Where n'est spécifiée, tous les enregistrements sont mis à jour.

Exemple

Cette requête modifie le code pays de tous les enregistrements de la table Clients.

```
UPDATE Clients SET Clients.Cli_pays = 'F'
```

Cette requête modifie dans la table Clients le code pays des enregistrements des clients dont la ville est 'Marseille'.

```
UPDATE Clients SET Clients.Cli_Pays = 'F'
  WHERE Clients.Cli_ville='Marseille'
```

L'instruction DELETE

L'instruction DELETE correspond à une requête Suppression qui supprime des enregistrements dans la table mentionnée dans la clause FROM, selon les critères fournis (facultatifs).

Syntaxe

```
DELETE FROM table
  [WHERE critères]
```

» Si aucune clause Where n'est spécifiée, tous les enregistrements sont supprimés.

Exemple

Cette requête supprime les commandes antérieures à 2007.

```
DELETE FROM Commandes
WHERE ((Commandes.Cde_DateCde) < #1/1/2007#)
```

L'instruction INSERT INTO

L'instruction SQL INSERT INTO correspond à une requête Ajout qui ajoute des enregistrements dans une table.

Syntaxe

Requête Ajout avec un seul enregistrement :

```
INSERT INTO <TableDestination> [(champ1[, champ2[, ...]])]
VALUES (valeur1[, valeur[, ...]])
```

» Cette requête doit contenir le même nombre de valeurs, du même type et dans le même ordre, que de colonnes à valoriser.

Requête Ajout avec un ou plusieurs enregistrements :

```
INSERT INTO <TableDestination> [(champ1[, champ2[, ...]])]
SELECT [<Tablesource>.]champ1[, champ2[, ...]]
FROM <liste de tables>
```

» L'instruction SELECT doit renvoyer autant de valeurs, dans le même ordre et du même type que les colonnes à valoriser (champs de l'instruction INSERT).

Exemples

Ajout d'un enregistrement dans la table Clients.

```
INSERT INTO Clients ( Cli_CodeCLi, Cli_societe, Cli_ville,
Cli_pays )
```


© ENI Editions - All rigths reserved

```
VALUES ('ENI','Editions ENI', 'Nantes','F').
```

Ajout des enregistrements de la table Prospects dans la table Clients.

```
INSERT INTO Clients
SELECT Prospect.*
FROM Prospect
```

Autres instructions

Requêtes d'interrogation

TRANSFORM : Crée une requête analyse croisée. Cette instruction est utilisée lors de la création d'une requête à partir de l'assistant Requête analyse croisée d'Access.

UNION : Crée une requête qui fusionne les résultats de deux, ou plusieurs, requêtes ou tables indépendantes. Les structures des tables et/ou des requêtes fusionnées doivent être identiques.

Création et gestion de tables

CREATE TABLE : Crée une nouvelle table.

CREATE INDEX : Crée un nouvel index pour une table existante.

ALTER TABLE : Modifie la structure d'une table.

DROP TABLE : Supprime une table de la base de données.

DROP INDEX : Supprime un index d'une table.

Création et gestion d'utilisateurs et de groupes

CREATE USER : Crée un ou plusieurs nouveaux utilisateurs.

ADD USER : Ajoute un ou plusieurs utilisateurs à un groupe d'utilisateurs existant.

DROP USER : Supprime un ou plusieurs utilisateurs.

CREATE GROUP : Crée un ou plusieurs nouveaux groupes.

DROP GROUP : Supprime un ou plusieurs groupes d'utilisateurs.

GRANT : Attribue des privilèges spécifiques à un utilisateur ou à un groupe d'utilisateurs existant.

REVOKE : Retire des privilèges spécifiques à un utilisateur ou à un groupe d'utilisateurs existant.

» Vous pouvez obtenir de plus amples informations au sujet du langage SQL et de ses instructions, à partir de la rubrique "Référence de Microsoft jet SQL", du sommaire de l'aide Access.

© ENI Editions - All rigths reserved

Exemples d'utilisation de requêtes SQL

Afin de démontrer l'intérêt du langage SQL, les exemples suivants proposent deux procédures équivalentes : l'une utilise le langage SQL, l'autre non.

Pour réaliser ces exemples vous devez créer une table **Clients** avec un champ nommé **Cli_Societe**, et saisir quelques enregistrements dans cette table. Vous pouvez également importer la table **Clients** de la base exemple Comptoir.mdb livrée avec Access 2003 et renommer les champs **Société** en **Cli_Societe**, **Pays** en **Cli_Pays** et **Ville** en **Cli_Ville**.

Mise à jour d'enregistrements

Le code de l'exemple suivant permet de modifier le contenu d'un champ dans une table pour des enregistrements répondant à certains critères.

Code VBA utilisant une requête SQL de type UPDATE.

```
Private Sub Maj_Pays()
Dim cncDevis As ADODB.Connection
Dim strSQL As String
' Mise à jour du code pays dans la table clients
Set cncDevis = CurrentProject.Connection
strSQL = "UPDATE Clients SET " _
  & "Clients.CLI_PAYS = 'F' " _
  & "WHERE Clients.CLI_VILLE = 'Marseille'"
cncDevis.Execute strSQL
End Sub
```

Code VBA équivalent sans requête SQL.

```
Private Sub Maj_Pays1()
Dim cncDevis As ADODB.Connection
Dim rstClient As ADODB.Recordset

' Ouverture d'un jeu d'enregistrement
Set cncDevis = CurrentProject.Connection
Set rstClient = New ADODB.Recordset
rstClient.Open "CLIENTS", cncDevis, adOpenForwardOnly,
adLockOptimistic

' Parcours séquentiel de la table Clients
Do While Not rstClient.EOF
  If rstClient("CLI_VILLE") = "Marseille" Then
    rstClient("CLI_PAYS") = "F"

    rstClient.Update
  End If
  rstClient.MoveNext
Loop

End Sub
```

Chargement d'une liste déroulante

Le code de l'exemple ci-après permet d'afficher dans une liste déroulante (contrôle zone de liste) le nom des clients dont le nom de Société commence par une lettre donnée (la lettre est saisie dans une zone de texte).

Pour tester cet exemple vous devez :

- créer un formulaire nommé Clients.

- Ajouter les contrôles suivants dans ce formulaire :

- une zone de texte nommé **txtNomSoc**.

© ENI Editions - All rigths reserved

- Une zone de liste nommée **lstSoc**.

- Un bouton de commande **cmdListe1**.

Code VBA utilisant une requête SQL de type SELECT. Le contenu de la liste est le résultat de la requête SQL.

```
Private Sub cmdListe1_Click()
Dim strSQL As String

' Contrôle du nom saisi
If txtNomSoc = "" Then
  MsgBox "Vous devez saisir au moins une lettre",
vbExclamation
  txtNomSoc.SetFocus
  Exit Sub
End If

' Propriétés de la liste déroulante
' La liste contient une requête
Me.lstSoc.RowSourceType = "Table/Requête"
strSQL = "SELECT Clients.CLI_SOCIETE FROM Clients " _
  & "WHERE (Clients.CLI_SOCIETE Like '" & Me.txtNomSoc &
"*')"
Me.lstSoc.RowSource = strSQL
' Réactualisation des données de la liste
Me.lstSoc.Requery

End Sub
```

Code VBA équivalent sans requête SQL. La liste est renseignée à partir d'une liste de valeur.

```
Private Sub cmdListe1_Click()
Dim cncDevis As ADODB.Connection
Dim rstClient As ADODB.Recordset
Dim strNomSoc as String
' Contrôle du nom saisi
If txtNomSoc = "" Or IsNull(txtNomSoc) Then
  MsgBox "Vous devez saisir au moins une lettre",
vbExclamation
  txtNomSoc.SetFocus
  Exit Sub
End If

' Initialisation de la liste déroulante
' La liste contient une liste de valeurs
Me.lstSoc.RowSourceType = "Liste valeurs"
Me.lstSoc.RowSource = ""
Me.lstSoc.Requery
' Ouverture d'un jeu d'enregistrement
Set cncDevis = CurrentProject.Connection
Set rstClient = New ADODB.Recordset
rstClient.Open "CLIENTS", cncDevis, adOpenForwardOnly, _
  adLockOptimistic
' Parcours séquentiel de la table et ajout d'éléments à la
liste
Do While Not rstClient.EOF
  If Left(rstClient("CLI_SOCIETE"), 1) = Left(Me.txtNomSoc, _
    Len(Me.txtNomSoc)) Then
    strNomSoc = rstClient("CLI_SOCIETE")
    Me.LstSoc.AddItem strNomSoc
  End If
  rstClient.MoveNext
Loop
rstClient.close

End Sub
```

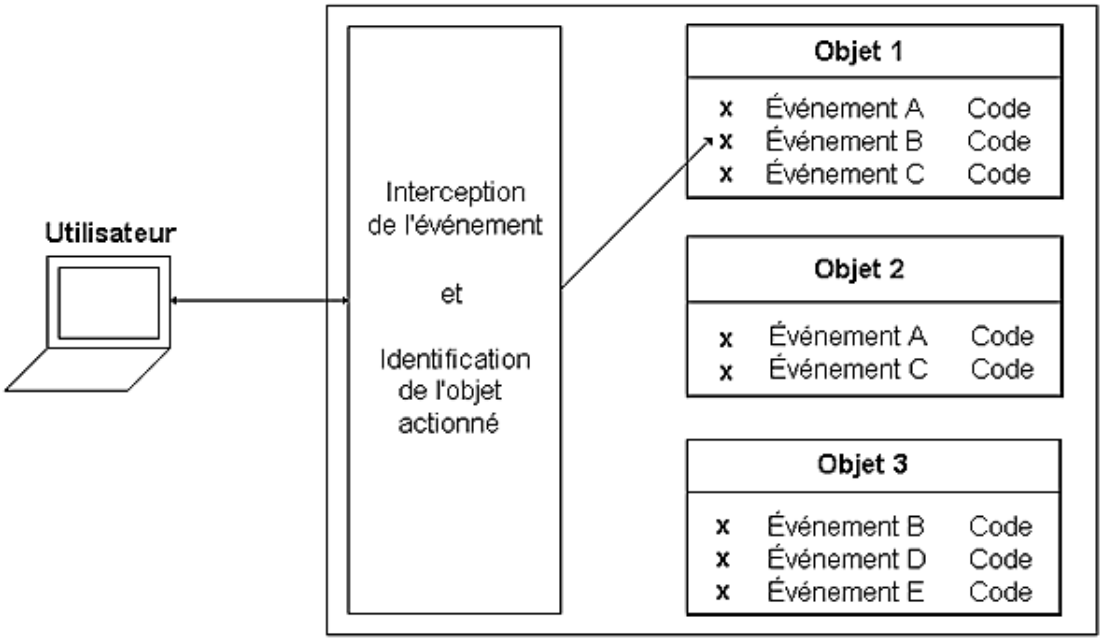
» Les exemples précédents permettent de constater que le code utilisant SQL est plus court et son exécution plus rapide, ce qui peut s'avérer très intéressant par exemple si la table client contient un nombre important d'enregistrements.

© ENI Editions - All rigths reserved

Présentation

Définition

Un **événement** est issu d'une action de l'utilisateur ou du système sur un objet. Il déclenche l'exécution du code correspondant à la procédure événementielle de l'objet activé.

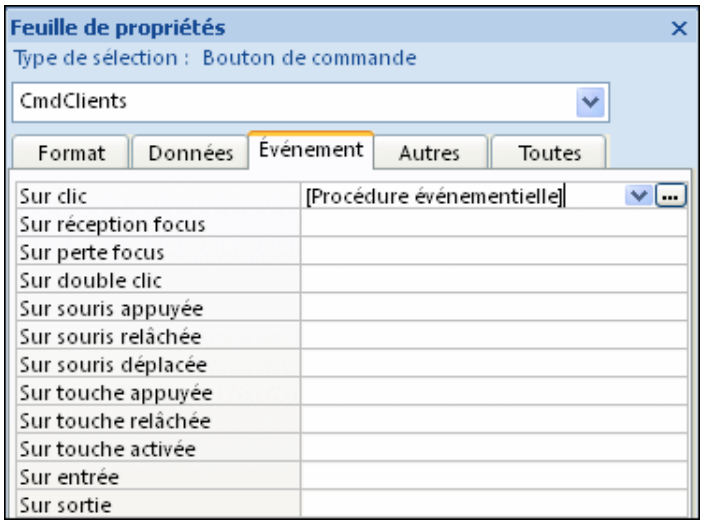


L'utilisation d'une procédure événementielle ou d'une macro permet d'ajouter la réponse propre à l'application à un événement se produisant dans un formulaire, un état ou un contrôle.

Association de code VBA à un événement

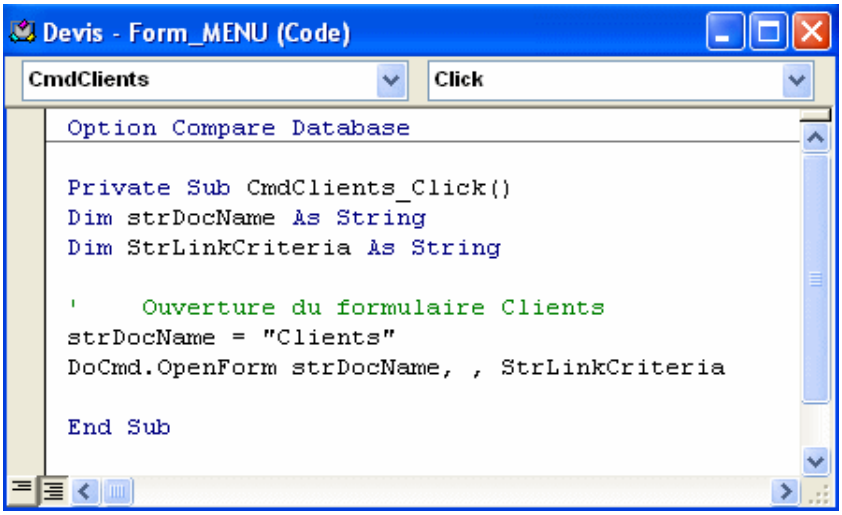
Vous pouvez affecter un code VBA à l'événement d'un objet de deux façons :

- à partir de l'onglet **Événement** de la fenêtre des propriétés de l'objet dans Access.



- À partir des listes déroulantes d'objets et d'événements dans l'éditeur Visual Basic

© ENI Editions - All rigths reserved



© ENI Editions - All rigths reserved

Catégories d'événements

Les listes d'événements suivantes sont classées par type et comportent :

- le nom de l'événement,

- le nom de la propriété correspondante dans la fenêtre des propriétés de l'objet (entre parenthèses),

- la description de l'événement,

- la mention Annulation indique si l'événement peut être annulé ou pas. L'annulation d'un événement fait l'objet du paragraphe Annulation d'un événement.

Evénements de type Fenêtre

Open : (Sur ouverture)
Se produit à l'ouverture du formulaire mais avant que le premier enregistrement soit affiché.
Se produit à l'ouverture d'un état, mais avant son impression.

Annulation : Oui

Load : (Sur chargement)
Se produit à l'ouverture du formulaire lorsque le premier enregistrement est affiché.

Annulation : Non

Resize : (Sur redimensionnement)
Se produit lors du premier affichage du formulaire et lors des modifications de sa taille.

Annulation : Non

UnLoad : (Sur libération)
Se produit lors de la fermeture et lorsque les enregistrements sont libérés, mais avant que le formulaire ne disparaisse.

Annulation :Oui

Close : (Sur fermeture)
Se produit lorsque le formulaire ou l'état disparaît à la fermeture.

Annulation : Non

Evénements de type Focus

Les événements de type "focus" peuvent s'appliquer aux formulaires (Activate, Deactivate, GotFocus, LostFocus) ou aux contrôles (Enter, Exit, Set- Focus, LostFocus).
On dit qu'un objet a le focus lorsqu'il peut recevoir une saisie de l'utilisateur via des actions à la souris ou au clavier. Les objets qui ont le focus sont appelés objets actifs (ActiveForm ou ActiveControl).

» La méthode **SetFocus** permet de donner le focus à un contrôle ou un formulaire.

Activate : (Sur activé)
Se produit lorsque la fenêtre du formulaire ou de l'état devient active.

Annulation : Non

Deactivate : (Sur désactivé)
Se produit lors de l'activation d'une autre fenêtre, mais avant qu'elle ne soit active.
Se produit également lors de la fermeture de la fenêtre.

© ENI Editions - All rigths reserved

Annulation : Non

Enter : (Sur entrée)
Se produit avant qu'un contrôle ne prenne le focus à partir d'un autre contrôle ou à l'ouverture du formulaire.

Annulation : Non

Exit : (Sur sortie)
Se produit avant qu'un contrôle ne perde le focus au profit d'un autre contrôle du formulaire ou d'un autre formulaire.

Annulation : Oui

GotFocus : (Sur réception focus)
Se produit lorsqu'un contrôle ou un formulaire sans contrôle actif reçoit le focus.

Annulation : Non

LostFocus : (Sur perte focus)
Se produit lorsqu'un contrôle ou un formulaire perd le focus.

Annulation : Non

Evénements de type Données

Les événements de type "données" s'appliquent aux formulaires associés à une source de données (propriété RecordSource renseignée à partir d'un nom de table ou d'une requête) et aux contrôles associés à un champ de la source de données du formulaire (propriété ControlSource renseignée).

AfterInsert : (Après insertion)
Se produit après l'ajout d'un nouvel enregistrement dans la table.

Annulation : Non

AfterUpdate : (Après MAJ)
Se produit après la mise à jour d'un contrôle ou d'un enregistrement avec des données modifiées.

Annulation : Non

AfterDelConfirm : (Après suppression)
Se produit après que l'utilisateur ait répondu à la demande de suppression des enregistrements.

Annulation : Non

BeforeInsert : (Avant insertion)
Se produit lors de la frappe du premier caractère d'un nouvel enregistrement, mais avant l'ajout réel de l'enregistrement.

Annulation : Oui

BeforeUpdate : (Avant MAJ)
Se produit avant la mise à jour d'un contrôle ou d'un enregistrement avec des données modifiées.

Annulation : Oui

BeforeDelConfirm : (Avant suppression)
Se produit après que l'utilisateur ait supprimé des enregistrements, et avant qu'Access ne demande confirmation.

Annulation : Oui

NotInList : (Sur absence dans liste)

© ENI Editions - All rigths reserved

Se produit lors de la saisie d'une valeur ne figurant pas dans une liste et lorsque la propriété limitée à une liste est vraie.

Annulation : Non

Current : (Sur activation)
Se produit lorsque le focus passe d'un enregistrement à l'autre.
Se produit également à l'ouverture avant que le premier enregistrement ne devienne celui en cours.

Annulation : Non

Change : (Sur changement)
Se produit lors de la modification du contenu d'une zone texte ou liste modifiable (clavier ou macro/Visual Basic).

Annulation : Non

Updated : (Sur MAJ)
Se produit lorsque les données d'un objet OLE ont été modifiées.

Annulation : Non

Delete : (Sur suppression)
Se produit avant la suppression effective. Lors de l'appui sur [Suppr] par exemple.

Annulation : Oui

Dirty : Se produit lorsque le contenu d'un formulaire ou la partie texte d'une zone de liste modifiable change.
Se produit également lors du passage d'une page à une autre dans un contrôle Onglet.

Annulation : Oui

Evénements de type Souris

Click : (Sur clic)
Se produit lors de l'appui sur le bouton gauche de la souris sur un contrôle ou une zone vierge d'un formulaire.

Annulation : Non

DblClick : (Sur double clic)
Se produit lors de deux appuis sur le bouton gauche de la souris sur un contrôle ou une zone vierge d'un formulaire.

Annulation : Oui

MouseDown : (Sur souris appuyée)
Se produit lors de l'appui sur un bouton de la souris sur un contrôle ou un formulaire.

Annulation : Oui

MouseMove : (Sur souris déplacée)
Se produit lorsque l'utilisateur déplace la souris sur un formulaire ou un contrôle.

Annulation : Non

MouseUp : (Sur souris relâchée)
Se produit lorsque l'utilisateur relâche le bouton de la souris sur un contrôle ou un formulaire.

Annulation : Non

Evénements de type Clavier

© ENI Editions - All rigths reserved

KeyDown : (Sur touche appuyée)
Se produit lors de l'appui sur une touche ou lors de l'exécution de **SendKeys** ou **EnvoiTouches**.
Il est possible cependant d'empêcher l'objet de recevoir la touche en attribuant la valeur 0 à l'argument **Keycode** de la procédure **KeyDown** de l'objet.

Annulation : Non

KeyPress : (Sur touche activée)
Se produit lors de l'appui sur une touche émettant un caractère ANSI standard ou lors de l'exécution de **SendKeys** ou **EnvoiTouches**.
Il est possible cependant d'empêcher l'objet de recevoir la touche en attribuant la valeur 0 à l'argument **keyAscii** de la procédure **KeyPress** de l'objet.

Annulation : Oui (par macro ou VBA)

KeyUp : (Sur touche relâchée)
Se produit lorsque l'utilisateur relâche la touche ou lors de l'exécution de **SendKeys** ou **EnvoiTouches**.
Il est possible cependant d'empêcher l'objet de recevoir la touche en attribuant la valeur 0 à l'argument **KeyCode** de la procédure **KeyUp** de l'objet.

Annulation : Non

» La propriété **KeyPreview** (aperçu des touches) d'un formulaire détermine si les procédures événementielles de clavier du niveau formulaire sont appelées avant celles du niveau contrôle. Vous pouvez utiliser la propriété **KeyPreview** pour créer une procédure de gestion du clavier au niveau d'un formulaire.

Exemple

L'exemple suivant montre comment afficher un formulaire lorsque l'utilisateur tape sur la touche [F1], et ce quel que soit le contrôle actif.

```
Private Sub Form_Load()
KeyPreview = True
End Sub

Private Sub Form_KeyDown(KeyCode As Integer, Shift As
Integer)
' Si la touche est F1, le formulaire Recherche est
affiché
' KeyCode est réinitialisé pour ne pas afficher l'aide
Access
If KeyCode = vbKeyF1 Then
  DoCmd.OpenForm "Recherche"
  KeyCode = 0
End If
End Sub
```

Événements de type Impression

Ces événements s'appliquent aux états Access.

Format : (Au formatage)
Se produit lorsque Microsoft Access détermine quelles données appartiennent à quelles sections.

Annulation : Oui

Print : (Sur impression)
Se produit lors de la mise en forme des données d'une section pour impression.

Annulation : Oui

Retreat : (Au reformatage)
Se produit lorsque Microsoft Access retourne à une section précédente pendant la mise en forme d'un état.

© ENI Editions - All rigths reserved

Annulation : Non

NoData : (Sur aucune donnée)
Se produit lorsque Microsoft Access a mis en forme un état en vue de son impression et que celui-ci ne contient aucune donnée.

Annulation : Oui

Page : (Sur la page)
Se produit après que Microsoft Access ait mis en forme une page d'un état en vue de son impression, mais avant que la page ne soit imprimée.

Annulation : Non

» Les événements Format, Print et Retreat s'appliquent aux sections des états.

Événements de type Filtre

ApplyFilter : (Sur filtre appliqué)
Se produit lorsque l'utilisateur applique un filtre.

Annulation : Oui

Filter : (Sur filtre)
Se produit lorsque l'utilisateur crée un filtre.

Annulation : Oui

Événements de type Erreur et Minuterie

Error : (Sur erreur)
Se produit lorsqu'une erreur survient.

Annulation : Non

Timer : (Sur minuterie)
Se produit lorsqu'un délai spécifié s'est écoulé.

Annulation : Non

© ENI Editions - All rigths reserved

Annulation d'un événement

Dans certains cas, il est souhaitable de pouvoir annuler un événement. Pour cela, il existe deux façons de procéder :

- en spécifiant le nom d'une macro, contenant l'action AnnulerEvénement, comme paramètre de la propriété de type événement correspondante, par exemple pour annuler la mise à jour d'un contrôle ou pour placer le nom de la macro contenant l'action **AnnulerEvénement** sur sa propriété **Avant MAJ**. L'action **AnnulerEvénement** en Visual Basic utilise la méthode **CancelEvent** de l'objet **Docmd**.

- en donnant la valeur **True** à l'argument **Cancel** d'une procédure événementielle.

Exemple

*Si la date est incorrecte, l'événement **Exit** (Sortie) est annulé : le curseur reste positionné sur le champ.*

```
Private Sub txtDateFin_Exit(Cancel As Integer)

If IsNull(txtDateFin) Then Exit Sub

' La date doit être correcte
If Not IsDate(txtDateFin) Then
  MsgBox "Date incorrecte", vbCritical
  Cancel = True
  Exit Sub
End If

' La date de fin doit être >= date de début
If DateValue(txtDateFin) < DateValue(txtDateDeb) Then
    MsgBox "Date de fin antérieure à la date de début",
vbCritical
  Cancel = True
  Exit Sub
End If

End Sub
```

» Tous les événements ne sont pas annulables (voir tableaux des catégories d'événements précédents).

© ENI Editions - All rigths reserved

Séquences types d'événements

Une action utilisateur ou système génère directement un événement, mais plusieurs événements sous-jacents peuvent être activés.

Voici quelques séquences types d'enchaînements d'événements :

Ouverture d'un formulaire :

Open → Load → Resize → Activate → Current

Fermeture d'un formulaire :

UnLoad → Deactivate → Close

Entrée dans un contrôle :

Enter → GotFocus

Sortie d'un contrôle :

Exit → LostFocus

Passage d'un formulaire à un autre :

Deactivate (form1) → Activate (form2)

Modification d'une zone de texte :

KeyDown → KeyPress → Change → KeyUp

Mise à jour d'un contrôle :

KeyDown → KeyPress → Change → KeyUp → Exit (Zone1) → LostFocus (Zone1) → Enter (Zone2) → GotFocus (Zone2)

Mise à jour d'un enregistrement :

KeyDown → KeyPress → Change → KeyUp → BeforeUpdate (Zone de texte) → AfterUpdate (Zone de texte) → BeforeUpdate (Formulaire) → AfterUpdate (Formulaire)

Introduction dans le premier champ d'un nouvel enregistrement :

KeyDown → KeyPress → BeforeUpdate (Formulaire) → Change → KeyUp

Suppression d'enregistrements :

Delete → Current → BeforeDelConfirm → AfterDelConfirm

© ENI Editions - All rigths reserved

Les événements de mise à jour

Dans Microsoft Access les événements de mise à jour interviennent à deux niveaux :

- mise à jour d'un contrôle,

- mise à jour d'un enregistrement.

Dans le cas d'une mise à jour d'un contrôle, ce sont les événements **BeforeUpdate** et **AfterUpdate** du contrôle lui-même qui sont exécutés.

Dans le cas d'une mise à jour d'enregistrement, ce sont les événements **BeforeUpdate** et **AfterUpdate** du formulaire qui sont exécutés.

Les événements "mise à jour des contrôles" sont exécutés avant les événements "mise à jour du formulaire".

© ENI Editions - All rigths reserved

Les différents types d'erreur

On distingue différents types d'erreur dans le langage VBA :

- les erreurs de syntaxe,
- les erreurs de compilation,
- les erreurs d'exécution,
- les erreurs de logique.

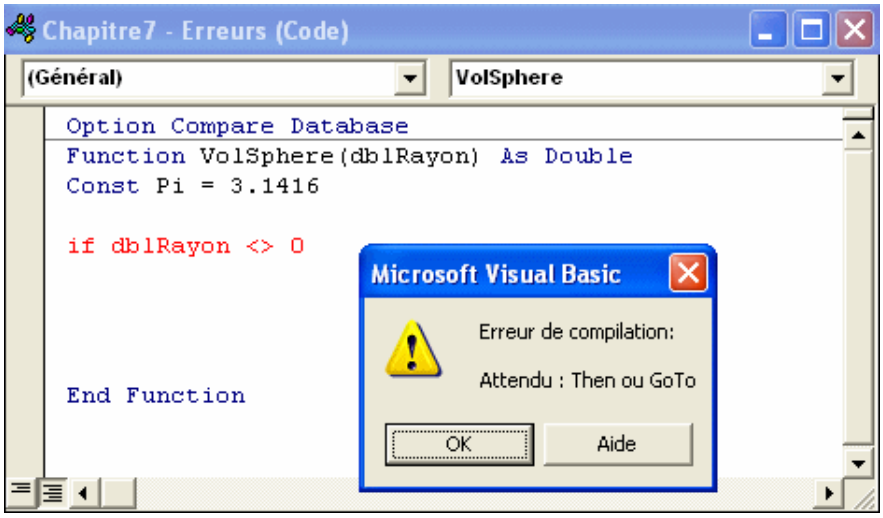
Les erreurs de syntaxe

Les erreurs de syntaxe peuvent être décelées automatiquement lors de la saisie du code dans VBA.

Pour activer la vérification de syntaxe, dans le menu **Outils** :

- Sélectionnez **Options**.
- Puis sélectionnez l'onglet **Éditeur** et cochez la case **Vérification automatique de la syntaxe**.

Exemple



» Les erreurs de syntaxe non corrigées provoqueront une erreur de compilation (cf. message affiché).

Les erreurs de compilation

Les erreurs de compilation sont décelées lorsque Access tente de compiler votre code.

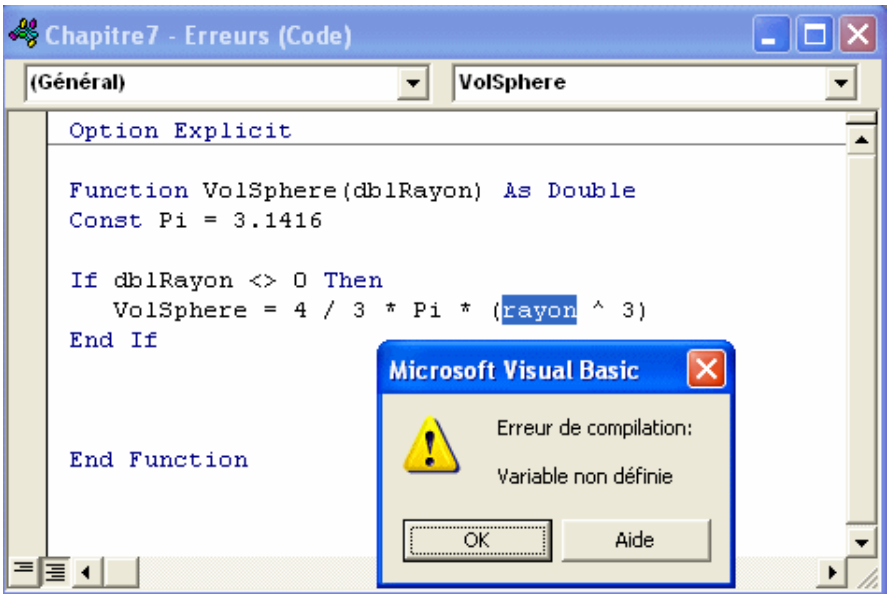
Le code VBA peut être compilé de deux façons :

- à la demande de l'utilisateur en sélectionnant l'option **Compiler** du menu **Débogage**. Dans ce cas le code est entièrement compilé.
- Automatiquement **lors de l'exécution du code**. Dans ce cas, le code contenu dans les procédures n'est compilé que lors du premier appel de la procédure. Les procédures non appelées ne seront pas compilées.

» Il est recommandé de compiler le programme avant de l'exécuter afin de gagner du temps sur la mise au point.

© ENI Editions - All rigths reserved

Exemple



Il est possible d'éviter les erreurs d'exécution dues à des variables non déclarées en utilisant l'instruction **Option Explicit**. Si vous tentez d'utiliser une variable non déclarée, une erreur se produit lors de la compilation.

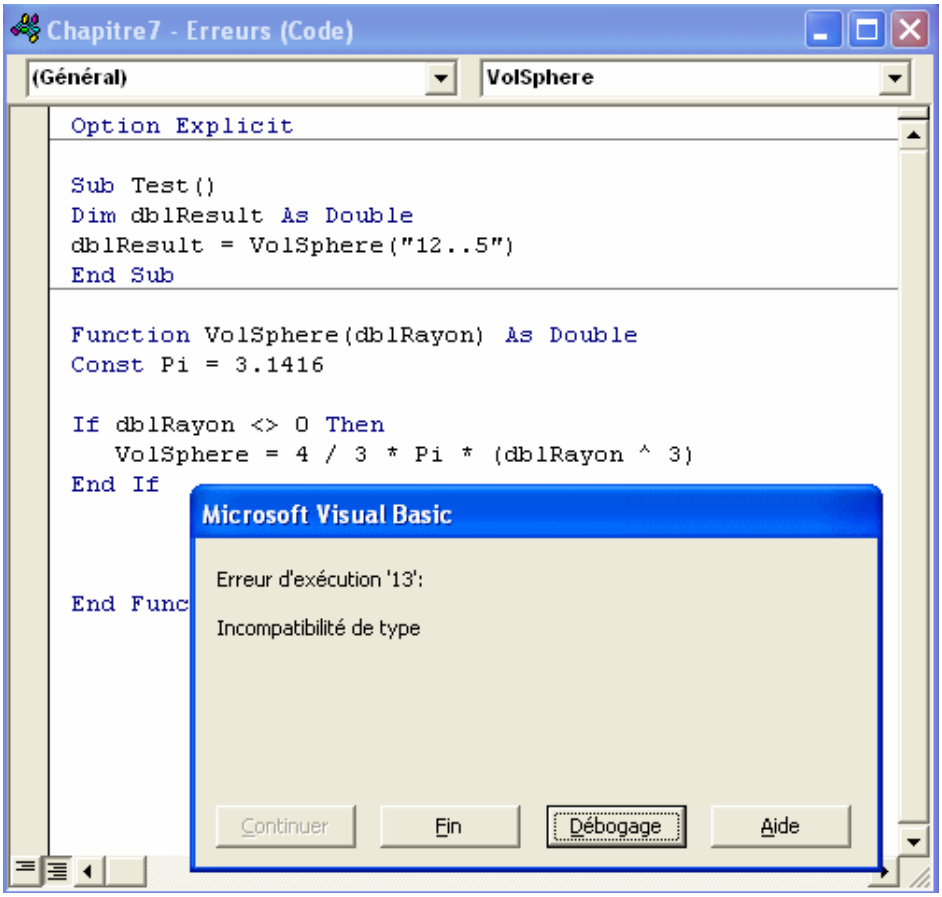
Les erreurs d'exécution

Les erreurs d'exécution sont décelées lorsque Access lance l'exécution de votre code. Une instruction, une opération, un appel de fonction... invalides provoquent une erreur d'exécution. Par exemple, l'utilisation d'une variable non déclarée ou un calcul effectué sur une variable comportant une valeur non numérique peuvent provoquer une erreur d'exécution.

Exemple

La variable dblRayon transmise comme argument contient une valeur non numérique.

© ENI Editions - All rigths reserved



Les erreurs de logique

Les erreurs de logique sont liées à des erreurs de raisonnement ou à une mauvaise traduction d'un raisonnement en code VBA. Par exemple un algorithme de calcul peut produire une erreur de résultat si, lors de sa transcription en VBA, une opération est omise ou mal traduite, ou si l'algorithme est erroné.

Les erreurs de logique sont les plus difficiles à détecter. Elles n'entraînent généralement pas d'erreur d'exécution, mais produisent un résultat différent de celui attendu.

Dans l'exemple cité précédemment, si la valeur de Pi est fausse ou la formule de calcul du volume incorrecte, la valeur retournée par **VolSphere** sera erronée.

© ENI Editions - All rigths reserved

Débogage avec VBA

Présentation

Le mode débogage permet, entre autres, de détecter et de corriger interactivement les erreurs autorisant ainsi la poursuite de l'exécution du programme.

Lorsqu'une erreur d'exécution se produit, un code erreur est affiché, l'exécution du programme est interrompue par Access et le mode débogage est activé.

Avec les différents outils de débogage, on peut :

- exécuter le code pas à pas,

- ajouter des points d'arrêt,

- connaître à tout moment la valeur d'une variable ou d'une expression,

- modifier interactivement le code,

- etc.

La barre d'outils Débogage

La barre d'outils **Débogage** permet d'accéder directement aux différents outils de débogage.



1. **Mode création** : Active ou désactive le mode création.

2. **Exécuter** (**F5**) : Exécute le code de la procédure en cours, la feuille **UserForm** active ou une macro.

3. **Arrêt** (**Ctrl Pause**) : Interrompt l'exécution du programme en cours et passe en mode Arrêt.

4. **Réinitialiser** : Efface le contenu des variables et réinitialise le projet.

5. **Basculer le point d'arrêt** (**F9**) : Définit ou supprime un point d'arrêt sur la ligne en cours ; le code est exécuté jusqu'au point d'arrêt, puis passe en mode débogage.

6. **Pas à pas détaillé** (**F8]** : Exécute le code, instruction par instruction, en parcourant la procédure en cours et des procédures appelées.

7. **Pas à pas principal** (**Shift F8**) : Exécute le code, instruction par instruction, uniquement pour la procédure en cours (les instructions des procédures appelées sont exécutées en continu).

8. **Pas à pas sortant** (**Ctrl Shift F8**) : Exécute en continu les lignes restantes de la procédure en cours.

9. **Fenêtre variable locales** : Affiche les valeurs des variables locales de la procédure.

10. **Fenêtre exécution** (raccourci-clavier **Ctrl G**) : Affiche la fenêtre exécution permettant d'exécuter interactivement une instruction.

11. **Fenêtre espions** : Affiche la liste des variables espions.

12. **Espion express** (**Shift F9**) : Affiche la valeur actuelle de l'expression sélectionnée.

© ENI Editions - All rigths reserved

13. **Pile des appels** : Affiche la liste des appels de procédure dont l'exécution est en cours.

L'objet Debug

L'objet **Debug** permet d'envoyer des données pour affichage dans la fenêtre **Exécution** au moment de l'exécution.

Méthodes

Print : Affiche du texte ou le résultat de l'évaluation d'une expression dans la fenêtre **Exécution**.

Assert : Suspend l'exécution de manière conditionnelle, à la ligne où apparaît la méthode.

Exemple

```
VolSphere = 4 / 3 * Pi * ((dblRayon) ^ 3)
Debug.print dblRayon
Debug.Assert VolSphere > 0
```


© ENI Editions - All rigths reserved

Gestion des erreurs par VBA

Principe

Lorsqu'une erreur d'exécution se produit, un code erreur est généré par Access. La gestion des erreurs consiste à identifier l'erreur en interceptant son code et à la traiter pour, par exemple :

- afficher un message plus explicite que celui défini par le système,

- corriger éventuellement l'erreur.

Le système de gestion des erreurs d'Access offre deux possibilités pour intercepter l'erreur selon l'endroit où elle s'est produite :

- dans la procédure événementielle **Error** d'un formulaire ou d'un état, si l'erreur a été provoquée au niveau de l'interface de Microsoft Access ou du moteur Microsoft Jet,

- en utilisant l'instruction **On Error** si l'erreur a eu lieu dans du code VBA.

» Afin de pouvoir traiter les erreurs, il est nécessaire d'en connaître la liste. La liste des codes erreur est détaillée en Annexe A.

La procédure événementielle Error

La procédure événementielle **Error** possède deux arguments :

```
Sub Form_Error (DataErr As Integer, Response As Integer)
```

`DataErr` : Contient le code de l'erreur.

`Response` : Permet d'afficher (`acDataErrDisplay`) ou non (`acData ErrContinue`) le message d'erreur standard d'Access.

Exemple

L'exemple suivant permet d'intercepter une erreur et de remplacer le message standard d'Access (ex : un index ou une clé principale ne peut contenir de valeur nulle : correspondant au numéro d'erreur 3058), par un message spécifique.

```
Sub Form_Error (DataErr As Integer,Response As Integer)
  'Index Null
  If DataErr = 3058 Then
    MsgBox "Un employé doit avoir un numéro"
     'le message standard d'Access ne s'affiche pas
    Response = acDataErrContinue
  End If
End Sub
```

L'instruction On Error

On Error GoTo

L'instruction **On Error GoTo** indique une séquence d'instructions à exécuter en cas d'erreur. Cette instruction doit être placée avant les instructions pouvant déclencher une erreur.

Syntaxe

```
On Error {GoTo <étiquette>| GoTo 0}
```

`<étiquette>` : Nom de l'étiquette indiquant le début de la séquence d'instructions. Une étiquette est composée d'un nom commençant par une lettre et du signe deux-points (:).
Ex : Gest_erreur :
La séquence d'instructions appelée pour gérer les erreurs est placée en général à la fin de la procédure ou de la

© ENI Editions - All rigths reserved

fonction et précédée d'une instruction **Exit Sub** ou **Exit Function** afin d'éviter son exécution s'il n'y a pas d'erreur.

`0` : Désactive le traitement d'erreurs.

D'autre part, dans la séquence d'instructions de traitement d'erreurs, il est nécessaire d'indiquer à quel endroit doit reprendre l'exécution de la procédure ou fonction après le traitement de l'erreur. Pour cela, il faut utiliser l'une des instructions suivantes :

`Resume` : Reprend l'exécution du programme à l'instruction qui a provoqué l'erreur.

`Resume Next` : Reprend l'exécution du programme à l'instruction qui suit celle qui a provoqué l'erreur.

`Resume <étiquette>` : Reprend l'exécution du programme à l'étiquette définie.

Exemple

```
Sub MaProcédure()
  On Error GoTo Erreur
  ...
  Exit Sub
  Erreur:
     Select Case Err
        Case 11
        'Division par Zéro
           MsgBox ...
           ...
           Resume Next
        Case 13
        'Types incompatibles
           ...
     End Select
End Sub
```

L'objet Err

L'objet **Err** contient des informations permettant de connaître l'origine de l'erreur.

Propriétés

Description : Renvoie une chaine de caractères expliquant l'origine de l'erreur.

HelpContext : Renvoie l'identificateur de contexte associé à une rubrique d'un fichier d'aide.

HelpFile : Renvoie une chaîne de caractères contenant le chemin d'accès complet au fichier d'aide.

LastDLLError : Renvoie un code d'erreur système produit par un appel à une bibliothèque de liaisons dynamiques.

Number : Renvoie une valeur numérique indiquant le numéro de l'erreur.

Source : Renvoie une chaîne de caractères contenant le nom de l'objet ou de l'application qui a généré l'erreur.

Méthodes

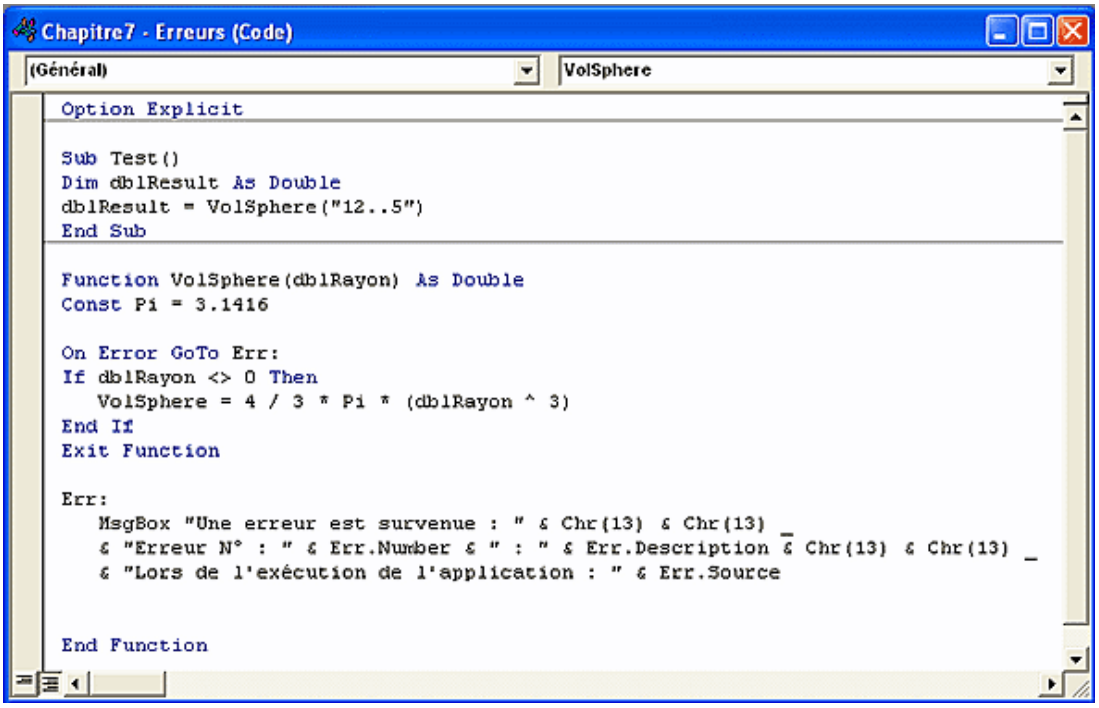
Clear : Efface de manière explicite le contenu de l'objet **Err**.

Raise : Permet de générer des erreurs d'exécution.

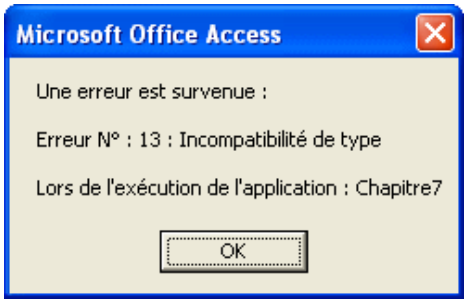
Exemple

Le code suivant permet d'afficher un message donnant des informations sur la nature de l'erreur.

© ENI Editions - All rigths reserved



Exemple de boîte de message renvoyée lorsqu'une erreur se produit :



© ENI Editions - All rigths reserved

Présentation

Le langage VBA permet de personnaliser vos formulaires et états lors de leur exécution. Vous pourrez, par exemple, effectuer les opérations suivantes :

- Modifier l'apparence d'un contrôle dans un formulaire ou un état (couleur, taille, position...).

- Ajouter ou supprimer des contrôles dans un formulaire ou un état.

- Enchaîner des formulaires à partir de boutons de commandes.

- Afficher ou imprimer un état,

- Appliquer des filtres sur les enregistrements d'un état ou d'un formulaire.

- Etc.

Les assistants "Boutons de commande" et "Zones de liste déroulante" génèrent automatiquement du code VBA dans des procédures événementielles. Ils constituent des premiers exemples simples de personnalisation de formulaires et d'états.

» Vous trouverez d'autres exemples de personnalisation de formulaires et d'états dans la "mini application" décrite au chapitre **Code d'une mini-application**.

© ENI Editions - All rigths reserved

Personnalisation des formulaires

Un objet **Form** fait partie de la collection **Forms** qui représente tous les formulaires actuellement ouverts. L'objet **ActiveForm** (**Screen.ActiveForm**) et l'objet **Me** désignent le formulaire actif.

» N'utilisez le mot réservé **Me** pour représenter un objet **Form** ou **Report** dans du code que si vous faites référence à ce formulaire ou état à partir du code dans le module de classe. Si vous y faites référence à partir d'un module standard ou d'un autre module du formulaire ou de l'état, vous devez utiliser la référence complète du formulaire ou de l'état.

Syntaxes

Pour faire référence à un formulaire, vous disposez des différentes syntaxes, permettant de faire référence à un objet d'une collection, suivantes :

```
Forms!<Nomformulaire>
    ex: Forms!Commandes
Forms![<Nomformulaire>]
    ex: Forms![Commandes]
Forms("<Nomformulaire>")
    ex: Forms("Commandes")
Forms(<Index>)
    ex: Forms(0)
```

» La syntaxe `Forms![<Nomformulaire>]` est la plus couramment utilisée.

» Si un nom de formulaire contient un espace, la première syntaxe ne fonctionne pas.
Utilisez toujours la même syntaxe afin de faciliter la lecture du code.
Il est préférable de référencer un formulaire spécifique par son nom plutôt que par son index, car ce dernier peut changer.

Pour faire référence à un sous-formulaire à partir d'un formulaire, vous devez utiliser le nom du contrôle contenant le sous-formulaire suivi du mot clé Form.

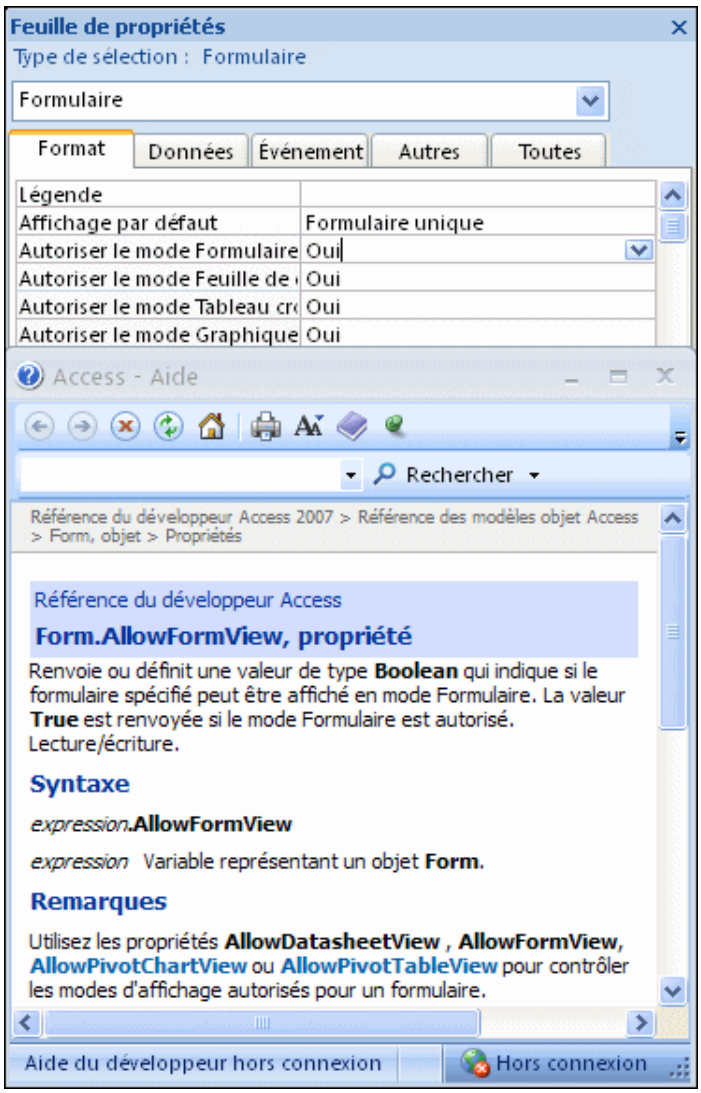
Exemple

```
[sFrmParticipant].Form!txtNbStag
[sFrmFormations].Form.RecordSource = strSQL
[sFrmFormations].Form.Requery
```

Propriétés disponibles en mode Création

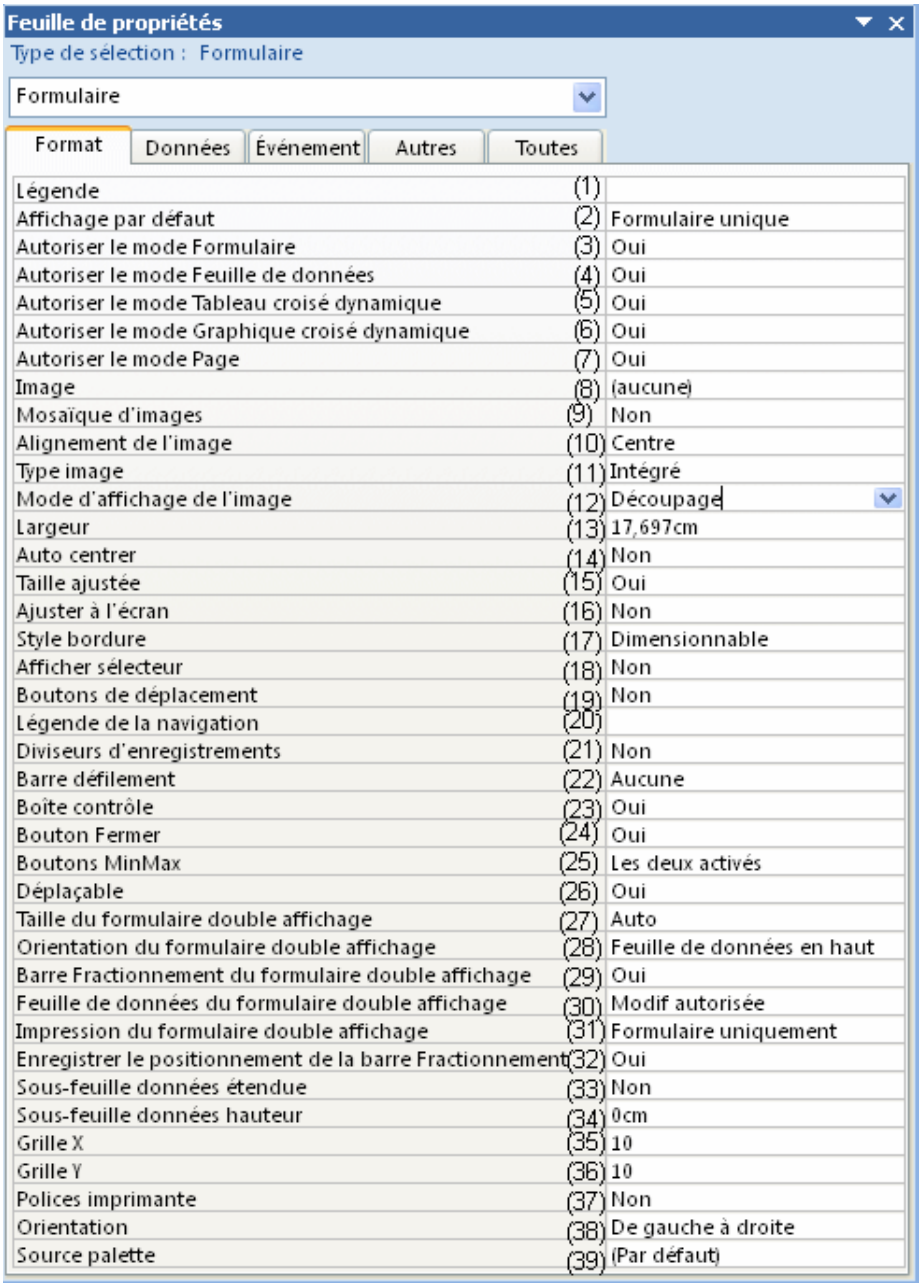
Pour connaître le nom VBA et la description d'une propriété disponible en mode création, vous devez vous positionner dans la fenêtre **Propriétés** de l'objet **Formulaire** et cliquer sur la touche **F1**.

© ENI Editions - All rigths reserved



Propriétés de l'onglet Format

© ENI Editions - All rigths reserved



1. Caption — Chaîne de caractères
2. DefaultView — Octet
   0 Formulaire unique (mode simple)
   1 Formulaires continus (mode continu)
   2 Feuille de données
   3 Tableau croisé dynamique
   4 Graphique croisé dynamique
   5 Formulaire double affichage
3. AllowFormView — Booléen
4. AllowDatasheetView — Booléen
5. AllowPivotTableView — Booléen
6. AllowPivotChartView — Booléen
7. AllowLayoutView — Booléen
8. Picture — Chaîne de caractères

© ENI Editions - All rigths reserved

9. PictureTiling — Booléen
10. PictureAlignment — Octet
    - 0 Haut à gauche
    - 1 Haut à droite
    - 2 Centré
    - 3 Bas à gauche
    - 4 Bas à droite
    - 5 Centre du formulaire
11. PictureType — Octet
    - 0 Intégré (ou incoroporé)
    - 1 Attaché (ou lié)
12. PictureSizeMode — Octet
    - 0 Découpage
    - 1 Echelle
    - 2 Zoom
13. Width — Réel simple
14. AutoCenter — Booléen
15. AutoResize — Booléen
16. FitToScreen — Booléen
17. BorderStyle — Octet
    - 0 Aucun
    - 1 Fin
    - 2 Dimensionnable
    - 3 Trait double fixe
18. RecordSelectors — Booléen
19. NavigationButtons — Booléen
20. NavigationCaption — Booléen
21. DividingLines — Booléen
22. ScrollBars — Octet
    - 0 Aucun
    - 1 Horizontale
    - 2 Verticale
    - 3 Les deux
23. ControlBox — Booléen
24. CloseButton — Booléen
25. MinMaxButtons — Octet
    - 0 Aucun
    - 1 Min activé
    - 2 Max activé
    - 3 Les deux activés
26. Moveable — Booléen
27. SplitFormSize — Entier long
28. SplitFormOrientation — Octet
    - 0 En haut
    - 1 En bas

© ENI Editions - All rigths reserved

2 A gauche

3 A droite

29. SplitFormSplitterBar — Booléen

30. SplitFormDatasheet — Octet

0 Modif autorisée

1 Lecture seule

31. SplitFormPrinting — Octet

0 Formulaire uniquement

1 Feuille de données uniquement

32. SplitFormSplitterBarSave — Booléen

33. SubdatasheetExpanded — Booléen

34. SubdatasheetHeight — Booléen

35. GridX — Entier

36. GridY — Entier

37. LayoutForPrint — Booléen

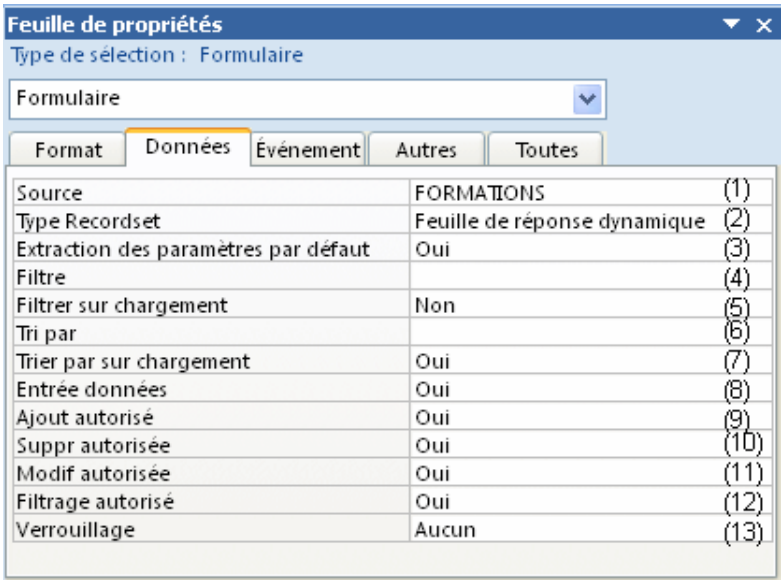
38. Orientation

0 De gauche à droite

1 De droite à gauche

39. PaletteSource — Chaîne de caractères

Propriétés de l'onglet Données



1. RecordSource — Chaîne de caractères

2. TypeRecordSet — Octet

0 Feuille de réponse dynamique

1 Feuille rép. Dyn (MAJ globale)

2 Instantané

3. FetchDefaults — Booléen

4. Filter — String

5. FilterOnLoad — Booléen

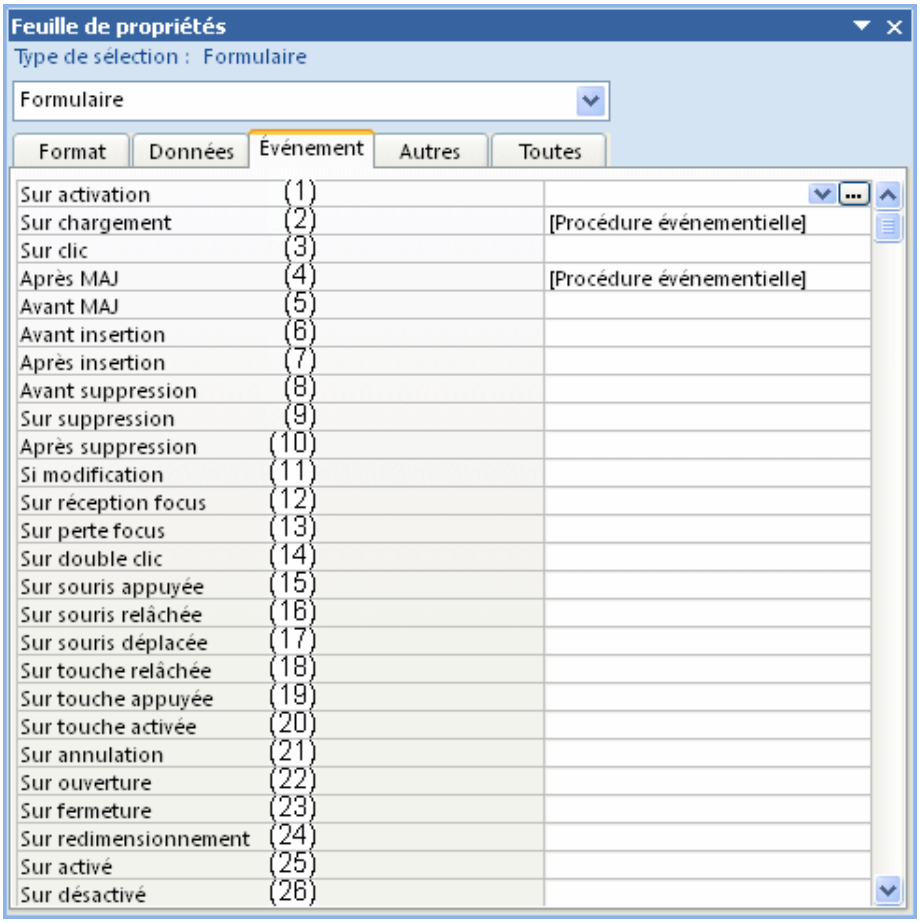
6. OrderBy — String

7. OrderByOnLoad — Booléen

© ENI Editions - All rigths reserved

8.	DataEntry	Booléen
9.	AllowAdditions	Booléen
10.	AllowDeletions	Booléen
11.	AllowEdits	Booléen
12.	AllowFilters	Booléen
13.	RecordLocks	Booléen

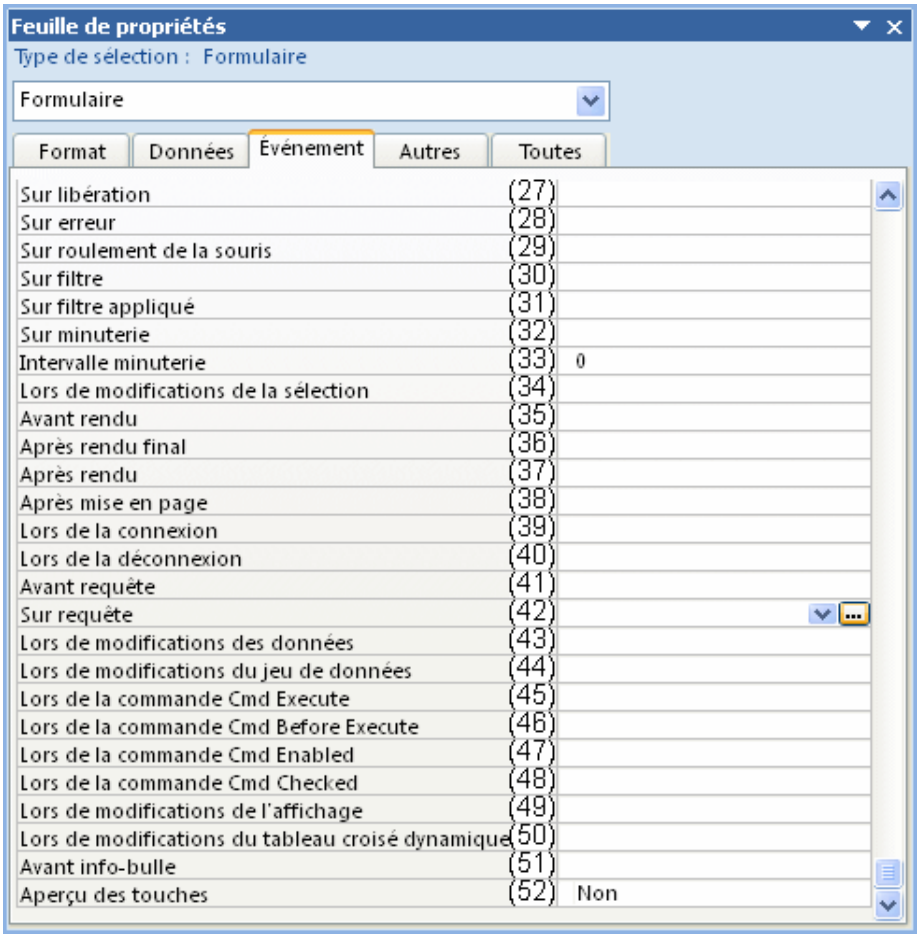
Propriétés de l'onglet Événement



1.	OnCurrent	Chaîne de caractères
2.	OnLoad	Chaîne de caractères
3.	OnClick	Chaîne de caractères
4.	BeforeUpdate	Chaîne de caractères
5.	AfterUpdate	Chaîne de caractères
6.	BeforeInsert	Chaîne de caractères
7.	AfterInsert	Chaîne de caractères
8.	BeforeDelConfirm	Chaîne de caractères
9.	OnDelete	Chaîne de caractères
10.	AfterDelConfirm	Chaîne de caractères
11.	OnDirty	Chaîne de caractères
12.	OnGotFocus	Chaîne de caractères
13.	OnLostFocus	Chaîne de caractères
14.	OnDblClick	Chaîne de caractères

© ENI Editions - All rigths reserved

15.	OnMouseDown	Chaîne de caractères
16.	OnMouseUp	Chaîne de caractères
17.	OnMouseMove	Chaîne de caractères
18.	OnKeyUp	Chaîne de caractères
19.	OnKeyDown	Chaîne de caractères
20.	OnKeyPress	Chaîne de caractères
21.	OnUndo	Chaîne de caractères
22.	OnOpen	Chaîne de caractères
23.	OnClose	Chaîne de caractères
24.	OnResize	Chaîne de caractères
25.	OnActivate	Chaîne de caractères
26.	OnDeactivate	Chaîne de caractères

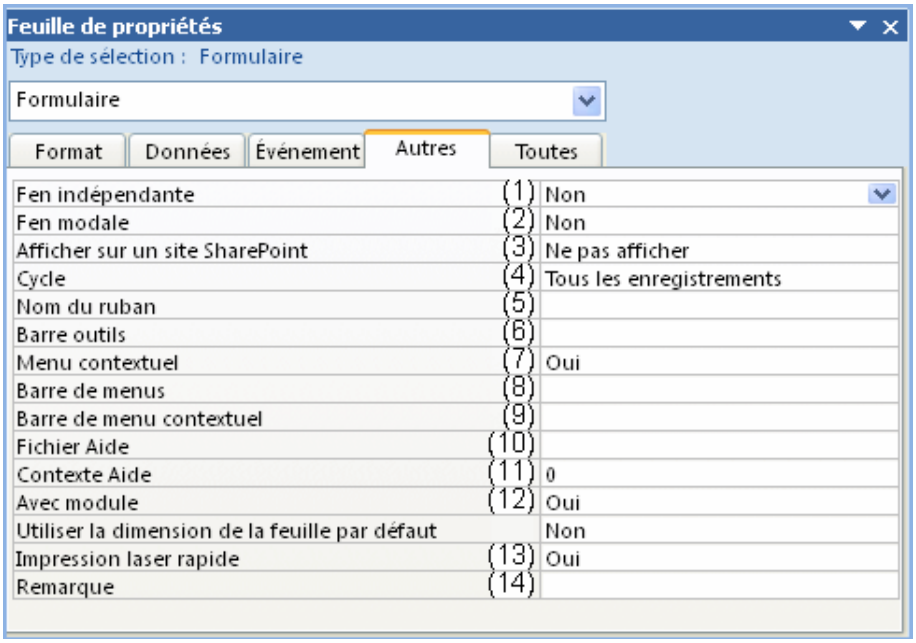


27.	OnUnload	Chaîne de caractères
28.	OnError	Chaîne de caractères
29.	MouseWheel	Chaîne de caractères
30.	OnFilter	Chaîne de caractères
31.	OnApplyFilter	Chaîne de caractères
32.	OnTimer	Chaîne de caractères
33.	TimerInterval	Chaîne de caractères
34.	SelectionChange	Chaîne de caractères
35.	BeforeRender	Chaîne de caractères
36.	AfterFinalRender	Chaîne de caractères

© ENI Editions - All rigths reserved

37. AfterRender — Chaîne de caractères
38. AfterLayout — Chaîne de caractères
39. OnConnect — Chaîne de caractères
40. OnDisconnect — Chaîne de caractères
41. BeforeQuery — Chaîne de caractères
42. Query — Chaîne de caractères
43. DataChange — Chaîne de caractères
44. DataSetChange — Chaîne de caractères
45. CommandExecute — Chaîne de caractères
46. CommandBeforeExecute — Chaîne de caractères
47. CommandChecked — Chaîne de caractères
48. DataSetChange — Chaîne de caractères
49. ViewChange — Chaîne de caractères
50. PivotTableChange — Chaîne de caractères
51. BeforeScreenTip — Chaîne de caractères
52. KeyPreview — Chaîne de caractères

Propriétés de l'onglet Autres



1. Popup — Booléen
2. Modal — Booléen
3. DisplayOnSharePointSite — Octet
   0 Ne pas afficher
   1 Suivre le paramètre de la table
4. Cycle — Octet
   0 Tous les enregistrements
   1 Enregistrement en cours
   2 Page en cours
5. RibbonName — Chaîne de caractères
6. ToolBar — Chaîne de caractères

© ENI Editions - All rigths reserved

7.	ShortcutMenu	Booléen
8.	MenuBar	Chaîne de caractères
9.	ShortcutMenuBar	Chaîne de caractères
10.	HelpFile	Chaîne de caractères
11.	HelpContextID	Entier long
12.	HasModule	Booléen
13.	FastLaserPrinting	Booléen
14.	Tag	Chaîne de caractères

Propriétés non disponibles en mode Création

Propriétés se rapportant au mode d'affichage du formulaire

CurrentView : Retourne le mode d'affichage en cours du formulaire.

OpenArgs : Retourne l'expression chaîne spécifiée dans l'argument **OpenArgs** de la méthode **OpenForm** qui a ouvert le formulaire.

Page : Spécifie le numéro de la page en cours.

Section : Identifie une section et permet d'accéder aux propriétés de cette section.

SelHeight : Spécifie ou détermine le nombre de lignes sélectionnées (enregistrements) dans le rectangle de sélection en cours.

SelLeft : Spécifie ou détermine quelle colonne (champ) figure le plus à gauche dans le rectangle de sélection en cours.

SelTop : Spécifie ou détermine quelle ligne (enregistrement) figure tout en haut du rectangle de sélection.

SelWidth : Spécifie ou détermine le nombre de colonnes (champ) sélectionnées dans le rectangle de sélection en cours.

Propriétés se rapportant aux enregistrements et à leur mise à jour

BatchUpdates : Indique si le formulaire spécifié prend en charge les mises à jour par lots de transactions.

BookMark : Définit un signet qui identifie de façon unique un enregistrement.

CurrentRecord : Permet d'identifier l'enregistrement en cours.

Dirty : Permet de déterminer si l'enregistrement en cours a subi des modifications depuis sa dernière sauvegarde.

NewRecord : Détermine si l'enregistrement en cours est un nouvel enregistrement.

RecordSet : Détermine ou renvoie l'objet DAO **Recordset** représentant la source d'enregistrement d'un formulaire.

RecordSourceQualifer : Renvoie ou définit une chaîne de caractères indiquant le nom du propriétaire de la source d'enregistrements SQL Server pour le formulaire.

Propriétés se rapportant à la présentation du formulaire

CurrentSectionLeft : Détermine la distance en **twips** entre le coin supérieur gauche de la section en cours et le coin supérieur gauche du formulaire (twip : unité de mesure équivalente à 1/20è de point ou 1/1440è de pouce. Un centimètre contient 567 twips).

CurrentSectionTop : Représente la distance en **twips** entre le coin supérieur de la section en cours et le coin

© ENI Editions - All rigths reserved

supérieur du formulaire.

DataSheetBackColor : Spécifie ou détermine la couleur d'arrière-plan de la totalité d'une feuille de données.

DatasheetBorderLineStyle : Indique le style de trait à utiliser pour la bordure de la feuille de données.

DatasheetCellsEffect : Permet de spécifier si les effets spéciaux s'appliquent aux cellules d'une feuille de données.

DatasheetColumnHeaderUnderlineStyle : Indique le style de trait à utiliser pour le bord inférieur des en-têtes de colonne de la feuille de données.

DataSheetFontHeight : Spécifie la taille en points utilisée pour afficher et imprimer les noms des champs et les données en mode Feuille de données.

DataSheetFontItalic : Met en italique les noms de champs et les données en mode Feuille de données.

DataSheetFontName : Spécifie la police utilisée pour afficher et imprimer les noms des champs et les données en mode Feuille de données.

DataSheetFontUnderline : Souligne les noms de champs et les données en mode Feuille de données.

DataSheetFontWeight : Spécifie l'épaisseur de la police utilisée pour afficher et imprimer les caractères des noms de champs et des données en mode Feuille de données.

DataSheetForeColor : Spécifie ou détermine la couleur du texte de la totalité d'une Feuille de données.

DatasheetGridlinesBehavior : Permet de spécifier le type de quadrillage devant apparaître en mode Feuille de données.

DatasheetGridlinesColor : Permet de spécifier la couleur du quadrillage d'une feuille de données.

HorizontalDatasheetGridlineStyle : Indique le quadrillage horizontal à utiliser pour la bordure de la feuille de données.

InsideHeight : Détermine la hauteur (en twips) de la fenêtre contenant un formulaire.

InsideWidth : Détermine la largeur (en twips) de la fenêtre contenant un formulaire.

Pages : Indique le nombre total de pages.

Painting : Permet de spécifier si le formulaire doit être redessiné.

PaintPalette : Spécifie la palette devant être utilisée par le formulaire.

PictureData : Permet de copier l'image du formulaire dans un autre objet qui reconnaît la propriété Image (**Picture**).

VerticalDatasheetGridlineStyle : Indique le quadrillage vertical à utiliser pour la bordure de la feuille de données.

WindowHeight : Indique la hauteur d'un formulaire exprimée en twips.

WindowLeft : Indique la position à l'écran, en twips, du bord gauche du formulaire.

WindowTop : Indique la position à l'écran, en twips, du bord supérieur du formulaire.

WindowWidth : Renvoie la largeur d'un formulaire exprimée en twips.

Propriétés renvoyant un objet

© ENI Editions - All rigths reserved

Application : Permet d'accéder à l'objet Application d'Access.

ChartSpace : Renvoie un objet ChartSpace qui représente l'espace graphique.

Form : Permet de faire référence un formulaire associé à un contrôle sous-formulaire.

Module : Permet de spécifier un module de formulaire. Les méthodes InsertLines, DeleteLines et ReplaceLine de l'objet Module permettent de modifier le contenu d'un module de code.

Parent : Permet de faire référence à l'objet père.

PivotTable : Renvoie un objet **PivotTable** qui représente le conteneur d'une liste de tableau croisé dynamique.

Printer : Renvoie ou définit un objet Printer qui représente l'imprimante par défaut du système actuel.

RecordsetClone : Permet de faire référence à l'objet **Recordset** du formulaire.

Propriétés relatives à l'impression du formulaire

PrtDevMode : Permet de définir ou d'obtenir des informations relatives au mode du périphérique d'impression spécifié.

PrtDevNames : Permet de définir ou d'obtenir des informations relatives à l'imprimante choisie.

PrtMip : Définit ou retourne des informations sur le mode du périphérique spécifié.

UseDefaultPrinter : Indique si le formulaire utilise l'imprimante par défaut du système.

Méthodes des objets Form

Méthodes

GotoPage : Déplace le focus sur le premier contrôle de la page indiquée.

Move : Déplace et/ou redimensionne le formulaire aux coordonnées indiquées par les valeurs des arguments.

Recalc : Recalcule immédiatement les contrôles du formulaire.

Refresh : Met à jour les données du formulaire afin qu'elles contiennent les dernières modifications éventuellement effectuées par d'autres utilisateurs.

Repaint : Effectue toutes les mises à jour en attente du formulaire.

Requery : Met à jour les données du formulaire spécifié.

SetFocus : Donne le focus au formulaire.

UnDo : Restaure un formulaire tel qu'il était avant d'être modifié.

Exemple

Affichage de la fiche Client en fonction du nom de la Société sélectionnée dans une zone de liste.

```
Private Sub lstRechCli_AfterUpdate()
Dim rs As Object

' Recherche l'enregistrement correspondant
' au client sélectionné
Set rs = Me.Recordset.Clone
rs.FindFirst "[Cli_CodeCLi] = '" & Me![lstRechCli] & "'"
```

© ENI Editions - All rigths reserved

```
If Not rs.EOF Then Me.Bookmark = rs.Bookmark

End Sub
```

Affectation d'un filtre en fonction du contenu d'une case à cocher : sélection des clients français, étrangers ou de tous les clients.

```
Private Sub chkPays_AfterUpdate()

Select Case chkPays

  ' Clients français
   Case 1
    Me.FilterOn = True
    Me.Filter = "[Cli_Pays] = 'France'"
    Me.Refresh

   ' Clients hors France
  Case 2
    Me.FilterOn = True
    Me.Filter = "[Cli_Pays] <> 'France'"
    Me.Refresh
  ' Tous les clients
  Case 3
    Me.FilterOn = False
    Me.Refresh

End Select
End Sub
```


© ENI Editions - All rigths reserved

Personnalisation des états

Un objet **Report** fait partie de la collection **Reports** qui représente tous les états actuellement ouverts. L'objet **ActiveReport** (**Screen.ActiveReport**) et l'objet **Me** désignent l'état actif.

Syntaxes

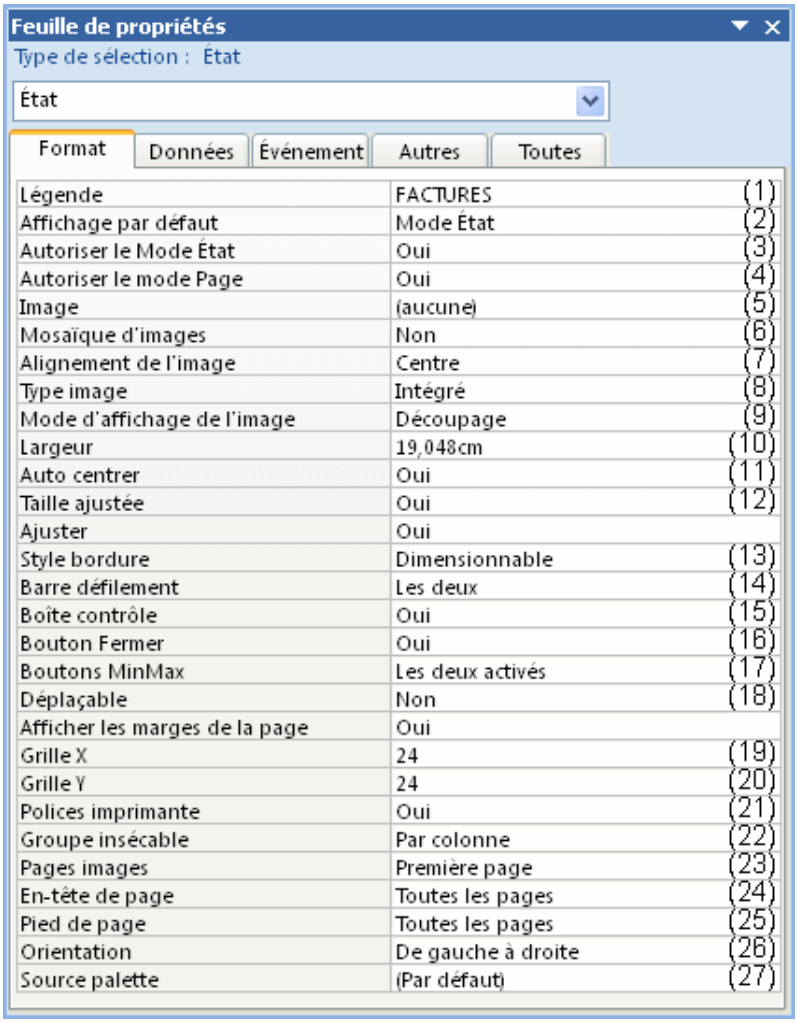
Pour faire référence à un état, vous disposez des mêmes syntaxes que pour les formulaires.

Exemple

```
Reports![ListeCde]
Reports("ListeCde")
Reports![ListeCde]![DateCde]
```

Propriétés disponibles en mode Création

Propriétés de l'onglet Propriétés



1. Caption — Chaîne de caractères
2. DefaultView — Octet
   - 0 Mode simple
   - 1 Mode continu
   - 2 Feuille de données
   - 3 Tableau croisé dynamique

© ENI Editions - All rigths reserved

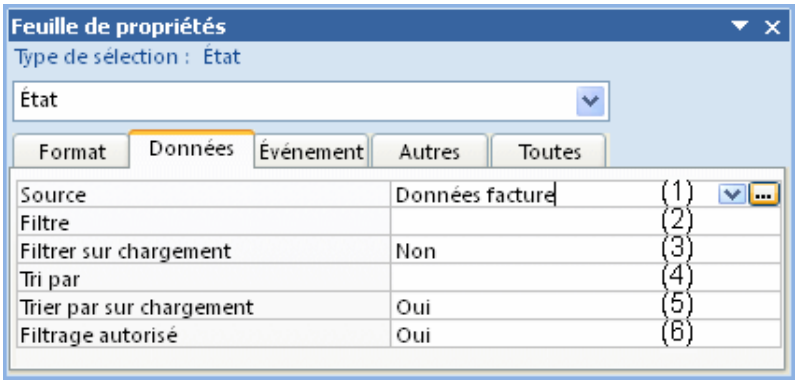
4 Graphique croisé dynamique
5 Formulaire double affichage

3. AlloReportView — Booléen
4. AllowLayoutView — Booléen
5. Picture — Chaîne de caractères
6. PictureTiling — Booléen
7. PictureAlignement — Octet
   0 Supérieur gauche
   1 Supérieur droit
   2 Centré
   3 Inférieur gauche
   4 Inférieur droit
   5 Centre formulaire
8. PictureType — Octet
   0 Intégré (ou incorporé)
   1 Attaché (ou lié)
9. PictureSizeMode — Octet
   0 Découpage
   1 Echelle
   2 Zoom
10. Width — Entier
11. AutoCenter — Booléen
12. AutoResize — Booléen
13. BorderStyle — Octet
   0 Aucun
   1 Fin
   2 Dimensionnable
   3 Trait double fixe
14. ScrollBars — Octet
   0 Aucun
   1 Horizontale
   2 Verticale
   3 Les deux
15. ControlBox — Booléen
16. CloseButton — Booléen
17. MinMaxButtons — Octet
   0 Aucun
   1 Min activé
   2 Max activé
   3 Les deux activés
18. Moveable — Booléen
19. GridX — Entier
20. GridY — Entier
21. LayoutForPrint — Booléen
22. GrpKeepTogether — Octet

© ENI Editions - All rigths reserved

	0 Par page	
	1 Par colonne	
23.	PicturePages	Octet
	0 Toutes les pages	
	1 Première page	
	2 Pas de page	
24.	PageHeader	Octet
	0 Toutes les pages	
	1 Sans en-tête d'état	
	2 Sans pied d'état	
	3 Sans en-tête/pied d'état	
25.	PageFooter	Octet
	0 Toutes les pages	
	1 Sans en-tête d'état	
	2 Sans pied d'état	
	3 Sans en-tête/pied d'état	
26.	Orientation	Octet
	0 De gauche à droite	
	1 De droite à gauche	
27.	PaletteSource	Chaine de caractères

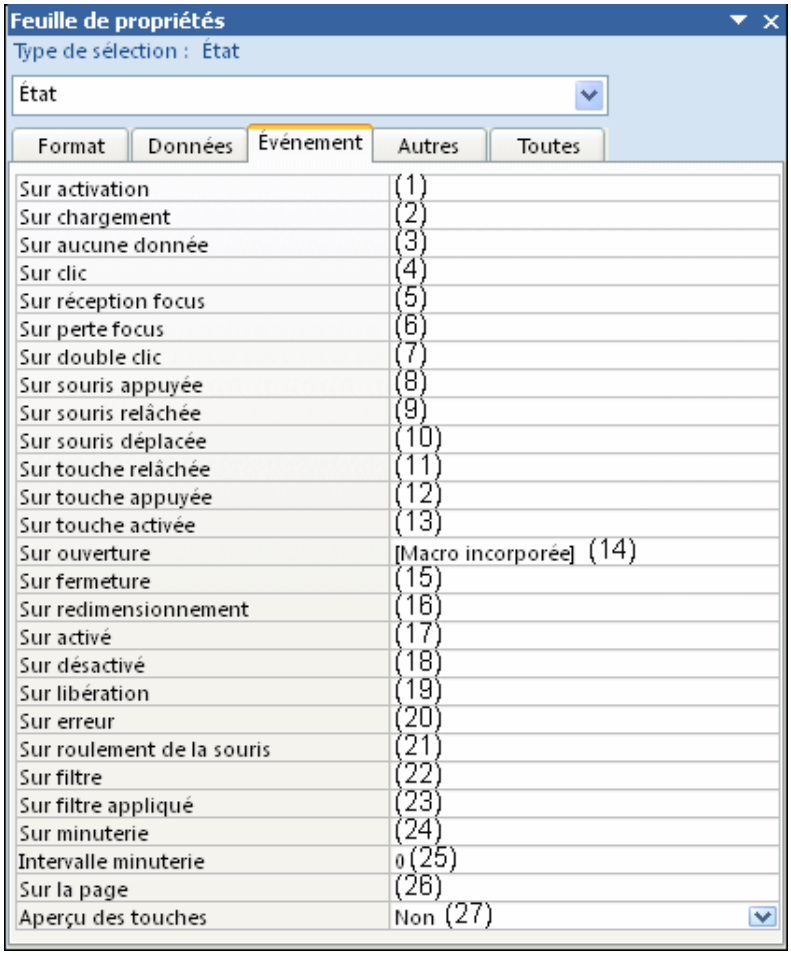
Propriétés de l'onglet Données



1.	RecordSource	Chaîne de caractères
2.	Filter	Chaîne de caractères
3.	FilterOnLoad	Booléen
4.	OrderBy	String
5.	OrderByOnLoad	Booléen
6.	AllowFilters	Booléen

Propriétés de l'onglet Événement

© ENI Editions - All rigths reserved

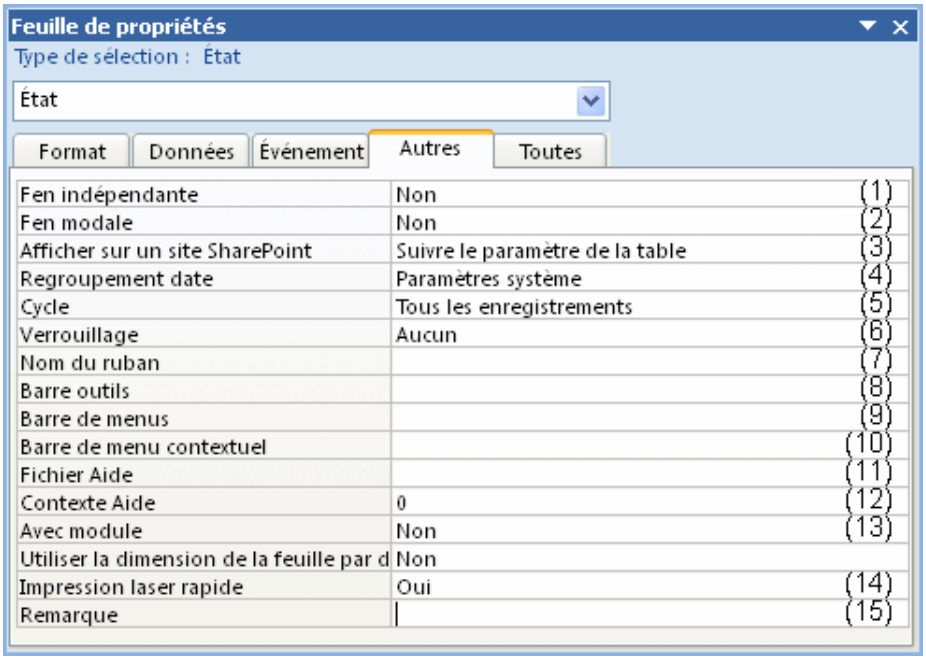


1. OnCurrent — Chaîne de caractères
2. OnLoad — Chaîne de caractères
3. OnNoData — Chaîne de caractères
4. OnClick — Chaîne de caractères
5. OnGotFocus — Chaîne de caractères
6. OnLostFocus — Chaîne de caractères
7. OnDblClick — Chaîne de caractères
8. OnMouseDown — Chaîne de caractères
9. OnMouseUp — Chaîne de caractères
10. OnMouseMove — Chaîne de caractères
11. OnKeyUp — Chaîne de caractères
12. OnKeyDown — Chaîne de caractères
13. OnKeyPress — Chaîne de caractères
14. OnOpen — Chaîne de caractères
15. OnClose — Chaîne de caractères
16. OnResize — Chaîne de caractères
17. OnActivate — Chaîne de caractères
18. OnDeactivate — Chaîne de caractères
19. OnUnload — Chaîne de caractères
20. OnError — Chaîne de caractères
21. MouseWheel — Chaîne de caractères
22. OnFilter — Chaîne de caractères

© ENI Editions - All rigths reserved

23.	OnApplyFilter	Chaîne de caractères
24.	OnTimer	Chaîne de caractères
25.	TimerInterval	Chaîne de caractères
26.	OnPage	Chaîne de caractères
27.	KeyPreview	Booléen

Propriétés de l'onglet Autres



1. PopUp — Booléen
2. Modal — Booléen
3. DisplayOnSharePointSite — Octet
   - 0 Ne pas afficher
   - 1 Suivre le paramètre de la table
4. DateGrouping
   - 0 Paramètres US
   - 1 Paramètres système
5. Cycle
   - 0 Tous les enregistrements
   - 1 Enregistrement en cours
   - 2 Page en cours
6. RecordLocks — Booléen
7. RibbonName — Chaîne de caractères
8. ToolBar — Chaîne de caractères
9. MenuBar — Chaîne de caractères
10. ShortcutMenuBar — Chaîne de caractères
11. HelpFile — Chaîne de caractères
12. HelpContextID — Entier long
13. HasModule — Booléen
14. FastLaserPrinting — Booléen
15. Tag — Chaîne de caractères

© ENI Editions - All rigths reserved

Propriétés non disponibles en mode création

Propriétés se rapportant au mode d'affichage de l'état

Moveable : Indique si l'état peut être déplacé par l'utilisateur.

Page : Indique le numéro de page en cours.

Propriétés se rapportant aux enregistrements et à leur mise à jour.

CurrentRecord : Identifie l'enregistrement en cours.

Dirty : Permet de déterminer si l'enregistrement en cours a subi des modifications depuis sa dernière sauvegarde.

HasData : Indique si l'état est dépendant d'un jeu d'enregistrements vide.

RecordSourceQualifier : Renvoie ou définit une chaîne de caractères indiquant le nom du propriétaire de la source d'enregistrements SQL Server pour l'état

Propriétés se rapportant à la présentation de l'état

DrawMode : Spécifie comment le crayon (la couleur utilisée pour le dessin) affecte les couleurs d'arrière-plan sur l'état lorsque vous utilisez les méthodes **Line**, **Circle** ou **PSet** pour dessiner sur un état à imprimer.

DrawStyle : Spécifie le style de trait lorsque vous utilisez les méthodes **Line** et **Circle** pour imprimer des traits sur un état.

DrawWidth : Spécifie la largeur de trait pour les méthodes **Line**, **Circle** et **PSet** lorsque vous imprimez des lignes sur un état.

FillColor : Permet de spécifier la couleur de la trame des rectangles et des cercles dessinés dans un état à l'aide des méthodes **Line** et **Circle**.

FillStyle : Spécifie si un cercle ou un trait dessiné à l'aide de la méthode **Circle** ou **Line** est transparent, opaque ou avec un motif de trame.

FontBold : Spécifie si une police doit apparaître en gras lors de l'impression de contrôles d'états ou lors de l'utilisation de la méthode Print dans un état.

Left : Définit la position gauche de l'état.

Painting : Permet de spécifier si l'état doit être redessiné.

PaintPalette : Spécifie la palette devant être utilisée par l'état.

ScaleLeft : Spécifie les unités des coordonnées horizontales représentant le bord gauche d'une page lorsque vous utilisez les méthodes **Circle**, **Line**, **PSet** ou **Print** pour l'impression ou l'aperçu d'un état ou pour son enregistrement sous forme de fichier.

ScaleMode : Spécifie l'unité de mesure pour les coordonnées d'une page quand vous utilisez les méthodes **Circle**, **Line**, **PSet** ou **Print** pour l'impression ou l'aperçu d'un état ou son enregistrement sous forme de fichier.

ScaleTop : Spécifie les unités des coordonnées verticales représentant le bord supérieur d'une page lorsque vous utilisez les méthodes **Circle**, **Line**, **PSet** ou **Print** pour l'impression ou l'aperçu d'un état ou pour son enregistrement sous forme de fichier.

ScaleWidth : Spécifie le nombre d'unités utilisées dans la largeur de la page lorsque vous utilisez les méthodes **Circle**, **Line**, **PSet** ou **Print** pour l'impression ou l'aperçu d'un état ou son enregistrement sous forme de fichier.

© ENI Editions - All rigths reserved

Top : Définit la position haute de l'état.

WindowLeft : Indique la position à l'écran, en twips, du bord gauche de l'état.

WindowTop : Indique la position à l'écran, en twips, du bord supérieur de l'état.

Propriétés renvoyant un objet

Application : Permet d'accéder à l'objet **Application** d'Access.

Module : Permet de spécifier un module d'état. Les méthodes **Insert-Lines**, **DeleteLines** et **ReplaceLine** de l'objet **Module** permettent de modifier le contenu d'un module de code.

Parent : Permet de faire référence à l'objet père.

Printer : Renvoie ou définit un objet **Printer** qui représente l'imprimante par défaut du système actuel.

Recordset : Détermine ou renvoie l'objet DAO **Recordset** représentant les enregistrements d'un état.

Shape : Renvoie une chaîne représentant la commande de type **Shape** permettant d'effectuer le tri et le regroupement de l'état spécifié. En lecture seule.

Report : Permet de faire référence à l'état associé à un contrôle sous-état.

Propriétés relatives à l'impression de l'état

CurrentX : Permet de spécifier la coordonnée horizontale du début de la prochaine méthode d'impression ou de dessin d'un état.

CurrentY : Permet de spécifier la coordonnée verticale du début de la prochaine méthode d'impression ou de dessin d'un état.

MoveLayout : Spécifie si Microsoft Access doit passer à la position d'impression suivante sur la page.

NextRecord : Spécifie si une section doit passer à l'enregistrement suivant.

Pages : Indique le nombre total de pages.

PrintCount : Renvoie le nombre d'évaluations de la propriété SurImpression (**OnPrint**) pour la section en cours d'un état.

PrintSection : Spécifie si une section doit être imprimée.

PrtDevMode : Permet de définir ou de restituer des informations relatives au mode du périphérique d'impression spécifié.

PrtDevNames : Permet de définir ou de retourner des informations relatives à l'imprimante choisie.

PrtMip : Définit ou retourne des informations sur le mode du périphérique spécifié.

UseDefaultPrinter : Indique si l'état utilise l'imprimante par défaut du système.

Autres propriétés

FormatCount : Permet de déterminer le nombre d'évaluations de la propriété AuFormatage (**OnFormat**) pour la section en cours dans un état.

GroupLevel : Fait référence au niveau de regroupement sur lequel porte le regroupement ou le tri d'un état.

Hwnd : Retourne le descripteur (une valeur unique de type Entier long) attribué à la fenêtre en cours par

© ENI Editions - All rigths reserved

Microsoft Windows.

PictureData : Permet de copier l'image de l'état dans un autre objet qui reconnaît la propriété Image (**Picture**).

Section : Identifie une section et permet d'accéder à ses propriétés.

Méthodes

Circle : Permet de dessiner un cercle ou une ellipse.

Line : Permet de dessiner des lignes ou des rectangles.

Move : Déplace et/ou redimensionne l'état aux coordonnées indiquées par les valeurs des arguments.

Print : Envoie le texte défini à l'objet **Report**.

Pset : Détermine un point de l'état pour modifier sa couleur.

Scale : Définit le système de coordonnées.

TextHeight : Retourne la hauteur d'une chaîne à imprimer.

TextWidth : Retourne la largeur d'une chaîne à imprimer.

© ENI Editions - All rigths reserved

Les objets Control

Un objet **Control** fait partie de la collection **Controls** qui représente tous les contrôles d'un formulaire, d'un état ou d'un sous-formulaire. L'objet **ActiveControl** (**Screen.ActiveControl** ou **<NomFormulaire>.ActiveControl**) désigne le contrôle actif.

Syntaxes

Vous pouvez référencer un contrôle de façon implicite :

```
<Formulaire>|<Etat>!<NomContrôle> ex: Me!DateCde
<Formulaire>|<Etat>![<NomContrôle>] ex: Me![DateCde]
<Formulaire>|<Etat>("<NomContrôle>") ex: Me("DateCde")
<Formulaire>|<Etat>.<NomContrôle> ex: Me.DateCde
```

ou explicitement comme membre de la collection **Controls** :

```
<Formulaire>|<Etat>.Controls{("<contrôle>")|!<contrôle>|
(<index contrôle>)}
ex:
    Me.Controls!DateCde
    Me.Controls![DateCde]
    Me.Controls("DateCde")
    Me.Controls(0)
```

Chaque contrôle a ses propriétés, méthodes et événements particuliers. Seules les propriétés et méthodes communes à tous les contrôles sont ici répertoriées.

» Pour référencer un contrôle du formulaire ou de l'état actif, le mot clé Me n'est pas obligatoire. Vous pouvez directement utiliser le nom du contrôle.

Propriétés communes à la plupart des contrôles

Application : Permet d'accéder à l'objet Application de Microsoft Access.

Form : Permet de faire référence au formulaire contenant le contrôle.

Hyperlink : Renvoie une référence à un objet lien hypertexte.

Object : Permet de retourner une référence à un objet ActiveX associé à un objet OLE lié ou incorporé dans un contrôle.

ObjectVerbs : Permet de déterminer la liste des actions OLE prises en charge par un objet OLE.

OldValue : Renvoie la valeur non modifiée d'un contrôle dépendant (ancienne valeur).

Parent : Permet de faire référence à l'objet père d'un contrôle.

Report : Permet de faire référence à l'état contenant le contrôle.

Méthodes communes à la plupart des contrôles

Requery : Recalcule la valeur d'un contrôle.

SetFocus : Donne le focus à un contrôle.

SizeToFit : Ajuste un contrôle au texte ou à l'image qu'il contient.

UnDo : Restaure un contrôle tel qu'il était avant d'être modifié.

La méthode Move

© ENI Editions - All rigths reserved

La méthode **Move** est disponible pour la plupart de contrôles (**CheckBox**, **ComboBox**, **CommandButton**, **Image**, **Label**, **Line**, **TexteBox**...).

Elle permet de déplacer et/ou redimensionner le contrôle en fonction des coordonnées indiquées par les valeurs des arguments.

Syntaxe de la méthode Move

```
<expression>.Move(<Gauche>, <Haut>, <Largeur>, <Hauteur>)
```

`<expression>` : Désigne un contrôle, état ou formulaire.

`Gauche` : Obligatoire. Position à l'écran, en twips, du bord gauche de l'objet par rapport au bord gauche de la fenêtre de Microsoft Access.

`Haut` : Facultatif. Position à l'écran, en twips, du bord supérieur de l'objet par rapport au bord supérieur de la fenêtre de Microsoft Access.

`Largeur` : Facultatif. Largeur désirée, en twips, de l'objet.

`Hauteur` : Facultatif. Hauteur désirée, en twips, de l'objet.

© ENI Editions - All rigths reserved

Les contrôles d'Access

Présentation

Microsoft Access 2007 est pourvu d'un nombre intéressant de contrôles facilitant l'élaboration d'une application.

Ces contrôles sont placés dans des formulaires ou dans des états de façon à aboutir à l'interface désirée. Chacun d'eux possède ses propres caractéristiques (propriétés), ses propres comportements (méthodes) ainsi qu'une liste d'événements auxquels ils peuvent réagir, tout cela accessible par l'intermédiaire de l'explorateur d'objets.

On distingue deux types de contrôles :

- les contrôles **intrinsèques**, directement intégrés au moteur Access,

- les contrôles **ActiveX**, chacun d'eux faisant l'objet d'un fichier OCX.

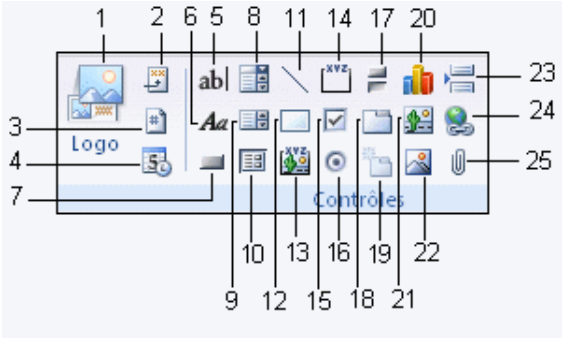
Les contrôles intrinsèques figurent tous dans la boîte à outils.

Les contrôles ActiveX peuvent être sélectionnés par l'intermédiaire de l'outil  de la boîte à outils, qui liste tous les contrôles ActiveX présents sur le système. Ils fonctionnent de la même manière que les contrôles intrinsèques et sont implantés dans le système par l'installation de nouveaux logiciels ou par l'achat de bibliothèques de contrôles ActiveX.

La version Office 2007 Professionnel permet notamment l'utilisation du contrôle Calendrier qui facilite l'affichage et la mise à jour d'un calendrier.

Liste des contrôles Access 2007

Pour accéder aux contrôles depuis un formulaire ou un état ouvert en Mode Création, activez l'onglet **Créer** du ruban : la listes des contrôles et des outils de mise en forme est alors disponible depuis le groupe **Contrôles**.



	Nom du contrôle	Type d'objet VBA
1.	Logo	Image
2.	Titre	TextBox
3.	Numéro de page	TextBox
4.	Date et Heure	TextBox
5.	Zone de texte	TextBox
6.	Etiquette	Label
7.	Bouton de commande	CommandButton
8.	Zone de liste déroulante	ComboBox
9.	Zone de liste	ListBox
10.	Sous-Formulaire / Sous-Etat	SubForm / SubReport

© ENI Editions - All rigths reserved

11.	Trait	Line
12.	Rectangle	Rectangle
13.	Cadre d'objet dépendant	BoundObjectFrame
14.	Groupe d'options	OptionGroup
15.	Case à cocher	CheckBox
16.	Case d'option	OptionButton
17.	Bouton bascule	ToggleButton
18.	Contrôle onglet	TabControl
19.	Page	Page
20.	Graphique	Chart
21.	Cadre d'objet indépendant	UnboundObjectFrame
22.	Image	Image
23.	Saut de page	PageBreak
24.	Lien hypertexte	Label
25.	Pièce jointe	Attachment

© ENI Editions - All rigths reserved

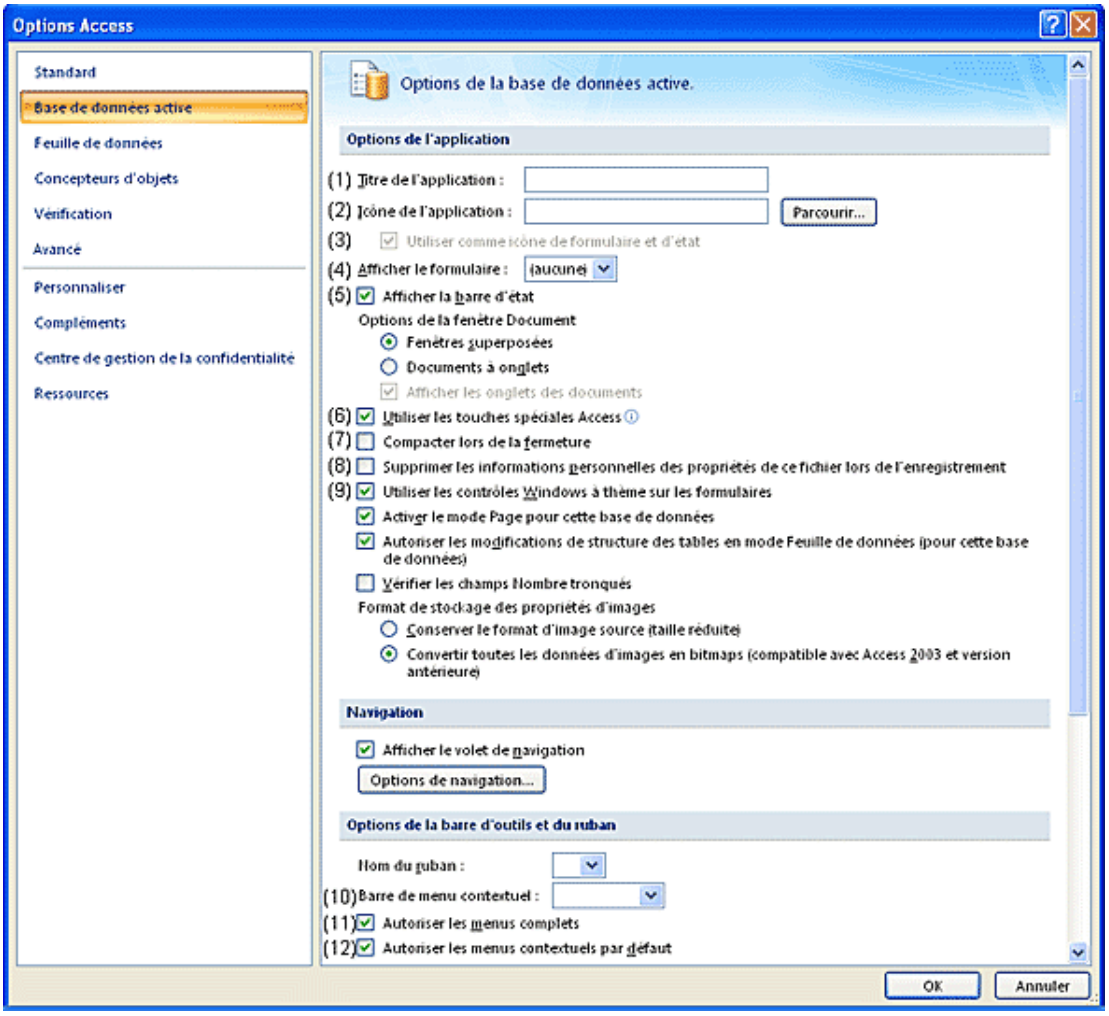
Les options de démarrage

Les options de démarrage d'Access 2007 permettent de définir l'aspect et le comportement d'une application lors de son ouverture.

Pour modifier les options de démarrage :

- Cliquez sur le bouton **Microsoft Office** , puis sur le bouton **Options Access** (en bas de la boîte de dialogue).
- Sélectionnez la catégorie **Base de données active**.

La boîte de dialogue suivante est alors affichée :



Certaines de ces options correspondent à des propriétés de la base de données courantes (objet Database) et peuvent être modifiées au moyen du langage VBA :

	Propriété VBA	Type
1.	AppTitle	String
2.	AppIcon	String
3.	UseAppIconForFrmRpt	Booléen
4.	StartupForm	String
5.	StartupShowStatusBar	Booléen
6.	AllowSpecialKeys	Booléen
7.	Auto Compact	Booléen

© ENI Editions - All rigths reserved

8.	Remove Personal Information	Booléen
9.	Themed Form Controls	Booléen
10.	StartupShortcutMenuBar	String
11.	AllowFullMenus	Booléen
12.	AllowShortcutMenus	Booléen

Pour modifier une propriété à l'aide du langage VBA, vous devez utiliser la collection **Properties** de l'objet Database. Si la propriété n'a pas déjà été définie, vous devez la créer au moyen de la méthode **CreateProperty**.

L'exemple suivant permet de modifier certaines options de démarrage.

```
Sub modifieOptions()
  Dim intX As Integer
' Titre de l'application
    intX = AddAppProperty("AppTitle", DB_Text, _
    "Gestion des Formations")
' Icône de l'application
    intX = AddAppProperty("AppIcon", DB_Text,
"C:\Formations\Formation.bmp")
' Option "Utiliser comme icône de formulaire ou
d'état"
    intX = AddAppProperty("UseAppIconForFrmRpt", DB_BOOLEAN,
1)
' Option "Compacter lors de la fermeture"
    intX = AddAppProperty("Auto Compact", DB_BOOLEAN, 1)
End Sub
```

```
Function AddAppProperty(strName As String, _
    varType As Variant, varValue As Variant) As Integer
  Dim dbs As Object, prp As Variant
  Const conPropNotFoundError = 3270

' Modifie la valeur de la propriété
Set dbs = CurrentDb
  On Error GoTo AddProp_Err
  dbs.Properties(strName) = varValue
  AddAppProperty = True
AddProp_Bye:
  Exit Function
AddProp_Err:
' En cas d'erreur, ajoute la propriété
' au moyen de la méthode CreateProperty
  If Err = conPropNotFoundError Then
    Set prp = dbs.CreateProperty(strName, varType, varValue)
    dbs.Properties.Append prp
    Resume
  Else
    AddAppProperty = False
    Resume AddProp_Bye
  End If
End Function
```


© ENI Editions - All rigths reserved

Personnalisation du ruban

Présentation

Dans Access 2007, il est possible de personnaliser intégralement l'apparence du ruban au moyen du langage XML.

Ce langage permet :

- de masquer les onglets existants,

- de créer de nouveaux onglets, groupes de commandes et commandes,

- de personnaliser la présentation des commandes à l'aide d'icônes,

- d'associer des macros ou du code VBA aux commandes,

- d'afficher des commandes intégrées à Access.

Le code XML peut être stocké à différents emplacements mais il est fortement recommandé de le stocker dans une table système de la base de données. Les rubans créés en langage XML peuvent être associés à l'application Access ou uniquement à certains formulaires et états.

Configuration d'Access pour la personnalisation du ruban

Afin de pouvoir créer et tester au mieux votre code XML, il est recommandé de modifier les options suivantes :

Affichage des tables systèmes dans le volet de navigation

Par défaut, les tables systèmes ne sont pas affichées dans le volet de navigation. Par conséquent vous ne pourrez pas accéder à la table système permettant de stocker votre code XML.

Pour modifier cette option :

- Cliquez avec le bouton droit sur la barre de navigation située en haut du **volet de navigation**.
- Sélectionnez l'option **Options de navigation** dans le menu contextuel.
- Dans la boîte de dialogue **Options de navigation**, activez l'option **Afficher les objets système**, puis cliquez sur le bouton de commande **OK**.

Affichage des messages d'erreurs contenues dans le code XML

Par défaut, les messages d'erreurs détectés dans le code XML ne sont pas affichés. De ce fait, si votre code XML contient des erreurs, le ruban ne pourra être affiché et il sera difficile d'identifier la cause de l'erreur.

Pour modifier cette option :

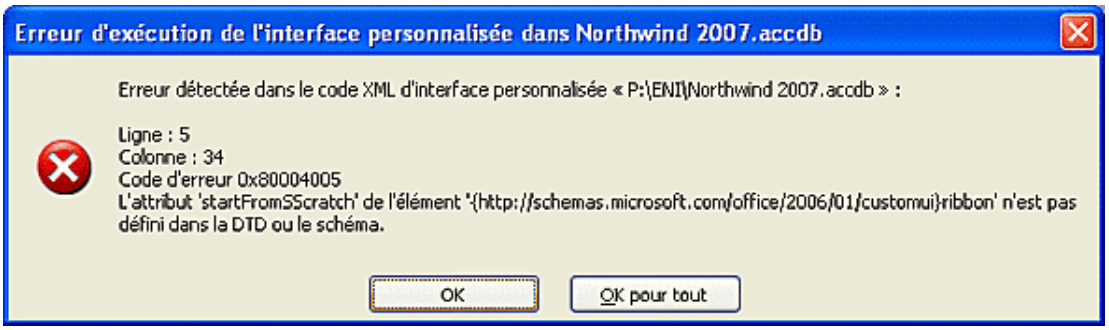
- Cliquez sur le bouton **Microsoft Office**  , puis sur le bouton **Options Access** (situé en bas de la boîte de dialogue).
- Sélectionnez la catégorie **Avancé** dans le menu de gauche.
- Activez l'option **Afficher les erreurs du complément d'interface utilisateur** situé sous **Général**.
- Cliquez sur le bouton de commande **OK**.

Si une erreur est détectée lors de l'exécution du code XML permettant de générer le ruban personnalisé, une message d'erreur est affiché. Le message indique les numéros de ligne et de colonne du code concerné par

© ENI Editions - All rigths reserved

l'erreur ainsi que le descriptif de l'erreur.

Exemple :



Création d'une table système USysRibbons

La table système **USysRibbons** permet de stocker le code XML permettant de créer un ou plusieurs rubans.

Pour créer cette table :

- Activez l'onglet **Créer**.
- Dans le groupe **Tables**, cliquez sur **Création de table**.
- Insérez les champs suivants :

RibbonId NuméroAuto

RibbonName Texte sur 255 caractères

RibbonXML Mémo

- Sélectionnez le champ **RibbonId** et cliquez sur la commande **Clé primaire** (située dans le groupe **Outils** de l'onglet **Création**).
- Cliquez sur **Enregistrer** depuis la barre d'outils **Accès rapide**.
- Nommez la nouvelle table **UsysRibbons**.

Ajout du code XML de personnalisation à la table USysRibbons

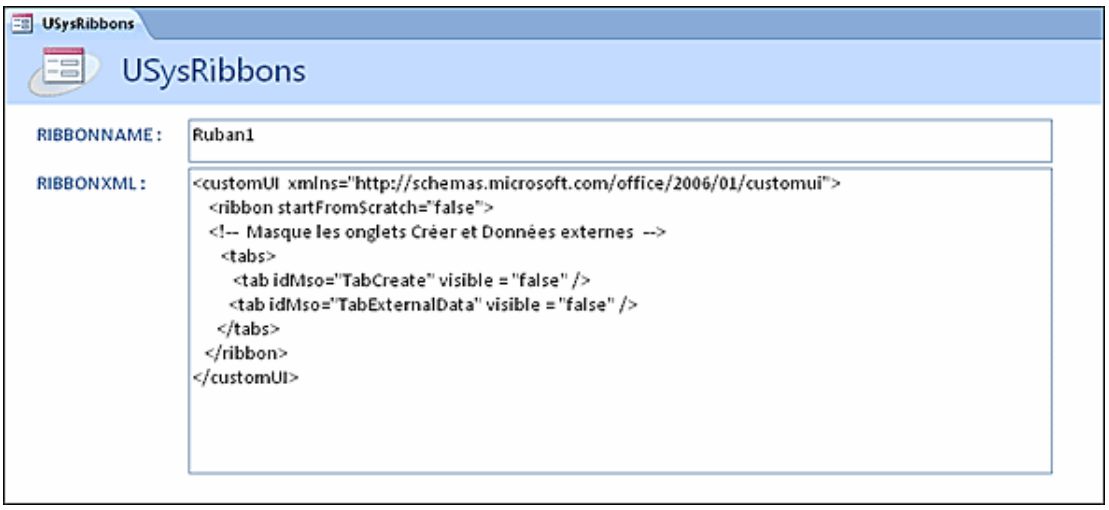
Pour associer du code XML à votre table **UsysRibbons**, la solution la plus simple est de générer un formulaire de saisie. Pour cela :

- Sélectionnez la table **UsysRibbons** dans le volet de navigation.
- Activez l'onglet **Créer**, puis cliquez sur **Formulaire** dans le groupe **Formulaires**.
- Saisissez le nom du ruban et le code XML associé dans le formulaire.

Exemple :

*Le code XML suivant permet de masquer les onglets **Créer** et **Données externes** du Ruban :*

© ENI Editions - All rigths reserved



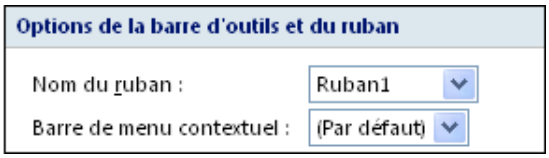
» Afin de faciliter la saisie et la lecture du code XML, vous pouvez utiliser un éditeur XML (ex : Visual Web Developper Express téléchargeable depuis le site de Microsoft) pour créer votre code et le copier ensuite dans la table UsysRibbon.

Association du ruban à l'application active

- Cliquez sur le bouton **Microsoft Office** , puis sur le bouton **Options Access** (situé en bas de la boîte de dialogue).

- Sélectionnez la catégorie **Base de données active** dans le menu de gauche.

- Inscrivez le nom du ruban (ex : ruban1) dans la liste **Nom du ruban** située sous **Options de la barre d'outils et du ruban**.



- Cliquez sur le bouton de commande **OK**. Le message suivant est alors affiché :



Fermez votre base de données et ouvrez-la à nouveau afin d'activer le nouveau ruban.

» Le code XML du ruban est exécuté à l'ouverture de la base de données. Pour tester les modifications apportées à votre code XML, vous devez systématiquement fermer puis rouvrir votre base de données.

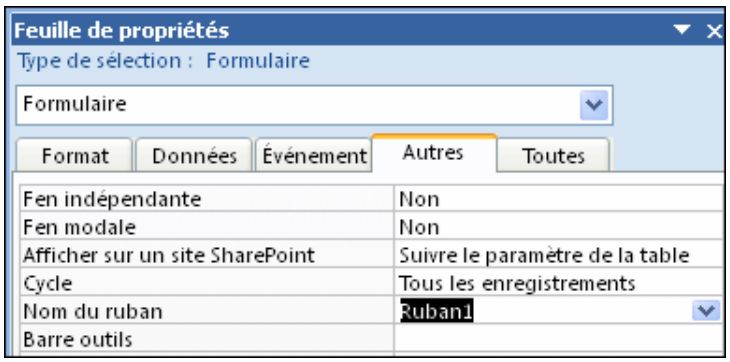
Association du ruban à un formulaire ou un état

- Ouvrez le formulaire ou l'état en **Mode création**.

- Affichez la feuille de propriété du formulaire ou de l'état.

- Sous l'onglet **Autre** de la feuille de propriété, cliquez sur la liste **Nom du ruban** et sélectionnez le ruban à afficher lors de l'ouverture du formulaire ou de l'état.

© ENI Editions - All rigths reserved



- Enregistrez puis fermez votre formulaire.

- Ouvrez le formulaire en Mode Formulaire: le ruban sélectionné est alors affiché. À la fermeture du formulaire, le ruban associé à l'application sera restauré.

© ENI Editions - All rigths reserved

Présentation du langage XML

Le langage XML

Le langage XML (eXtensible Markup Language) est le langage standard de description et d'échange de données, utilisé notamment sur le web.

Tout comme le langage HTML, le langage XML utilise des balises (ou "tags") pour délimiter un contenu. Une balise est facilement identifiable car elle commence par le caractère < et se termine par le caractère > (les balises de fin de contenu commencent par /).

Eléments XML utilisés pour personnaliser le ruban

Les principales balises XML utilisées dans le code XML d'un ruban

Nom de balise	Description
<customUI>	Balise principale d'un ruban personnalisé
<ribbon>	Balise contenant la description du ruban
<OfficeMenu>	Menu accessible depuis le bouton Microsoft Office
<tab>	Onglet dans le ruban
<group>	Groupe dans un onglet dans le ruban
<labelControl>	Etiquette (titre ou intitulé)
<button>	Bouton de commande
<splitButton>	Bouton de menu
<toggleButton>	Bouton à bascule
<menu>	Menu déroulant
<dynamicMenu>	Menu déroulant dynamique
<gallery>	Galerie ou palette de choix
<dialogBoxLauncher>	Bouton déclenchant l'ouverture d'une boîte de dialogue

Les propriétés associées aux objets du ruban

Les propriétés suivantes permettent de personnaliser les différents contrôles affichés dans le ruban (bouton de commande, menu déroulant…)

Propriété	Description
Id	Identifiant unique d'un contrôle personnalisé.
idMso	Identifiant unique d'un contrôle standard Microsoft.
enabled	"true" si le contrôle est accessbile, "false" sinon. La valeur par défaut est "true".
insertBeforeMso	Place l'objet avant un contrôle Office existant.
insertAfterMso	Place l'objet après un contrôle Office existant.
imageMso	Nom de l'image à utiliser pour le contrôle.
itemSize	Taille des éléments du menu.
label	Texte affiché sur le contrôle.
onAction	Macro ou code VBA associé au contrôle.
screentip	Titre de l'info-bulle du contrôle.
supertip	Texte apparaissant dans l'info-bulle du contrôle.
size	Taille du contrôle ("normal" ou "large").
showImage	"true" si l'image du contrôle est affichée, "false" sinon. La valeur par défaut est "true".

© ENI Editions - All rigths reserved

showLabel	"true" si l'intitulé du contrôle est affichée, "false" sinon. La valeur par défaut est "true".
visible	"true" si le contrôle est visible, "false" sinon. La valeur par défaut est "true".

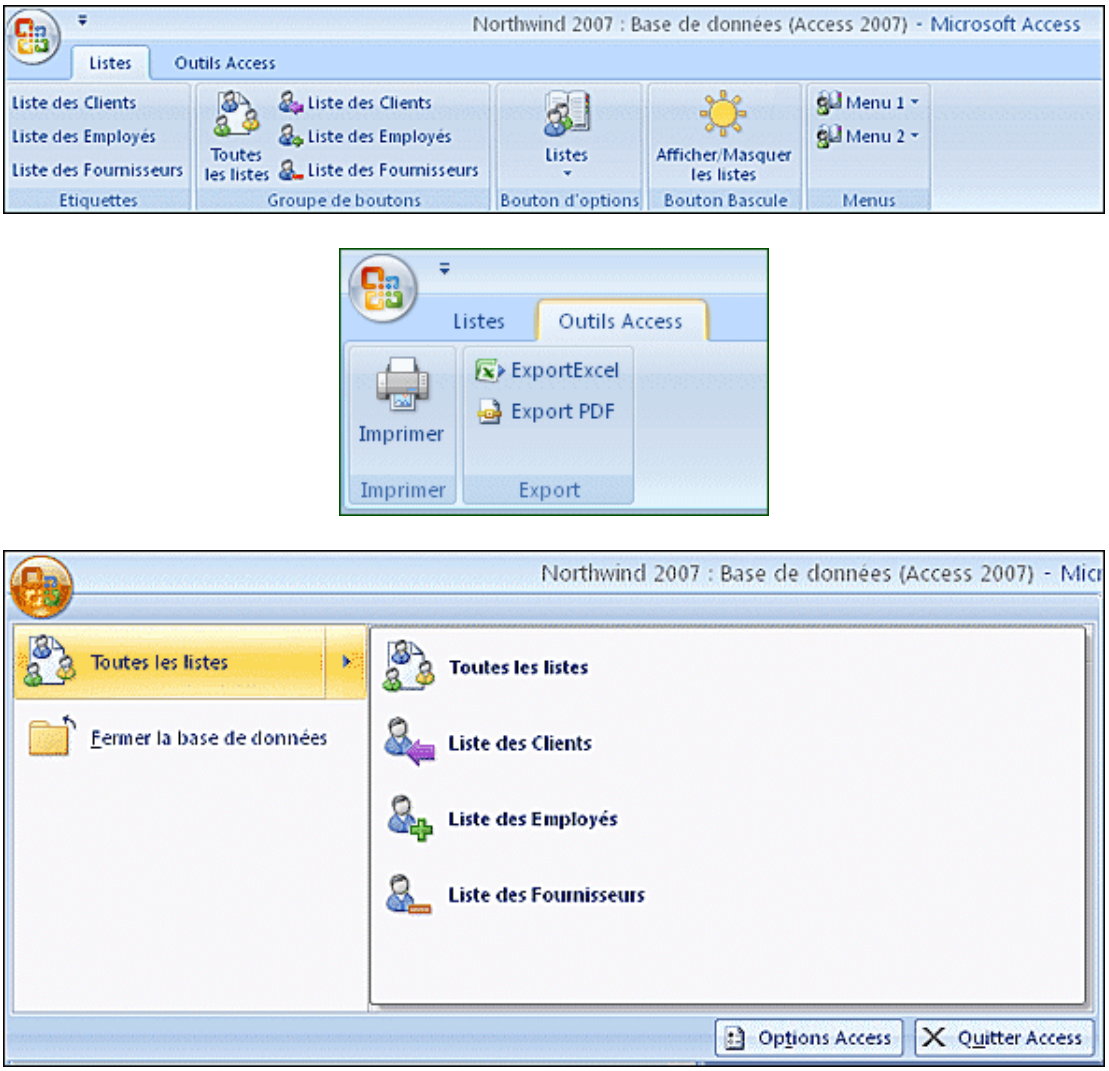
Les propriétés suivantes permettent de personnaliser le ruban :

OnLoad	Macro ou code VBA associé au chargement du ruban.
startFromScratch	"true" si le ruban personnalisé doit remplacer le ruban standard d'Access. Si "false", les nouveaux onglets s'ajoutent aux onglets du ruban Access.

© ENI Editions - All rigths reserved

Exemple de ruban personnalisé

L'exemple de code XML décrit dans ce chapitre permet de créer le ruban présenté ci-dessous :



Code XML du ruban personnalisé

```
<customUI
xmlns="http://schemas.microsoft.com/office/2006/01/customui"
onLoad="Initialisation">
ribbon startFromScratch="true">
' Modification du menu Microsoft Office
  <officeMenu>
' Création d'un bouton de menu
  <splitButton id="spltListe1"
  insertBeforeMso="FileCloseDatabase">
    <menu id="spltMenu"
  itemSize="large" label="Listes de formulaires">
    <button id="Option1" label="Toutes les listes"
      imageMso="FileCreateDocumentWorkspace"
      onAction="OuvreListes"/>
    <button id="Option2" label="Liste des Clients"
      imageMso="DirectRepliesTo"
      onAction="OuvreListeClients"/>
    <button id="Option3" label="Liste des Employés"
      imageMso="DistributionListAddNewMember"
      onAction="OuvreListeEmployes"/>
        <button id="Option4" label="Liste des Fournisseurs"
      imageMso="DistributionListRemoveMember"
    onAction="OuvreListeFournisseurs"/>
    </menu>
  </splitButton>
```

© ENI Editions - All rigths reserved

```
' Masque les boutons standards du menu Microsoft Office
  <button idMso="FileNewDatabase" visible="false"/>
  <button idMso="FileOpenDatabase" visible="false"/>
  <button idMso="FileSaveAsMenuAccess" visible="false"/>
</officeMenu>

' Création d'onglets personnalisés
  <tabs>
   ' Onglet Listes
  <tab id="Listes" label="Listes">

' Groupe Etiquette
  <group id="grpListes1" label="Etiquettes">
    <labelControl id="lbl1" label="Liste des Clients"/>
    <labelControl id="lbl2" label="Liste des Employés"/>
    <labelControl id="lbl3" label="Liste des Fournisseurs"/>
  </group>

' Groupe de boutons
  <group id="grpBoutons" label="Groupe de boutons">
    <button id="btn1" label="Toutes les listes"
      imageMso="FileCreateDocumentWorkspace"
      onAction="OuvreListes"
      size="large"/>

    <button id="btn2" label="Liste des Clients"
      imageMso="DirectRepliesTo"
      onAction="OuvreListeClients"
      size="normal"/>

    <button id="btn3" label="Liste des Employés"
      imageMso="DistributionListAddNewMember"
      onAction="OuvreListeEmployes"
      size="normal"/>

    <button id="btn4" label="Liste des Fournisseurs"
      imageMso="DistributionListRemoveMember"
      onAction="OuvreListeFournisseurs"
      size="normal"/>
</group>

' Groupe bouton d'options
  <group id="grpSplit" label="Bouton d'options">
  <splitButton id="spl1" size="large">
  <button id="btnSplit" label="Listes"
    onAction = "OuvreListes"
    imageMso="DistributionListSelectMembers"/>
  <menu id="mnuSplit" label="Menu" itemSize="large">
  <button id="btnSplit1" label="Listes des Clients"
    imageMso="DirectRepliesTo"
    onAction = "OuvreListeClients"/>
  <button id="btnSplit2" label="Liste des Employés"
    imageMso="DistributionListAddNewMember"
    onAction = "OuvreListeEmployes"/>
  <button id="btnSplit3" label="Liste des Fournisseurs"
    imageMso="DistributionListRemoveMember"
    onAction = "OuvreListeFournisseurs"/>
  <button id="btnSplit4" label="Toutes les listes"
    imageMso="DistributionListSelectMembers"
    onAction = "OuvreListes"/>
    </menu>
  </splitButton>
</group>

' Groupe bouton bascule
  <group id="grptb1" label="Bouton Bascule">
    <toggleButton id="tb1" label="Afficher/Masquer les listes"
      imageMso="PictureBrightnessGallery"
      size="large"
      onAction = "AffichageListes">
  </group>

' Groupe Menus
  <group id="grpMenus" label="Menus">
  <menu id="menu1" label="Menu 1"
    imageMso="DistributionListSelectMembers">
  <button id="btnMenu1"
    label="Liste des Clients"
    onAction = "OuvreListeClients"/>
```


© ENI Editions - All rigths reserved

```
  <button id="btnMenu2"
    label="Liste des Employés "
    onAction = "OuvreListeEmployes"/>
  <button id="btnMenu3"
    label="Liste des Fournisseurs"
    onAction = "OuvreListeFournisseurs"/>
  <button id="btnMenu4"
    label="Toutes les listes"
    onAction = "OuvreListes"/>
  </menu>

  <menu id="menu2"
    label="Menu 2"
    itemSize="large"
    imageMso="DistributionListSelectMembers">
  <button id="btnMenu5"
    label="Liste des Clients"
    imageMso="DirectRepliesTo"
    onAction = "OuvreListeClients"/>
   <button id="btnMenu6"
     label="Liste des Employés "
     imageMso="DistributionListAddNewMember"
     onAction = "OuvreListeEmployes"/>
  <button id="btnMenu7"
     label="Liste des Fournisseurs"
    imageMso="DistributionListRemoveMember"
     onAction = "OuvreListeFournisseurs"/>
   <button id="btnMenu8"
    label="Toutes les listes"
     imageMso="DistributionListSelectMembers"
    onAction = "OuvreListes"/>
  </menu>
  </group>
</tab>

' Onglet Outils Access
   <tab id="tabAccess" label="Outils Access">
  <group idMso="GroupPrintPreviewPrintAccess"/>
    <group id="grpExport" label="Export">
    <button idMso="ExportExcel" label="ExportExcel"
      screentip="Export Excel"
      supertip="Cliquez ici pour exporter au format Excel"/>
    <button idMso="FileSaveAsPdfOrXps" label="Export PDF"
      screentip="Export PDF"
      supertip="Cliquez ici pour exporter au format PDF"/>
    </group>
    </tab>
  </tabs>
  </ribbon>
</customUI>
```

Code VBA appelé par les commandes du ruban personnalisé

Le code VBA suivant est appelé au moyen des propriétés OnAction des différents contrôles.

```
Option Compare Database
Option Explicit
_____________________________________________________________________

Sub Initialisation(ruban As IRibbonUI)
' Message de bienvenue
    MsgBox "Bienvenue dans l'application NorthWind 2007"
' Verrouille le volet de navigation
    DoCmd.LockNavigationPane True
End Sub
_____________________________________________________________________

Sub OuvreListes(ctl As IRibbonControl)
' Ouvre les 3 formulaires Listes
    DoCmd.OpenForm "Liste des clients"
    DoCmd.OpenForm "Liste des employés"
    DoCmd.OpenForm "Liste des fournisseurs"
End Sub
_____________________________________________________________________

Sub OuvreListeClients(ctl As IRibbonControl)
    DoCmd.OpenForm "Liste des clients"
```

© ENI Editions - All rigths reserved

```
End Sub
_____________________________________________________________________________

Sub OuvreListeEmployes(ctl As IRibbonControl)
    DoCmd.OpenForm "Liste des employés"
End Sub
_____________________________________________________________________________

Sub OuvreListeFournisseurs(ctl As IRibbonControl)
    DoCmd.OpenForm "Liste des fournisseurs"
End Sub
_____________________________________________________________________________

Sub AffichageListes(ctl As IRibbonControl, blnActif As Boolean)
' Affiche ou masque les formulaires
    If blnActif Then
    DoCmd.OpenForm "Liste des clients"
    DoCmd.OpenForm "Liste des employés"
    DoCmd.OpenForm "Liste des fournisseurs
  Else
    DoCmd.Close acForm, "Liste des clients"
    DoCmd.Close acForm, "Liste des employés"
    DoCmd.Close acForm, "Liste des fournisseurs"
  End If
End Sub
```

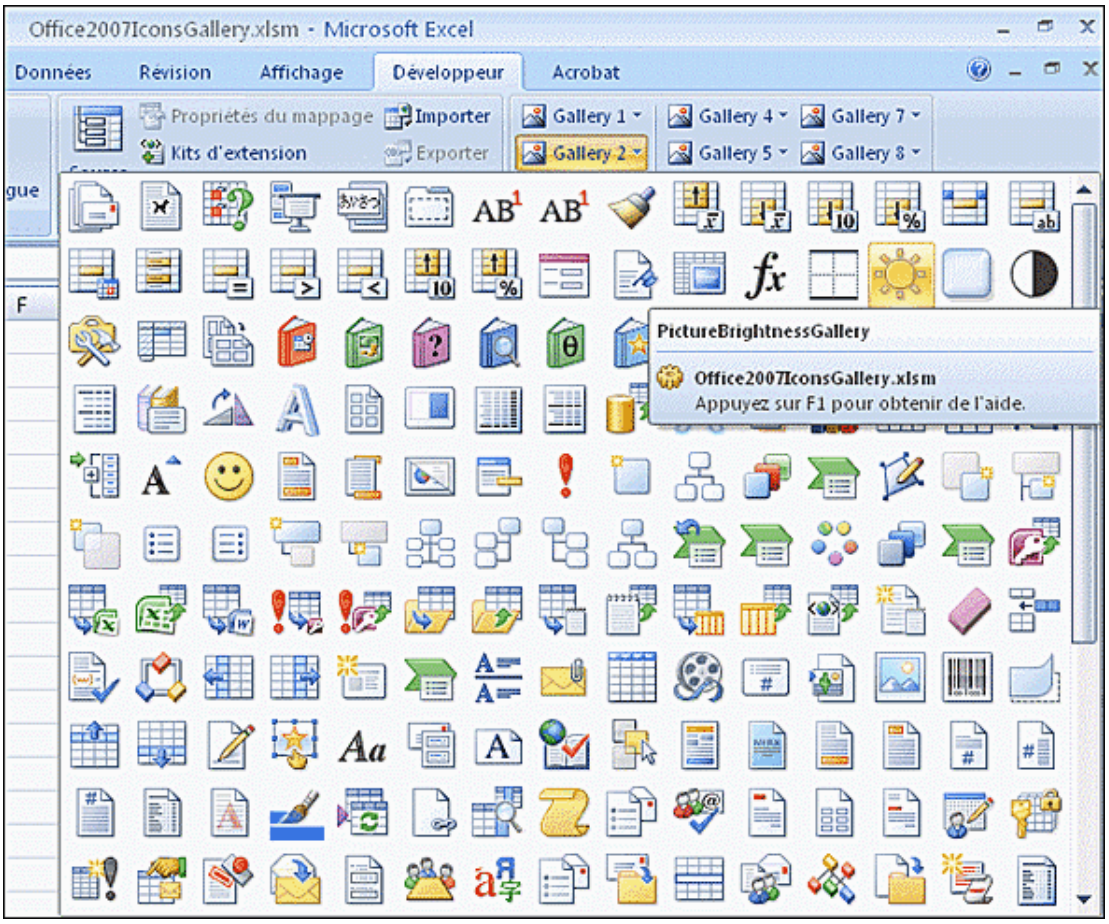

© ENI Editions - All rigths reserved

Images de la galerie des icônes Microsoft Office

Les noms des images utilisées pour personnaliser le ruban (ex : FileCreateDocumentWorkspace, DirectRepliesTo...), correspondent à l'intitulé de l'icône dans la galerie des icônes Microsoft Office.

Pour connaître la liste des noms d'icônes de la galerie Microsoft Office :

- Ouvrez dans Excel 2007 le fichier Office2007IconsGallery.xlsm (fichier livré avec les exemples ou téléchargeable sur le site de Microsoft).
- Activez l'onglet **Développeur**.
- Cliquez sur l'un des boutons "Galerie" pour afficher la liste des icônes.
- Lorsque vous survolez une icône, son nom est affiché dans l'info-bulle associée :



Si l'onglet Développeur n'est pas affiché dans Excel :

- Cliquez sur le bouton **Microsoft Office**  , puis sur le bouton **Options Excel** (situé en bas de la boîte de dialogue).
- Sélectionnez la catégorie **Standard** dans le menu de gauche.
- Activez l'option **Afficher l'onglet Développeur** dans le ruban situé sous **Meilleurs options pour travailler avec Excel**.
- Cliquez sur le bouton **OK** : l'onglet **Développeur** a été ajouté au ruban d'Excel, à droite de l'onglet **Affichage**.

© ENI Editions - All rigths reserved

La technologie Automation

Présentation

Automation, appelée également **OLE** (*Object Linking and Embedding*) ou OLE Automation, est une technologie permettant de manipuler les objets d'une autre application directement à partir d'Access ou de VBA Access.

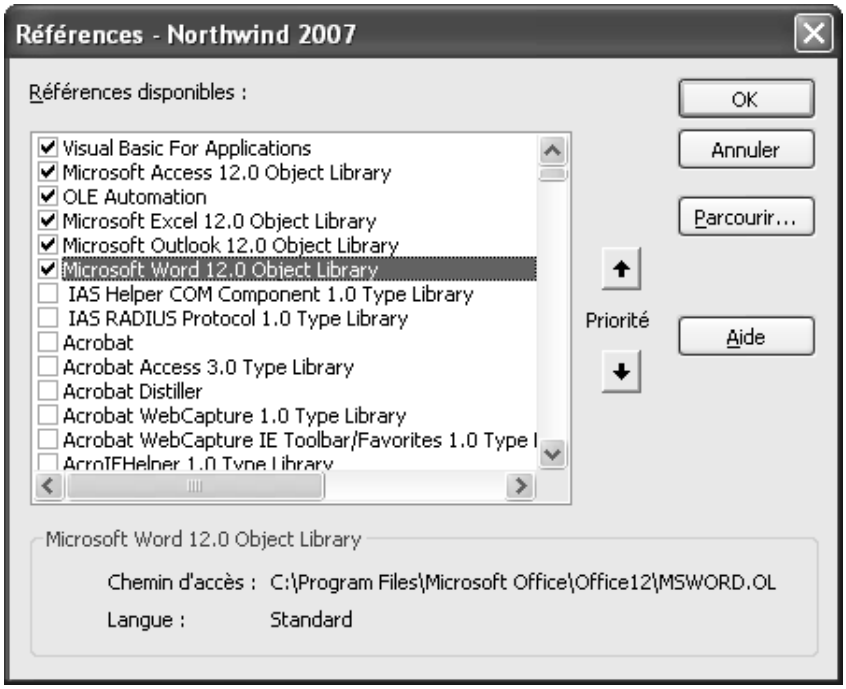
Pour fonctionner, **Automation** nécessite un client et un serveur appelé **serveur OLE**. Le serveur est l'application ou le composant qui fournit les services au client. Le client (appelé également contrôleur) utilise ces services pour piloter l'application serveur et manipuler ses objets. Par exemple si vous lancez un publipostage Word à partir de VBA Access, Access est le client et Word le serveur OLE.

Une **bibliothèque d'objets** est un fichier, doté généralement d'une extension olb, fournissant les informations permettant de manipuler les objets mis à disposition par un serveur. Vous pouvez utiliser l'Explorateur d'objets pour examiner le contenu d'une bibliothèque d'objets.

Pour avoir accès aux objets d'une autre application, vous devez référencer sa bibliothèque d'objets de la façon suivante :

- Dans un module, sélectionnez le menu **Outils - Références**.

La boîte de dialogue **Références** s'affiche avec tous les serveurs OLE enregistrés dans la base de registre.



- Activez la référence voulue.

» À l'aide de l'explorateur d'objets, il est simple de consulter la liste des objets, méthodes et propriétés d'un serveur OLE.

Utilisation de la technologie Automation

Pour pouvoir manipuler les objets d'une autre application, vous devez procéder de la façon suivante :

- définissez dans le code VBA une variable objet,

- utilisez les fonctions **CreateObject** ou **GetObject** pour faire référence à l'objet.

Exemple

© ENI Editions - All rigths reserved

Lancement de Word.

```
Dim AppWord as Object
Set AppWord = CreateObject("Word.Application")
```

ou

```
Dim appWord As New Word.Application
```

Référence à un document Word existant.

```
Dim DocWord As New Word.Document
Set DocWord = GetObject("C:\Clients\relance.docx")
```

Les paragraphes suivants décrivent comment piloter différents logiciels de la suite Microsoft Office en utilisant la technologie Automation.

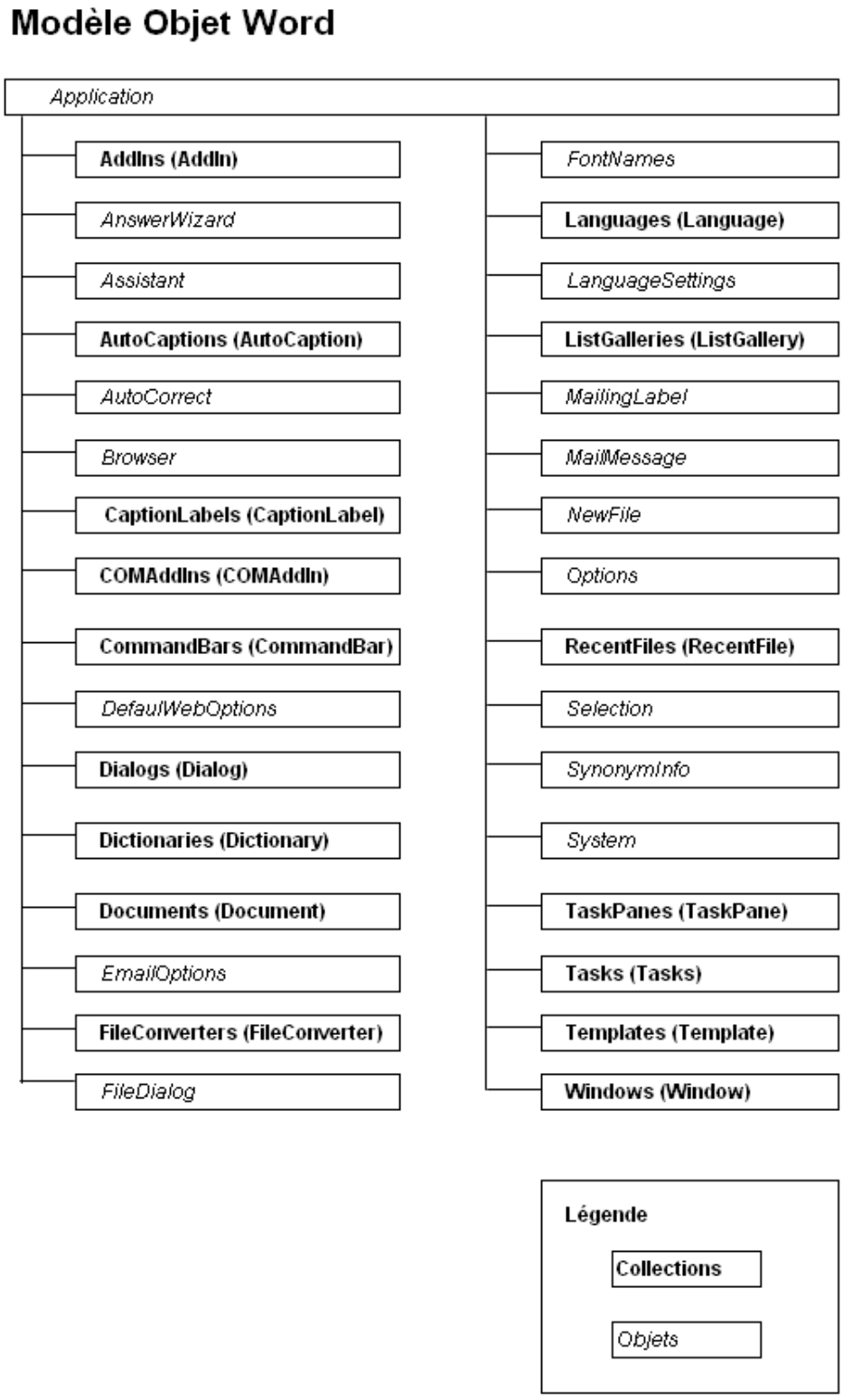
» Les objets, collections, méthodes et propriétés des modèles objet de la suite Microsoft Office étant très nombreux, seuls les plus utilisés sont décrits dans les paragraphes suivants.

© ENI Editions - All rigths reserved

Piloter Word depuis Access

Le modèle Objet Word

Extrait du modèle Objet Word :



Objets et collections Word

Documents : Collection de tous les classeurs (objets **Document**) ouverts.

© ENI Editions - All rigths reserved

Dictionaries : Collection de tous les dictionnaires personnels actifs (objets **Dictionary**).

EmailOptions : Objet contenant les attributs globaux que Microsoft Word utilise lorsque vous créez et modifiez des messages électroniques et des réponses à ces messages.

FileConverters : Collection de tous les convertisseurs de fichier (objets FileConvert) disponibles pour ouvrir et enregistrer des fichiers.

FontNames : Objet contenant la liste des noms de toutes les polices disponibles.

Languages : Collection des langues (objets **Language**) utilisées dans Word pour les vérifications linguistiques et la mise en forme.

ListGalleries : Collection des trois premiers onglets (objets **ListGallery**) de la boîte de dialogue **Puces et numéros**.

MailMessage : Objet représentant le message électronique actif si Word est votre éditeur de courrier électronique.

Options : Représente les options d'application et de document de Word. La plupart des propriétés de l'objet **Options** correspondent à des éléments de la boîte de dialogue **Options** (menu **Outils**).

Selection : Représente la sélection en cours d'une fenêtre ou d'un volet.

System : Contient des informations sur le système de votre ordinateur.

Templates : Collection de tous les modèles (objets **Template**) actuellement disponibles.

La collection Documents

Méthodes

La collection **Documents** est constituée de tous les objets **Document** ouverts dans Word.

Add : Crée un nouveau Document et l'ajoute à la collection **Documents**.
Ex : `Documents.Add Template:="Normal"`

Close : Ferme tous les documents Word ouverts.
Ex : `Documents.Close`

Open : Ouvre le **Document** spécifié et l'ajoute à la collection **Documents**.
Ex :
```
Documents.Open _
FileName:="C:\Clients\Relance.doc", _
ReadOnly:=True
```

Save : Enregistre tous les documents ouverts.
Ex : `Documents.Save`

Les objets Document

Un objet **Document** permet de faire référence à un document Word.
ActiveDocument désigne le document actif.

Méthodes

Activate : Active un document déjà ouvert.
Ex : `Documents("Achats.doc").Activate`

Close : Ferme un document Word ouvert.
Ex :
```
Documents("Achats.doc").Close
ou ActiveDocument.Close
```


© ENI Editions - All rigths reserved

PrintPreview : Affiche un document en mode Aperçu avant impression.
Ex : `ActiveDocument.PrintPreview.`

Range : Renvoie un objet **Range** (Plage).
Ex : `ActiveDocument.Range(0, 50).Bold = True`

Save : Enregistre un document.
Ex : `ActiveDocument.Save`

SaveAs : Enregistre un document sous un nouveau nom et/ou sous un nouveau format.
Ex : `ActiveDocument.SaveAs FileName:=strDocName`

Objets et collections

Characters : Collection des caractères situés dans un document, une plage ou une sélection.

MailMerge : Fusion et publipostage dans Word.

PageSetup : Description de la mise en page.
Ex : `ActiveDocument.PageSetup.RightMargin = InchesToPoints(1)`

Paragraphs : Collection des paragraphes d'un document.

SmartTags : Collection des balises actives (objets **SmartTag**) d'un document ou d'une plage de texte au sein d'un document.

Words : Collection des mots du document.
Ex : `ActiveDocument.Words.Count`

Exemples

Mise en gras des caractères d'un document Word.

```
Private Sub Word_EnGras()
Dim AppWord As New Word.Application

With AppWord
  .Documents.Open CurrentProject.Path & "\Relance.docx"
  .ActiveDocument.Select
  .Selection.Font.Bold = True
  .ActiveDocument.Close wdSaveChanges
  .Quit
End With

End Sub
```

Lancement d'un publipostage

Le code suivant permet de générer une lettre de relance pour les clients ayant une commande livrée et non réglée. Le publipostage fait appel à la requête "Commandes Non Réglées" de la base Clients.mdb.

```
Private Sub Relances()
Dim AppWord As Word.Application
Dim DocWord As Word.Document
Dim m_Provider As String

' Lancement de Word
Set AppWord = New Word.Application
AppWord.Visible = True

' Ouverture de la lettre type
Set DocWord = AppWord.Documents.Open(CurrentProject.Path & _
 "\Relance.docx")

' Provider = Base Access Clients.mdb
m_Provider = "Microsoft.ACE.OLEDB.12.0;Password="""";User
```

© ENI Editions - All rigths reserved

```
ID=Admin;" _
    & "Data Source=" & CurrentProject.Path & "\Clients.accdb;
" _
    & "Mode=Read;"

' Préparation du publipostage pour les commandes non
réglées
With DocWord.MailMerge
  .OpenDataSource Name:= _
    CurrentProject.Path & "\Clients.accdb", _
    ConfirmConversions:=False, _
    ReadOnly:=False, LinkToSource:=True,
AddToRecentFiles:=False, _
    Connection:=" & m_Provider & ", _
    SQLStatement:="SELECT * FROM `COMMANDES NON REGLEES`"
  .Destination = wdSendToNewDocument
  .SuppressBlankLines = True
  With .DataSource
    .FirstRecord = wdDefaultFirstRecord
    .LastRecord = wdDefaultLastRecord
  End With
' Lancement du publipostage
.Execute
End With

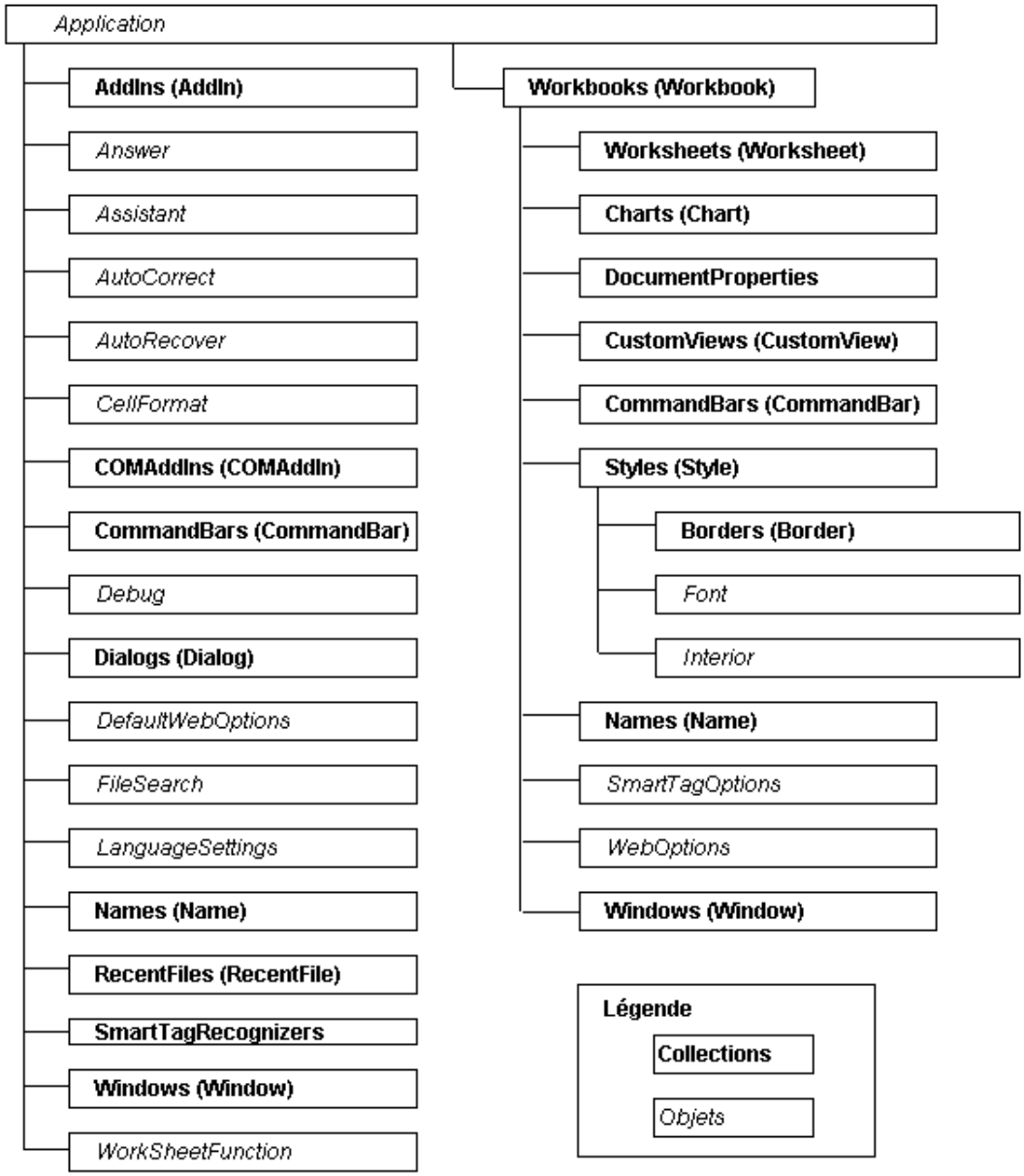
End Sub
```


© ENI Editions - All rigths reserved

Piloter Excel depuis Access

Le modèle Objet Excel

Extrait du modèle Objet Excel :



Objets et collections Excel

Workbooks : Collection de tous les classeurs (objets **Workbook**) ouverts.

Worksheets : Collection de toutes les feuilles de calculs (objets **Worksheet**) d'un classeur.

Charts : Collection de toutes les feuilles graphiques (objets **Chart**) d'un classeur.

CustomViews : Collection de vues personnalisées (objets **CustomView**) d'un classeur.

Names : Collection de tous les objets **Name** dans l'application ou le classeur. Chaque objet **Name** représente un nom défini pour une plage de cellules.

SmartTagOptions : Objet représentant les options relatives aux balises actives.

© ENI Editions - All rigths reserved

Ex : activation des balises actives
```
ActiveWorkbook.SmartTagOptions _
.EmbedSmartTags = True
```

Styles : Collection de toutes les descriptions de styles (objets **Style**) d'un classeur.

La collection Workbooks

La collection **Workbooks** est constituée de tous les classeurs ouverts.

Add : Crée un nouveau classeur qui devient le classeur actif et l'ajoute à la collection **Workbooks**.
Ex : `Workbooks.Add`

Close : Ferme tous les classeurs Excel ouverts.
Ex : `Worksheets.Close`

Open : Ouvre le classeur spécifié qui devient le classeur actif et l'ajoute à la collection Workbooks.
Ex : `Workbooks.Open Filename:="C:\Devis\devis.xls"`

Save : Enregistre tous les classeurs ouverts.
Ex : `Workbooks.Save`

Les objets Workbook

Un objet Workbook permet de faire référence à un classeur Excel.

La propriété ActiveWorkbook désigne le classeur actif.

Activate : Active un classeur déjà ouvert.
Ex : `Documents("Achats.xlsx").Activate`

Close : Ferme un classeur Excel ouvert.
Ex : `Documents("Achats.xlsx").Close`
ou `ActiveDocument.Close`

PrintPreview : Affiche la feuille active d'un classeur en mode Aperçu avant impression.
Ex : `ActiveWorkbook.PrintPreview`

Save : Enregistre un classeur.
Ex : `ActiveWorkbook.Save`

SaveAs : Enregistre un classeur sous un nouveau nom et/ou sous un nouveau format.
Ex : `ActiveWorkbook.SaveAs FileName:= "Devis.xlt"`

Ecriture de données dans des cellules Excel

Le code suivant permet d'afficher la liste des commandes non réglées.

```
Private Sub ExportXLS_Commandes()

' Objets Access
Dim dbsClients As DAO.Database
Dim rstCommandes As DAO.Recordset
Dim fld As DAO.Field

' Objets Excel
Dim appExcel As Excel.Application
Dim wbkCommandes As Excel.Workbook
Dim wksCommandes As Excel.Worksheet

' Variables de boucles
Dim intLig As Integer
Dim intCol As Integer

' Création du classeur Excel
```


© ENI Editions - All rigths reserved

```
Set appExcel = CreateObject("Excel.application")
Set wbkCommandes = appExcel.Workbooks.Add
Set wksCommandes = appExcel.ActiveSheet
appExcel.Visible = True

' Ouverture de la table Commandes
Set dbsClients = DBEngine.OpenDatabase(CurrentProject.Path _
    & "\Clients.accdb")
Set rstCommandes = dbsClients.OpenRecordset _
    ("COMMANDES NON REGLEES", dbOpenDynaset)

' Mise à jour de la feuille active
With wksCommandes

' En-têtes de colonnes renseignées à partir des noms
de champs
intCol = 1
  For Each fld In rstCommandes.Fields
    .Cells(1, intCol).Value = fld.Name
    intCol = intCol + 1
  Next fld

' Parcours des enregistrements
' Ajout d'une ligne par enregistrement
  intLig = 2
  Do While Not rstCommandes.EOF
    intCol = 1
    For Each fld In rstCommandes.Fields
      .Cells(intLig, intCol).Value = rstCommandes(intCol - 1)
      intCol = intCol + 1
    Next fld
    intLig = intLig + 1
    rstCommandes.MoveNext
  Loop
  ' Affectation d'un nom à la feuille Excel
  .Name = "Liste des commandes non réglées"
End With

' Fermeture des objets Access
rstCommandes.Close
dbsClients.Close

' Activation du classeur dans Excel
appExcel.ActiveWindow.Activate

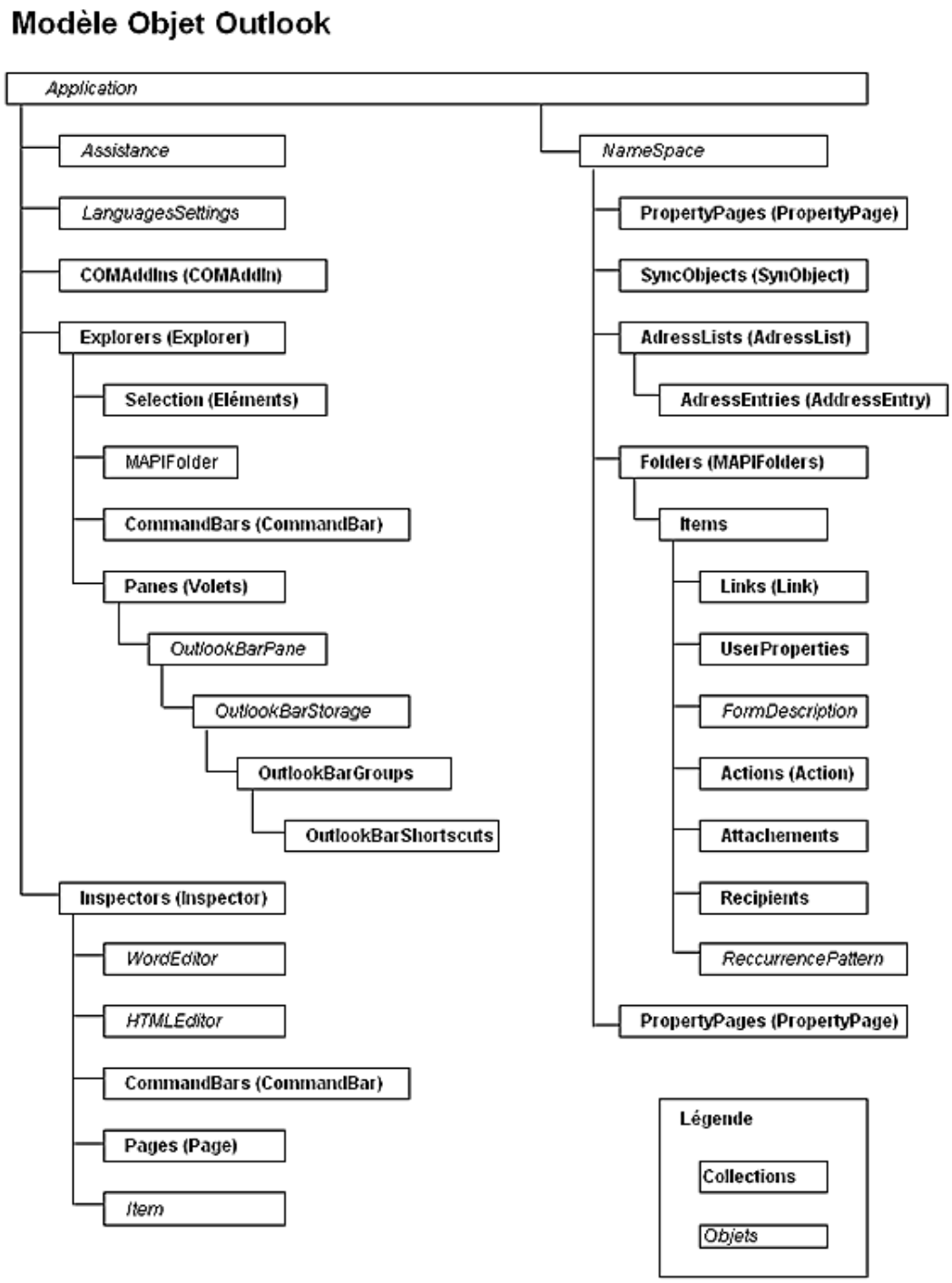
End Sub
```

© ENI Editions - All rigths reserved

Piloter Outlook depuis Access

Le modèle Objet Outlook

Extrait du modèle Objet Outlook :



Exemple : affichage des contacts Outlook

Cet exemple permet d'afficher la liste des contacts Outlook (nom et adresse email) dans une zone de liste d'un formulaire Access.

Pour tester cet exemple vous devez :

- créer un nouveau formulaire,

© ENI Editions - All rigths reserved

- ajouter au formulaire une zone de liste **lstContacts** et un bouton de commande **cmdContacts**.

```
Private Sub CmdContacts_Click()
Dim objOle As Outlook.Application
Dim objNamespace As Namespace
Dim objAddrList As AddressList
Dim objAddrEntries As AddressEntries
Dim objAdrEntry As AddressEntry
Dim i As Integer

' Initialisation de la liste déroulante
With Me.lstContacts
  .ColumnCount = 1
  .RowSourceType = "Liste valeurs"
  .RowSource = ""
End With

' Application Outlook
Set objOle = CreateObject("Outlook.Application")

' Ouverture du carnet d'adresses Contacts
Set objNamespace = objOle.GetNamespace("MAPI")
Set objAddrList = objNamespace.AddressLists("Contacts")
Set objAddrEntries = objAddrList.AddressEntries

' Accès au premier contact
Set objAdrEntry = objAddrEntries.GetFirst

' Ajout des contacts dans la liste
For i = 1 To objAddrEntries.Count
  With Me.lstContacts
    .AddItem objAdrEntry.Name
  End With

' Contact suivant
Set objAdrEntry = objAddrEntries.GetNext
Next i

End Sub
```


© ENI Editions - All rigths reserved

Le protocole DDE

DDE (*Dynamic Data Exchange*) est un protocole d'échange dynamique de données entre deux applications Windows, l'une appelée **Client** ou **Destination**, l'autre appelée **Serveur** ou **Source**. Le lien est rompu si l'une des deux applications vient à être fermée.

Une liaison DDE comprend trois phases principales :

- **initialisation** : l'application destination recherche l'application source et établit avec elle un canal de communication (liaison).

- **conversation** : des données sont échangées sur le canal.

- **clôture** : le canal de communication est fermé.

L'initialisation

- DDEInitiate

La fonction **DDEInitiate** permet d'initialiser la conversation entre deux applications. Elle renvoie le numéro de canal si la communication a pu être établie, sinon une erreur se produit.

Syntaxe

```
DDEInitiate (<application>,<sujet>)
```

`Application` : Nom de l'application Source.

`Sujet` : Groupe de données à utiliser.

Exemple

```
'crée un lien DDE avec la feuille feuil1 d'Excel
canal = DDEInitiate ("Excel","Feuil1")
```

La liaison

- DDE

La fonction **DDE** permet d'entamer une conversation **DDE** avec une autre application, de demander à cette dernière des éléments d'information pour les afficher dans un contrôle figurant sur un formulaire ou sur un état.

Syntaxe

```
DDE(<Application>, <Rubrique>, <Elément>)
```

`Application` : Expression chaîne identifiant une application.

`Rubrique` : Fichier document ou fichier de données.

`Elément` : Référence à une donnée de l'application Source.

Exemple

```
' Affecte le contenu de la cellule L1C1 à une zone
' de texte via sa propriété Source Contrôle.
=DDE("excel","classeur1","L1C1")
```

- DDESend

© ENI Editions - All rigths reserved

La fonction **DDESend** permet d'entamer une conversation **DDE** avec une autre application, et d'envoyer un élément d'information à cette application depuis un contrôle situé sur un formulaire ou sur un état.

Syntaxe

```
DDESend(<Application>, <Rubrique>, <Elément>, <Données>)
```

`Application` : Expression chaîne identifiant une application.

`Rubrique` : Fichier document ou fichier de données.

`Elément` : Référence à une donnée de l'application Source.

`Données` : Chaîne ou expression contenant les données à envoyer à application.

Exemple

```
' Affecte à la cellule L1C1 le contenu du contrôle
' Ville via la propriété Source Contrôle d'une zone '
de texte.
=DDESend("excel","classeur1","L1C1",[Ville])
```

- DDEPoke

L'instruction **DDEPoke** envoie des informations à l'application Source sur un canal ouvert.

Syntaxe

```
DDEPoke(<Numéro de canal>, <Élément>, <Donnée>)
```

`Numéro de canal` : N° de canal retourné par la fonction **DDEInitiate**.

`Élément` : Référence à une donnée de l'application Source.

`Donnée` : Donnée à envoyer à l'application Source.

Exemple

```
 'envoie la valeur 15 dans la cellule L1C1

DDEPoke canal,"L1C1","15"
```

- DDERequest

La fonction **DDERequest** demande des informations à l'application Source.

Syntaxe

```
DDERequest(<Numéro de canal>, <Élément>)
```

`Numéro de canal` : N° du canal retourné par la fonction **DDEInitiate**.

`Élément` : Référence à une donnée de l'application Source.

Exemple

```
'affecte le contenu de la cellule L1C1
'à la variable montant
montant = DDERequest (canal,"L1C1")
```

- DDEExecute

© ENI Editions - All rigths reserved

L'instruction **DDEExecute** envoie une chaîne de commandes à l'autre application en cours de liaison.

Syntaxe

```
DDEExecute(<Numéro de canal>, <Commande>)
```

`Numéro de canal` : N° du canal retourné par la fonction **DDEInitiate**.

`Commande` : Chaîne contenant une commande reconnue par l'autre application.

Exemple

```
'Crée une nouvelle feuille de calcul.
DDEExecute Canal,"[New(1)]"
```

La clôture

- DDETerminate

L'instruction **DDETerminate** ferme le canal de communication DDE.

Syntaxe

```
DDETerminate(<Numéro de canal>)
```

`Numéro de canal` : N° du canal retourné par la fonction **DDEInitiate**.

Exemple

```
DDETerminate canal
```

- DDETerminateAll

L'instruction **DDETerminateAll** ferme tous les canaux de communication DDE.

Syntaxe

```
DDETerminateAll
```

© ENI Editions - All rigths reserved

Import et export de données au format XML

Le langage XML (*eXtensible Markup Language*) est le langage standard de description et d'échange de données sur le Web, tandis que HTML (*HyperText Markup Language*) est le langage standard de création et d'affichage de pages Web.

XSL (*eXtensible Stylesheet Language*) est une instance de XML permettant de contrôler la présentation des documents XML.

Microsoft Access 2007 offre la possibilité d'importer ou d'exporter des données sous la forme d'un fichier XML et des présentations sous la forme d'un fichier XSL.

En VBA, vous pouvez utiliser les méthodes **ImportXML** et **ExportXML** de l'objet **Application** pour importer ou exporter des données au format XML.

La méthode ExportXML

La méthode **ExportXML** permet d'exporter les données et/ou la présentation d'un objet Access spécifié. Il est possible d'exporter au format XML les objets Access suivants :

- tables,
- requêtes,
- formulaires,
- états,
- pages d'accès aux données.

Syntaxe

```
Application.ExportXML(<ObjectType>, <DataSource>,
[<DataTarget>], [<SchemaTarget>],
[<PresentationTarget>], [<ImageTarget>], [<Encoding>],
[<OtherFlags>])
```

`ObjectType` : Obligatoire. Type d'objet Access à exporter (**acExportDataAccessPage**, **acExportForm**, **acExportReport**...).

`DataSource` : Obligatoire. Chaîne de caractères indiquant le nom de l'objet Access à exporter.

`DataTarget` : Facultatif. Chaîne de caractères indiquant le nom de fichier et le chemin des données exportées. Si cet argument est omis, les données ne sont pas exportées.

`SchemaTarget` : Facultatif. Chaîne de caractères indiquant le nom de fichier et le chemin des informations sur les schémas exportés. Si cet argument est omis, ces informations sont incorporées dans le document des données.

`PresentationTarget` : Facultatif. Chaîne de caractères indiquant le nom de fichier et le chemin des informations de présentation exportées. Si cet argument est omis, ces informations ne sont pas exportées.

`ImageTarget` : Facultatif. Chaîne de caractères indiquant le chemin pour les images exportées. Si cet argument est omis, les images ne sont pas exportées.

`Encoding` : Facultatif. Indique l'encodage de texte à utiliser pour le fichier XML exporté.

`OtherFlags` : Facultatif. Entier long qui représente la somme des valeurs d'options décrites dans la liste suivante.

© ENI Editions - All rigths reserved

Valeur	Description
1	**Tables connexes** : inclut les différentes tables concernant l'objet spécifié par DataSource.
2	**Propriétés relationnelles** : crée des propriétés de schéma relationnel.
4	**Exécution à partir du serveur** : crée un wrapper ASP ; sinon, c'est un wrapper HTML par défaut. S'applique uniquement lors de l'exportation d'états.
8	**Propriétés spéciales** : crée des propriétés de schéma étendues.

La méthode ImportXML

La méthode **ImportXML** permet d'importer des données et/ou des informations de présentation dans une table Microsoft Access à partir d'un fichier XML.

Syntaxe

```
Application.ImportXML(<DataSource>, [<ImportOptions>])
```

`Datasource` : Chaîne de caractères contenant le nom et le chemin du fichier XML à importer.

`ImportOptions` : Facultatif. Définit les options relatives à l'import du fichier XML.

Constantes utilisables pour définir les options d'importation.

`acAppendData` : Si une table porte déjà le nom du fichier XML, les données sont ajoutées à la table, sinon la table est créée.

`acStructureAndData` : Valeur par défaut. Importe la structure et les données. Si une table porte déjà le nom du fichier XML, Access génère un nouveau nom de table.

`acStructureOnly` : Importe uniquement la structure. Si une table porte déjà le nom du fichier XML, Access génère un nouveau nom de table.

Exemple d'import/export XML

Dans l'exemple suivant, nous allons voir comment :

- exporter une table au format XML,
- afficher le fichier XML dans le navigateur Internet Explorer,
- exporter une présentation de formulaire,
- importer le fichier XML dans une nouvelle table après avoir modifié son contenu.

Pour réaliser cet exemple, vous devez créer un répertoire "C:\XML".

Étape 1 : Export de la table Clients

- Saisissez le code suivant dans un nouveau module appelé **ImportExportXML** et exécutez la procédure **ExportTabClient**.

```
Private Sub ExportTabClient()
' Export de la table Client
ExportTable ("Clients")
End Sub

Private Sub ExportTable(strTableName As String)
' Export des données (XML) et de leur présentation
(XSL)
```

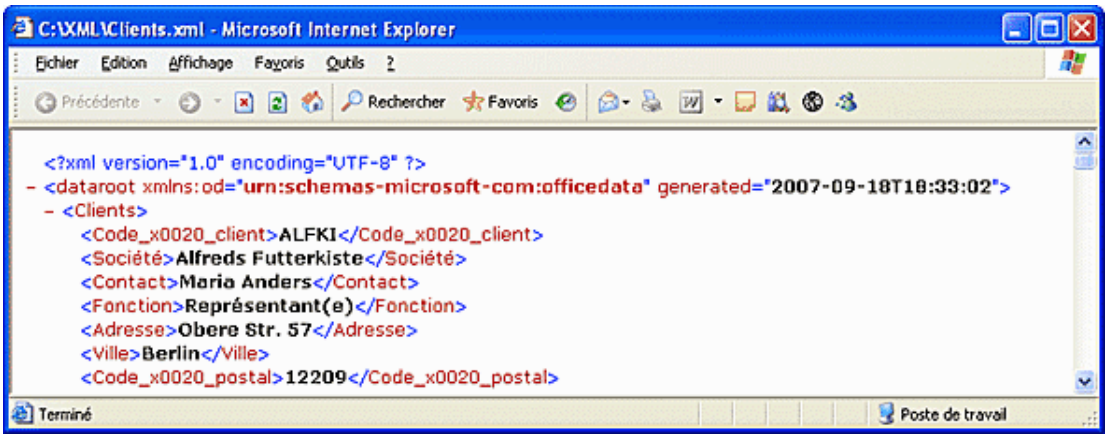

© ENI Editions - All rigths reserved

```
Application.ExportXML _
    ObjectType:=acExportTable, _
    DataSource:=strTableName, _
    DataTarget:="C:\XML\" & strTableName & ".xml", _
    PresentationTarget:="C:\XML\" & strTableName & ".xsl", _
    Encoding:=acUTF8
End Sub
```

Étape 2 : Lien entre les fichiers XML et XSL

Dans le répertoire XML, les trois fichiers suivants ont été créés : **clients.xml**, **clients.xsl** et **clients.htm**.

Si vous ouvrez le fichier **clients.xml** dans Internet Explorer, vous obtenez le résultat suivant :



Pour pouvoir afficher correctement la liste des clients, vous devez soit utiliser le fichier **clients.htm** qui fait le lien entre les fichiers xml et xsl, soit modifier le fichier **clients.xml** afin de lui associer le fichier **clients.xsl** (insérer la deuxième ligne).

» Pour modifier le code contenu dans les fichiers xml ou xsl, vous pouvez utiliser le bloc-note de Windows (Notepad.exe) ou tout autre éditeur de fichiers texte.

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="Clients.xsl"?>
<dataroot xmlns:od="urn:schemas-microsoft-com:officedata">
<Clients>
```

Étape 3 : Export d'une présentation de formulaire

- Saisissez le code suivant dans le module **ImportExportXML** et exécutez la procédure **ExportFormClient**.

```
Private Sub ExportFormClient()
' Export de la table Client
  ExportFormulaire ("Clients")
End Sub

Private Sub ExportFormulaire(strFormName As String)
  ' Export de de la présentation (XSL) d'un formulaire
    Application.ExportXML _
    ObjectType:=acExportForm, _
    DataSource:=strFormName, _
    PresentationTarget:="C:\XML\" & strFormName & ".xsl", _
    Encoding:=acUTF8

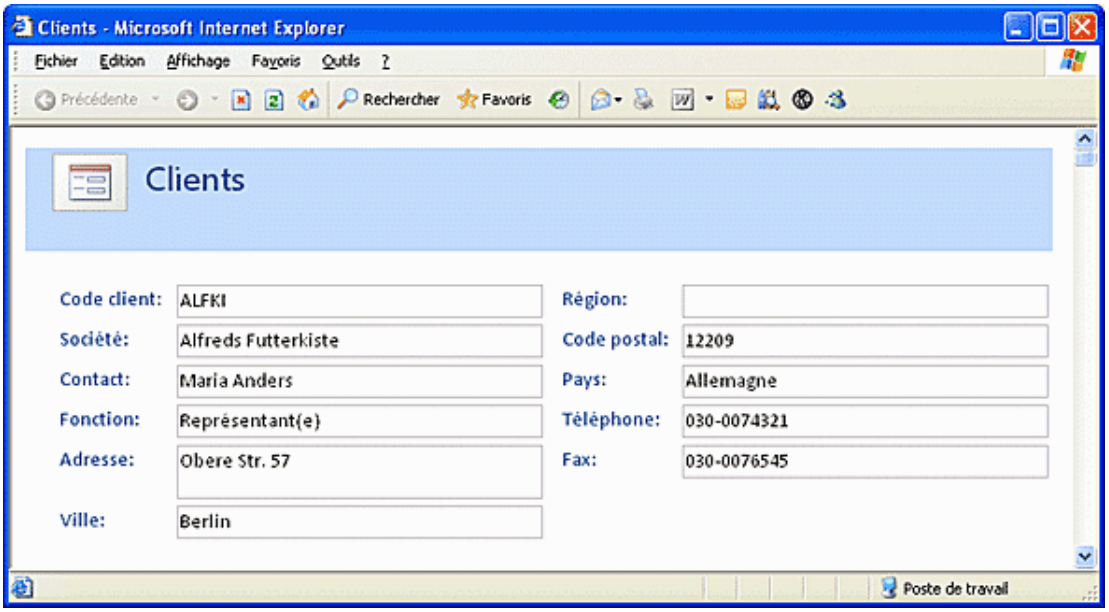
End Sub
```

Le fichier clients.xsl a été remplacé dans le répertoire C:\XML.

- Ouvrez le fichier **clients.xml** dans Internet Explorer.

Vous obtenez maintenant le résultat suivant :

© ENI Editions - All rigths reserved



Étape 4 : Import du fichier Clients.XML après modification

- Modifiez le fichier **clients.xml** de manière à ne conserver que le premier enregistrement.

```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="Clients.xsl"?>
<dataroot xmlns:od="urn:schemas-microsoft-com:officedata">
<Clients>
<Code_x0020_clientALFKI</Code_x0020_client>
<SociétéAlfreds Futterkiste</Société>
<Contact>Maria Anders</Contact>
<Fonction>Représentant(e)</Fonction>
<Adresse>Obere Str. 57</Adresse>
<Ville>Berlin</Ville>
<Code_x0020_postal>12209</Code_x0020_postal>
<Pays>Allemagne</Pays>
<Téléphone>030-0074321</Téléphone>
<Fax030-0076545</Fax>
</Clients>
</dataroot>
```

- Saisissez le code suivant dans le module **ImportExportXML**.

- Sauvegardez votre table clients sous un autre nom dans Access.

- Exécutez la procédure **ImportTableClient**.

```
Private Sub ImportTableClient()
' Import de la table Client
  ImportTable ("Clients")
End Sub

Private Sub ImportTable(strTableName As String)
' Import d'une table
  Application.ImportXML "C:\XML\" & strTableName & ".xml"

End Sub
```

» La table **Clients** est créée dans Access avec un seul enregistrement.

© ENI Editions - All rigths reserved

Exemple de création d'un fichier HTML

Vous pouvez à partir du langage VBA créer un fichier HTML. Pour cela vous devez utiliser l'objet **FileSystemObject**.

» Les méthodes et propriétés de cet objet sont détaillées dans le chapitre **Programmation Windows**.

L'exemple suivant permet de créer le fichier **ImageListe.htm** contenant toutes les images du répertoire sélectionné par l'utilisateur.

```
Private Sub CreeHTML()
Dim fso As Object
' Dossier
Dim fld As Object
' Collection Files
Dim fls As Object
' Objet File
Dim fl As Object

' Fichier Html
Dim fHtml As Object
Dim strFolder As String
Dim strHtml As String

' Sélection du dossier
strFolder = InputBox("Saisissez le chemin du dossier
contenant " _
& "les images", "Test VBA", "C:\")

' Recherche des fichiers images
Set fso = CreateObject("Scripting.FileSystemObject")
If fso.folderExists(strFolder) Then
  Set fld = fso.GetFolder(strFolder & "\")
  Set fls = fld.Files
  strHtml = "<html>"

  For Each fl In fls
    If Left(fl.Type, 5) = "Image" Then
      strHtml = strHtml & "<img src=" & "'" & strFolder & "\"
_
     & fl.Name & "'" height=" & x & 100 & x & ">" _
& fl.Name & "<BR>"
    End If
  Next
  ' Crée le fihier HTML
  strHtml = strHtml & "</html>"
  Set fHtml = fso.CreateTextFile(strFolder & "\" & _
  "ImageList.htm", True)
  fHtml.Write (strHtml)
Else
  MsgBox "Dossier " & strFolder & " non trouvé"
End If

End Sub
```

Aperçu dans Internet Explorer d'un fichier HTML créé par le code de l'exemple.

© ENI Editions - All rigths reserved



© ENI Editions - All rigths reserved

Présentation des API

L'interface de programmation Windows API (*Application Programming Interface*) offre des fonctions qui permettent de contrôler les aspects les plus internes du système d'exploitation. Vous pouvez étendre et personnaliser vos applications Access en appelant des fonctions Windows API à partir de VBA. Bien qu'Access continue à évoluer et que son langage de programmation natif (VBA) intègre de plus en plus de fonctions systèmes, il est nécessaire pour certaines tâches des fonctions de l'API.

Une API est un ensemble de fonctions que vous pouvez utiliser pour travailler avec un composant, une application ou le système d'exploitation. Elle se compose généralement d'une ou de plusieurs DLL (*Dynamic Link Library* ou bibliothèque de liaison dynamique).

L'API la plus couramment utilisée est l'**API Windows** qui inclut les DLL constituant le système d'exploitation Windows. Chaque application Windows interagit directement ou indirectement avec l'API Windows. Ceci garantit un comportement cohérent de toutes les applications fonctionnant sous Windows.

Les DLL de Windows les plus couramment utilisées sont les suivantes :

Kernel32.dll : Fonctions de bas niveau du système d'exploitation, telles que gestion de la mémoire et gestion des ressources.

User32.dll : Fonctions de gestion Windows, telles que traitement des messages, horloges, menus et communication.

GDI32.dll : Bibliothèque GDI (*Graphics Device Interface*) qui contient des fonctions de sortie vers les périphériques (graphisme, contexte d'affichage et gestion des polices).

D'autres API sont également disponibles comme par exemple l'interface **MAPI** (*Mail Application Programming Interface*) qui permet d'écrire des **applications de courrier électronique**.

Pour obtenir des informations sur les fonctions de l'API Windows, il existe deux sources d'informations :

- la **visionneuse d'API** (fichier exécutable ApiLoad.Exe inclus dans Microsoft Office XP Developer et dans Microsoft Visual Basic) permet d'afficher les constantes, déclarations et les types d'API. Les éléments sélectionnés peuvent être copiés de la Visionneuse d'API vers les applications VBA.
  Les informations de la visionneuse d'API sont contenues dans des fichiers texte (win32api.txt, mapi32.txt...) pouvant être exportés dans une base mdb pour en faciliter la consultation ultérieure.

- la **plate-forme Microsoft SDK** (Software Development Kit) contient une documentation complète de l'API Windows et est disponible gratuitement sur le site Microsoft Developer Network.

© ENI Editions - All rigths reserved

Appel d'une fonction de l'API Windows

Pour appeler une fonction de l'API Windows, vous devez la déclarer en utilisant l'instruction **Declare** dans la section Déclarations d'un module de votre projet (généralement un module dédié aux procédures générales de l'application).

Syntaxe de l'instruction Declare

Dans les versions 32 bits de Visual Basic, le respect des minuscules et majuscules est obligatoire pour les noms de fonctions et de procédures des DLL.

```
[Public|Private] Declare Sub <nom_proc> Lib "<Nom_DLL>"_
[Alias "<Nom_alias>"] [([liste_arguments])]

[Public|Private] Declare Function <Nom_fonc> _
Lib "<Nom_DLL>"_

[Alias "<Nom_alias>"] [([Liste_arguments])] [As <Type>]
```

`Nom_proc, Nom_fonc` : Nom de la procédure ou de la fonction tel qu'utilisé dans Visual Basic.

`Nom_DLL` : Nom de la DLL.

`Nom_alias` : Nom de la procédure ou de la fonction dans la DLL.

`Liste_arguments` : Paramètres à passer à la procédure (cf. chapitre **Le langage Visual Basic**).

`Type` : Type de la valeur du retour d'une procédure Function.

Certaines DLL ne fournissent pas de nom pour leurs procédures et leurs fonctions mais plutôt un nombre ordinal. La déclaration de telles procédures ou fonctions utilise la même syntaxe, mais il faut définir le nombre ordinal au niveau de l'Alias sous la forme d'un dièse (#) suivi du nombre (ex : Alias "#52").

Passage des arguments

Les fonctions et procédures des DLL sont écrites pour la plupart en langage C et utilisent sa syntaxe. Aussi, le passage d'arguments à une procédure ou à une fonction d'une DLL depuis Visual Basic, n'est pas toujours simple. Au niveau des DLL utilisant la syntaxe du C, tous les arguments sont passés par valeurs, sauf les tableaux.

Les chaînes en langage C sont considérées comme des tableaux de caractères.
Certains arguments de procédures de DLL peuvent accepter différents types de données (un peu comme les Variant), ils doivent être déclarés avec le type Any (ex : `variable As Any`). Visual Basic, pour ce type d'argument, considère que celui-ci est passé systématiquement par référence ; s'il y a lieu de le passer par valeur, il faut l'expliciter au niveau de l'appel (et non de la déclaration) de la procédure ou de la fonction avec la mention `ByVal`.

Par défaut, Visual Basic passe les arguments par Référence.

Tableau des déclarations des arguments

Déclaration en langage C	**Déclaration Visual Basic**	**Appel**
BOOL	ByVal <variable> As Boolean	Une expression dont l'évaluation produit une valeur de type Boolean.
long, DWORD, LONG, COLORREF	ByVal <variable> As Long	Une expression dont l'évaluation produit une valeur de type Long.
WORD	ByVal <variable> As Long	Une expression dont l'évaluation produit une valeur de type Long.
NULL	As Any ou ByVal <variable> As Long	ByVal Nothing ou ByVal 0$ ou vbNullString.

© ENI Editions - All rigths reserved

int, short	Byval <variable> As Integer	Une expression dont l'évaluation produit une valeur de type Integer.
char, BYTE	Byval <variable> As Byte	Une expression dont l'évaluation produit une valeur de type Byte.
Entier (INT, UINT) 32 bits avec compilateur 32 bits	ByVal <variable> As Long	Une expression dont l'évaluation produit une valeur de type Long.
Identificateur Windows (hWnd, hDC, hMenu...)	ByVal <variable> As Long	Une expression dont l'évaluation produit une valeur de type Long.
Vide (valeur de renvoi de fonction)	Sub <procédure>	Sans objet.
Pointeur vers une chaîne (LPSTR)	ByVal <variable> As String	Une expression dont l'évaluation produit une valeur de type String.
Pointeur vers un entier long (LPDWORD)	<variable> As Long	Une variable de type Long.
Pointeur vers un vide	<variable> As Any	Une variable (utilisez ByVal pour passer une chaîne).
Pointeur vers un char	<variable> As Byte	Une expression dont l'évaluation produit une valeur de type Byte.
Pointeur vers un entier (LPINT)	<variable> As Long	Une variable de type Long.
Pointeur vers une structure	<variable> As <type>	Une variable de type définie par l'utilisateur.

© ENI Editions - All rigths reserved

Liste de fonctions API Windows

Cette liste comporte les fonctions de l'API Windows couramment utilisées. Des exemples d'utilisation de certaines de ces fonctions sont données dans le paragraphe suivant.

GetWindowsDirectory() : Renvoie le chemin complet du répertoire de Windows (applications, outils du système).

GetSystemDirectory() : Renvoie le chemin complet du répertoire système de Windows.

GetSystemInfo() : Renvoie tout un ensemble d'informations sur le système. Ces données sont stockées dans une structure de type SYSTEM_INFO.

GetActiveWindow() : Renvoie le handle de la fenêtre active.

FindWindow() : Renvoie le handle de la fenêtre en fonction de son nom et de la classe à partir de laquelle elle a été définie.

SetFocus() : Attribue le focus d'entrée à la fenêtre référencée par son handle.

GetPrivateProfileString() : Renvoie une option extraite d'un fichier **.Ini** à partir d'un nom de section et de clé.

WNetGetUser() : Renvoie le nom du login réseau utilisé pour la session en cours.

© ENI Editions - All rigths reserved

Exemples d'utilisation de fonctions API Windows

Récupération du répertoire Windows

Déclaration de la fonction API.

```
Private Declare Function GetPrivateProfileString Lib
"KERNEL32" _
        Alias "GetPrivateProfileStringA" _
            (ByVal lpApplicationName As String, _
            ByVal lpKeyName As Any, _
            ByVal lpDefault As String, _
            ByVal lpReturnedString As String, _
            ByVal nSize As Long, _
            ByVal lpFileName As String) As Long
```

Appel de la fonction API.

```
Function GetWinPath() As String

' Cette fonction VBA renvoie le répertoire Windows
Dim strResult As String
Dim strProfile As String

strResult = String(255, " ")
strProfile = GetWindowsDirectory(strResult, 255)

  ' Tronque la chaîne au premier caractère nul
If strProfile <> "" Then
  strResult = Trim(strResult)
  GetWinPath = Left(strResult, InStr(1, strResult,
vbNullChar) - 1)
Else
  GetWinPath = ""
End If
End Function
```

Lancement de l'application Excel si non active

Cet exemple permet de rechercher si l'application Excel est active, et de lancer son exécution si besoin est.

Deux fonctions API sont ici utilisées : **FindWindow** permet de rechercher la fenêtre Excel, **FindExecutable** de rechercher l'emplacement du fichier Excel.exe.

Déclaration de la fonction API.

```
' Cette fonction API recherche une fenêtre
Public Declare Function FindWindow Lib "user32" Alias _
"FindWindowA" (ByVal lpClassName As String, ByVal _
lpWindowName As String) As Long

' Cette fonction API recherche un fichier exécutable
Public Declare Function FindExecutable Lib "shell32.dll" _
Alias "FindExecutableA" (ByVal lpFile As String, _
ByVal lpDirectory As String, ByVal lpResult As String) As
Long
```

Appel des fonctions API.

```
Private Function LanceExcel() As Boolean
Dim strClassName As String
Dim strWindowName As String
Dim Hwnd As Long
Dim blnExe As Boolean
Dim strRepExcel As String
Dim Result As Long

' Initialisation
```

© ENI Editions - All rigths reserved

```
LanceExcel = False

' Recherche de la fenêtre Excel active
strClassName = vbNullString
strWindowName = "Microsoft Excel - Classeur1"
Hwnd = FindWindow(strClassName, strWindowName)

' Si non trouvée recherche du répertoire Excel
' et lancement de l'application Excel
If Hwnd = 0 Then
  strRepExcel = String$(255, 0)
  Result = FindExecutable("Excel.exe", "C:\", strRepExcel)
  If Result = 0 Then Exit Function
  blnExe = Shell(strRepExcel, vbNormalFocus)
  If Not blnExe Then Exit Function
End If
LanceExcel = True

End Function
```

Récupération d'une valeur dans un fichier .ini

Le fichier AppDevis.ini est constitué des trois lignes suivantes :

```
[BASE]
BasePath=C:\DEVIS
BaseName=DEVIS.ACCDB
```

L'exemple suivant permet de récupérer le nom et le chemin d'accès dans le fichier AppDevis.ini situé dans le répertoire Windows (en faisant appel à la function GetWinPath de l'exemple 1).

Déclaration de la fonction API.

```
Public Declare Function GetPrivateProfileString Lib
"kernel32" _
  Alias "GetPrivateProfileStringA" _
  (ByVal lpApplicationName As String, _
  ByVal lpKeyName As Any, _
  ByVal lpDefault As String, _
  ByVal lpReturnedString As String, _
  ByVal nSize As Long, _
  ByVal lpFileName As String) As Long
```

Appel de la fonction API.

```
Public Function LireCle(Section As String, Clé As String, _
Fichier As String) As String

' Cette fonction renvoie la valeur de la clé
' qu'on peut trouver dans un fichier .ini quelconque
Dim strResult As String
Dim strProfile As Integer
Dim i As Integer

strResult = String(255, " ")
strProfile = GetPrivateProfileString(Section, Clé, "", _
       strResult, 255, Fichier)
If strProfile = 0 Then
      LireCle = ""
      Exit Function
End If

' Tronque la chaine au premier caractère nul
strResult = Trim(strResult)
LireCle = Left(strResult, InStr(1, strResult, vbNullChar) -
1)

End Function
```

Appel de la fonction VBA LireCle.

© ENI Editions - All rigths reserved

```
Private Sub Init_Projet()
Dim strBaseName As String
Dim strPath As String
Dim strWinPath As String

' Recherche du nom et du répertoire de la base
strWinPath = GetWinPath()
If strWinPath = "" Then Exit Sub

' Recherche de valeurs dans Devis.ini
strBaseName = Trim(LireCle("BASE", "BASENAME", strWinPath _
& "\" & "Devis.ini"))
strPath = Trim(LireCle("BASE", "BASEPATH", strWinPath & "\" _
& "Devis.ini"))

End Sub
```

© ENI Editions - All rigths reserved

L'objet FileSystemObject

L'objet **FileSystemObject** donne accès au système de fichier d'un ordinateur. Il permet notamment de rechercher, créer, supprimer ou déplacer des fichiers ou des dossiers.

Méthodes

Méthode relatives aux fichiers

CopyFile : Copie un ou plusieurs fichiers d'un emplacement vers un autre.

CreateTextFile : Crée un fichier sur le disque portant le nom spécifié et renvoie un objet **TextStream** pouvant être utilisé pour lire ou écrire dans le fichier.

DeleteFile : Supprime un fichier spécifié.

FileExists : Renvoie un booléen indiquant si le fichier spécifié existe.

MoveFile : Déplace un ou plusieurs fichiers d'un dossier vers un autre.

OpenTextFile : Ouvre le fichier spécifié et renvoie un objet TextStream pouvant être utilisé pour lire le fichier ou y effectuer un ajout.

Méthode relatives aux dossiers

CopyFolder : Copie un dossier d'un emplacement vers un autre.

CreateFolder : Crée un dossier.

DeleteFolder : Supprime le dossier spécifié et son contenu.

FolderExists : Renvoie un booléen indiquant si le dossier spécifié existe.

MoveFolder : Déplace un ou plusieurs dossiers d'un emplacement vers un autre.

Méthode relatives aux lecteurs

DriveExists : Renvoie un booléen indiquant si le lecteur logique spécifié existe.

GetDrive : Renvoie un objet **Drive** correspondant au lecteur dans un chemin spécifié.

GetDriveName : Renvoie une chaîne contenant le nom du lecteur pour un chemin spécifié.

Propriétés

Drives : Renvoie une collection constituée de tous les objets **Drive** disponibles sur la machine locale.

Exemple

L'exemple suivant permet d'afficher la liste des lecteurs disponibles et l'espace disponible sur chacun d'eux.

```
Sub AfficheLecteur()
Dim fso As Object
Dim strMsg As String
Dim drv As Object
Dim drvs As Object

Set fso = CreateObject("Scripting.FileSystemObject")
Set drvs = fso.Drives
For Each drv In drvs
  If drv.isready Then
```

© ENI Editions - All rigths reserved

```
    strMsg = strMsg & drv.driveletter & ": " _
    & Format(EspaceDisponible(drv.driveletter & ":"), _
    "# ### ##0") & " KOctets" & vbCrLf
  End If
Next
MsgBox strMsg
End Sub
```


© ENI Editions - All rigths reserved

Présentation

Énoncé du problème

Le service Ressources Humaines d'une entreprise souhaite effectuer un suivi des formations auxquelles participent ses employés.

L'application Access, réalisée à cet effet et présentée dans ce chapitre, permet :

- de saisir les stages de formations,

- de rechercher les stages de formation en fonction d'un ensemble de critères,

- d'imprimer la liste des stages correspondants en rappelant les différents critères de sélection.

Base de données Formations

La base de données Formations contenant cette application est constituée de deux formulaires et d'un état :

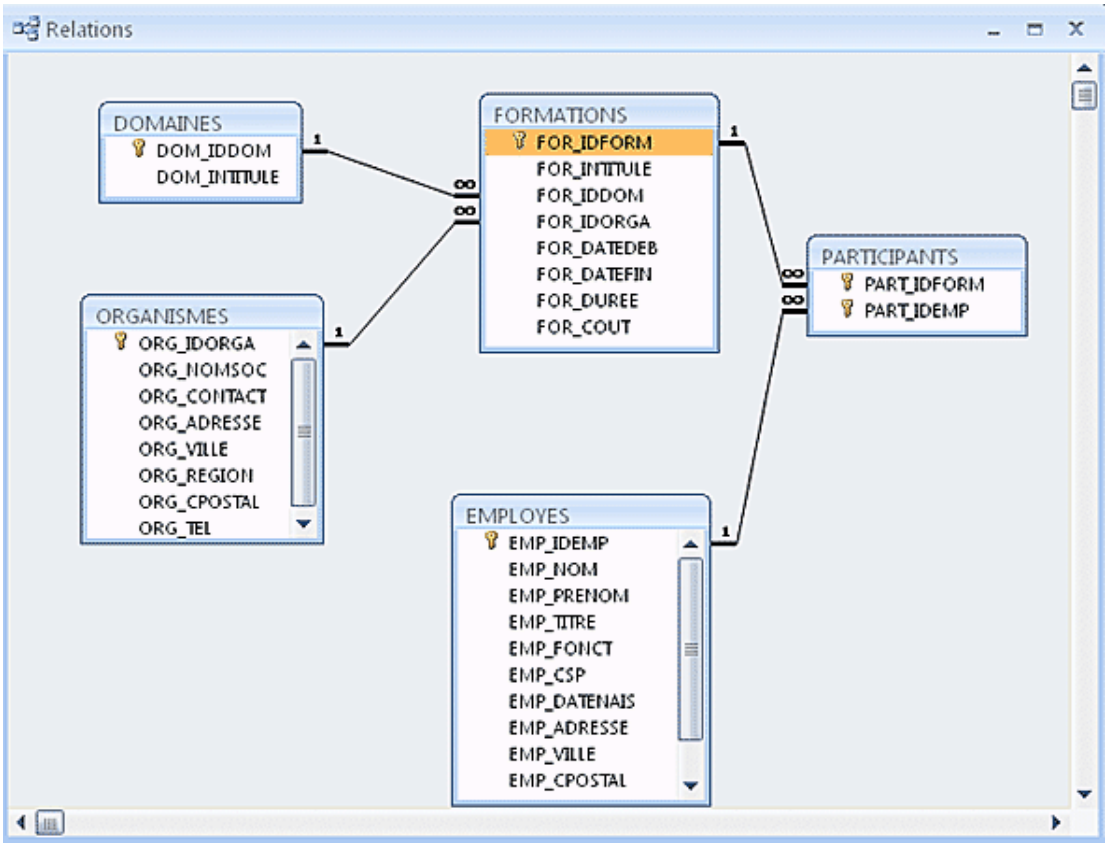
- le formulaire Formations permet de saisir, modifier et supprimer des stages de formation.

- Le formulaire Recherche permet de rechercher des stages en fonction de différents critères.

- L'état Formations affiche la liste des stages sélectionnés dans le formulaire Recherche.

» L'option **Effacer en cascade** est activée sur la relation entre les tables Formations et Participants.

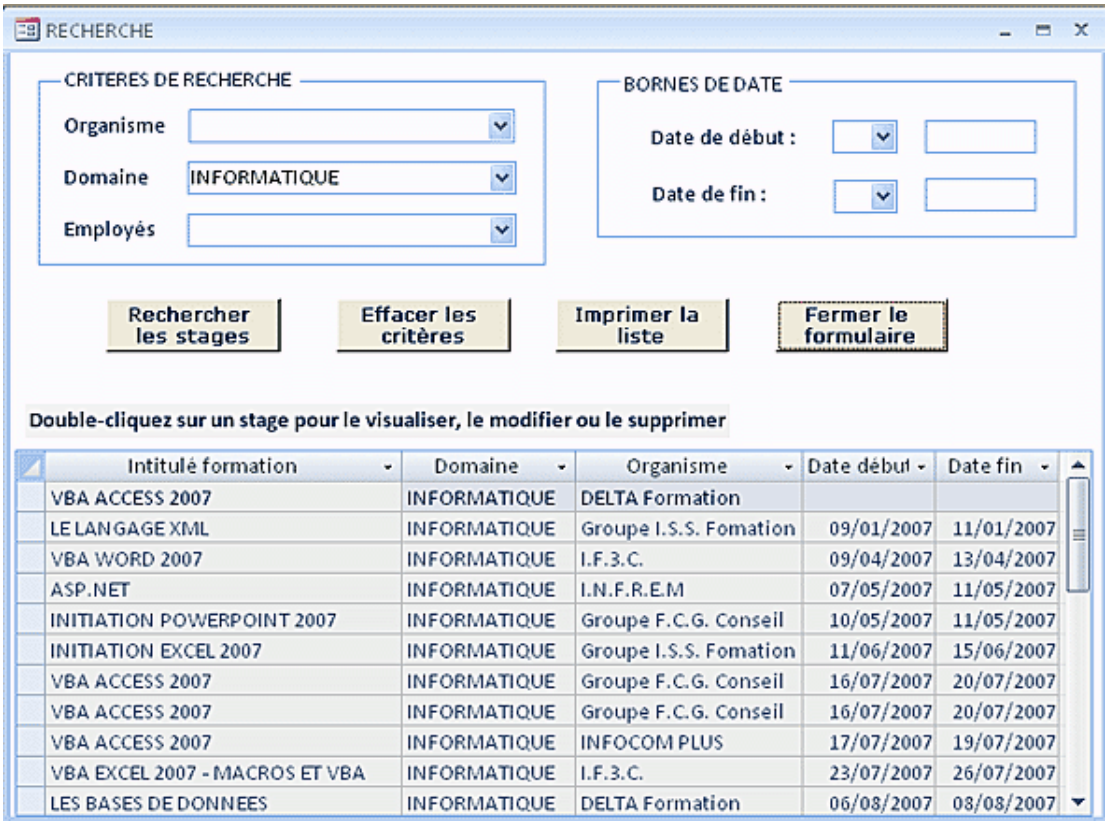
Modèle relationnel de la base



© ENI Editions - All rigths reserved

Formulaire "Formations"

Ce formulaire peut être soit ouvert directement à partir d'Access pour ajouter des stages, soit appelé à partir du formulaire "Recherche" pour modifier ou supprimer un stage existant.



Intitulé formation	Domaine	Organisme	Date début	Date fin
VBA ACCESS 2007	INFORMATIQUE	DELTA Formation		
LE LANGAGE XML	INFORMATIQUE	Groupe I.S.S. Fomation	09/01/2007	11/01/2007
VBA WORD 2007	INFORMATIQUE	I.F.3.C.	09/04/2007	13/04/2007
ASP.NET	INFORMATIQUE	I.N.F.R.E.M	07/05/2007	11/05/2007
INITIATION POWERPOINT 2007	INFORMATIQUE	Groupe F.C.G. Conseil	10/05/2007	11/05/2007
INITIATION EXCEL 2007	INFORMATIQUE	Groupe I.S.S. Fomation	11/06/2007	15/06/2007
VBA ACCESS 2007	INFORMATIQUE	Groupe F.C.G. Conseil	16/07/2007	20/07/2007
VBA ACCESS 2007	INFORMATIQUE	Groupe F.C.G. Conseil	16/07/2007	20/07/2007
VBA ACCESS 2007	INFORMATIQUE	INFOCOM PLUS	17/07/2007	19/07/2007
VBA EXCEL 2007 - MACROS ET VBA	INFORMATIQUE	I.F.3.C.	23/07/2007	26/07/2007
LES BASES DE DONNEES	INFORMATIQUE	DELTA Formation	06/08/2007	08/08/2007

La propriété source du formulaire est Formations (Table Formations).

Liste des contrôles

	Nom du contrôle	Description
1	txtInitulé	Zone de texte associée au champ FOR_INTITULE
2	cboOrganisme	Zone de liste déroulante basée sur la table Organismes et associée au champ FOR_IDORGA
3	cboDomaine	Zone de liste déroulante basée sur la table Domaines et associée au champ FOR_IDDOM
4	txtCout	Zone de texte associée au champ FOR_COUT
5	txtCoutStage	Zone de texte
6	txtDateDeb	Zone de texte au format Date associée au champ FOR_DATEDEB
7	txtDateFin	Zone de texte au format Date associée au champ FOR_DATEFIN
8	txtDuree	Zone de texte associée au champ FOR_DUREE
9	sFrmParticip	Contrôle sous-formulaire affichant le sous-formulaire Access SF Participants
10	txtNbStag	Zone de texte
11	cmdEnregistrer	Bouton de commande
12	cmdSupprimer	Bouton de commande
13	cmdFermer	Bouton de commande

© ENI Editions - All rigths reserved

Contrôle	Propriété spécifique
Form	Entrées Données = Oui Source = Formations
txtNbStag	Valeur par défaut = 0 Verrouillé = Oui Source Contrôle = [sFrmParticipant].[Formulaire]![txtNbStag]
txtCoutStag	Verrouillé = Oui

Code VBA

Le code VBA associé à ce formulaire montre les aspects suivants :

- synchronisation entre formulaire et sous-formulaire : lors de l'ajout ou de la suppression d'un participant, le nombre de stagiaires et le coût par stagiaires sont recalculés et réaffichés,

- gestion des doublons au niveau du sous-formulaire,

- utilisation d'un contrôle calendrier pour saisir des dates,

- appel d'une fonction pour effectuer des calculs sur les dates,

- ...

Code VBA général à l'application : procédures et déclarations communes

- Insérez le code suivant dans un module standard (ProcGene par exemple).

```
Option Compare Database
Option Explicit
' Critères de recherche
Public p_strCond As String
' N° de l'employé
Public p_lngEmp As Long
' Titre de l'état
Public p_strTitre As String
_____________________________________________________________
Function NbJourOuvres(Date1, Date2) As Integer
Dim NbJour, NbSem As Integer

' Calcule le nombre de jours ouvrés
' entre deux dates
NbJour = DateDiff("d", Date1, Date2) + 1
' Nombre de semaines entre les 2 dates
NbSem = DateDiff("ww", Date1, Date2)
NbJourOuvres = NbJour - (NbSem * 2)

End Function
```

Code VBA du formulaire

```
Option Explicit
Dim BlnMaj As Boolean
_____________________________________________________________
Private Sub cmdEnregistrer_Click()
Dim intRep As String

' Contrôle des champs obligatoires
If Not Ctrl_Info Then Exit Sub
If IsNull(txtDateDeb) Or IsNull(txtDateFin <> "") Then
  If MsgBox("Dates non<+>saisies, confirmez-vous " _
    & "l'enregistrement ?", vbQuestion & vbYesNo) = vbNo Then
    txtDateFin.SetFocus
  Exit Sub
 End If
End If
```


© ENI Editions - All rigths reserved

```
' Passe à la création du stage suivant
' si mode ajout de données
If Me.DataEntry Then
    txtDuree = "
    txtCoutStag = ""
    BlnMaj = False
    DoCmd.GoToRecord , , acNewRec
    txtIntitule.SetFocus
    CmdSupprimer.Enabled = False
    cmdEnregistrer.Enabled = False
Else
    DoCmd.Close
End If
End Sub
_____________________________________________________________________
Private Sub cmdFermer_Click()
' Teste si saisie d'un stage en cours
If txtIntitule <> "" Then
    MsgBox ("Enregistrez ou supprimez le stage avant de fermer")
Else
    DoCmd.Close
End If
End Sub
_____________________________________________________________________
Private Sub CmdSupprimer_Click()
On Error GoTo Err_Suppr
If BlnMaj Then

' Supprime l'enregistrement si modifié
    DoCmd.DoMenuItem acFormBar, acEditMenu, 8, , acMenuVer70
    DoCmd.DoMenuItem acFormBar, acEditMenu, 6, , acMenuVer70
Else
' Annule la saisie sinon
    DoCmd.DoMenuItem acFormBar, acEditMenu, acUndo, , acMenuVer70
End If
On Error GoTo 0
Exit Sub

Err_Suppr:
    MsgBox "Suppression impossible : " & Err.Number, vbCritical
    On Error GoTo 0
End Sub
_____________________________________________________________________
Private Sub Form_AfterUpdate()
' Indicateur de mise à jour
BlnMaj = True
End Sub


_____________________________________________________________________
Private Sub Form_Load()
' Désactive les boutons Supprimer
' et Enregistrer si mode ajout données
If Me.DataEntry Then
    cmdEnregistrer.Enabled = False
    CmdSupprimer.Enabled = False
Else
' Calcul du coût par stagiaire
    Me.Recalc
    If txtNbStag > 0 Then
      txtCoutStag = txtCout / txtNbStag
    End If
    BlnMaj = True
End If
End Sub
_____________________________________________________________________
Private Sub sFrmParticipant_Enter()
' Contrôle des informations obligatoires
Ctrl_Info
End Sub
_____________________________________________________________________
Private Sub txtCout_AfterUpdate()
' Calcul du coût / Stagiaire
If txtCout > 0 And txtNbStag > 0 Then
    txtCoutStag = txtCout / txtNbStag
End If
End Sub
_____________________________________________________________________
Private Sub txtDateDeb_LostFocus()
' Calcul de la durée du stage
```

© ENI Editions - All rigths reserved

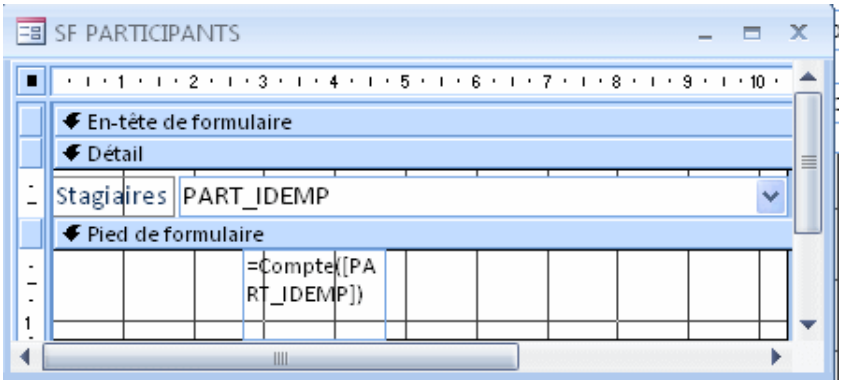
```
Calc_Durée
End Sub
_____________________________________________________________________
Private Sub txtDateFin_LostFocus()
' Calcul de la durée du stage
Calc_Durée
End Sub
_____________________________________________________________________
Private Sub txtIntitule_BeforeUpdate(Cancel As Integer)
' Active le bouton Supprimer
CmdSupprimer.Enabled = True
End Sub


_____________________________________________________________________
Private Function Ctrl_Info() As Boolean
' Contrôle si les champs obligatoires ont été saisis
If IsNull(txtIntitule) Then
    MsgBox "Intitulé obligatoire", vbCritical
    txtIntitule.SetFocus
    Exit Function
End If
If IsNull(cboOrganisme) Then
    MsgBox "Organisme obligatoire", vbCritical
    cboOrganisme.SetFocus
  Exit Function
End If
If IsNull(cboDomaine) Then
    MsgBox "Domaine obligatoire", vbCritical
    cboDomaine.SetFocus
    Exit Function
End If
If IsNull(txtCout) Then
    MsgBox "Montant du stage obligatoire", vbCritical
    txtCout.SetFocus
    Exit Function
End If
Ctrl_Info = True
CmdSupprimer.Enabled = True
cmdEnregistrer.Enabled = True
End Function
_____________________________________________________________________
Private Sub Calc_Durée()
Dim intDuree As Integer
' Si les deux dates sont renseignées
' Contrôle des dates et calcul de la durée
If txtDateDeb <> "" And txtDateFin <> "" Then
    intDuree = DateValue(txtDateFin) - DateValue(txtDateDeb)
    If intDuree < 0 Then
    MsgBox "Vous devez spécifier une date de début antérieure " _
    & "à la date de fin", vbCritical, "Gestion des Formations"
    Exit Sub
  Else
    txtDuree = NbJourOuvres(DateValue(txtDateDeb), _
        DateValue(txtDateFin))
End If
End If
End Sub
```

Sous-formulaire "SF Participants"

Le formulaire "**Formations**" et le sous-formulaire "**SF Participants**" sont liés sur les champs FOR_IDFORM et PART_IDFORM.

© ENI Editions - All rigths reserved



	Nom du contrôle	Description
1	cboEmploye	Zone de liste déroulante basée sur la table Employes associée au champ FOR_IDEMP
2	txtNbStag	Zone de liste indépendante

Contrôle	Propriété spécifique
Form	Source = Participants
txtNbStag	Source Contrôle = Compte([PART_IDEMP])

Code VBA

```
Private Sub cboEmploye_AfterUpdate()
' Si Doublons : la méthode Refresh échoue
On Error GoTo Doublons
Me.Refresh
On Error GoTo 0
' Recalcule le coût par stagiaire
' Si le formulaire principal est Formations
If UCase(Screen.ActiveForm.Name) = "FORMATIONS" Then
  CalcCoutStage
End If
Exit Sub

' Gestion des doublons : message d'erreur
Doublons:
If Err.Number = 3022 Then
  MsgBox "Participant déjà saisi"
Else
  MsgBox "Erreur lors de l'ajout du participant : " _
  & Err.Number, vbCritical
End If
' Remise à blanc du champ
cboEmploye = ""
End Sub
_____________________________________________________________
Private Sub Form_AfterDelConfirm(Status As Integer)
' Recalcule le coût par stagiaire
' Si le formulaire principal est Formations
If UCase(Screen.ActiveForm.Name) = "FORMATIONS" Then
  CalcCoutStage
End If
End Sub
_____________________________________________________________
Private Sub CalcCoutStage()
Dim frmFormat As Object

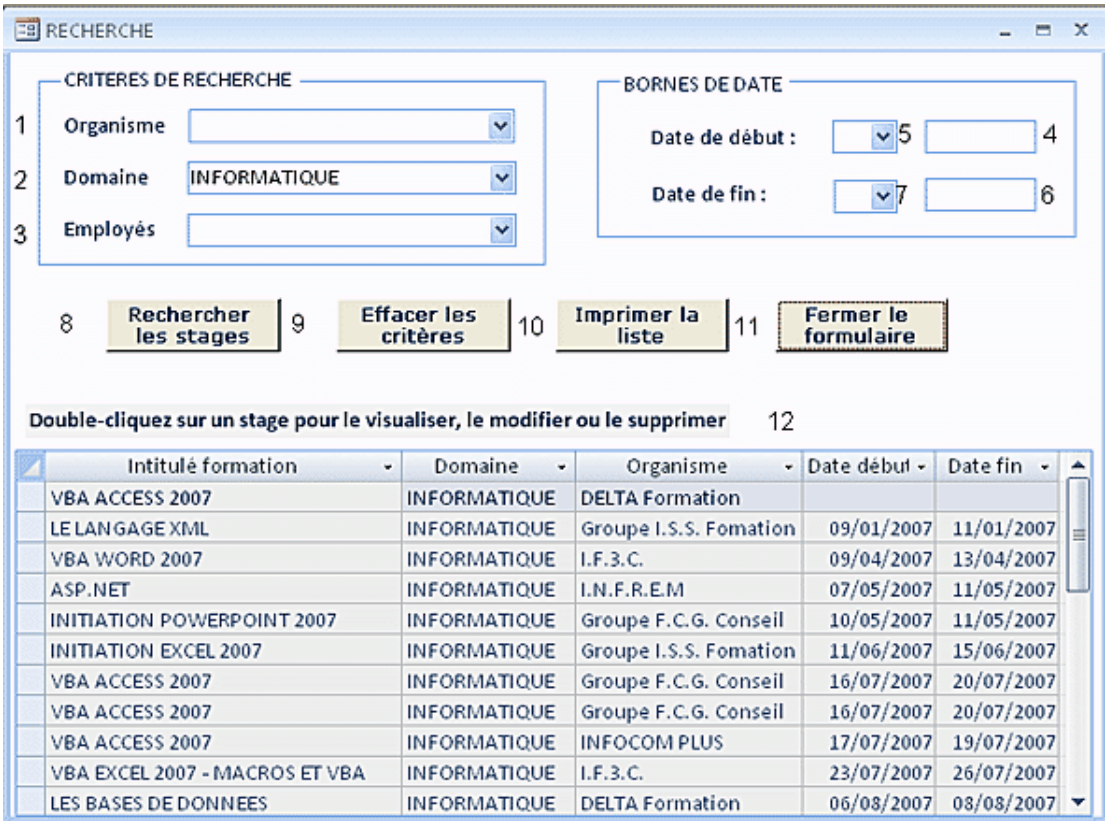
' Calcule le coût / Stagiaire
Set frmFormat = Forms![FORMATIONS]
frmFormat.Recalc
If frmFormat.txtCout <> 0 And txtNbStag <> 0 Then
  frmFormat.txtCoutStag = frmFormat.txtCout / txtNbStag
End If
' Active le bouton de commande Enregistrer
If txtNbStag <> 0 Then frmFormat.cmdEnregistrer.Enabled = True
End Sub
```

© ENI Editions - All rigths reserved

© ENI Editions - All rigths reserved

Formulaire "Recherche"

Ce formulaire permet d'afficher et d'imprimer la liste des stages répondant à certains critères. Ce formulaire n'est associé à aucune source de données.



Intitulé formation	Domaine	Organisme	Date début	Date fin
VBA ACCESS 2007	INFORMATIQUE	DELTA Formation		
LE LANGAGE XML	INFORMATIQUE	Groupe I.S.S. Fomation	09/01/2007	11/01/2007
VBA WORD 2007	INFORMATIQUE	I.F.3.C.	09/04/2007	13/04/2007
ASP.NET	INFORMATIQUE	I.N.F.R.E.M	07/05/2007	11/05/2007
INITIATION POWERPOINT 2007	INFORMATIQUE	Groupe F.C.G. Conseil	10/05/2007	11/05/2007
INITIATION EXCEL 2007	INFORMATIQUE	Groupe I.S.S. Fomation	11/06/2007	15/06/2007
VBA ACCESS 2007	INFORMATIQUE	Groupe F.C.G. Conseil	16/07/2007	20/07/2007
VBA ACCESS 2007	INFORMATIQUE	Groupe F.C.G. Conseil	16/07/2007	20/07/2007
VBA ACCESS 2007	INFORMATIQUE	INFOCOM PLUS	17/07/2007	19/07/2007
VBA EXCEL 2007 - MACROS ET VBA	INFORMATIQUE	I.F.3.C.	23/07/2007	26/07/2007
LES BASES DE DONNEES	INFORMATIQUE	DELTA Formation	06/08/2007	08/08/2007

Liste des contrôles

	Nom du contrôle	Description
1	cboOrganisme	Zone de liste déroulante basée sur la table Organismes
2	cboDomaine	Zone de liste déroulante basée sur la table Domaines
3	cboEmploye	Zone de liste déroulante basée sur la table Employes
4	txtDateDeb	Zone de texte au format Date
5	txtOperat1	Zone de liste déroulante contenant les valeurs "=",">","<"
6	txtDateFin	Zone de texte au format Date associée au champ FOR_DATEFIN
7	txtOperat2	Zone de liste déroulante contenant les valeurs "=",">","<"
8	cmdRechercher	Bouton de commande
9	cmdEffacer	Bouton de commande
10	cmdImprimer	Bouton de commande
11	cmdFermer	Bouton de commande
12	sFrmformation	Contrôle sous-formulaire affichant le sous formulaire Access SF Formations

Code VBA

Le code VBA associé à ce formulaire montre les aspects suivants :

- constitution d'une requête de type SQL à partir de critères saisis,

© ENI Editions - All rigths reserved

- affectation de la requête au sous-formulaire pour n'afficher que les enregistrements souhaités,

- affectation de la requête à un état pour n'imprimer que les enregistrements souhaités.

```
Option Compare Database
Option Explicit
Dim strSQL As String
_____________________________________________________________________
Private Sub cmdRechercher_Click()

' Crée la requête en fonction des critères saisis
If cboEmploye <> "" Then
  ' Tables formations et participants
  strSQL = "SELECT FOR_IDFORM, FOR_INTITULE, " _
  & "FOR_IDDOM, FOR_IDORGA, FOR_DATEDEB, " _
  & "FOR_DATEFIN, PART_IDEMP " _
  & "FROM FORMATIONS INNER JOIN PARTICIPANTS " _
  & "ON FORMATIONS.FOR_IDFORM = " _
  & "PARTICIPANTS.PART_IDFORM"
Else

' Table formation
  strSQL = "SELECT FOR_IDFORM, FOR_INTITULE, " _
  & "FOR_IDDOM, FOR_IDORGA, " _
  & "FOR_DATEDEB, FOR_DATEFIN FROM FORMATIONS "
End If

' Prépare la clause Where de la requête SQL
' en concaténant les critères
p_strCond = ""
p_lngEmp = 0
If cboDomaine <> "" Then
  p_strCond = p_strCond _
        & " AND FOR_IDDOM = " & cboDomaine
End If
If cboOrganisme <> "" Then
  p_strCond = p_strCond _
        & " AND FOR_IDORGA = " & cboOrganisme
End If
If cboEmploye <> "" Then
  p_strCond = p_strCond _
        & " AND PART_IDEMP = " & cboEmploye
  p_lngEmp = cboEmploye
End If

' Critères sur les dates
If cboOperat1 <> "" And txtDateDeb <> "" Then
  p_strCond = p_strCond & " AND (FOR_DATEDEB " & cboOperat1 _
  & " #" & Format(DateValue(txtDateDeb), "MM/DD/YY") & "#)"
End If
If cboOperat2 <> "" And txtDateFin <>"" Then
  p_strCond = p_strCond & " AND (FOR_DATEFIN " & cboOperat2 _
  & " #" & Format(DateValue(txtDateFin), "MM/DD/YY") & "#)"
End If

' Suppression du 1er AND
If p_strCond <> "" Then
  p_strCond = Right(p_strCond, (Len(p_strCond) - 4))
End If

' Affecte la requête SQL au sous-formulaire
If p_strCond <> "" Then
  strSQL = strSQL & " WHERE " & p_strCond & " ORDER BY FOR_DATEDEB"
Else
  strSQL = strSQL & " ORDER BY FOR_DATEDEB"
End If

SFrmFormations.Form.RecordSource = strSQL
SFrmFormations.Form.Requery

End Sub
_____________________________________________________________________
Private Sub cmdEffacer_Click()
Dim ctl As Control

' Efface les zones de texte et zones de listes
For Each ctl In Me.Controls
```


© ENI Editions - All rigths reserved

```
  If TypeOf ctl Is TextBox Or TypeOf ctl Is ComboBox Then
    ctl.Value = ""
  End If
Next
End Sub
```

```
Private Sub cmdImprimer_Click()

' Constitution du titre de l'état
p_strTitre = ""
If cboDomaine <> "" Then
  p_strTitre = p_strTitre & " - Domaine : " & cboDomaine.Column(1)
End If
If cboOrganisme <> "" Then
  p_strTitre = p_strTitre & " - Organisme : " & cboOrganisme.Column(1)
End If
If cboEmploye <> ""Then
  p_strTitre = p_strTitre & " - Employe : " & cboEmploye.Column(1)
End If

' Critères sur les dates
If cboOperat1 <> "" And txtDateDeb <> "" Then
  p_strTitre = p_strTitre & " - Date de début " & cboOperat1 _
    & " " & txtDateDeb
End If
If cboOperat2 <> "" And txtDateFin <> "" Then
  p_strTitre = p_strTitre & " - Date de fin " & cboOperat1 _
    & " " & txtDateDeb
End If
If p_strTitre <> "" Then
  p_strTitre = Right(p_strTitre, Len(p_strTitre) - 3)
End If

' Ouverture de l'état
DoCmd.OpenReport "Formations", acViewPreview

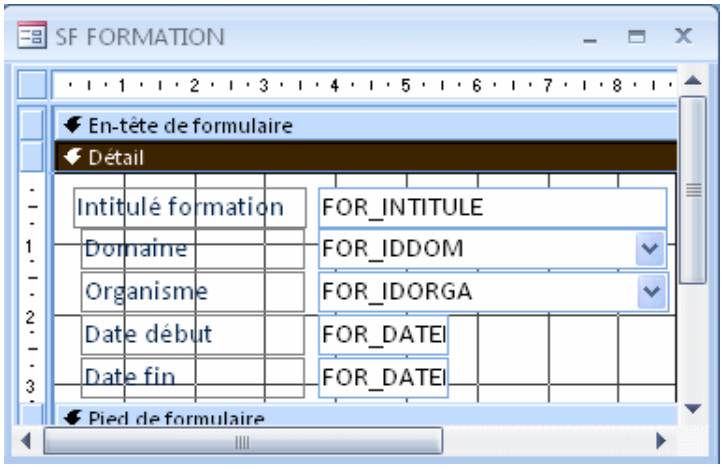
End Sub
```

```
Private Sub cmdFermer_Click()
DoCmd.Close
End Sub
```

Sous-formulaire "SF Formations"

Le sous-formulaire "SF Formations" est basé sur une requête effectuée à partir de la table "Formations".

La propriété RecordSource du formulaire est : "SELECT * FROM FORMATIONS".



Code VBA

```
Option Compare Database
Private Sub Form_DblClick(Cancel As Integer)
Dim lngIDForm As Long
' Affichage du formulaire Formations
IngIDForm = [FOR_IDFORM]
DoCmd.OpenForm "Formations", acNormal, , "[FOR_IDFORM]=" _
```

© ENI Editions - All rigths reserved

```
& lngIDForm, acFormEdit
End Sub
```

Etat "Formations"

L'état "SF Formations" est basé sur la table "Formations".



mardi 11 septembre 2007 15:17:42 — Page 1 sur 1

Liste des formations

Domaine : INFORMATIQUE

Intitulé formation	Domaine	Organisme	Date de début	Date de fin
VBA ACCESS 2007	INFORMATIQUE	DELTA Formation		
LE LANGAGE XML	INFORMATIQUE	Groupe I.S.I. Formation	09/01/2007	11/01/2007
VBA WORD 2007	INFORMATIQUE	I.F.B.C.	09/04/2007	13/04/2007
ASP.NET	INFORMATIQUE	I.N.F.R.E.M	07/05/2007	11/05/2007
INITIATION POWERPOINT 2007	INFORMATIQUE	Groupe F.C.G. Conseil	10/05/2007	11/05/2007
INITIATION EXCEL2007	INFORMATIQUE	Groupe I.S.I. Formation	11/06/2007	15/06/2007
VBA ACCESS 2007	INFORMATIQUE	Groupe F.C.G. Conseil	16/07/2007	20/07/2007
VBA ACCESS 2007	INFORMATIQUE	Groupe F.C.G. Conseil	16/07/2007	20/07/2007

Code VBA

```
Private Sub Report_Open(Cancel As Integer)

' Titre déterminé en fonction des critères de
sélection
txtCriteres.Caption = p_strTitre

' Modifie la source de données si un employé a été
sélectionné
If p_lngEmp <> 0 Then
  Me.RecordSource = "SELECT FORMATIONS.*, EMPLOYES.EMP_IDEMP,
" _
  & "PARTICIPANTS.PART_IDEMP " _
  & "FROM EMPLOYES INNER JOIN (FORMATIONS INNER JOIN " _
  & "PARTICIPANTS ON FORMATIONS.FOR_IDFORM =
PARTICIPANTS.PART_IDFORM)" _
  & "ON EMPLOYES.EMP_IDEMP = PARTICIPANTS.PART_IDEMP"
Else
  Me.RecordSource = "FORMATIONS"
End If
' Critères de sélection
Me.FilterOn = True
Me.Filter = p_strCond

End Sub
```


© ENI Editions - All rigths reserved

Fonctions et instructions VBA

Manipulation de chaînes de caractères

Fonctions

Format() : Mise en forme d'une chaîne.

Instr() : Recherche d'une sous-chaîne de caractères.

Lcase() : Conversion en minuscules.

Left() : Extraction de la partie gauche d'une chaîne de caractères.

Len() : Longueur d'une chaîne.

Ltrim() : Suppression des espaces à gauche.

Mid() : Renvoie une valeur de type Variant (String) contenant un nombre indiqué de caractères extraits d'une chaîne de caractères.

Right() : Extraction de la partie droite d'une chaîne de caractères.

Rtrim() : Suppression des espaces à droite.

Space() : Création d'une chaîne d'espaces.

StrComp() : Comparaison de chaînes.

StrConv : Renvoie une valeur de type Variant (String) convertie au format indiqué.

String() : Création d'une chaîne de caractères répétés.

Trim() : Suppression des espaces gauche/droite.

Ucase() : Conversion en majuscules.

Instructions

Lset : Alignement à gauche d'une chaîne de caractères.

Option Compare: Méthode de comparaison par défaut.

Rset : Alignement à droite d'une chaîne de caractères.

Contrôle du déroulement du programme

Fonctions

Choose() : Renvoie la valeur d'une liste de choix en fonction d'un indice.

DoEvents() : Arrête momentanément l'exécution afin que le système d'exploitation puisse traiter d'autres événements.

IIf() : Renvoie une valeur en fonction d'une condition.

Switch() : Renvoie la valeur associée à la première expression vraie.

Instructions

© ENI Editions - All rigths reserved

Call : Appelle une procédure Sub.

Do...Loop : Création d'une itération.

End : Terminaison de programme.

Exit Do : Sortie d'une itération.

Exit For : Sortie d'une itération.

For Each... Next : Permet de passer en revue tous les éléments d'une collection ou d'un tableau.

For... Next : Création d'une itération.

Function...
End Function : Définit le début et la fin d'une fonction.

GoSub... Return : Appel d'un sous-programme.

Goto : Effectue un branchement inconditionnel vers une ligne déterminée d'une procédure.

If... Then... Else : Création d'une alternative.

On... GoSub : Exécution de sous-programmes en fonction d'une variable.

On... Goto : Branchement vers plusieurs étiquettes de programme en fonction d'une variable.

Property Get: Déclaration d'une procédure Property Get.

Property Let: Déclaration d'une procédure Property Let.

Property Set: Déclaration d'une procédure Property Set.

Select Case : Création d'une alternative.

Stop : Interruption provisoire du code ACCESS BASIC.

Sub... End Sub: Définit le début et la fin d'une procédure.

While... Wend : Création d'une itération.

With : Permet d'exécuter une série d'instructions sur un objet.

Conversion de données

Fonctions

Asc() : Valeur Ascii d'un caractère.

Cbool() : Conversion d'une expression en Boolean.

CByte() : Conversion d'une expression en Byte.

CCur() : Conversion de chaîne de caractères nombre de type Currency.

Cdate() : Conversion d'une expression en Date.

CDbl() : Conversion de chaîne de caractères nombre de type double.

© ENI Editions - All rigths reserved

CDec() : Conversion d'une expression en Décimal.

Chr() : Caractère Ascii d'un nombre.

CInt() : Conversion d'une chaîne de caractères en un nombre de type entier.

CIng() : Conversion d'une chaîne de caractères en un nombre de type long.

CSng() : Conversion d'une chaîne de caractères en un nombre de type single.

CStr() : Conversion d'une expression au format String.

CVar() : Conversion d'une chaîne de caractères en un nombre de type variant.

CVErr : Renvoie un code d'erreur spécifié par l'utilisateur.

DateSerial() : Création d'un nombre date à partir de AA MM JJ.

DateValue() : Conversion d'une expression chaîne ou d'une expression qui représente une date en date.

Day() : Jour d'une date.

Hex() : Conversion d'un nombre en une chaîne le représentant en Hexadécimal.

Oct() : Conversion d'un nombre en une chaîne le représentant en Octal.

Str() :: Conversion de nombre en chaîne de caractères.

Val() : Conversion de chaîne de caractères en nombre.

Date et heure

Fonctions

Date() : Date et heure courante.

DateAdd() : Ajout à une date.

DateDiff() : Différence entre deux dates.

DatePart() : Extraction d'une partie de date.

DateSerial() : Retourne une date à partir du jour, du mois et de l'année.

DateValue() : Retourne une date à partir d'une chaîne.

Day() : Retourne le numéro du jour dans le mois.

Hour() : Retourne le numéro de l'heure (0 à 23) d'une expression heure.

IsDate() : Indique si l'argument Variant peut être converti en Date.

Minute() : Retourne le nombre de minutes (0 à 59) d'une expression heure.

Month() : Retourne le numéro du mois d'une date.

Now() : Date et heure courante.

Second() : Retourne le nombre de secondes (0 à 59) d'une expression heure.

© ENI Editions - All rigths reserved

Time() : Renvoie l'heure système.

Timer() : Nombre de secondes écoulées depuis Minuit.

TimeSerial() : Retourne le temps série d'une heure.

TimeValue() : Retourne une heure à partir d'une chaîne.

WeekDay() : Retourne le numéro du jour dans la semaine à partir d'une date.

Year() : Retourne la valeur de l'année à partir d'une date.

Instructions

Date : Permet de modifier la date système.

Time : Permet de modifier l'heure système.

Déclaration

Instructions

Const: Définit une constante symbolique.

Declare: Déclare une sous-routine de DLL.

Deftype: Définition d'un type par défaut.

Dim: Définit une variable.

Let : Affecte une valeur à une variable.

Option Base: Définit la plus petite valeur de l'indice d'un tableau.

Option Compare: Définit le mode de comparaison des fichiers textes.

Option Explicit : Impose la déclaration des variables.

Option Private Module : Interdit au contenu d'un module d'être référencé à l'extérieur de l'application.

Private: Définit une variable ou une procédure comme étant privée.

Public: Définit une variable globale.

Redim : Redéfinit les dimensions d'un tableau dynamique.

Set : Affecte un objet à une variable.

Static: Définit une variable statique.

Type: Définit des variables structurées utilisateurs.

Domaine

Fonctions

DAvg() : Moyenne d'un domaine.

© ENI Editions - All rigths reserved

DCount(): Dénombrement sur un domaine.

DFirst() : Premier d'un domaine.

DLast() : Dernier d'un domaine.

DLookUp() : Recherche dans un domaine.

DMax() : Maximum d'un domaine.

DMin() : Minimum d'un domaine.

DStDevP() : Ecart type d'une population d'un domaine.

DStDev() : Ecart type d'un échantillon d'un domaine.

DSum() : Somme d'un domaine.

DVar() : Variance d'un échantillon d'un domaine.

DVarP() : Variance d'une population d'un domaine.

Échange dynamique de données

Fonctions

DDE() : Crée un canal DDE et demande d'une information.

DDEInitiate() : Envoi d'une commande via un canal DDE.

DDERequest() : Demande d'une information via un canal DDE.

DDESend() : Envoi d'une information via un canal DDE.

Instructions

DDEExecute : Utilise un canal DDE ouvert pour envoyer une commande à une application.

DDEPoke : Utilise un canal DDE ouvert pour envoyer des données à une application.

DDETerminate : Fermeture d'un canal DDE.

DDETerminate All : Fermeture de tous les canaux DDE ouverts.

Entrées/sorties fichiers

Fonctions

Curdir() : Retourne le dernier répertoire utilisé dans la boîte de dialogue "Ouvrir une base de données".

Dir() : Recherche de fichiers correspondants à un chemin et un nom générique.

Eof() : Fin de fichier.

FileAttr() : Renvoie le mode de gestion d'un fichier ouvert.

FileDateTime() : Retourne la date et l'heure de création ou de dernière modification d'un fichier.

FileLen() : Retourne la taille d'un fichier.

© ENI Editions - All rigths reserved

FreeFile() : Renvoie le prochain numéro de fichier disponible.

GetAttr() : Retourne les attributs d'un fichier ou d'un répertoire.

Loc() : Position actuelle dans le fichier.

Lof() : Renvoie la longueur d'un fichier.

Seek() : Renvoie la position courante dans un fichier.

Spc(): Déplacement de x espaces dans une instruction Print.

Tab(): Déplacement de x tabulations dans une instruction Print.

Instructions

Chdir : Changement de répertoire courant.

Chdrive : Changement d'unité de disque courante.

Close : Fermeture d'un fichier.

FileCopy : Permet la copie de fichier.

Get : Lecture de données dans un fichier séquentiel.

Input : Lecture dans un fichier séquentiel.

Kill : Suppression de fichier du disque.

Line Input : Lecture d'une ligne dans un fichier séquentiel.

Lock : Verrouillage d'accès à un segment de fichier.

Mkdir : Création d'un répertoire.

Name : Affectation d'un nouveau nom à un fichier disque.

Open : Ouverture d'un fichier.

Print : Ecriture de données dans un fichier séquentiel.

Put : Ecriture d'une variable dans un fichier.

Reset : Fermeture de tous les fichiers.

RmDir : Suppression d'un répertoire.

SetAttr : Modification des attributs d'un fichier.

Unlock : Déverrouillage d'accès à un segment de fichier.

Width : Affectation d'une largeur de ligne en sortie sur un fichier.

Write : Écriture de données dans un fichier séquentiel.

Finance

Fonctions

© ENI Editions - All rigths reserved

DDB() : Amortissement d'un bien au cours d'une période spécifique en utilisant la méthode d'amortissement dégressif à taux double ou toute autre méthode précisée.

FV() : Valeur future d'une annuité sur la base d'une série de versements périodiques constants et d'un taux constant.

IPmt() : Montant de l'intérêt pour une période donnée sur la base de versements périodiques constants et d'un taux d'intérêt fixe.

IRR() : Taux de rentabilité interne d'une série de versements.

MIRR() : Taux de rentabilité interne modifié d'une série de versements.

Nper() : Nombre de périodes pour une annuité sur la base de versements périodiques constants et d'un taux d'intérêt constant.

NPV() : Valeur actuelle nette d'un investissement sur la base d'une série de versements périodiques et d'un taux.

Pmt() : Versement pour une annuité sur la base d'une série de versements périodiques constants et d'un taux constant.

PPmt() : Versement principal pour une période donnée d'une annuité sur la base de remboursements périodiques constants et d'un taux d'intérêt constant.

PV() : Valeur présente d'une annuité sur la base de versements périodiques constants et d'un taux d'intérêt constant.

RATE() : Taux d'intérêt par période d'une annuité.

SLN() : Amortissement constant d'un bien durant une période déterminée.

SYD() : Amortissement d'un bien pour une période donnée de manière proportionnelle à l'ordre numérique inverse des années.

Gestion d'erreurs

Fonctions

CVErr : Renvoie la variable Error.

Err() : Code d'erreur sous la forme d'un integer.

Error() : Retourne le message standard correspondant à un numéro d'erreur.

IsError() : Indique si une expression est une valeur d'erreur.

Instructions

Error : Simule une erreur.

On error : Choix du branchement à effectuer en cas d'erreur.

Resume : Choix du branchement à effectuer après traitement d'erreur.

Graphiques dans les états imprimés

Fonctions

QBColor() : Renvoie la valeur d'une couleur RVB en fonction d'un numéro de couleur.

© ENI Editions - All rigths reserved

RGB() : Renvoie un entier long représentant une couleur RVB.

Liaisons et incorporation d'objets

Fonctions

CreateObject() : Crée et renvoie une référence à un objet ActiveX.

GetObject() : Renvoie une référence à un objet fourni par un composant ActiveX.

Manipulation d'objets

Méthodes

CreateControl : Crée un contrôle dans un formulaire ouvert.

CreateForm : Crée un formulaire.

CreateReport : Crée un état.

CreateReportControl : Crée un contrôle dans un état ouvert.

DeleteControl: Détruit un contrôle de formulaire.

DeleteReportControl : Détruit un contrôle d'état.

Mathématiques

Fonctions

Abs() : Valeur absolue d'un nombre.

Atn() : Arctangente d'un nombre.

Cos() : Cosinus d'un angle en radians.

Exp() : Exponentielle.

Fix() : Partie entière d'un nombre. Pour les nombres négatifs, renvoie le premier nombre entier négatif supérieur ou égal.

Int() : Partie entière d'un nombre. Pour les nombres négatifs, renvoie le premier nombre entier négatif inférieur ou égal.

Log() : Logarithme népérien.

Rnd() : Renvoie un nombre aléatoire.

Sgn() : Valeur indiquant le signe d'un nombre.

Sin() : Sinus d'un angle en radians.

Sqr() : Racine carrée d'un nombre.

Tan() : Tangente d'un angle en radians.

Instructions

Randomize : Initialisation du générateur aléatoire.

© ENI Editions - All rigths reserved

SQL

Fonctions

Avg() : Retourne la moyenne des valeurs d'un champ d'une série d'enregistrements.

Count() : Dénombrement d'enregistrements (contrôles ou expressions SQL).

First() : Retourne la valeur d'un champ du premier enregistrement d'une requête, formulaire ou état.

Last() : Retourne la valeur d'un champ du dernier enregistrement d'une requête, formulaire ou état.

Max() : Retourne le maximum des valeurs d'un champ d'une série d'enregistrements.

Min() : Retourne le minimum des valeurs d'un champ d'une série d'enregistrements.

StDev(), StDevP() : Retourne une estimation de l'écart type d'une population ou d'un échantillon de population.

Sum() : Retourne la somme des valeurs d'un champ d'une série d'enregistrements.

Var(), VarP() : Retourne une estimation de la variance d'une population ou d'un échantillon de population.

Tableaux

Fonctions

IsArray() : Renvoie une valeur de type Boolean (booléen) qui indique si une variable est un tableau.

Array() : Renvoie une variable de type Variant contenant un tableau.

Instructions

Erase : Réinitialisation de tableau statique et libération de la mémoire allouée à un tableau dynamique.

Lbound, Ubound : Recherche des bornes inférieures et supérieures d'un tableau.

Option Base : Fixe la borne inférieure des tableaux.

ReDim : Redimensionnement dynamique de tableaux.

Vérification de variables

Fonctions

IsDate() : Teste si une variable est de type date.

IsEmpty() : Teste si une variable est vide.

IsMissing() : Indique si un argument facultatif a été passé à une procédure.

IsNull() : Teste si une variable est Null.

IsNumeric() : Teste si une variable est de type numérique.

IsObject() : Indique si une expression fait référence à un objet.

TypeName() : Renvoie une chaîne indiquant le type d'une variable.

VarType() : Détermine le type de données source d'une variable de type variant.

© ENI Editions - All rigths reserved

Fonctions et instructions diverses

Fonctions

Command() : Contient la partie argument de la ligne de commande utilisée pour lancer Microsoft Access.

CreateGroupLevel() : Création d'un niveau de regroupement dans un état.

CurrentUser() : Renvoie le nom de l'utilisateur actuellement connecté.

DoEvents() : Rend le processeur provisoirement à WINDOWS pour qu'il puisse traiter les événements.

Environ() : Renvoie la chaîne associée à une variable d'environnement.

GetAllSettings : Renvoie la liste des clés (et leurs valeurs respectives) figurant dans une entrée d'application de la base de registre de Windows.

GetSetting : Renvoie une valeur de clé de l'entrée du registre Windows d'une application.

InputBox : Affiche une invite dans une boîte de dialogue, puis renvoie le contenu de la zone de texte sous la forme d'une valeur de type String.

MsgBox() : Affichage d'une boîte de message.

Shell() : Exécution d'un programme.

StrReverse : Renvoie une chaîne contenant des caractères dont l'ordre a été inversé par rapport à une chaîne donnée.

Instructions

AppActivate : Activation d'une fenêtre d'application Windows.

Beep: Émet un Bip.

SendKeys : Envoi d'une séquence de touches clavier.

DeleteSetting : Supprime une section ou un paramètre de clé d'une entrée de registre Windows.

Rem : Permet d'inclure des remarques explicatives dans un programme.

SaveSetting : Sauvegarde ou crée une entrée d'application dans le registre Windows.

Codes erreurs récupérables

Des erreurs récupérables peuvent survenir lors de l'exécution d'une application. Certaines d'entre elles peuvent également se produire lors du développement ou de la compilation. Vous pouvez tester et répondre à des erreurs récupérables à l'aide de l'instruction On Error et de l'objet Err.

Code Erreur	Description
3	Return sans GoSub.
5	Appel de procédure incorrect.
6	Dépassement de capacité.
7	Mémoire insuffisante.

© ENI Editions - All rigths reserved

9	Indice en dehors de la plage.
10	Ce tableau est fixe ou temporairement verrouillé.
11	Division par zéro.
13	Type incompatible.
14	Espace de chaîne insuffisant.
16	Expression trop complexe.
17	Impossible d'exécuter l'opération requise.
18	Interruption par l'utilisateur.
20	Reprise sans erreur.
28	Espace pile insuffisant.
35	Sub, Function ou Property non définie.
47	Trop de clients d'application pour la DLL.
48	Erreur de chargement de la DLL.
49	Convention d'appel de DLL incorrecte.
51	Erreur interne.
52	Nom ou numéro de fichier incorrect.
53	Fichier introuvable.
54	Mode d'accès au fichier incorrect.
55	Fichier déjà ouvert.
57	Erreur d'entrée/sortie de périphérique.
58	Ce fichier existe déjà.
59	Longueur d'enregistrement incorrecte.
61	Disque plein.
62	L'entrée dépasse la fin de fichier.
63	Numéro d'enregistrement incorrect.
67	Trop de fichiers.
68	Périphérique non disponible.
70	Permission refusée.
71	Disque non prêt.
74	Impossible de renommer avec un lecteur différent.

© ENI Editions - All rigths reserved

75 Erreur dans le chemin d'accès.

76 Chemin d'accès introuvable.

91 Variable objet ou variable bloc With non définie.

92 Boucle For non initialisée.

93 Format de chaîne incorrect.

94 Utilisation incorrecte de Null.

97 Impossible d'appeler une procédure Friend sur un objet qui n'est pas une instance de classe de définition.

98 Un appel de propriété ou de méthode ne peut pas faire référence à un objet privé ni en tant qu'argument ni en tant que valeur renvoyée.

298 Impossible de charger la DLL système.

320 Impossible d'utiliser des noms de périphériques de caractères dans les noms de fichiers spécifiés.

321 Format de fichier incorrect.

322 Impossible de créer le fichier temporaire nécessaire.

325 Format incorrect dans le fichier ressource.

327 La valeur de l'objet de données nommé est introuvable.

328 Paramètre incorrect. Impossible d'écrire des tableaux.

335 Impossible d'accéder au registre système.

336 Le composant n'est pas correctement enregistré.

337 Composant introuvable.

338 Exécution incorrecte du composant.

360 Objet déjà chargé.

361 Impossible de charger ou de décharger cet objet.

363 Le contrôle spécifié est introuvable.

364 L'objet était déchargé.

365 Impossible de décharger dans ce contexte.

368 Le fichier spécifié est périmé. Ce programme requiert une version plus récente.

371 L'objet spécifié ne peut être utilisé come propriétaire de la feuille pour la méthode Show.

380 Valeur de propriété incorrecte.

381 Index du tableau de propriétés incorrect.

382 Property Set ne peut pas être exécutée en mode exécution.

© ENI Editions - All rigths reserved

383 Property Set ne peut être utilisée avec une propriété en lecture seule.

385 Index de table de propriété requis.

387 Property Set non autorisée.

393 Property Get ne peut pas être exécutée en mode exécution.

394 Property Get ne peut pas être exécutée sur une propriété en écriture seule.

400 Feuille déjà affichée; affichage modal impossible.

402 Le code doit d'abord fermer la feuille modale de premier plan.

419 Autorisation d'utiliser l'objet refusée.

422 Propriété introuvable.

423 Propriété ou méthode introuvable.

424 Objet requis.

425 Utilisation d'objet incorrecte.

429 Un composant ne peut pas créer l'objet ou fournir une référence à cet objet.

430 Cette classe ne gère pas Automation.

432 Nom de fichier ou de classe non trouvé pendant une opération Automation.

438 Cet objet ne gère pas cette propriété ou méthode.

440 Erreur Automation.

442 La connexion à la bibliothèque de types ou d'objets pour un processus distant a été perdue.

443 L'objet Automation n'a pas de valeur par défaut.

445 L'objet ne gère pas cette action.

446 L'objet ne gère pas les arguments nommés.

447 L'objet ne gère pas les paramètres régionaux en cours.

448 Argument nommé introuvable.

449 Argument non facultatif ou affectation de propriété incorrecte.

450 Nombre d'arguments incorrect ou affectation de propriété non valide.

451 Cet objet n'est pas une collection.

452 Numéro incorrect.

453 Fonction DLL spécifiée introuvable.

454 Ressource de code introuvable.

455 Erreur de verrouillage de ressource de code.

© ENI Editions - All rigths reserved

457 Cette clé est déjà associée à un élément de cette collection.

458 Cette variable utilise un type non géré par Visual Basic.

459 Ce composant ne gère pas le jeu d'événements.

460 Format de Presse-papiers incorrect.

461 Méthode ou membre de données introuvable.

462 Le serveur distant n'existe pas ou n'est pas disponible.

463 La classe n'est pas enregistrée sur la machine locale.

480 Impossible de créer une image en mode AutoRedraw.

481 Image incorrecte.

482 Erreur d'imprimante.

483 Le gestionnaire d'imprimante ne gère pas la propriété spécifiée.

484 Problème pour obtenir des informations du gestionnaire d'imprimante du système. Assurez-vous que le gestionnaire d'imprimante est installé correctement.

485 Type d'image incorrect.

486 Impossible d'imprimer l'image de feuille sur ce type d'imprimante.

520 Impossible de vider le Presse-papiers.

521 Impossible d'ouvrir le Presse-papiers.

735 Impossible d'enregistrer le fichier dans un répertoire TEMP.

744 Le texte recherché est introuvable.

746 Remplacements trop longs.

31001 Mémoire insuffisante.

31004 Pas d'objet.

31018 Classe non définie.

31027 Impossible d'activer l'objet.

31032 Impossible de créer un objet incorporé.

31036 Erreur à l'enregistrement dans le fichier.

31037 Erreur de chargement à partir du fichier.

© ENI Editions - All rigths reserved

Constantes VBA

Visual Basic pour Applications permet de définir des constantes afin d'améliorer la lisibilité du code et d'en faciliter la maintenance.

Vous pouvez également utiliser les constantes VBA (constantes intrinsèques) suivantes partout dans votre code.

Constantes de couleur

Constante	Valeur	Description
vbBlack	0x0	Noir
vbRed	0xFF	Rouge
vbGreen	0xFF00	Vert
vbYellow	0xFFFF	Jaune
vbBlue	0x0FF0000	Bleu
vbMagenta	0x0FF00FF	Magenta
vbCyan	0x0FFFF00	Cyan
vbWhite	0x0FFFFFF	Blanc

Constantes de date

Constante	Valeur	Description
vbSunday	1	Dimanche
vbMonday	2	Lundi
vbTuesday	3	Mardi
vbWednesday	4	Mercredi
vbThursday	5	Jeudi
vbFriday	6	Vendredi
vbSaturday	7	Samedi

Constantes des touches clavier lettres et chiffres

Les valeurs des touches A à Z sont les mêmes que leurs équivalents ASCII.

Constante	Valeur	Description
vbKeyA	65	Touche A
vbKeyB	66	Touche B
vbKeyC	67	Touche C
...	...	...
vbKeyZ	90	Touche Z

Les valeurs des touches 0 à 9 sont les mêmes que leurs équivalents ASCII.

Constante	Valeur	Description
vbKey0	48	Touche 0
vbKey1	49	Touche 1
vbKey2	50	Touche 2
...	...	...
vbKey9	57	Touche 9

© ENI Editions - All rigths reserved

Constantes de touches de fonction

Constante	Valeur	Description
vbKeyF1	0x70	Touche F1
vbKeyF2	0x71	Touche F2
...	...	...
vbKeyF11	0x7A	Touche F11
...	...	...
vbKeyF16	0x7F	Touche F16

Constantes de touches diverses

Constante	Valeur	Description
vbKeyCancel	0x3	Touche ANNULER
vbKeyBack	0x8	Touche **Retour arrière**
vbKeyTab	0x9	Touche **Tab**
vbKeyClear	0xC	Touche EFFACER
vbKeyReturn	0xD	Touche **Entrée**
vbKeyShift	0x10	Touche **Shift**
vbKeyControl	0x11	Touche **Ctrl**
vbKeyMenu	0x12	Touche MENU
vbKeyPause	0x13	Touche PAUSE
vbKeyCapital	0x14	Touche **Caps Lock**
vbKeyEscape	0x1B	Touche **Echap**
vbKeySpace	0x20	Touche ESPACE
vbKeyPageUp	0x21	Touche **Pg Up**
vbKeyPageDown	0x22	Touche **Pg Dn**
vbKeyEnd	0x23	Touche **Fin**
vbKeyHome	0x24	Touche **Début**
vbKeyUp	0x25	Touche **Flèche haut**
vbKeyDown	0x26	Touche **Flèche bas**
vbKeyRight	0x27	Touche **Flèche droite**
vbKeyLeft	0x28	Touche **Flèche gauche**
vbKeySelect	0x29	Touche Sélection
vbKeyPrint	0x2A	Touche **Impr écran**
vbKeyExecute	0x2B	Touche EXÉCUTER
vbKeySnapshot	0x2C	Touche SNAPSHOT
vbKeyInsert	0x2D	Touche **Ins**
vbKeyDelete	0x2E	Touche **Suppr**
vbKeyHelp	0x2F	Touche AIDE
vbKeyNumlock	0x90	Touche **Caps lock**
vbKeyLButton	0x1	Bouton gauche de la souris
vbKeyRButton	0x2	Bouton droit de la souris

© ENI Editions - All rigths reserved

Raccourcis-clavier

F1 : Aide sur le mot clé sur lequel se trouve le curseur.

F5 : Exécute une procédure.

F8 : Mode Pas à pas détaillé.

Shift F8 : Mode Pas à pas principal.

Ctrl Shift F8: Mode Pas à pas sortant.

Ctrl F8 : Exécute jusqu'au curseur.

Shift F9 : Affiche la valeur en cours.

F9 : Affiche ou désactiver le point d'arrêt.

Ctrl Shift F9 : Supprime tous les points d'arrêt.

Ctrl F9 : Définit l'instruction suivante.

© ENI Editions - All rigths reserved